



(12) **United States Patent**
Li et al.

(10) **Patent No.:** **US 12,389,041 B2**
(45) **Date of Patent:** ***Aug. 12, 2025**

(54) **ENCODING AND DECODING WITH MERGE MODE AND BLOCK PARTITION INDEX**

(71) Applicant: **Panasonic Intellectual Property Corporation of America**, Torrance, CA (US)

(72) Inventors: **Jing Ya Li**, Singapore (SG); **Che Wei Kuo**, Singapore (SG); **Chong Soon Lim**, Singapore (SG); **Chu Tong Wang**, Singapore (SG); **Han Boon Teo**, Singapore (SG); **Hai Wei Sun**, Singapore (SG); **Kiyofumi Abe**, Osaka (JP); **Takahiro Nishi**, Nara (JP); **Tadamasa Toma**, Osaka (JP); **Yusuke Kato**, Osaka (JP)

(73) Assignee: **Panasonic intellectual Property Corporation of America**, Torrance, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/526,683**

(22) Filed: **Dec. 1, 2023**

(65) **Prior Publication Data**

US 2024/0114169 A1 Apr. 4, 2024

Related U.S. Application Data

(63) Continuation of application No. 17/727,299, filed on Apr. 22, 2022, now Pat. No. 11,877,009, which is a (Continued)

(51) **Int. Cl.**
H04N 19/66 (2014.01)
H04N 19/122 (2014.01)

(Continued)

(52) **U.S. Cl.**
CPC **H04N 19/66** (2014.11); **H04N 19/122** (2014.11); **H04N 19/176** (2014.11); **H04N 19/103** (2014.11); **H04N 19/96** (2014.11)

(58) **Field of Classification Search**
CPC H04N 19/66; H04N 19/176; H04N 19/122
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2019/0364279 A1 11/2019 Yasugi et al.
2020/0366901 A1* 11/2020 Ye H04N 19/44
(Continued)

OTHER PUBLICATIONS

H.265 (ISO/IEC 23008-2 HEVC (High Efficiency Video Coding)), "Information technology—High efficiency coding and media delivery in heterogeneous environments—Part 2: High efficiency video coding," International Standard, First Edition, Dec. 1, 2013. (312 pages).

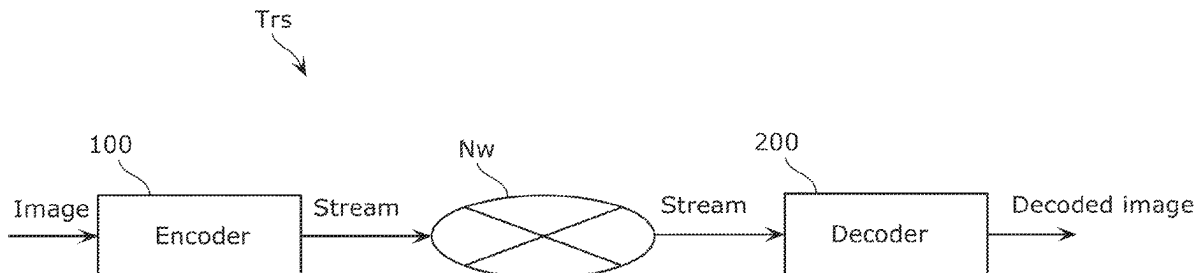
Primary Examiner — Richard T Torrente

(74) *Attorney, Agent, or Firm* — Seed IP Law Group LLP

(57) **ABSTRACT**

An encoder determines, based on a width and a height of a block, whether or not to disable a prediction mode in which the block is split along a partitioning line defined by a distance and an angle and then prediction is performed; and encodes the block with the prediction mode disabled or not disabled according to a result of the determination on whether or not to disable the prediction mode. Here, the distance is the shortest distance between the center of the block and the partitioning line, and the angle is an angle representing a direction from the center of the block toward the partitioning line in the shortest distance. The encoder determines to disable the prediction mode when (i) a width-to-height ratio is at least 8 or (ii) a height-to-width ratio is at least 8.

2 Claims, 119 Drawing Sheets



Related U.S. Application Data

continuation of application No. 17/726,248, filed on Apr. 21, 2022, now Pat. No. 11,722,699, which is a continuation of application No. 17/725,084, filed on Apr. 20, 2022, now Pat. No. 11,849,147, which is a continuation of application No. 17/724,298, filed on Apr. 19, 2022, now Pat. No. 11,750,842, which is a continuation of application No. 17/723,324, filed on Apr. 18, 2022, now Pat. No. 11,838,550, which is a continuation of application No. 17/116,137, filed on Dec. 9, 2020, now Pat. No. 11,363,299.

(60) Provisional application No. 62/947,185, filed on Dec. 12, 2019.

(51) **Int. Cl.**

H04N 19/176 (2014.01)
H04N 19/103 (2014.01)
H04N 19/96 (2014.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2020/0374528	A1 *	11/2020	Huang		H04N 19/103
2021/0051335	A1 *	2/2021	Liao		H04N 19/503
2021/0105477	A1 *	4/2021	Koo		H04N 19/91
2021/0120265	A1 *	4/2021	Lee		H04N 19/54

* cited by examiner

FIG. 1

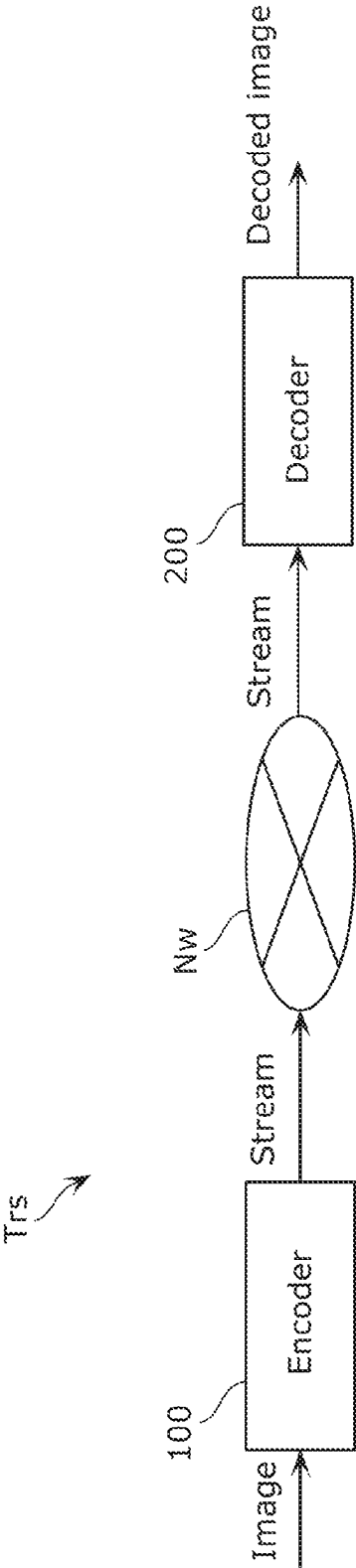


FIG. 2

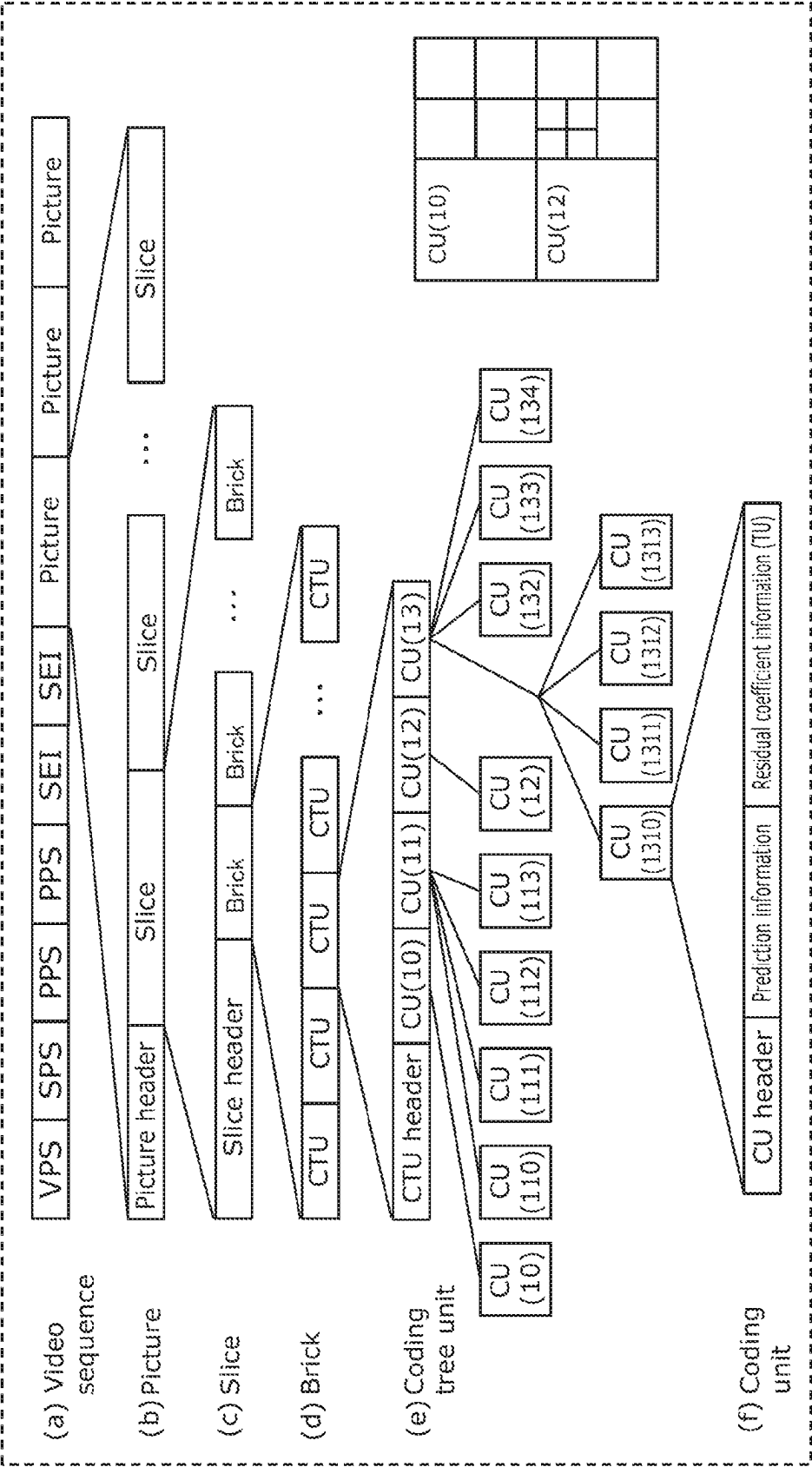


FIG. 3

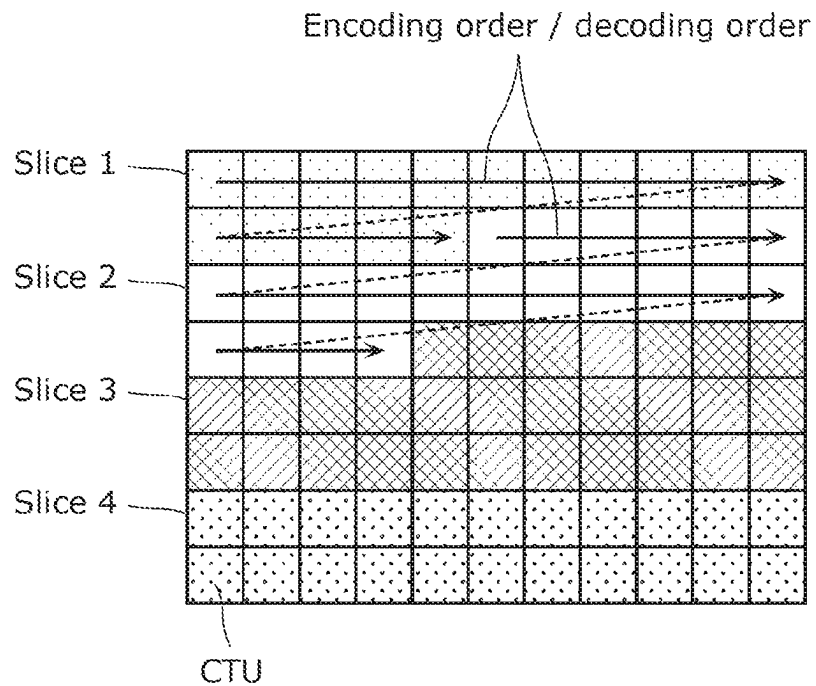


FIG. 4

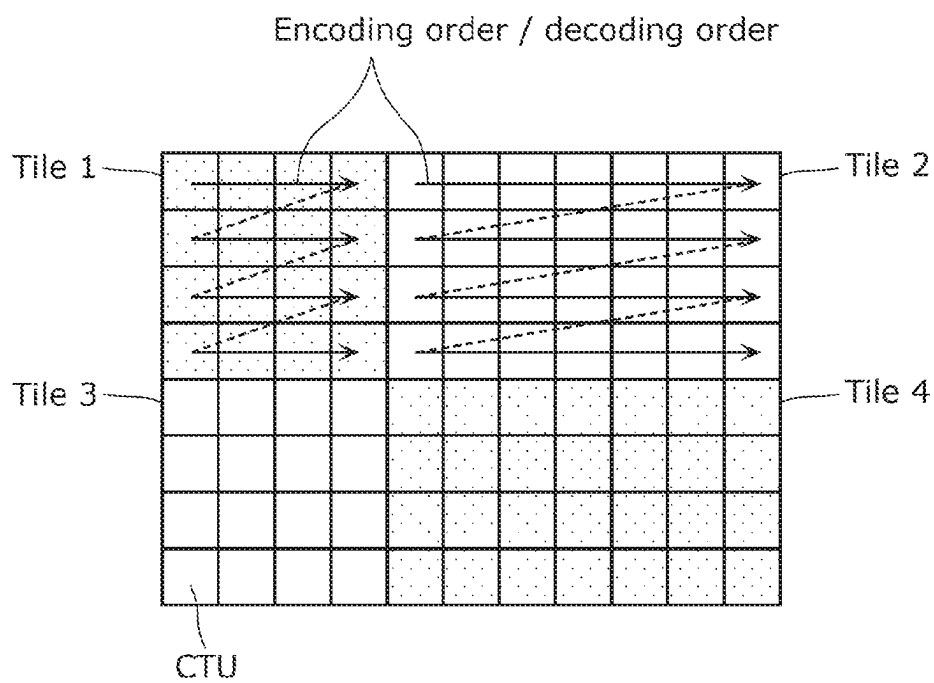


FIG. 5

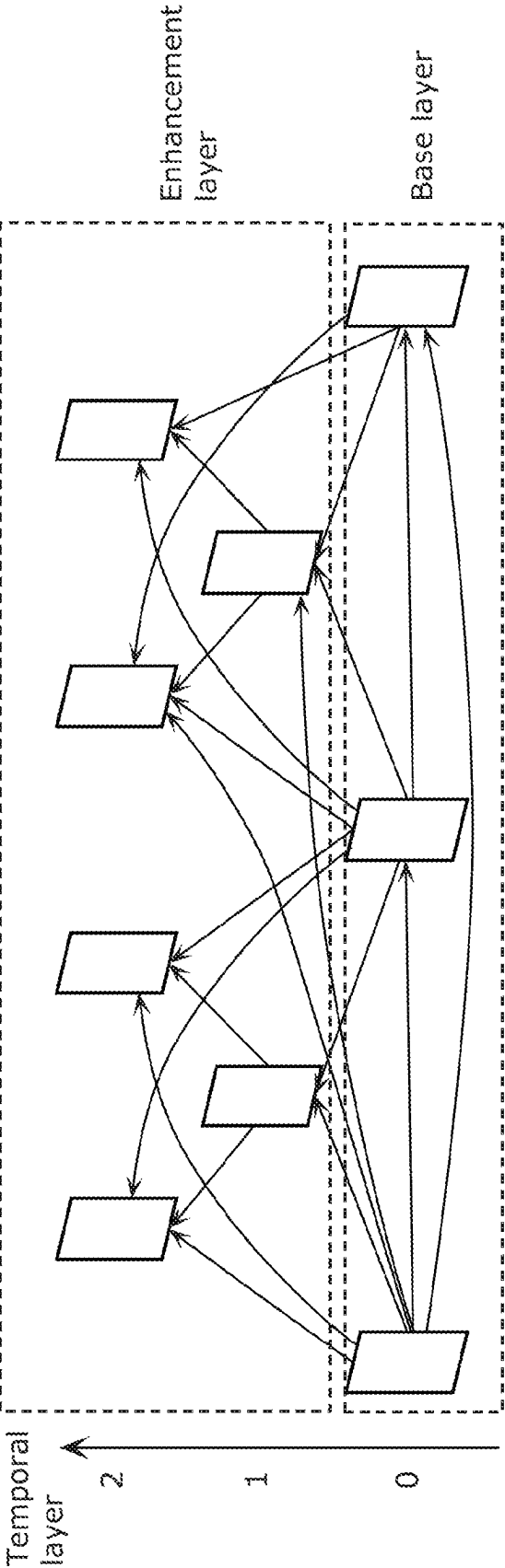


FIG. 6

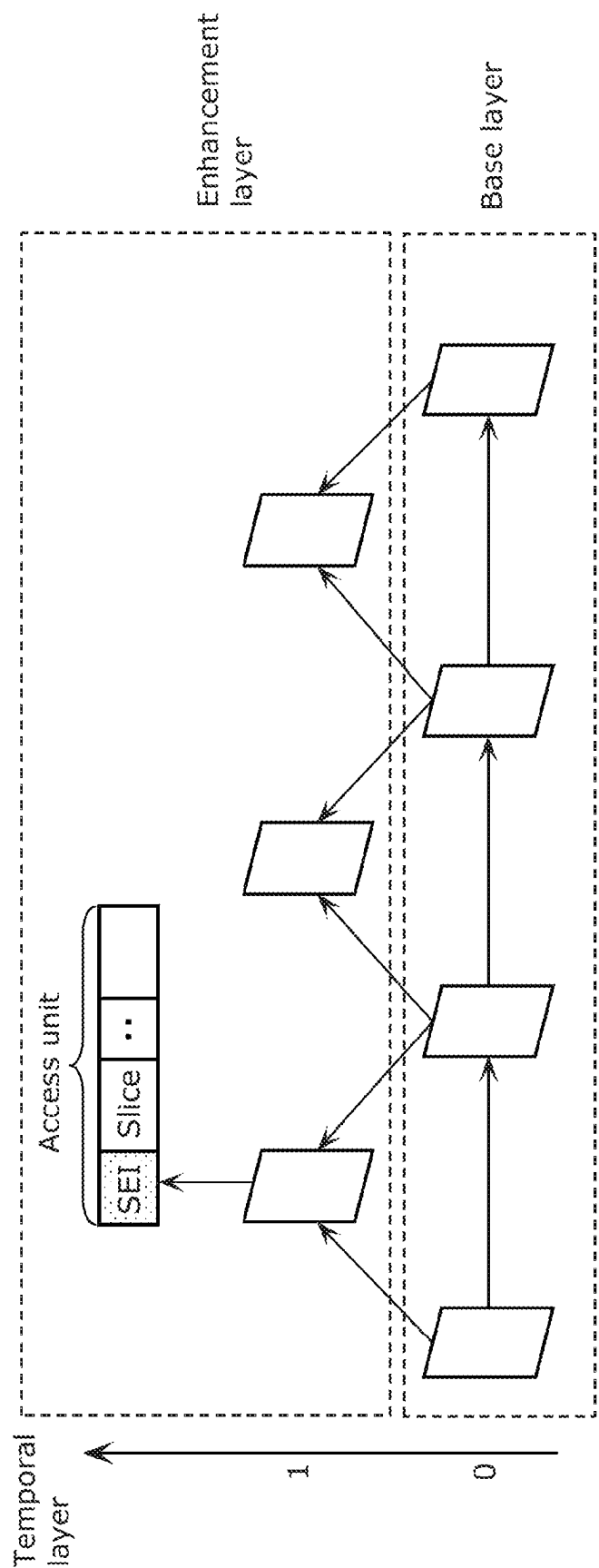


FIG. 7

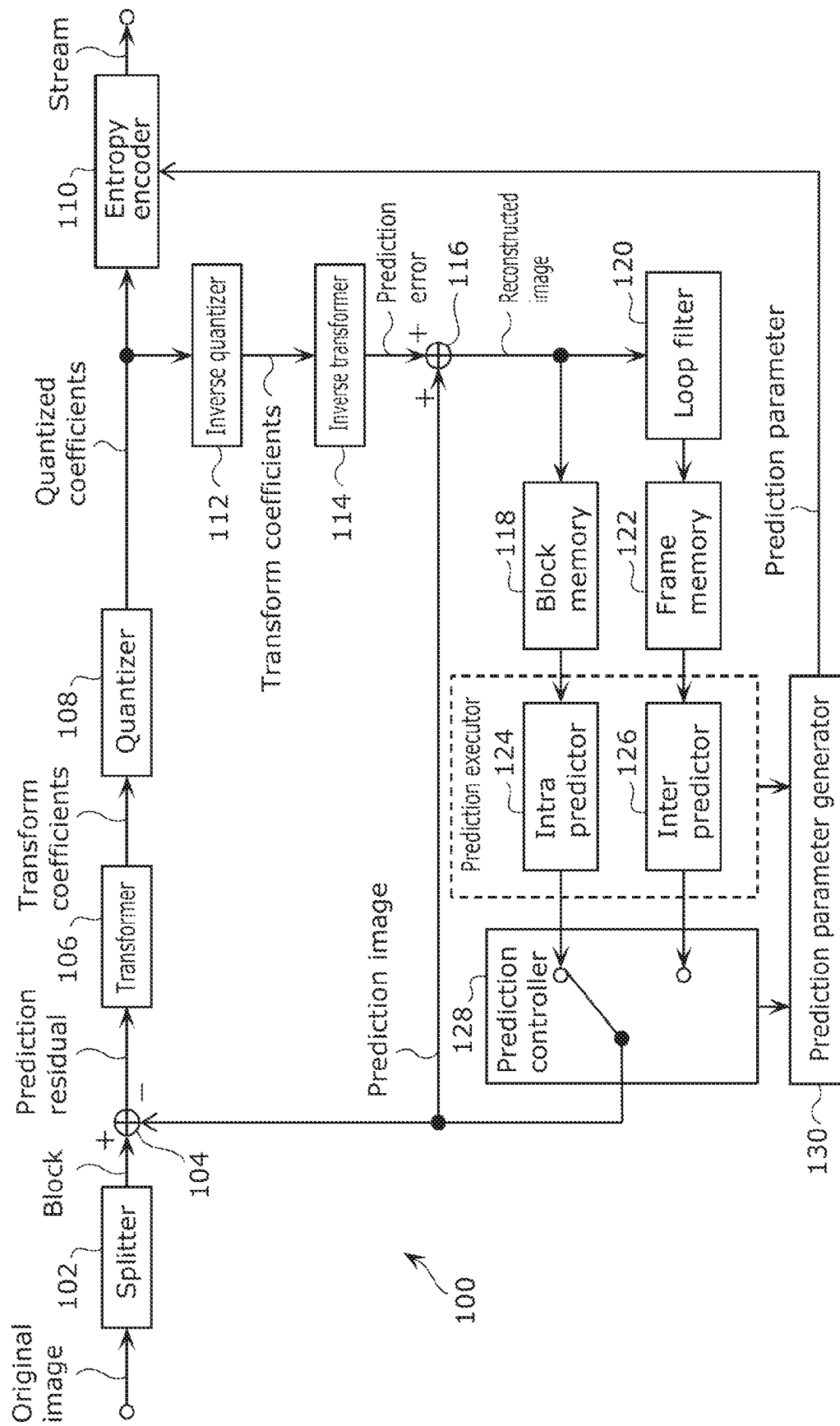


FIG. 8

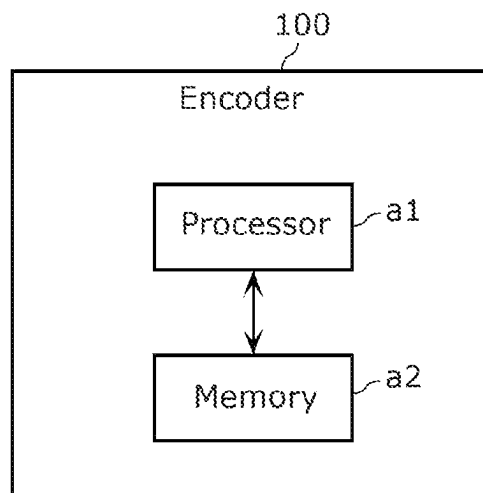


FIG. 9

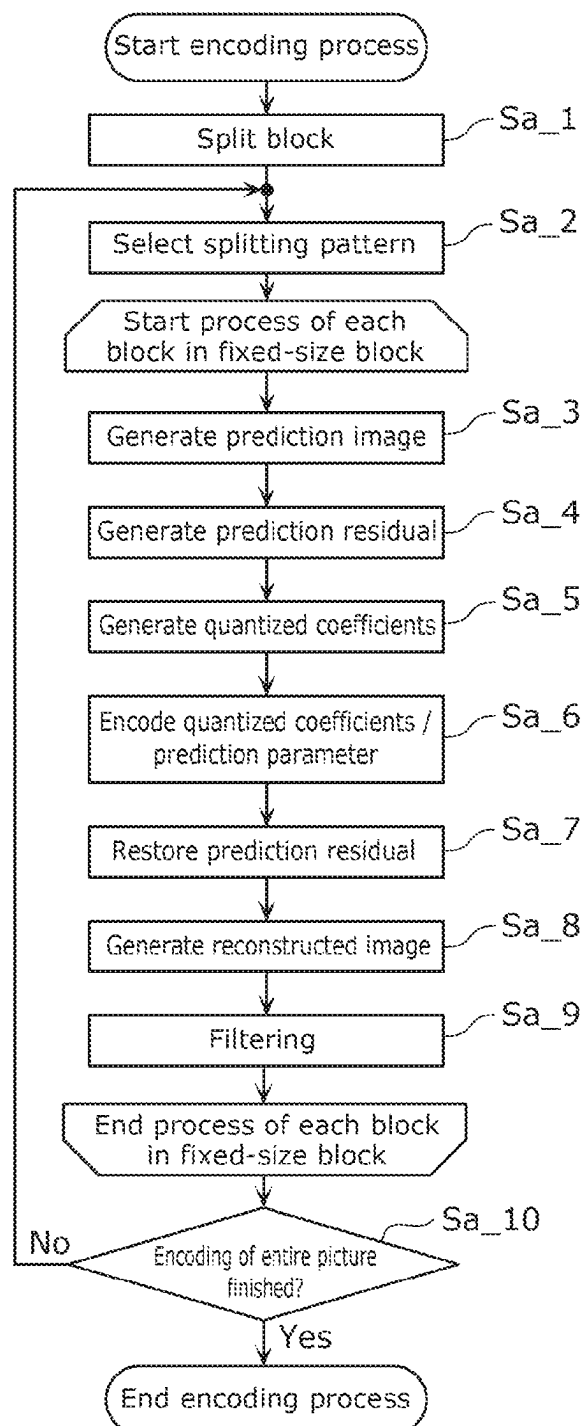


FIG. 10

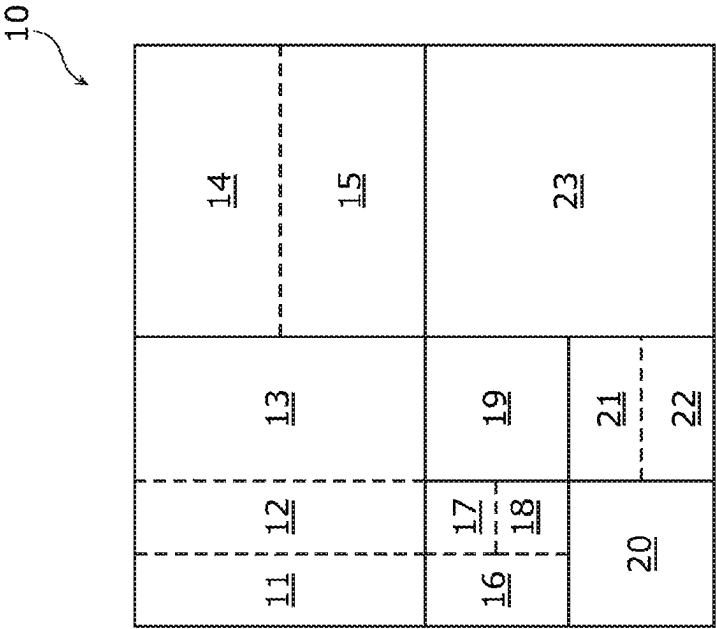


FIG. 11

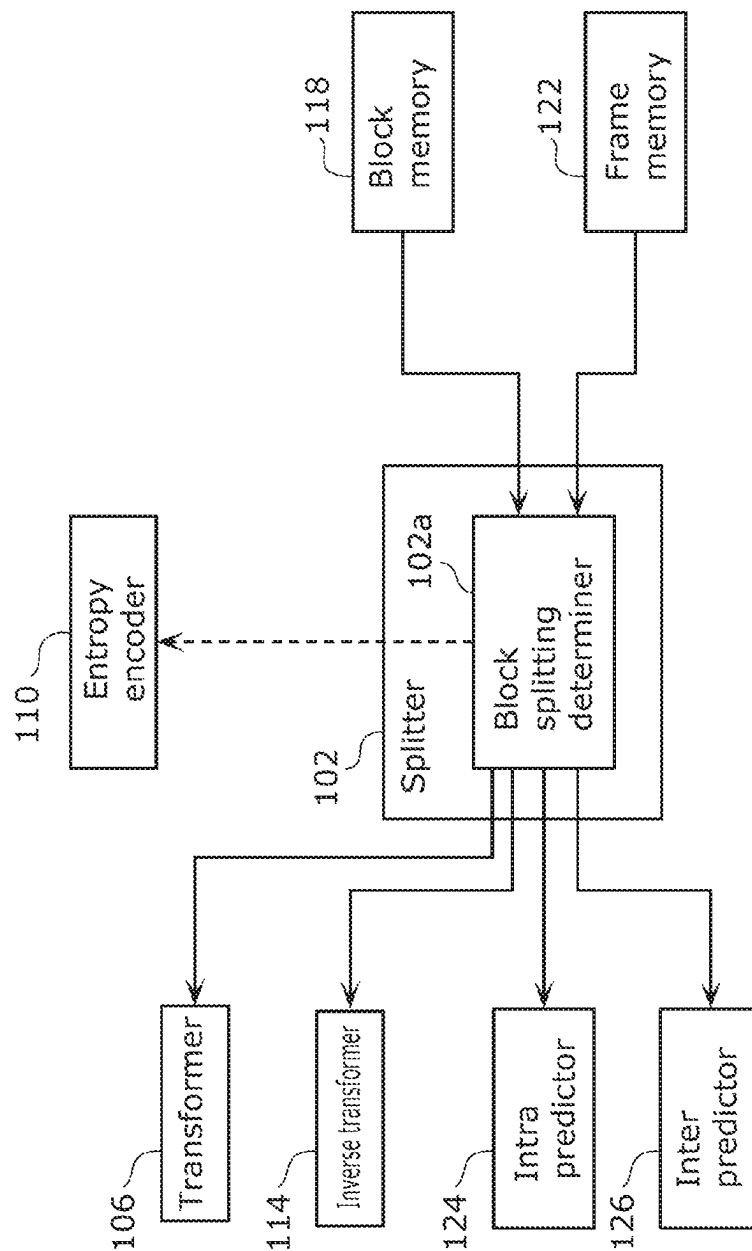


FIG. 12

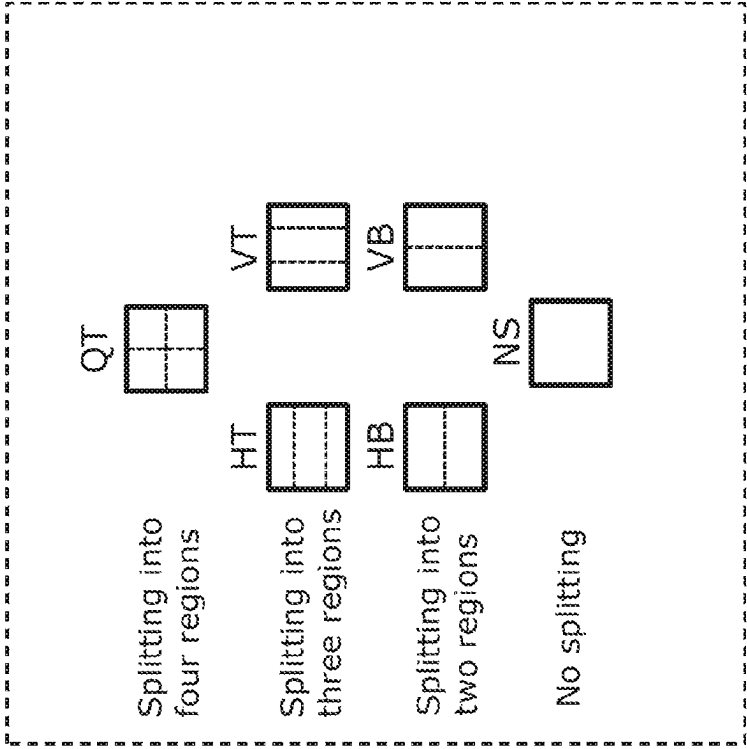


FIG. 13A

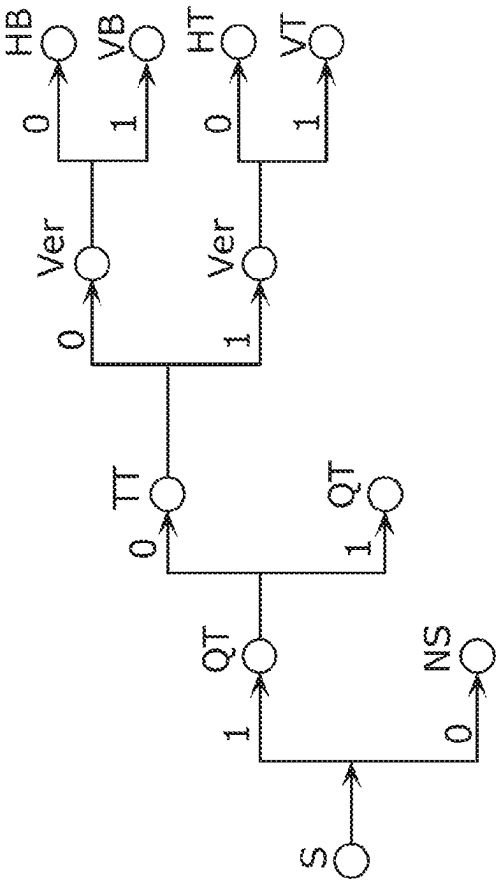


FIG. 13B

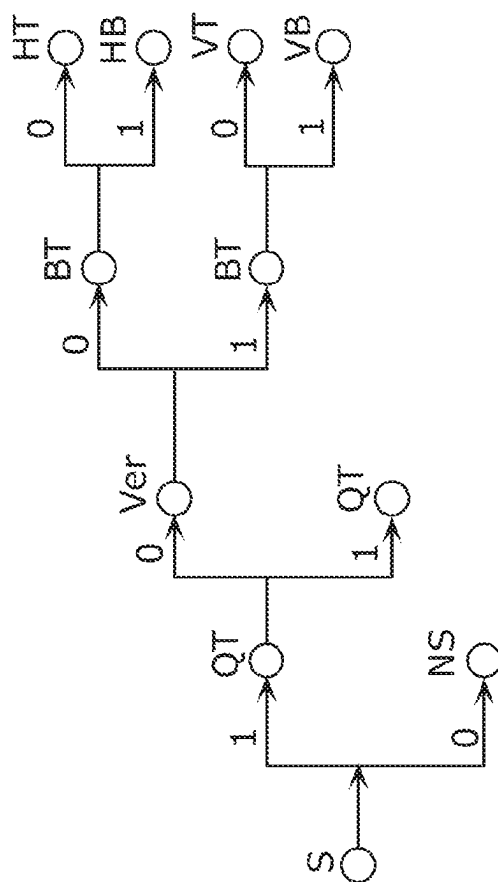


FIG. 14

Transform type	Basis function $T_i(j)$, $i, j=0, 1, \dots, N-1$
DCT-II	$T_i(j) = \omega_0 \cdot \sqrt{\frac{2}{N}} \cdot \cos\left(\frac{n \cdot i \cdot (2j + 1)}{2N}\right)$ $\text{where } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i = 0 \\ 1 & i \neq 0 \end{cases}$
DCT-V	$T_i(j) = \omega_0 \cdot \omega_1 \cdot \sqrt{\frac{2}{2N-1}} \cdot \cos\left(\frac{2n \cdot i \cdot j}{2N-1}\right)$ $\text{where } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i = 0 \\ 1 & i \neq 0 \end{cases}, \omega_1 = \begin{cases} \sqrt{\frac{2}{N}} & j = 0 \\ 1 & j \neq 0 \end{cases}$
DCT-VIII	$T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \cos\left(\frac{n \cdot (2i+1) \cdot (2j+1)}{4N+2}\right)$
DST-I	$T_i(j) = \sqrt{\frac{2}{N+1}} \cdot \sin\left(\frac{n \cdot (i+1) \cdot (j+1)}{N+1}\right)$
DST-VII	$T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \sin\left(\frac{n \cdot (2i+1) \cdot (j+1)}{2N+1}\right)$

FIG. 15

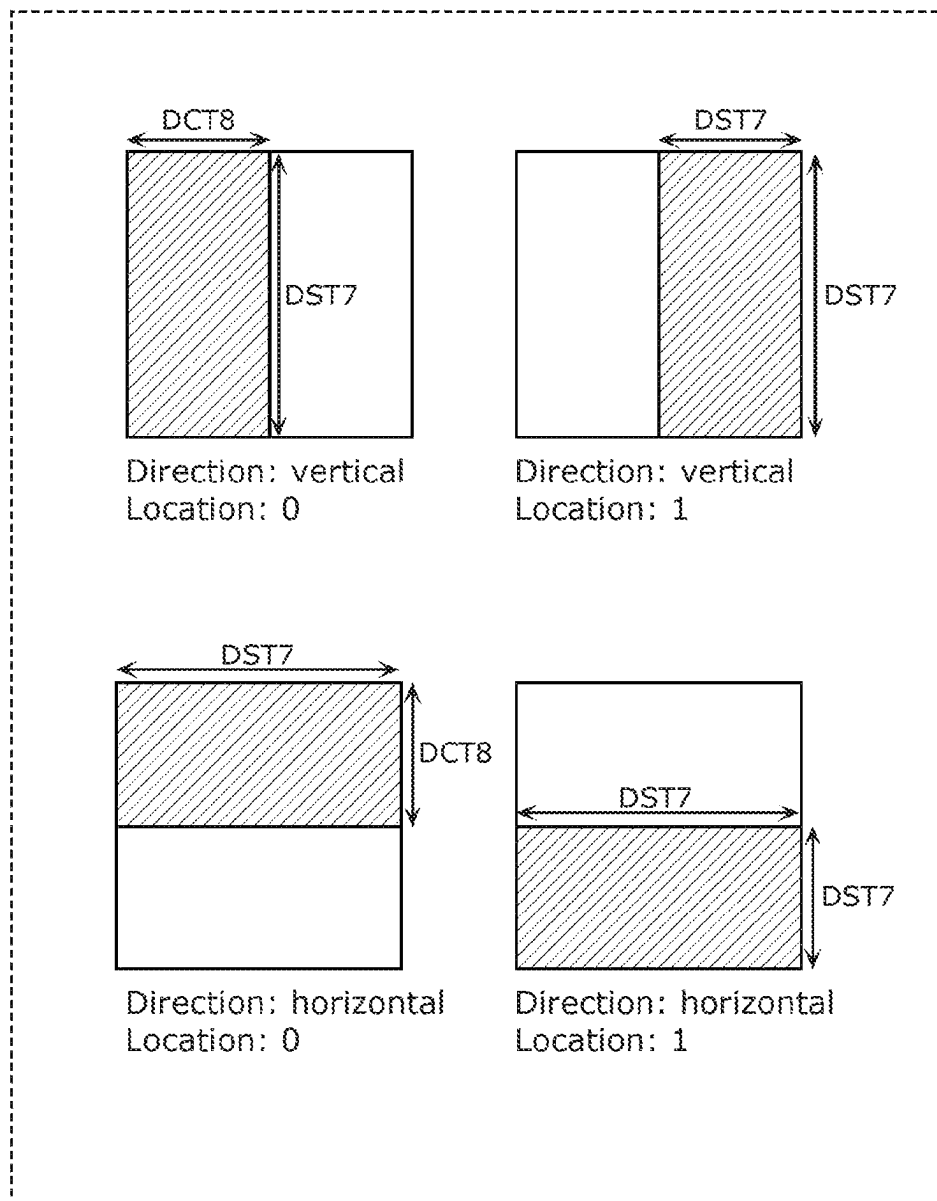


FIG. 16

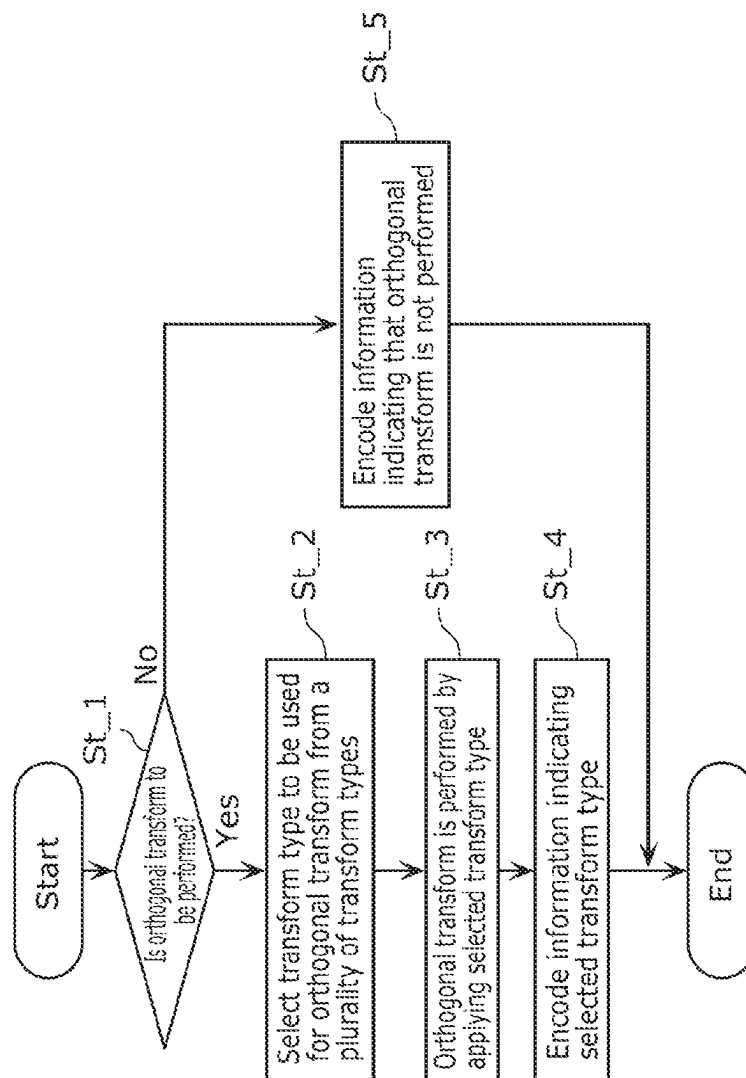


FIG. 17

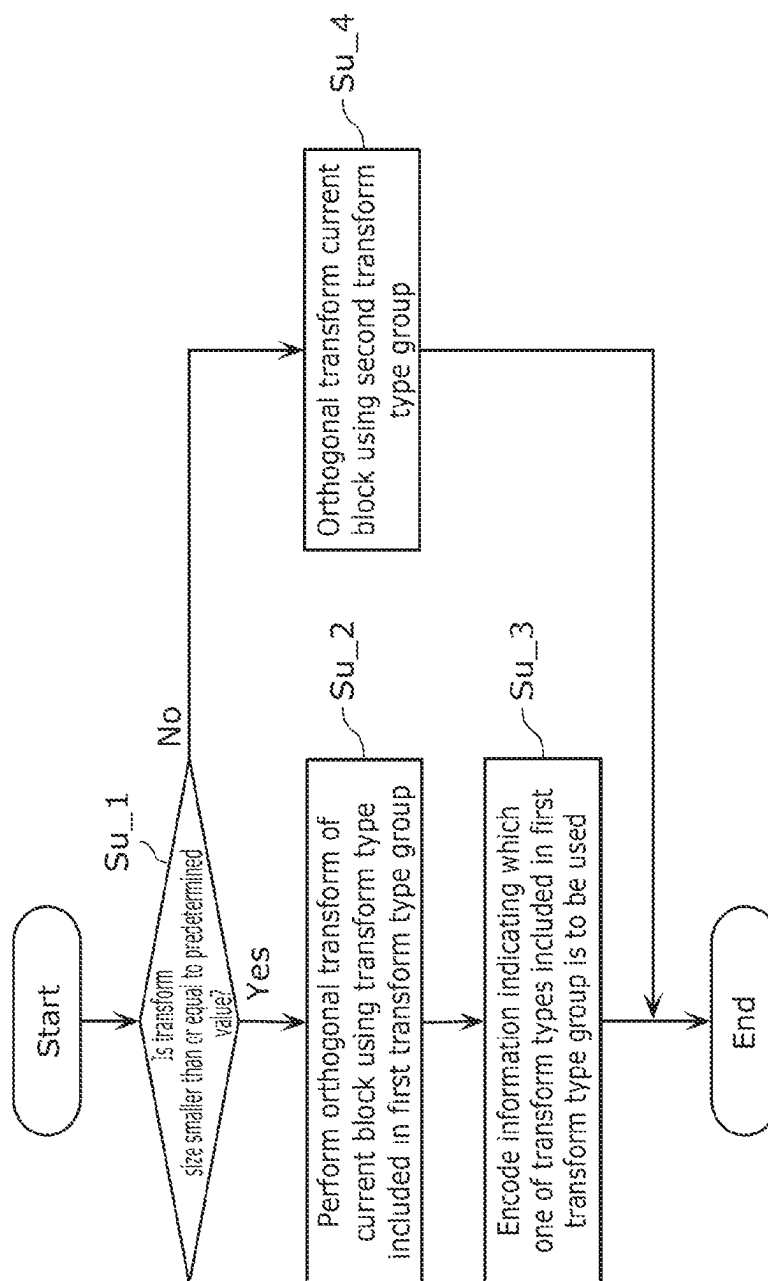


FIG. 18

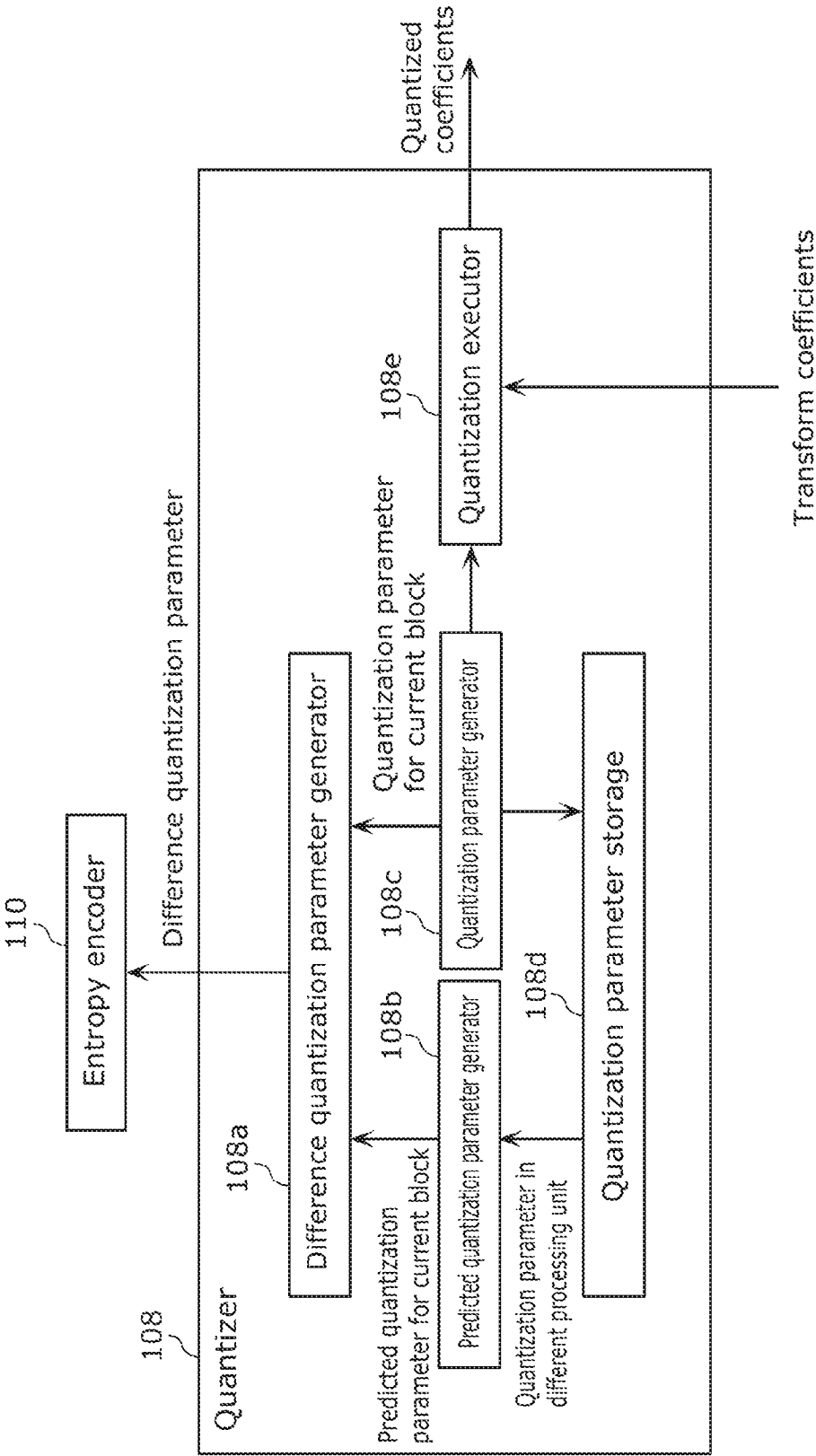


FIG. 19

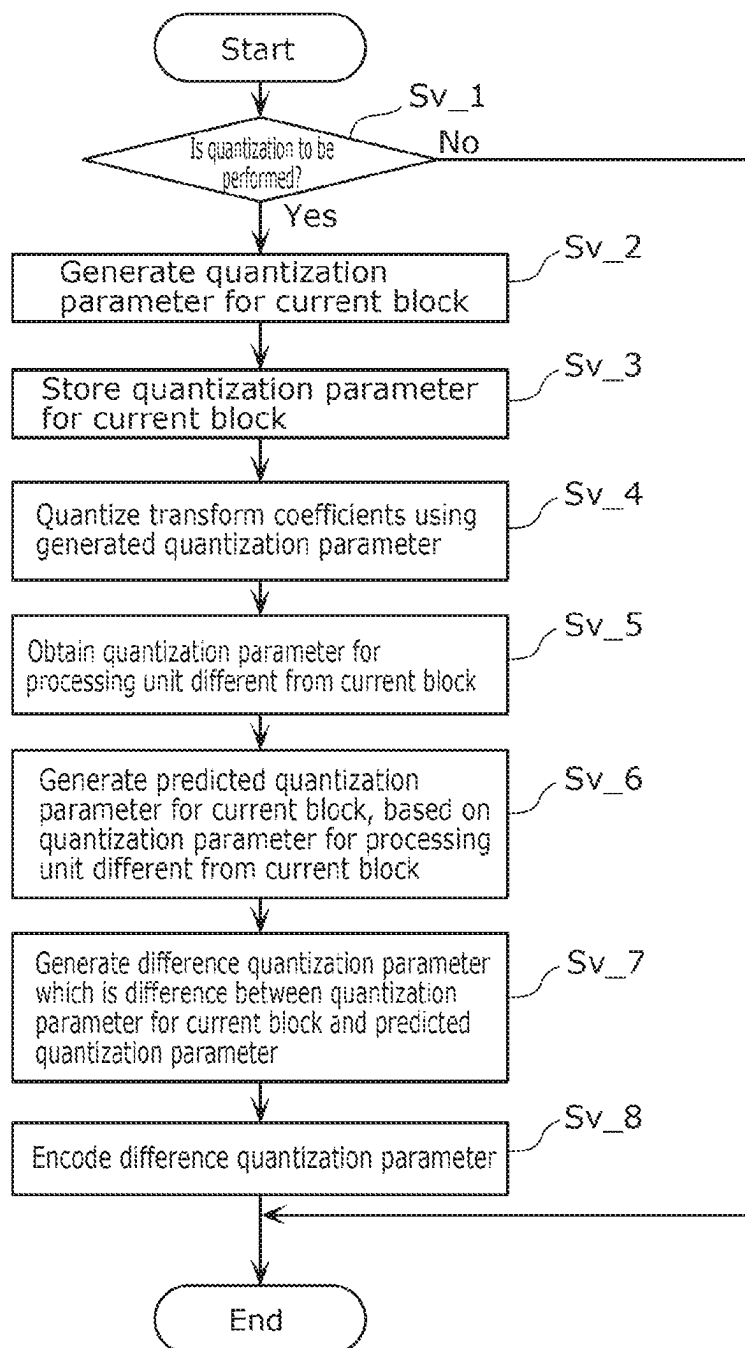


FIG. 20

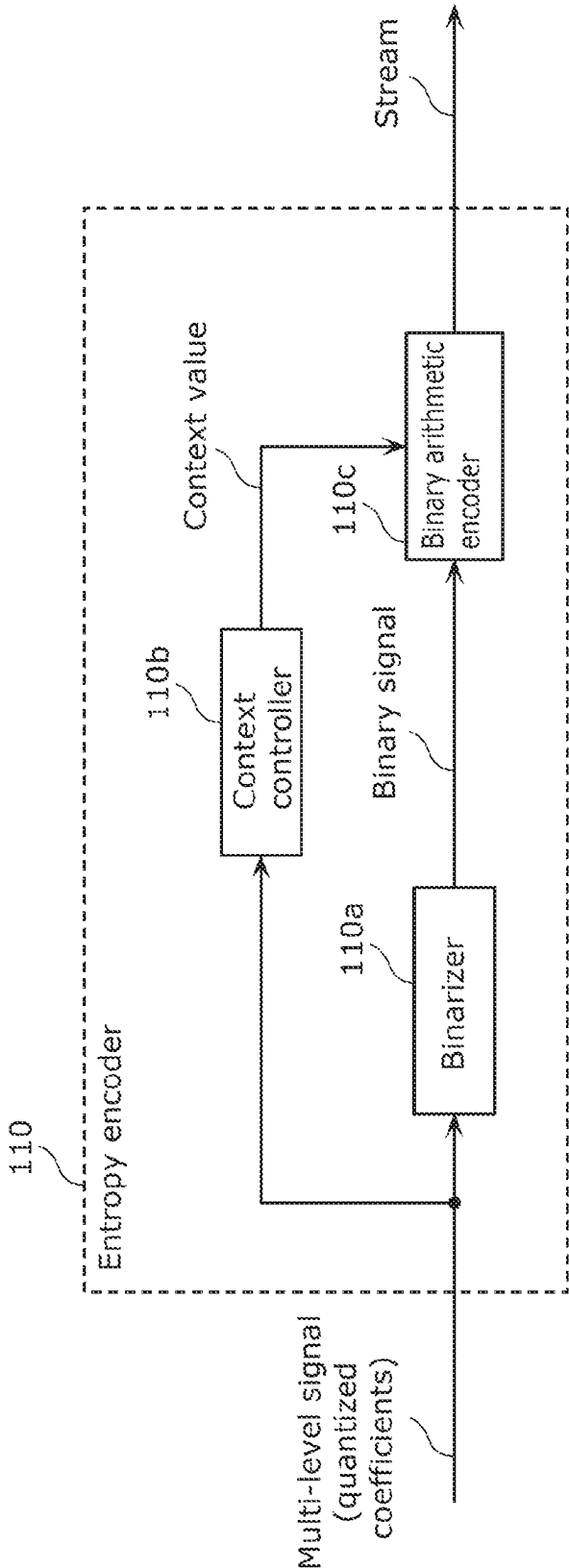


FIG. 21

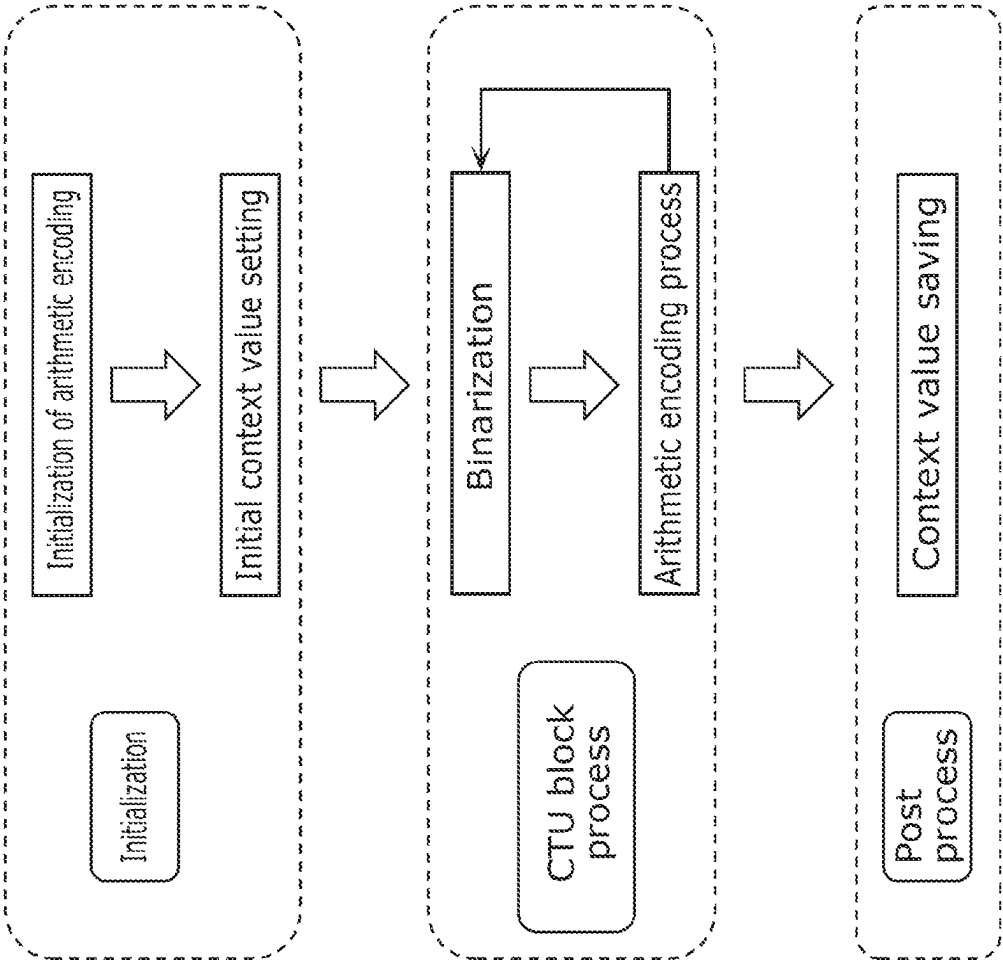


FIG. 22

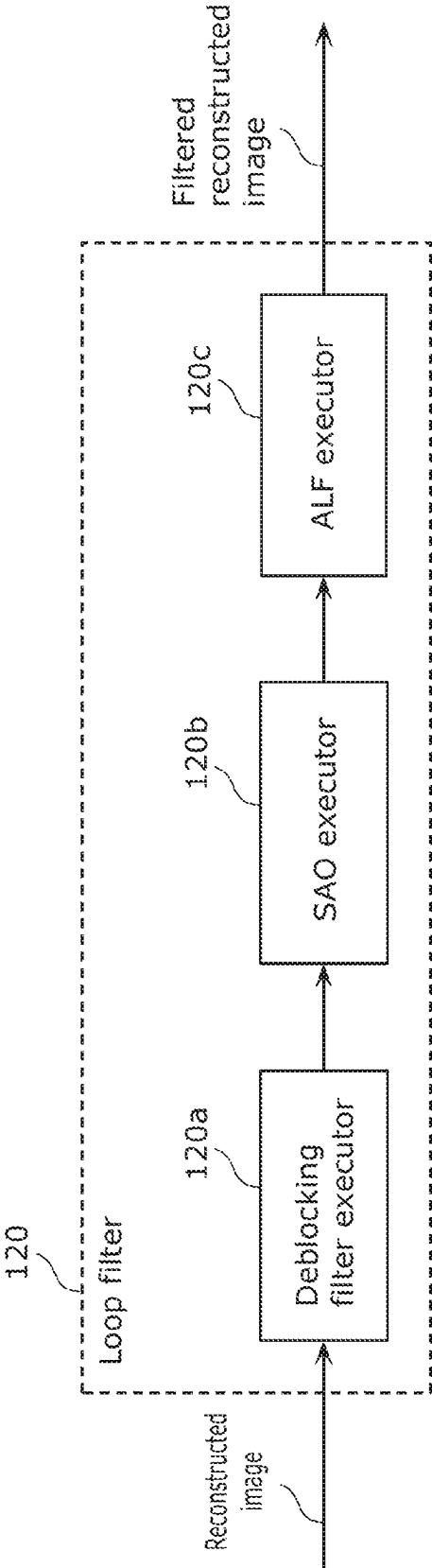


FIG. 23A

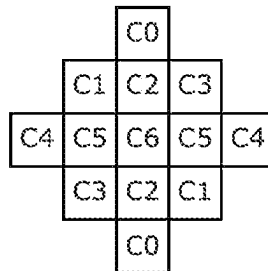


FIG. 23B

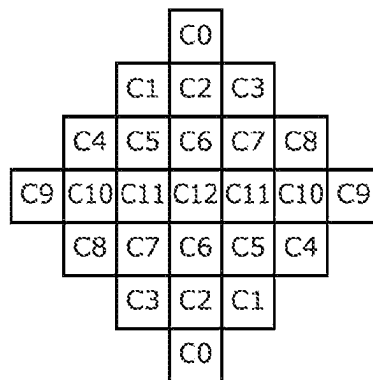


FIG. 23C

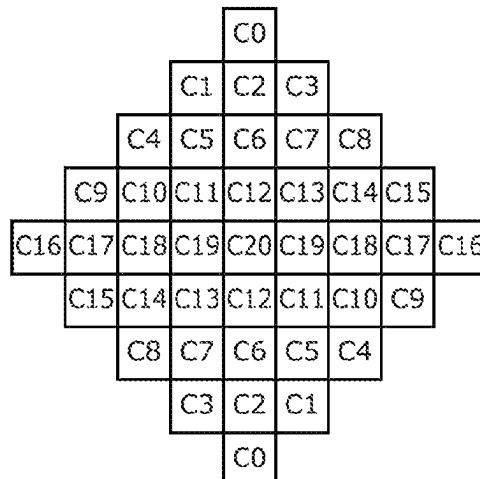


FIG. 23D

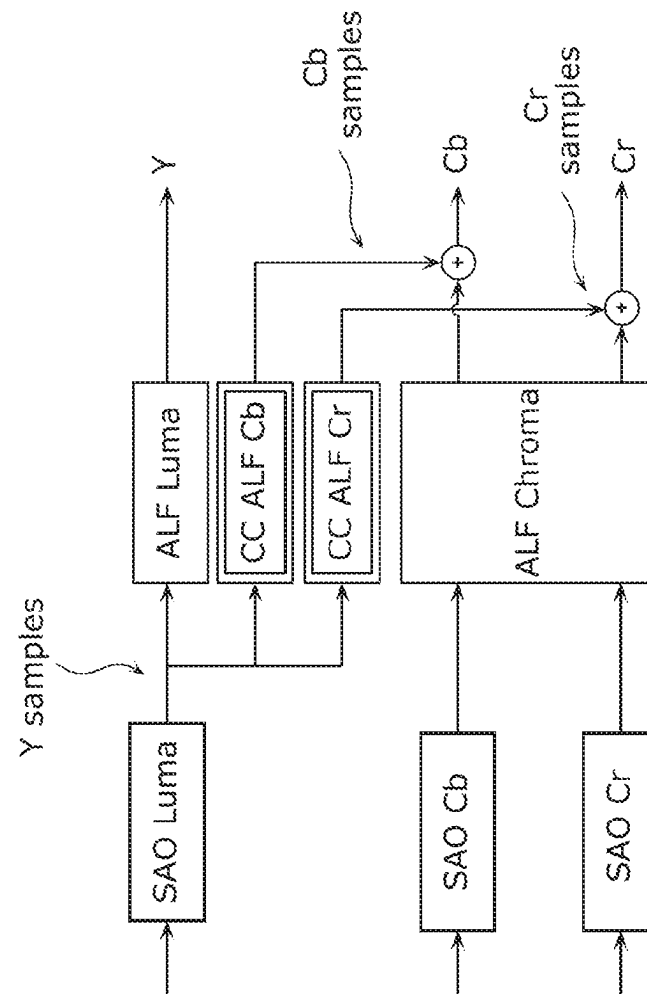


FIG. 23E

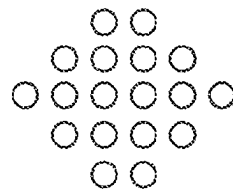


FIG. 23F

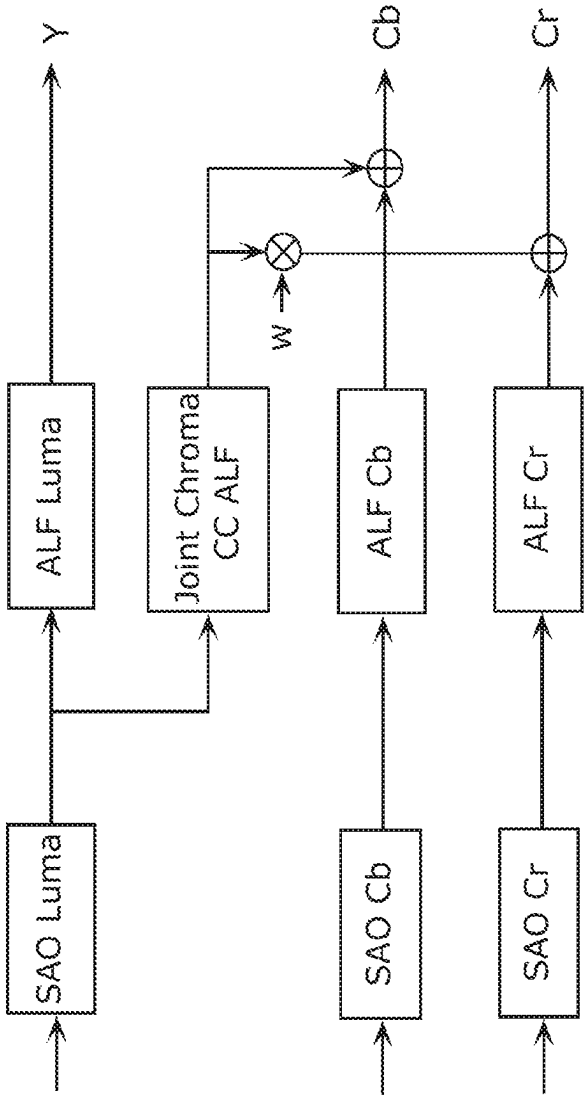


FIG. 23G

Index	1	2	3	4	5	6	7
w	1/4	1/2	3/4	1	4	2	4/3

FIG. 24

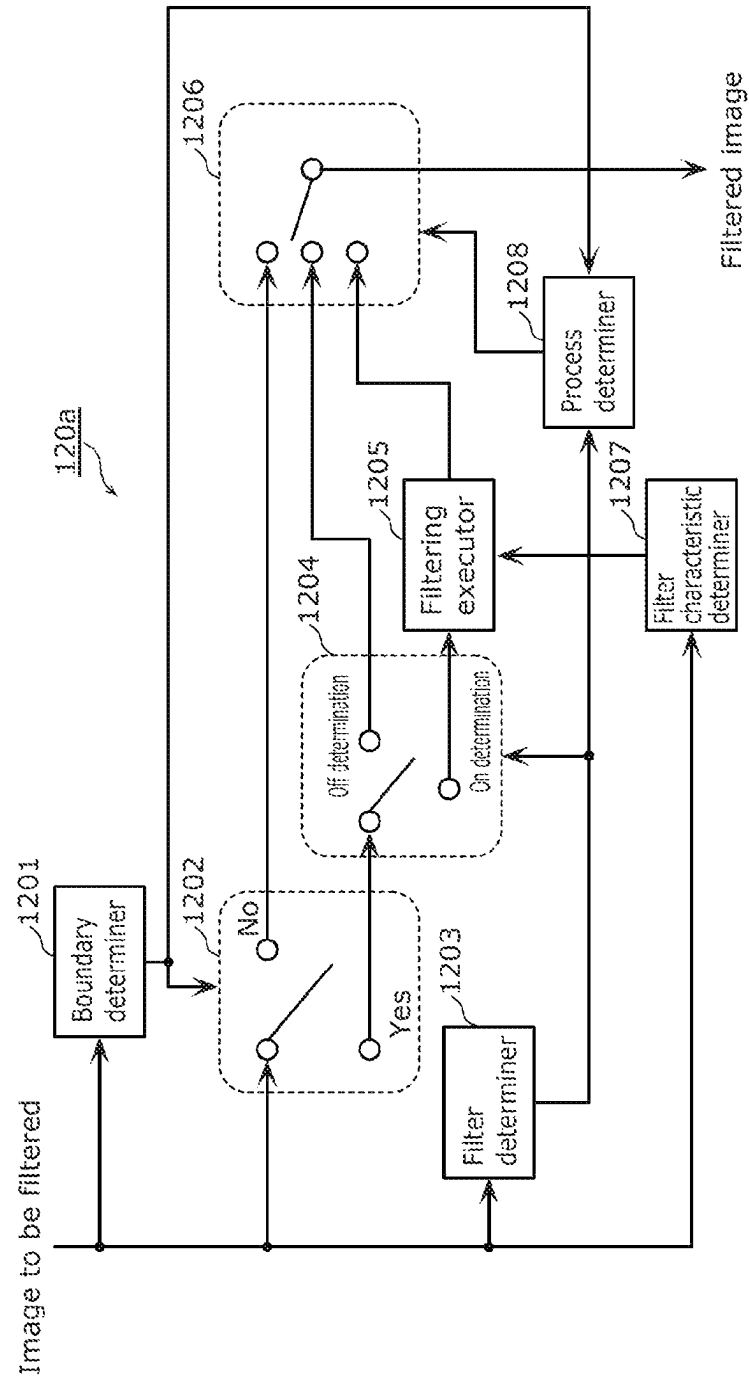


FIG. 25

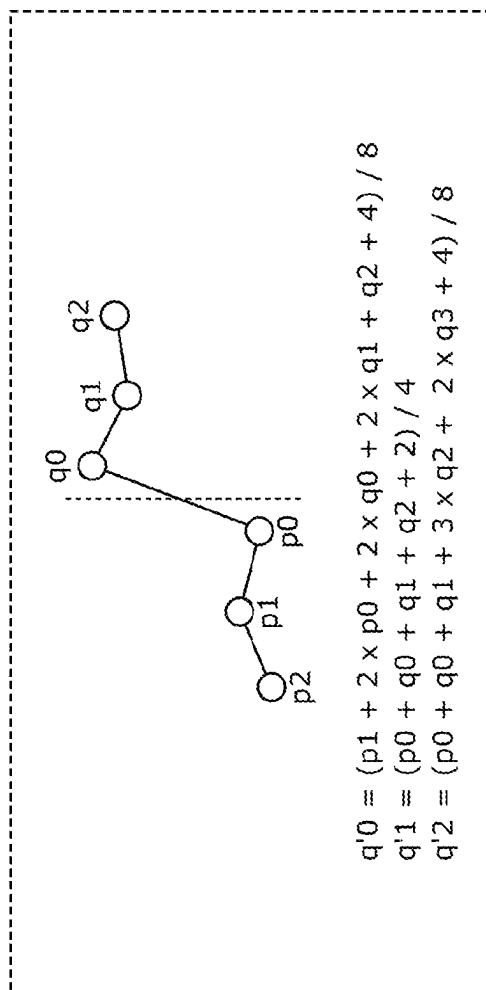


FIG. 26

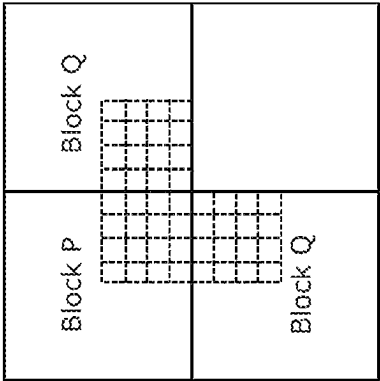


FIG. 27

Conditions for blocks p and q located across boundary for which Bs value is to be determined	Bs values
- At least one of the blocks is intra prediction block	2
- At least one of the blocks includes non-zero transform coefficient and has a side which coincides with boundary between orthogonal transform blocks	1
- Absolute value of difference between motion vectors of the two blocks across boundary corresponds to one or more pixels	1
- Reference images for motion compensation of the two blocks across boundary or the numbers of motion vectors of the two blocks are different	1
- The others	0

FIG. 28

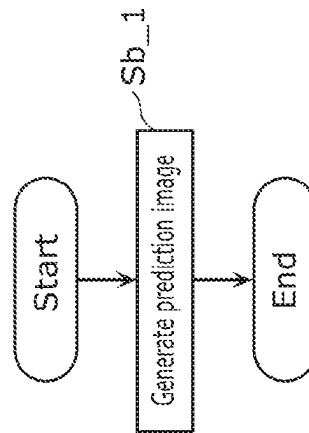


FIG. 29

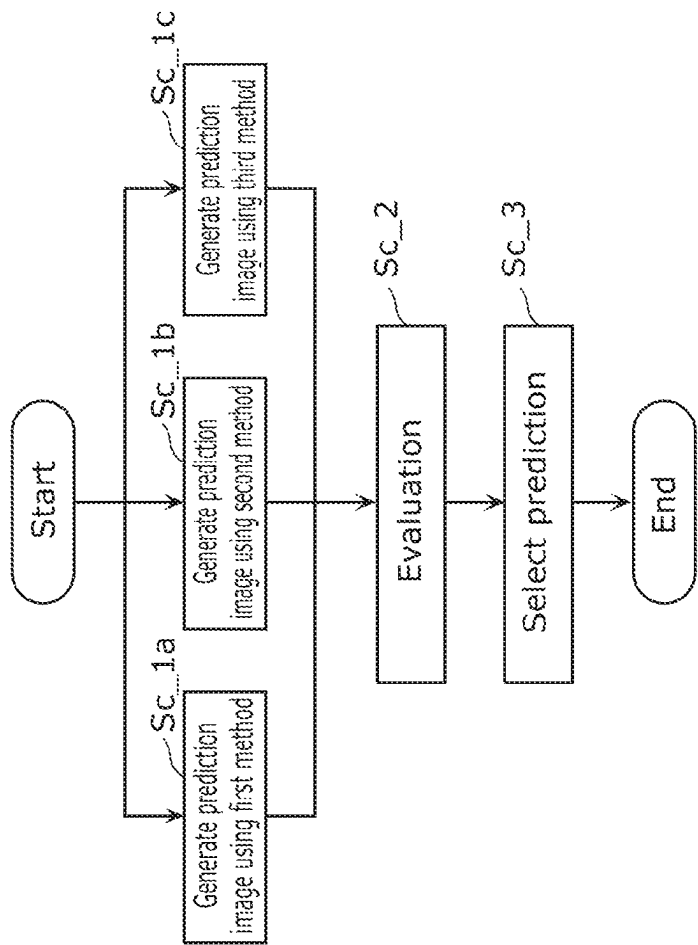


FIG. 30

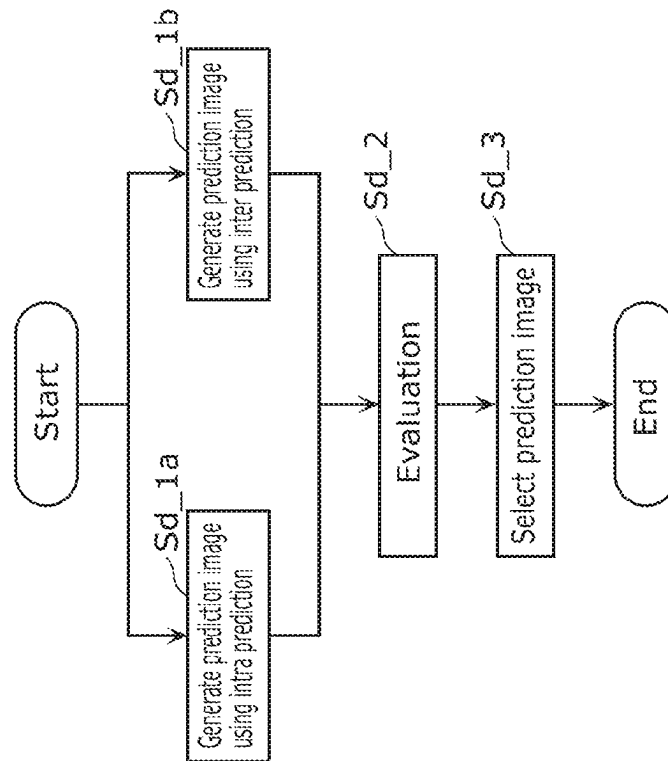


FIG. 31

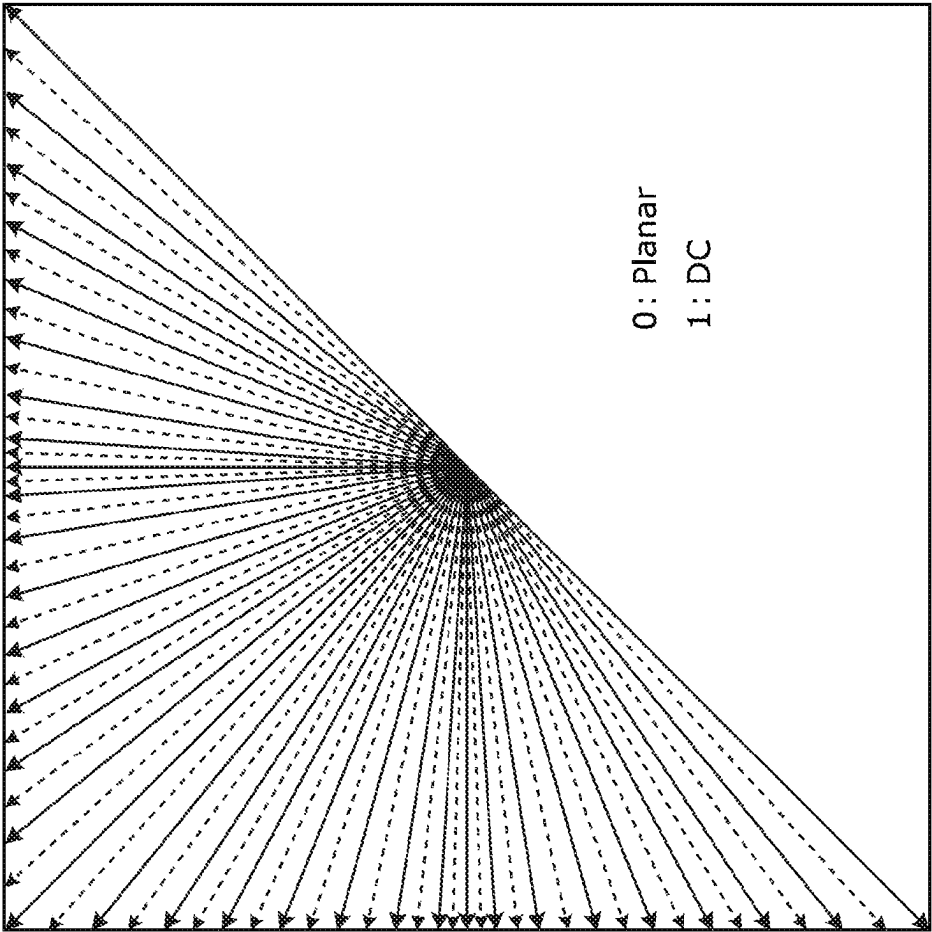


FIG. 32

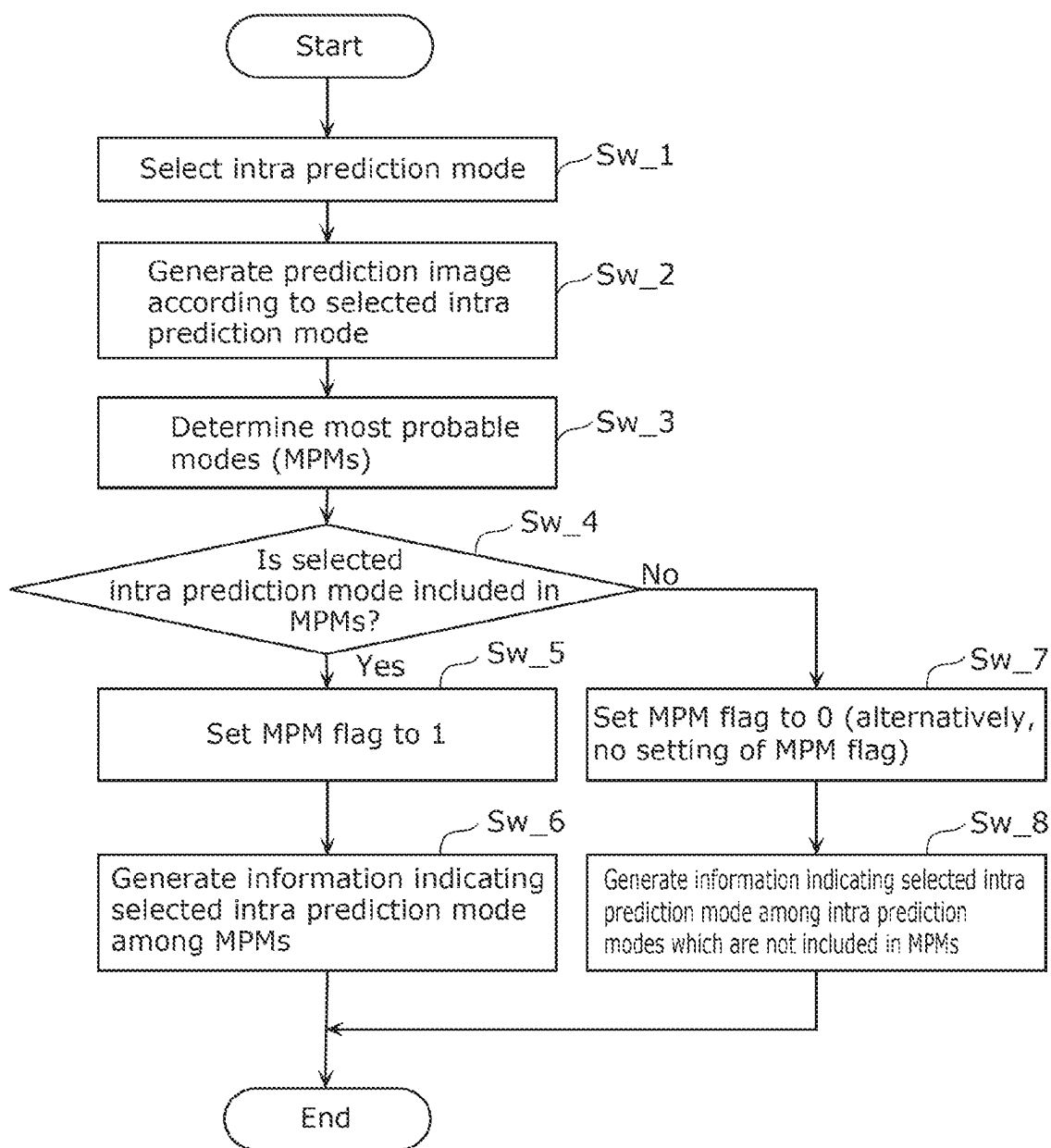


FIG. 33

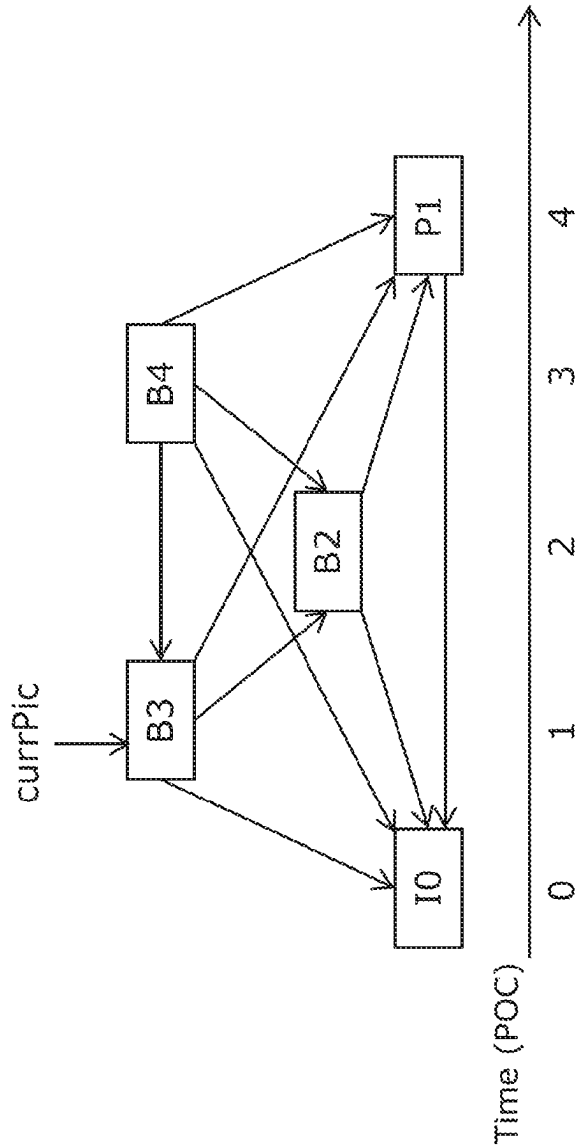


FIG. 34

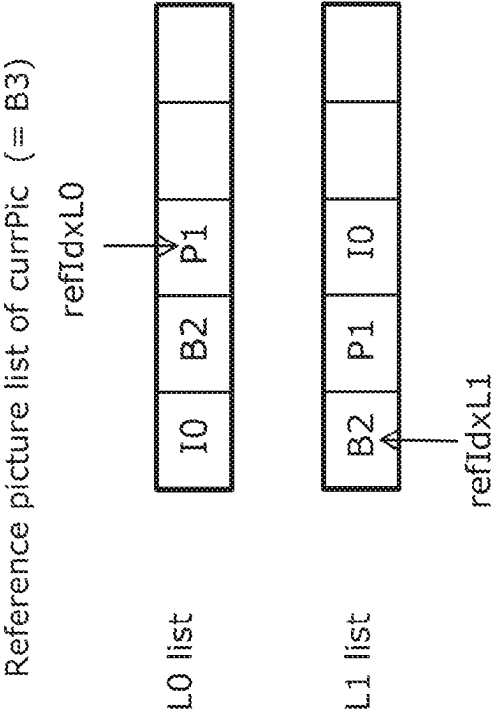


FIG. 35

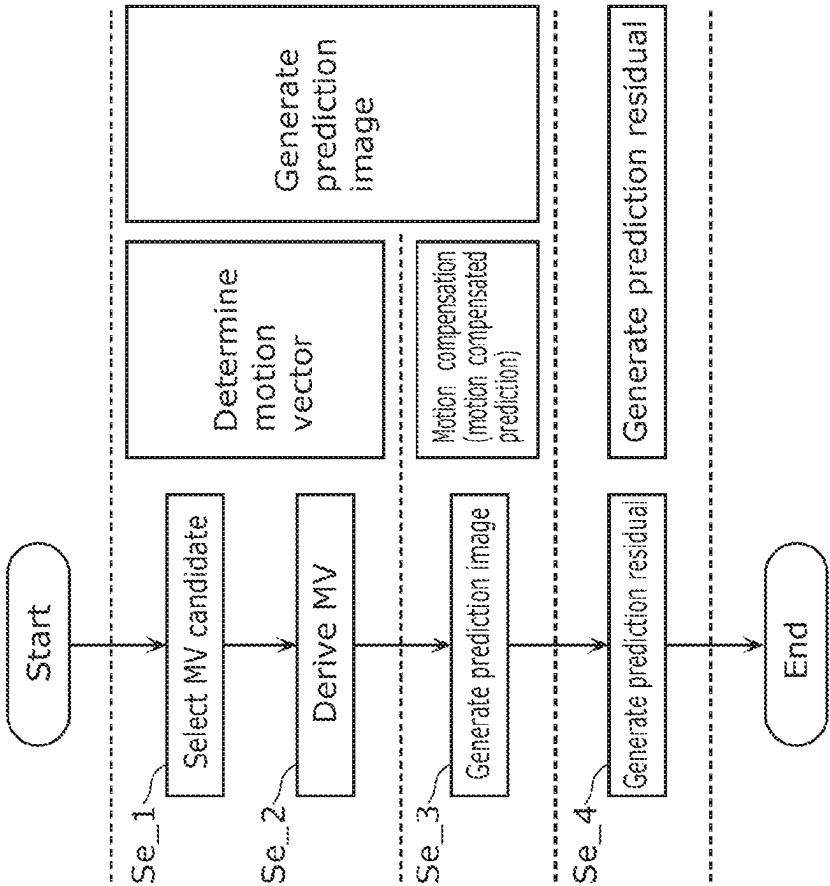


FIG. 36

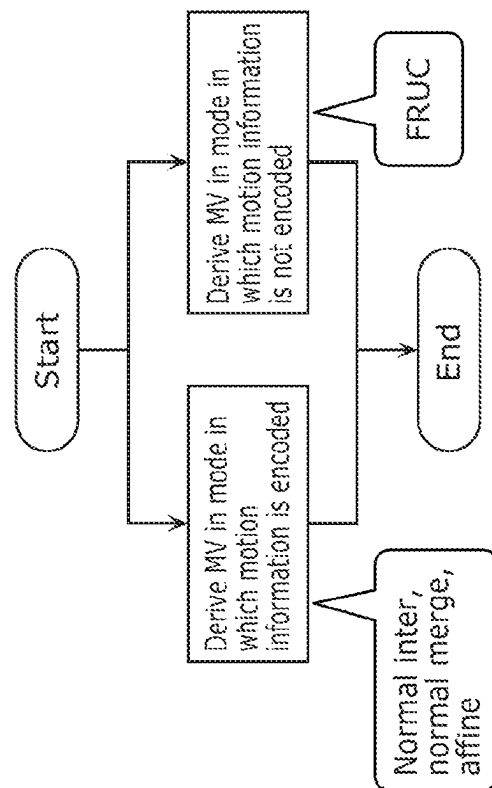


FIG. 37

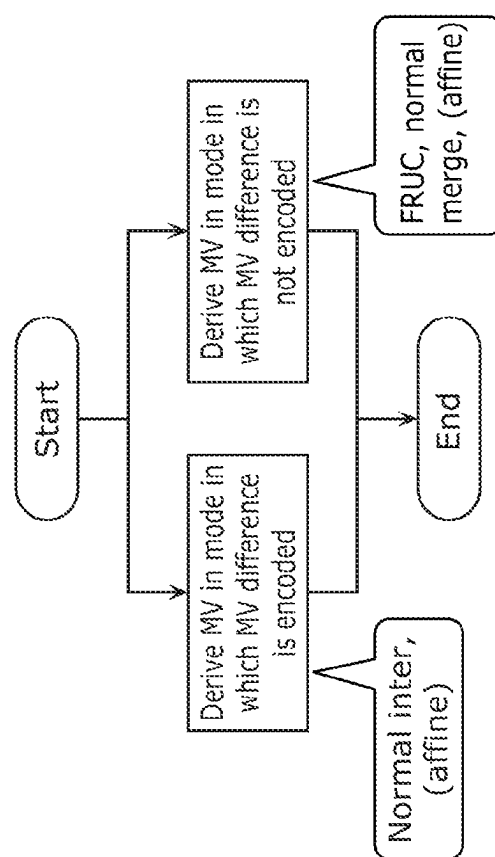


FIG. 38A

MV derivation mode		Motion information	MV difference
merge mode	inter mode	○	○
	other than MMVD mode	○	×
	MMVD mode	○	△
FRUC mode		×	×

FIG. 38B

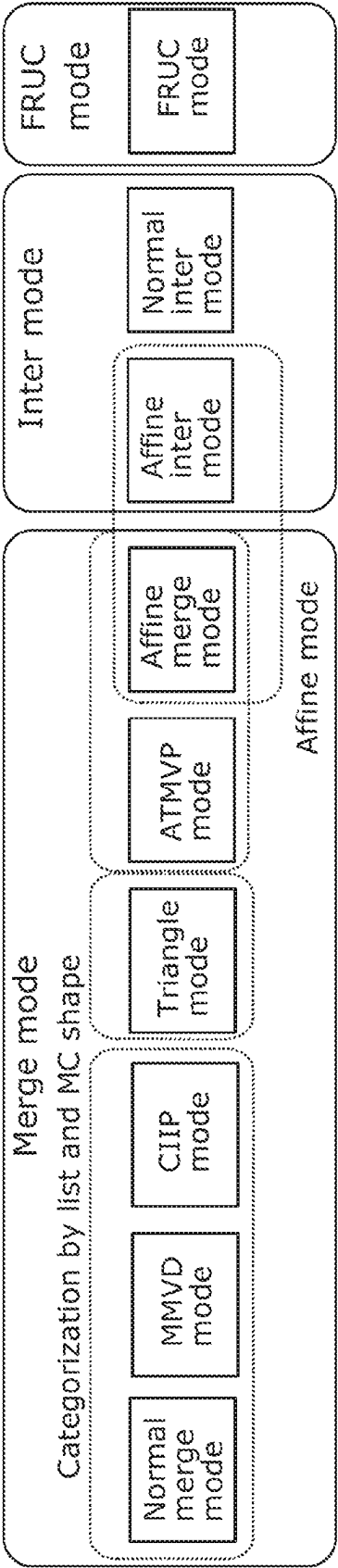


FIG. 39

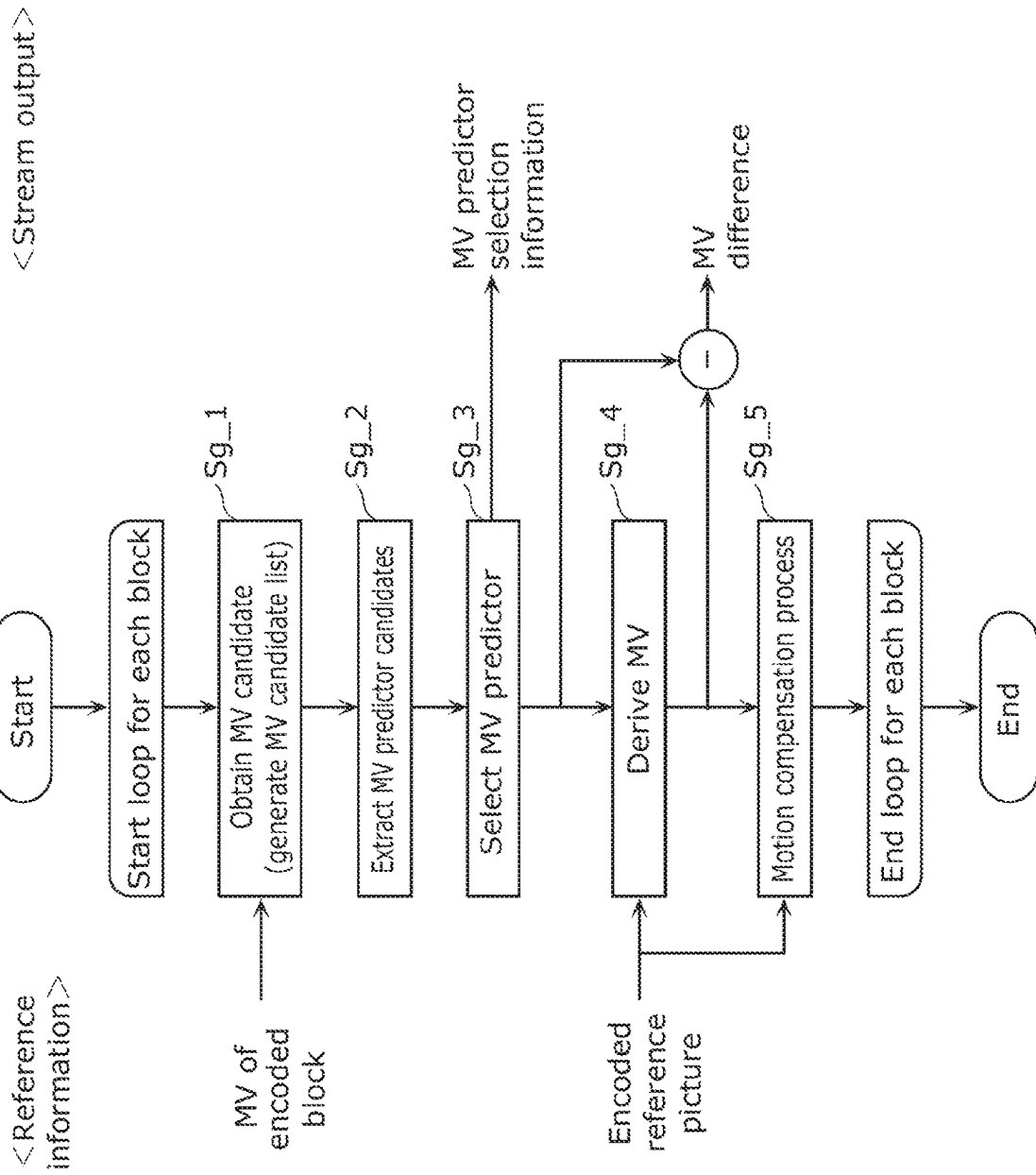


FIG. 40

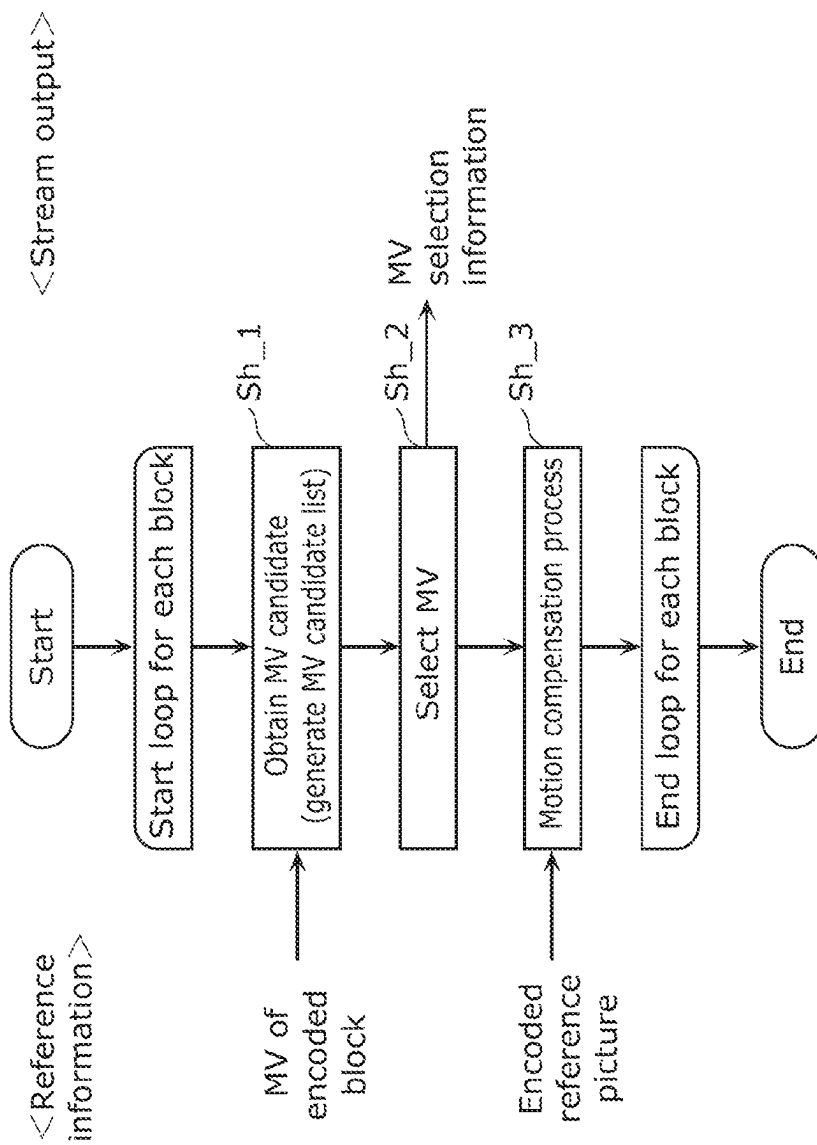


FIG. 41

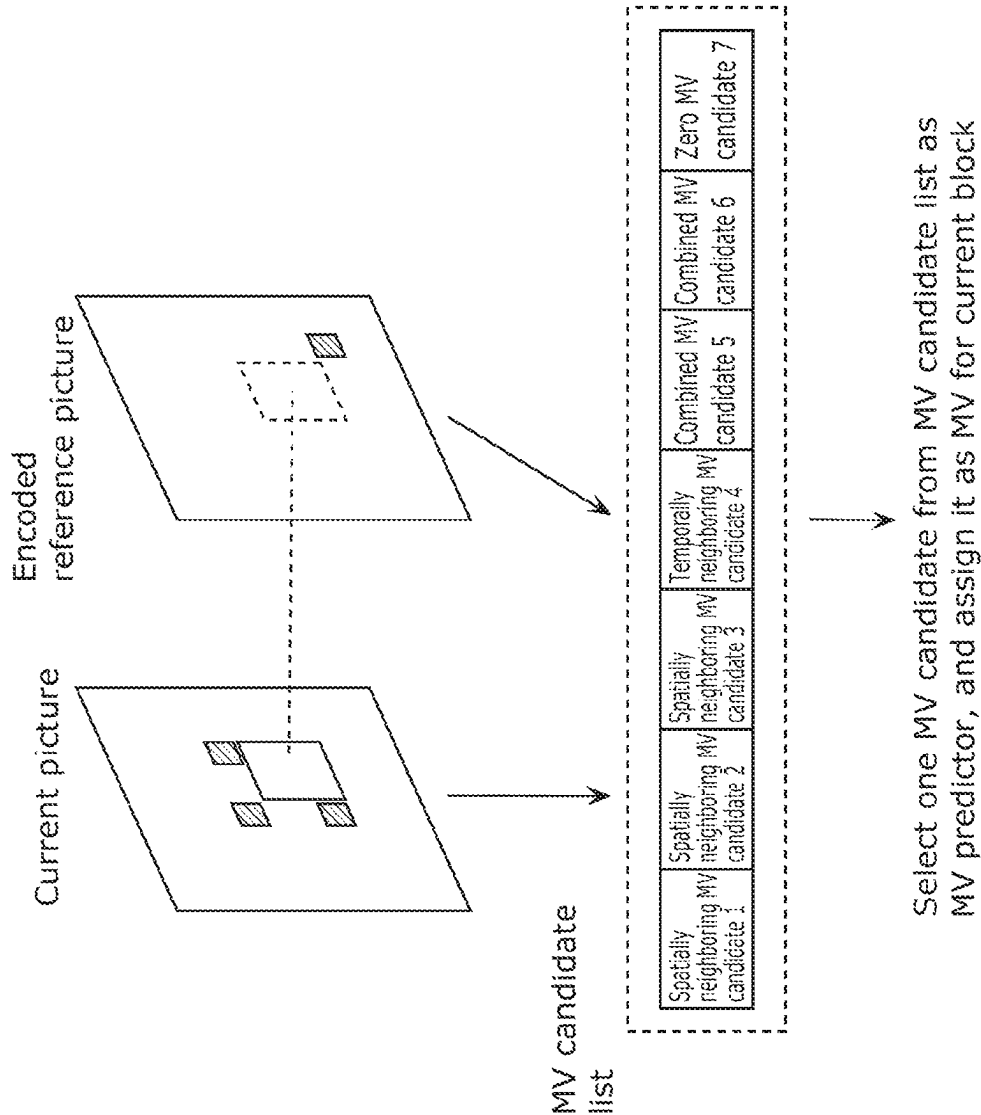


FIG. 42

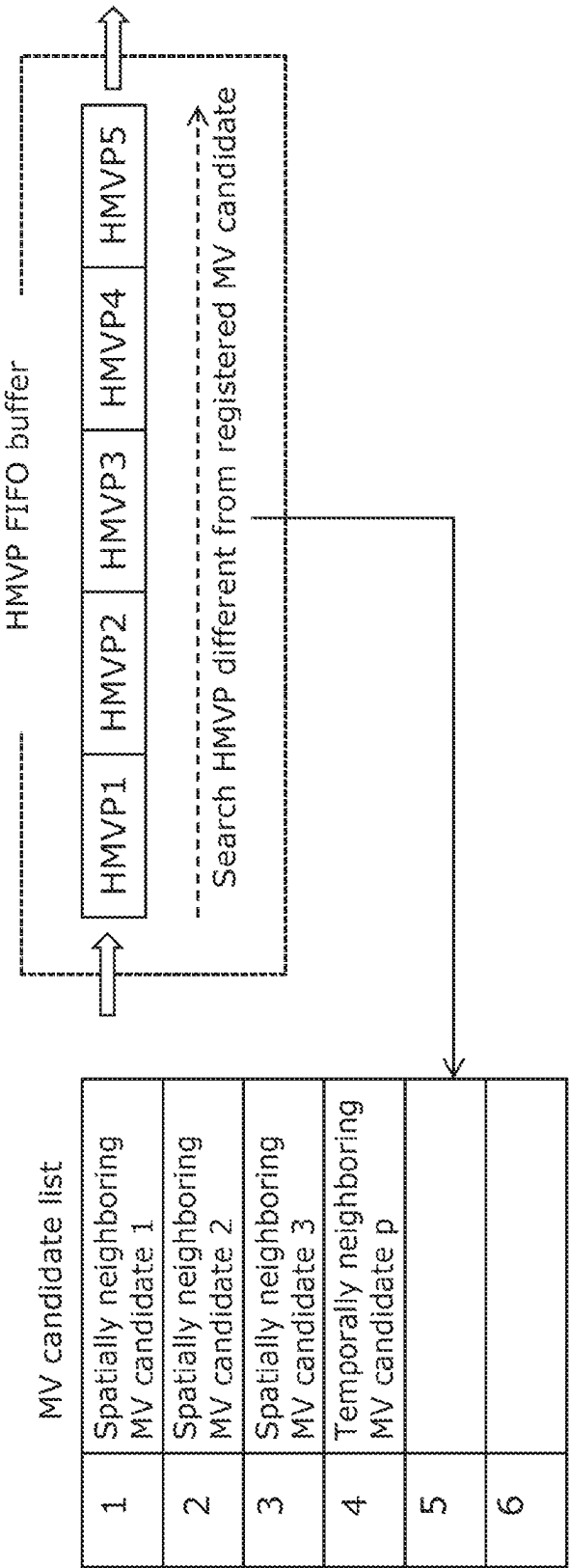


FIG. 43

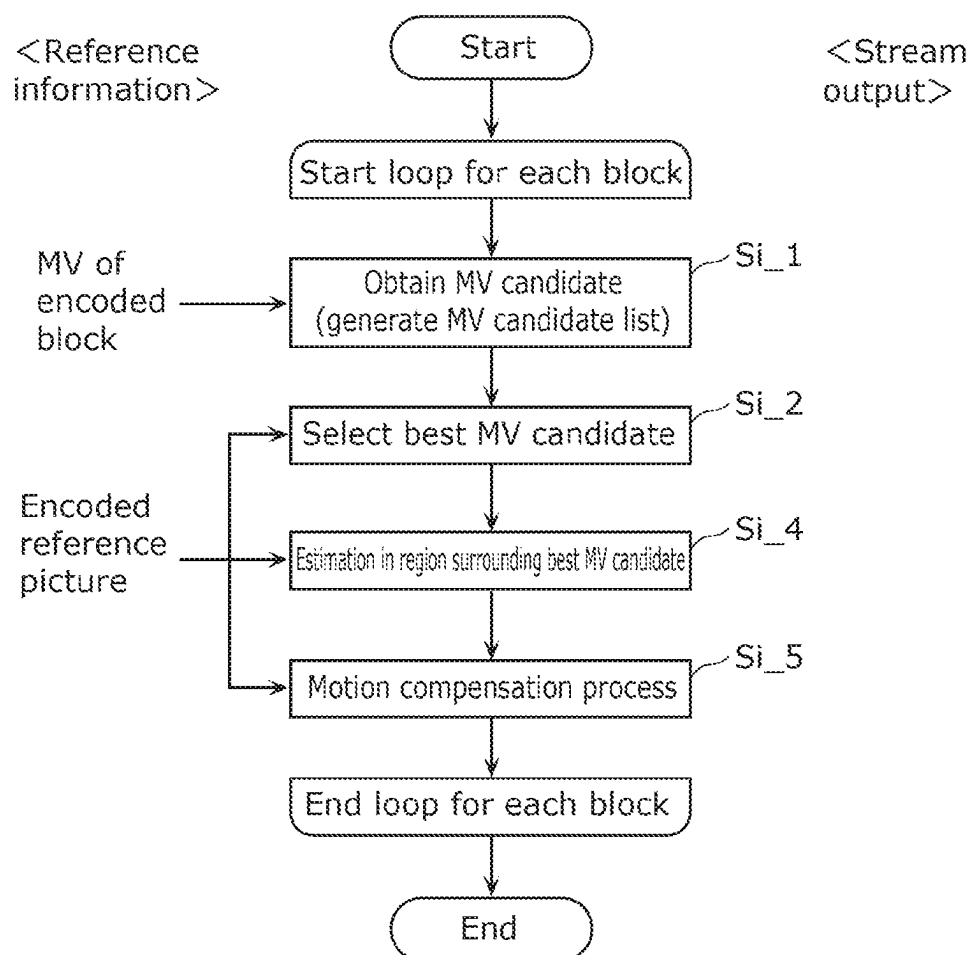


FIG. 44

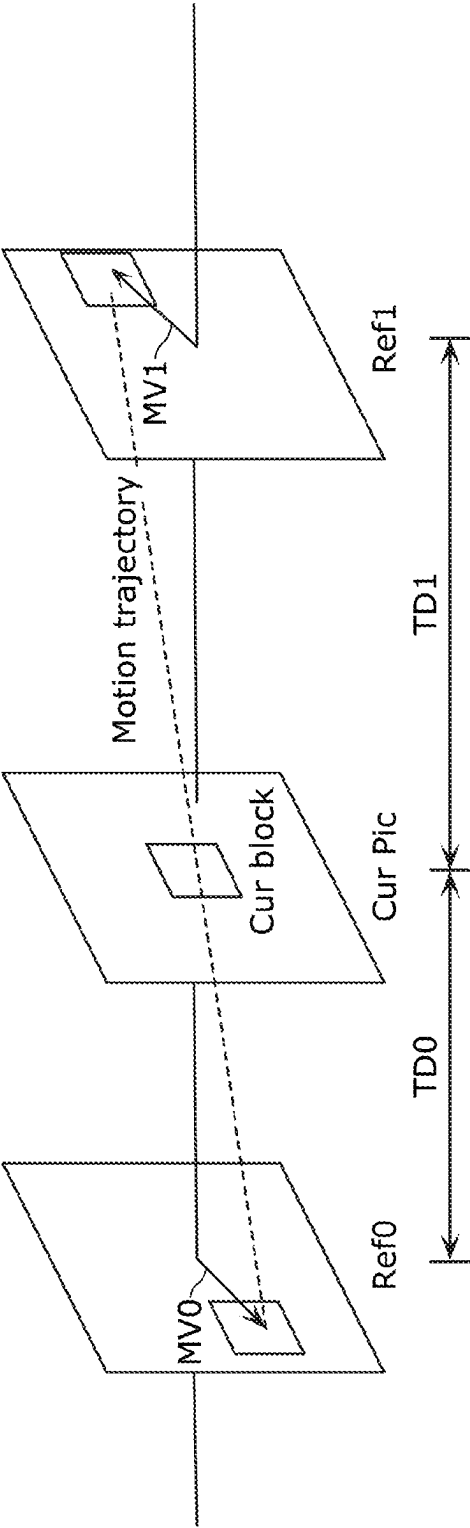


FIG. 45

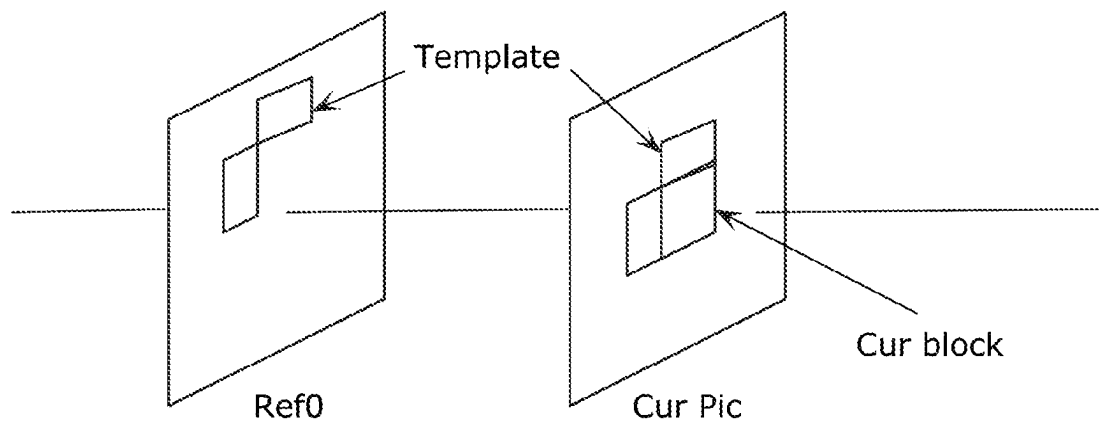


FIG. 46A

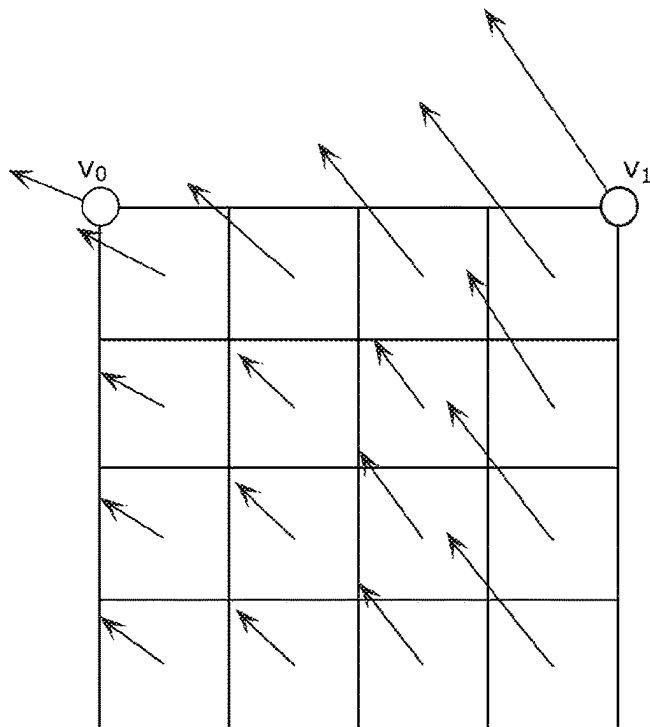


FIG. 46B

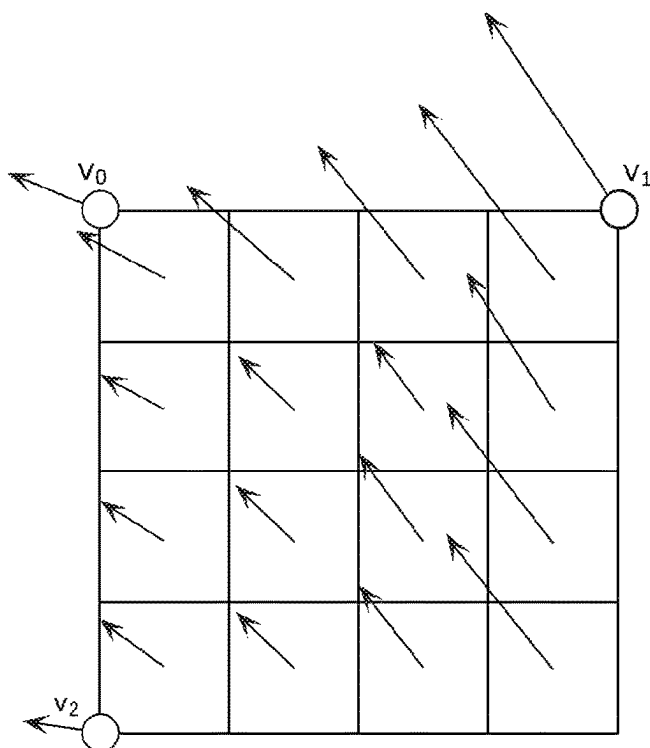


FIG. 47A

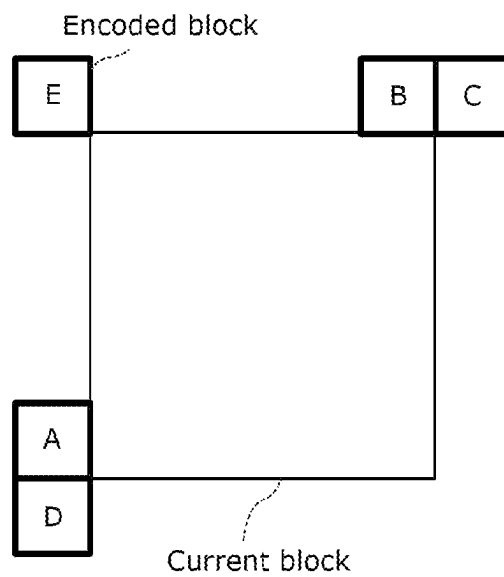


FIG. 47B

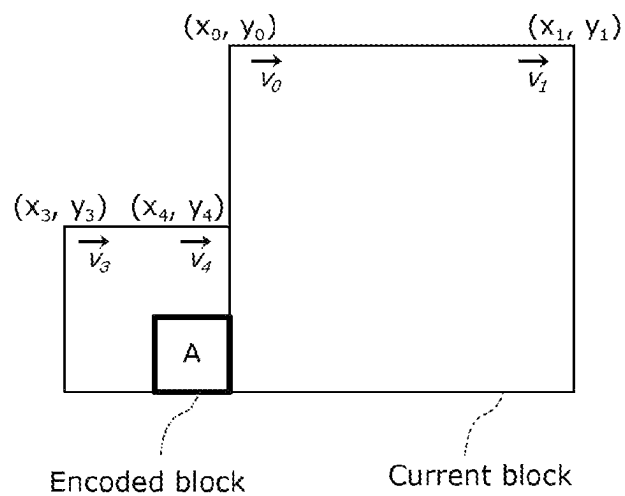


FIG. 47C

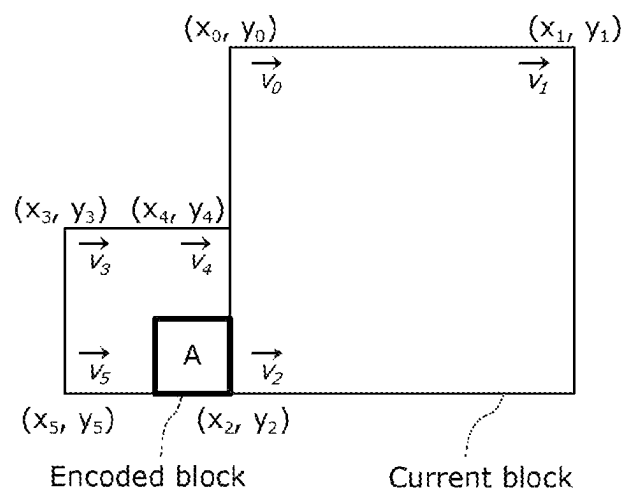


FIG. 48A

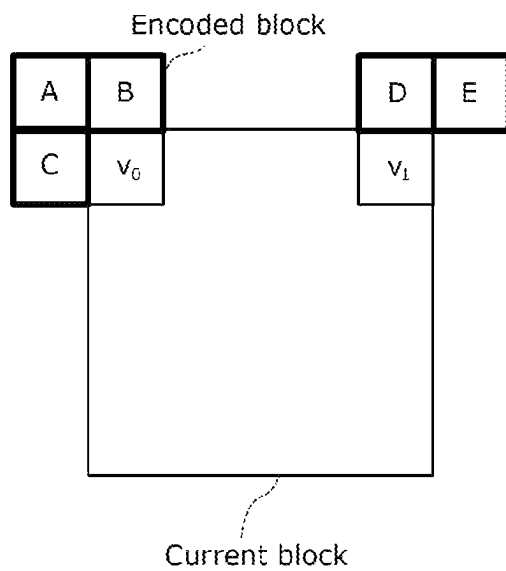


FIG. 48B

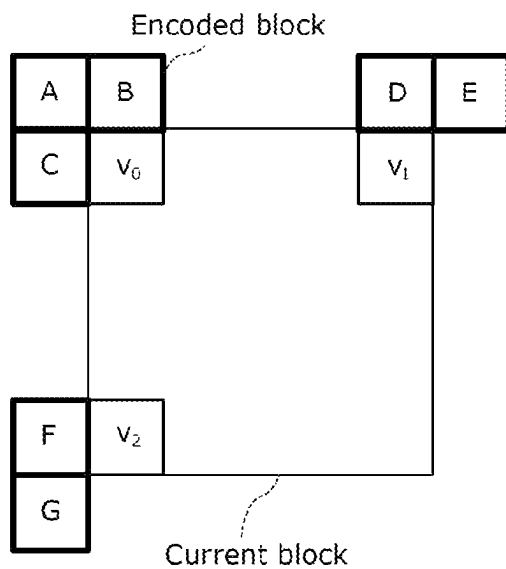


FIG. 49A

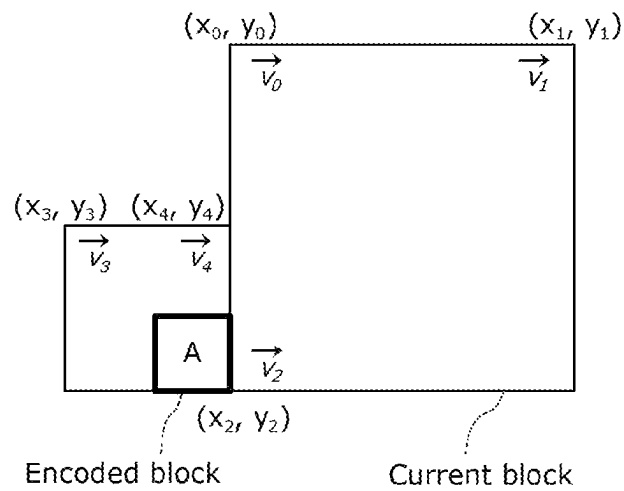


FIG. 49B

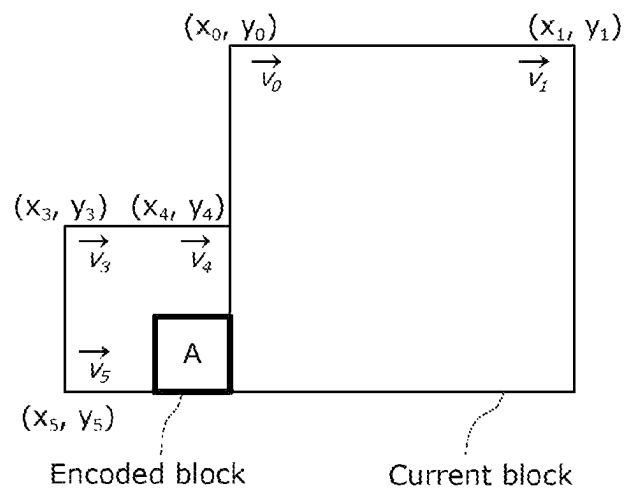


FIG. 50

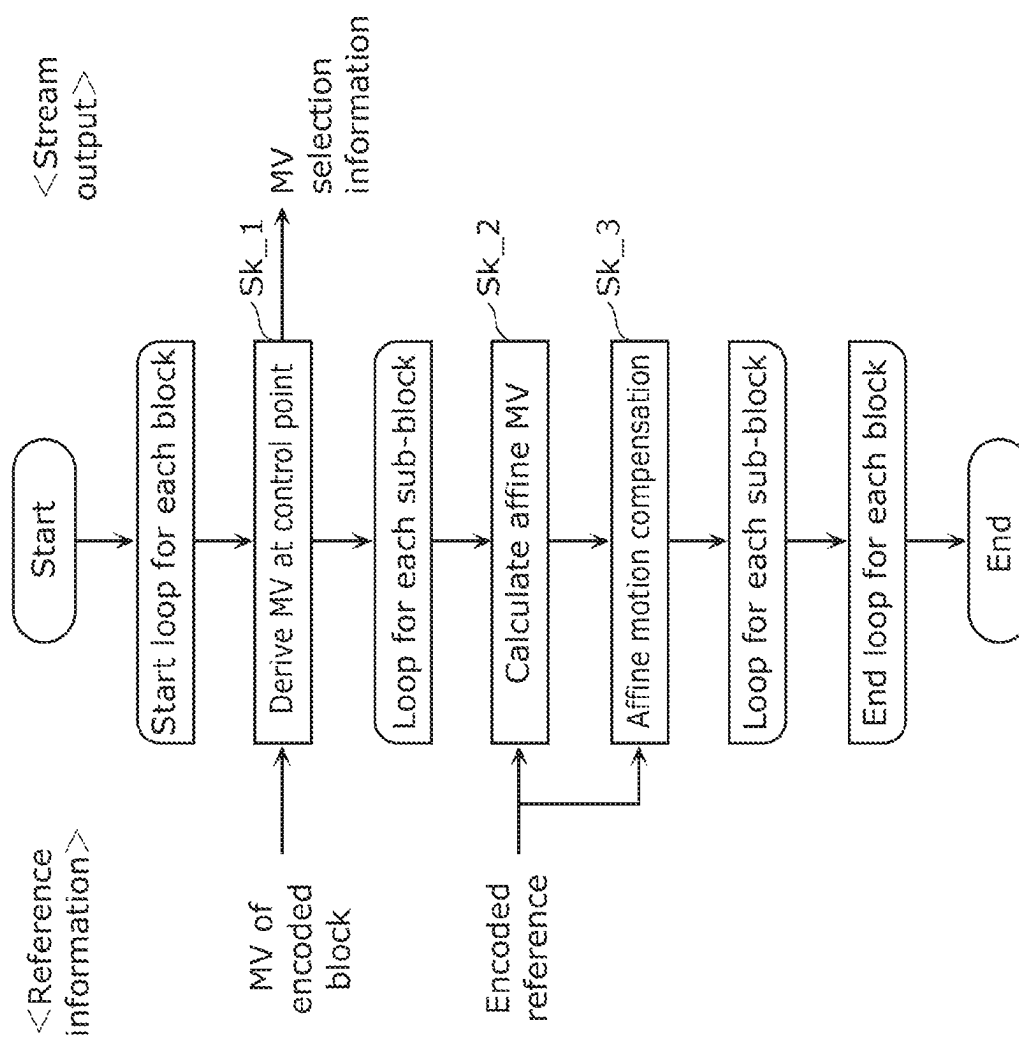


FIG. 51

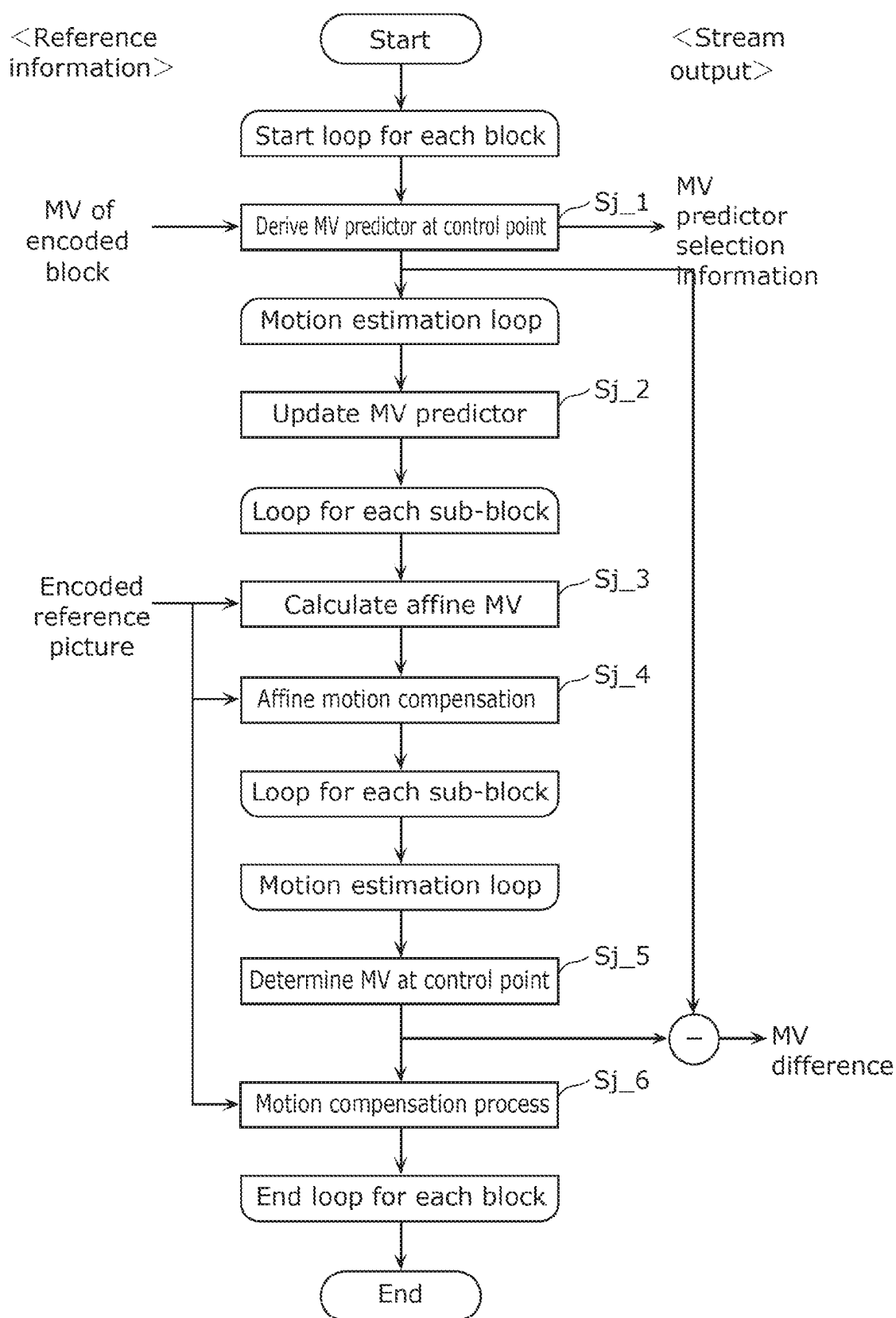


FIG. 52A

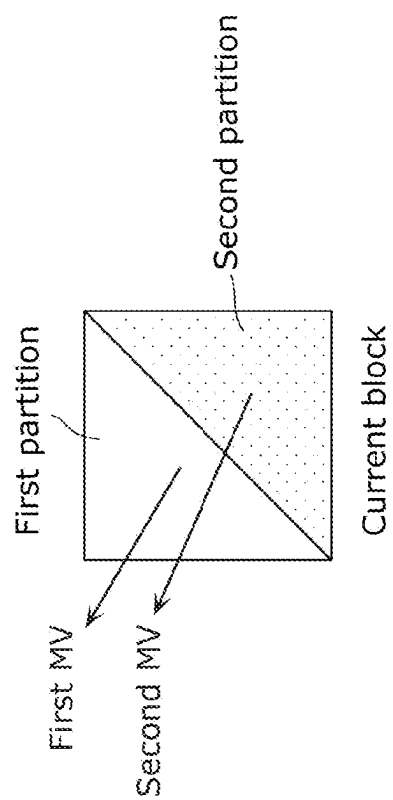


FIG. 52B

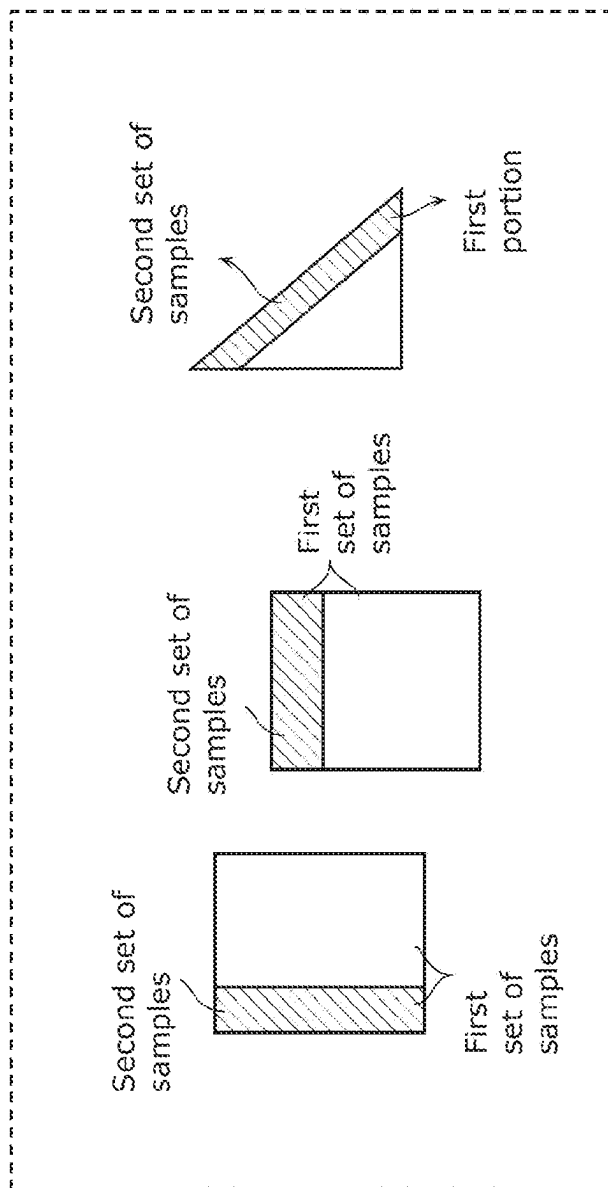


FIG. 52C

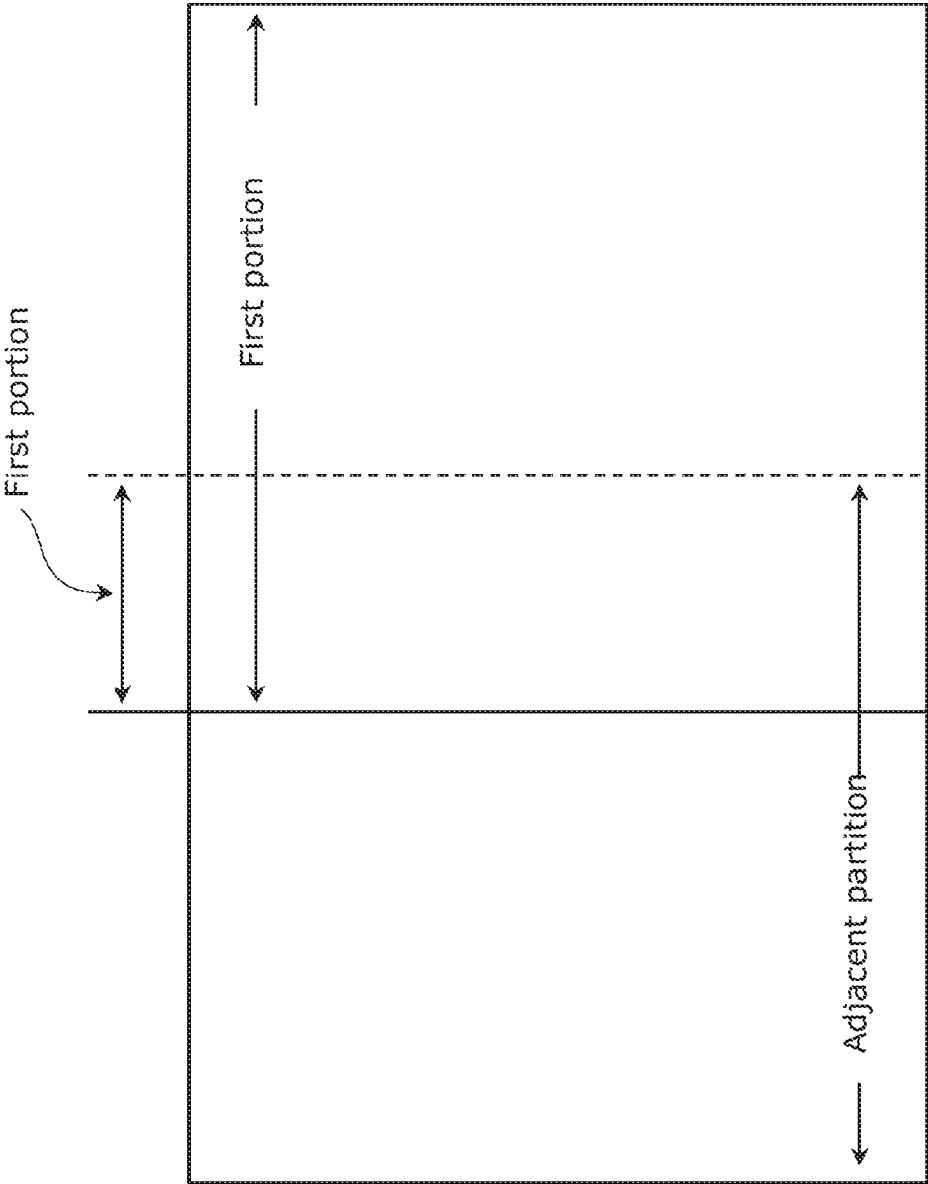


FIG. 53

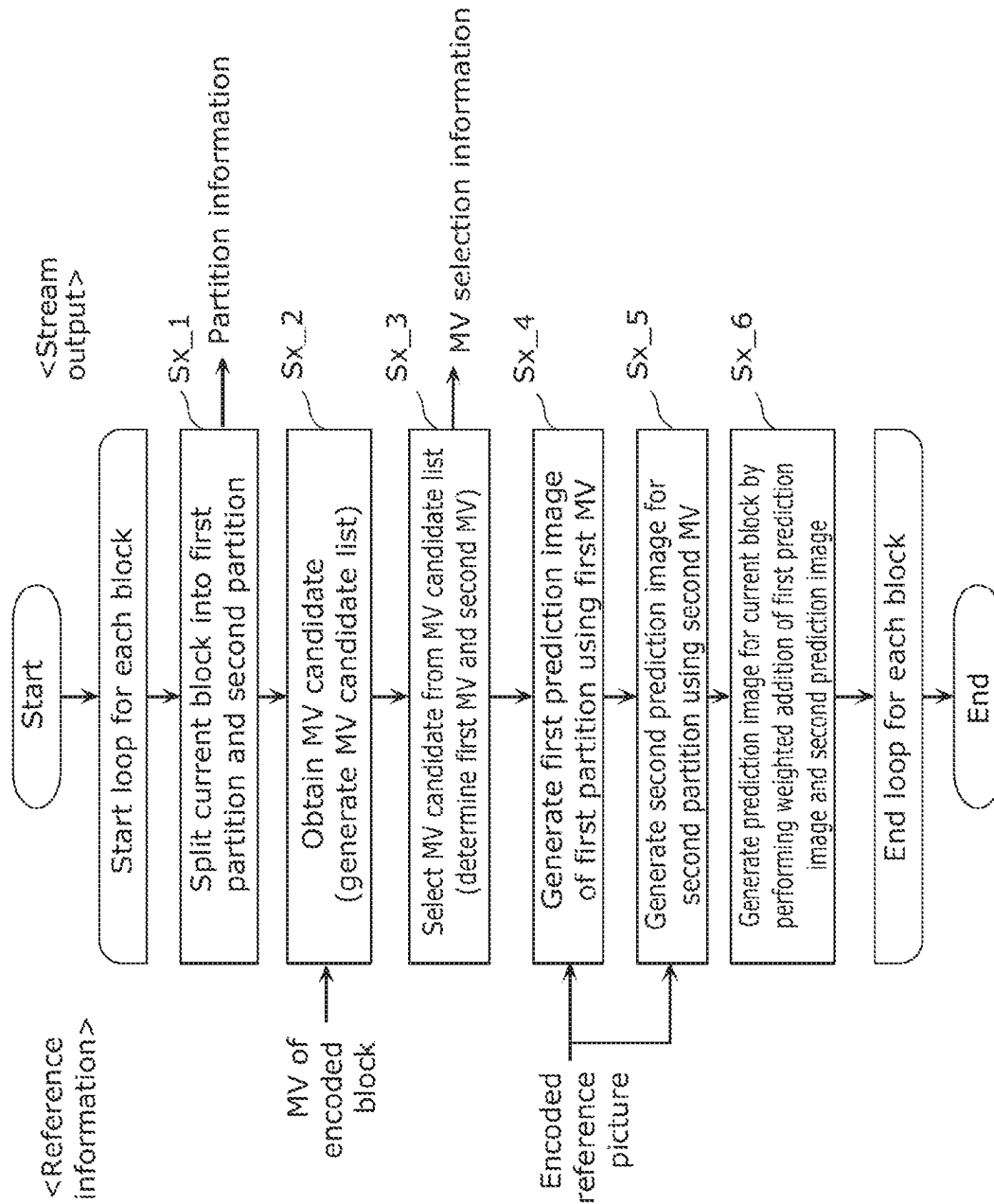


FIG. 54

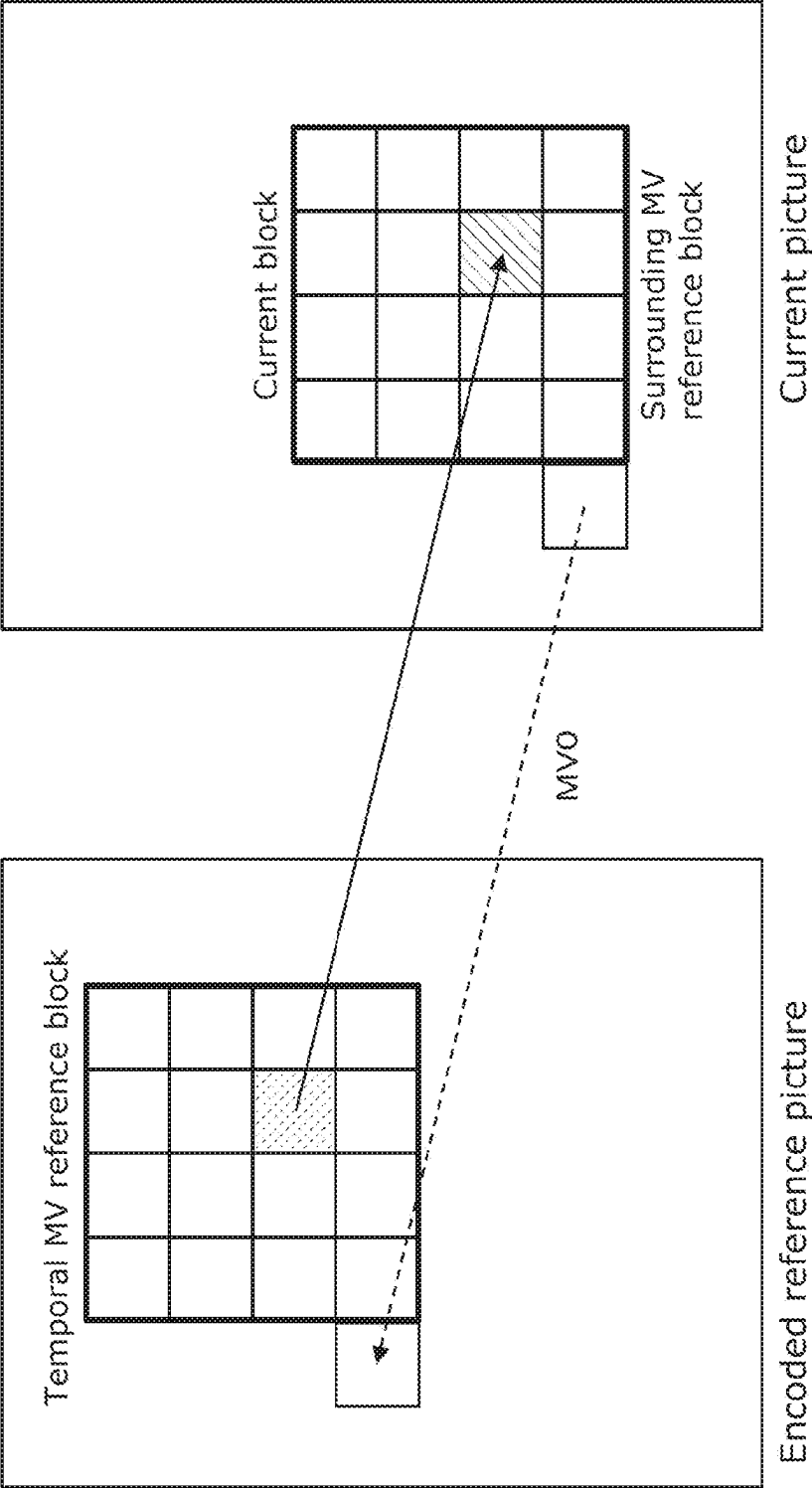


FIG. 55

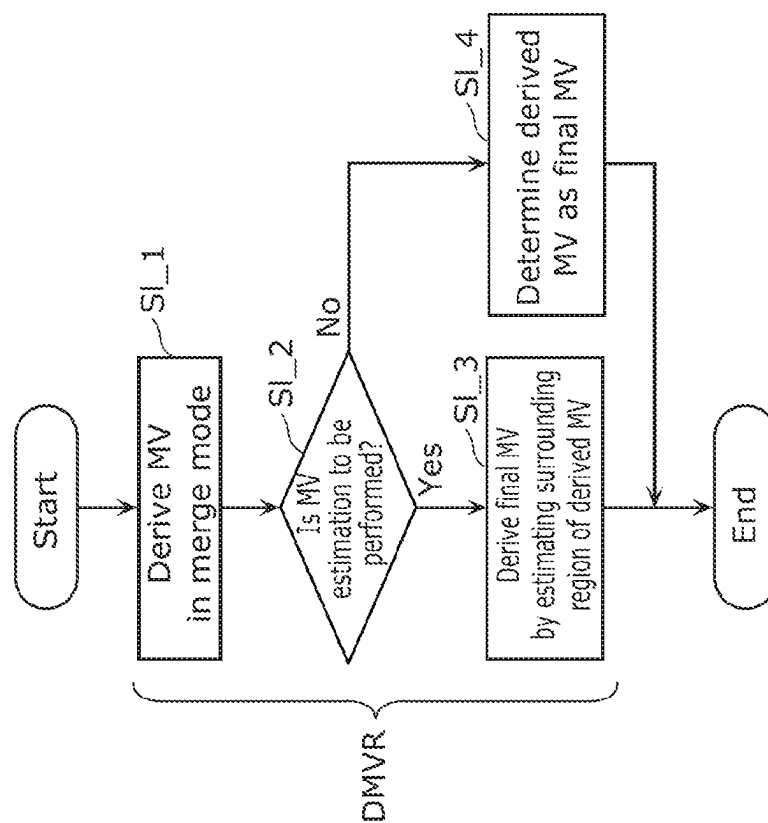


FIG. 56

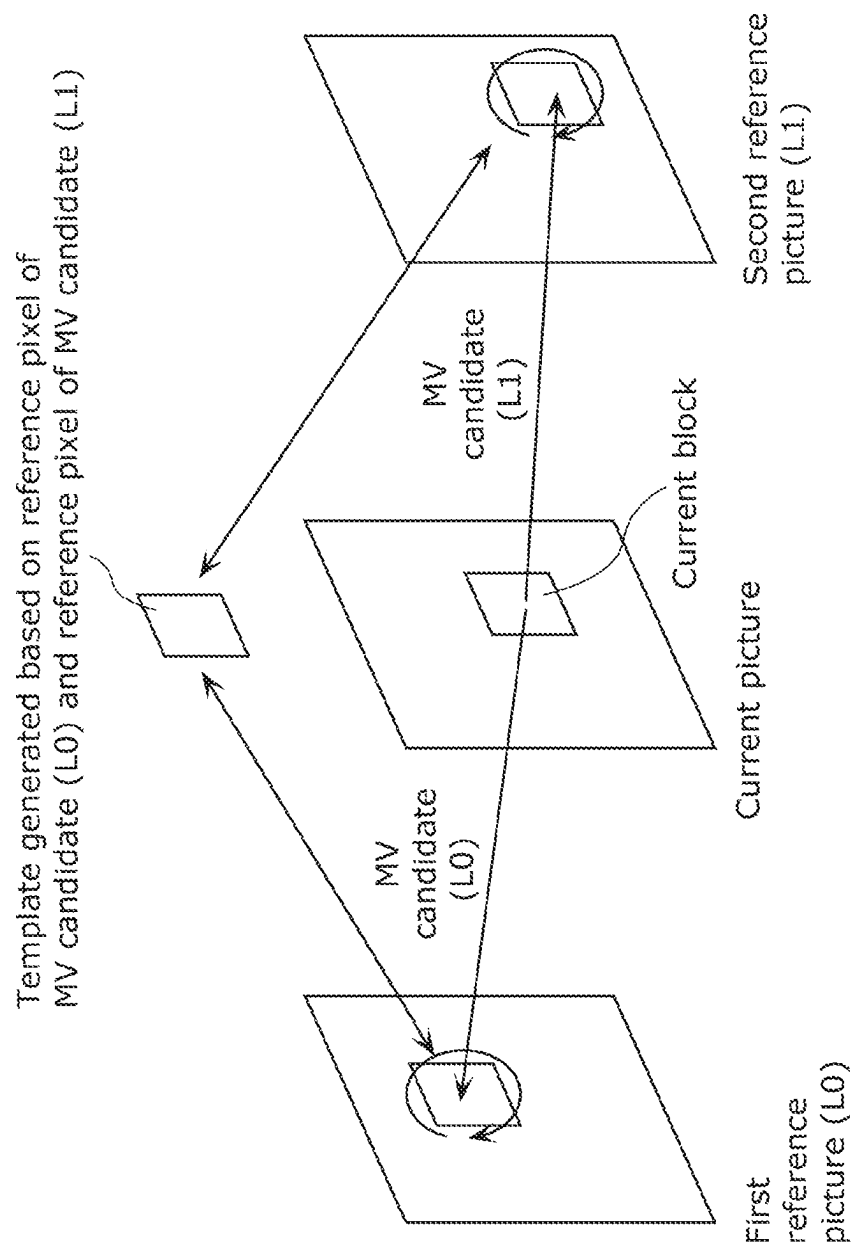


FIG. 57

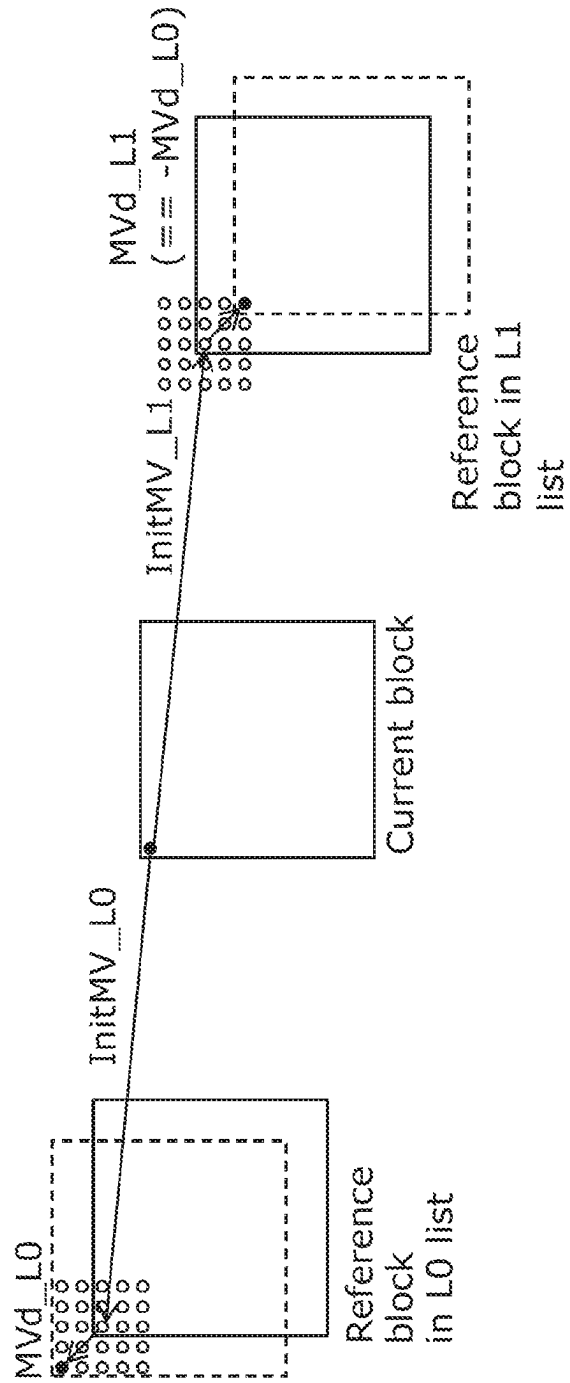


FIG. 58A

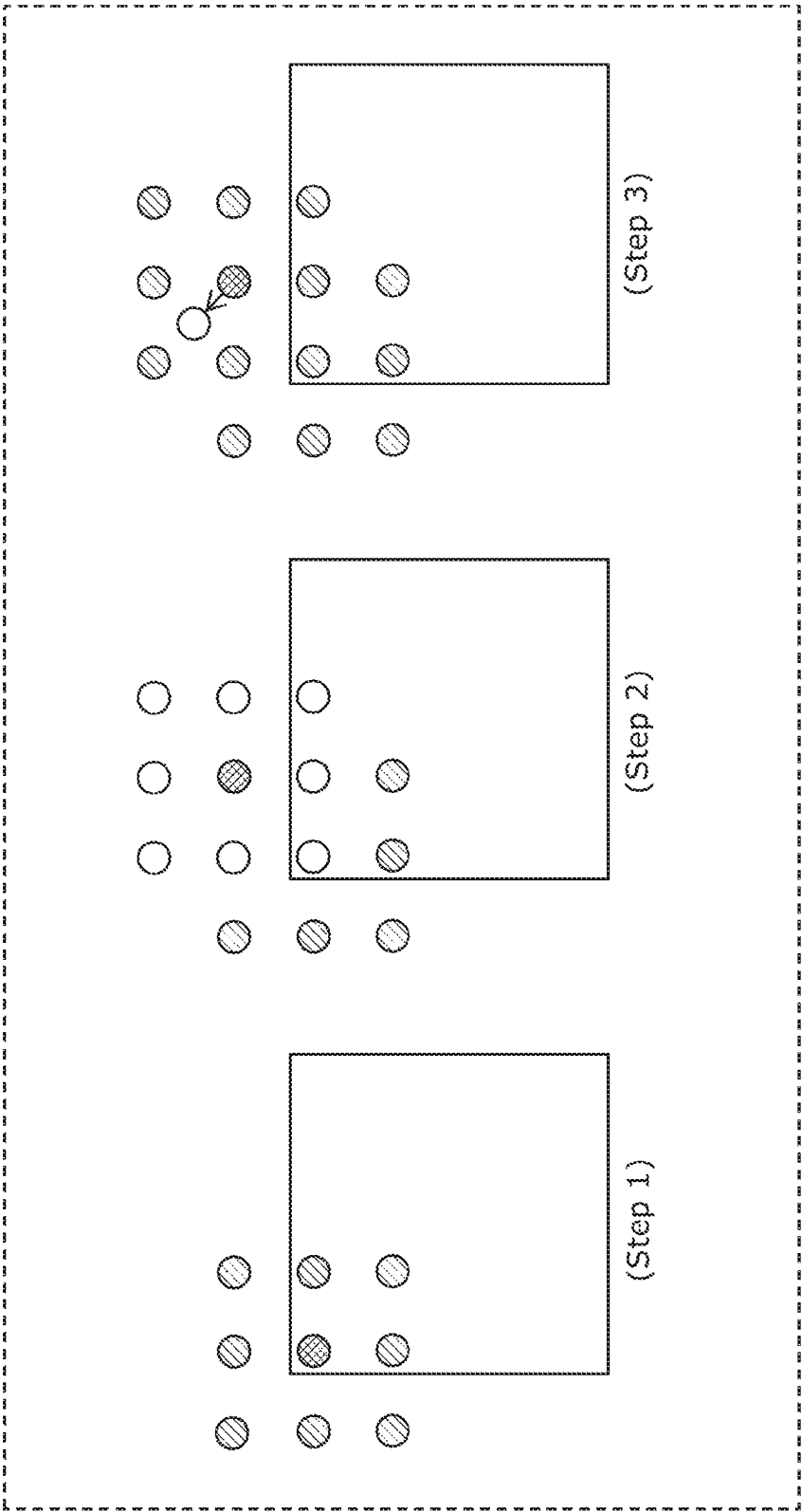


FIG. 58B

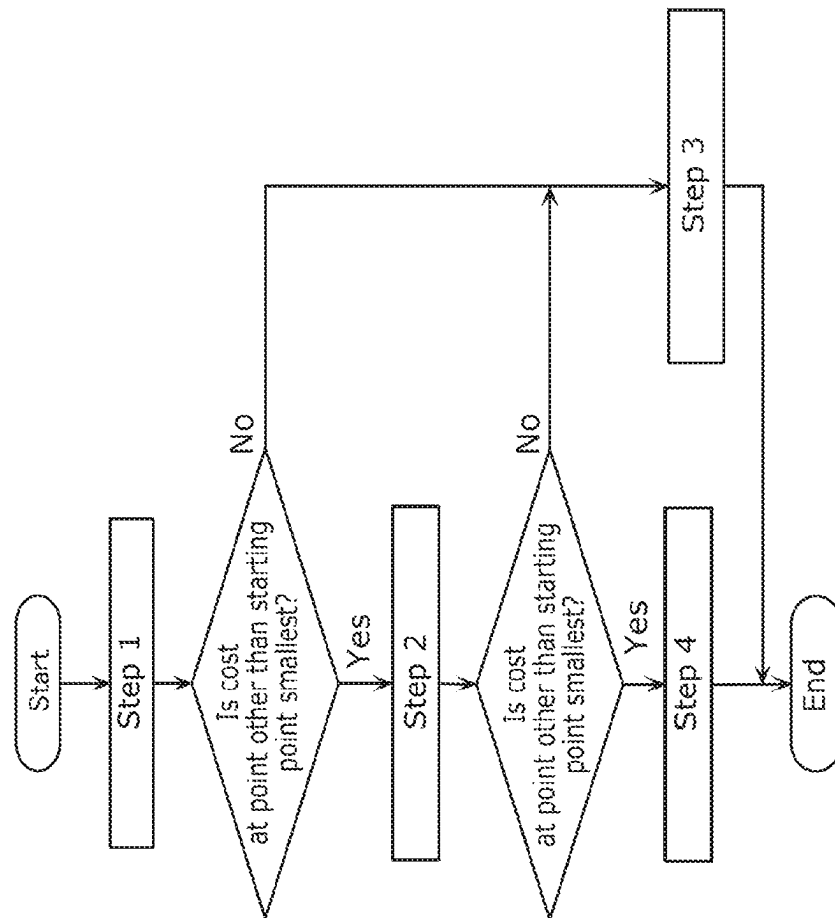


FIG. 59

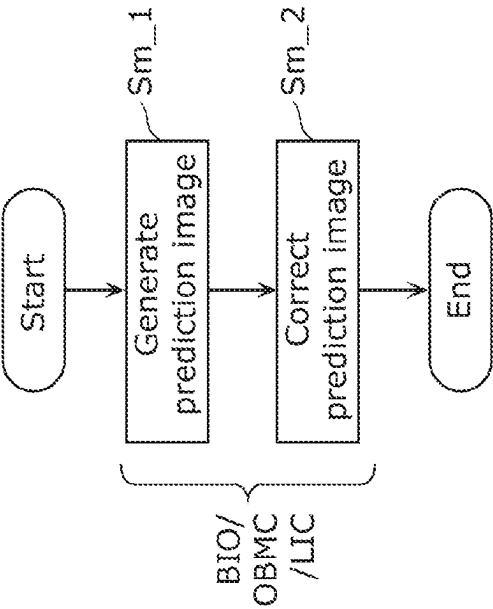


FIG. 60

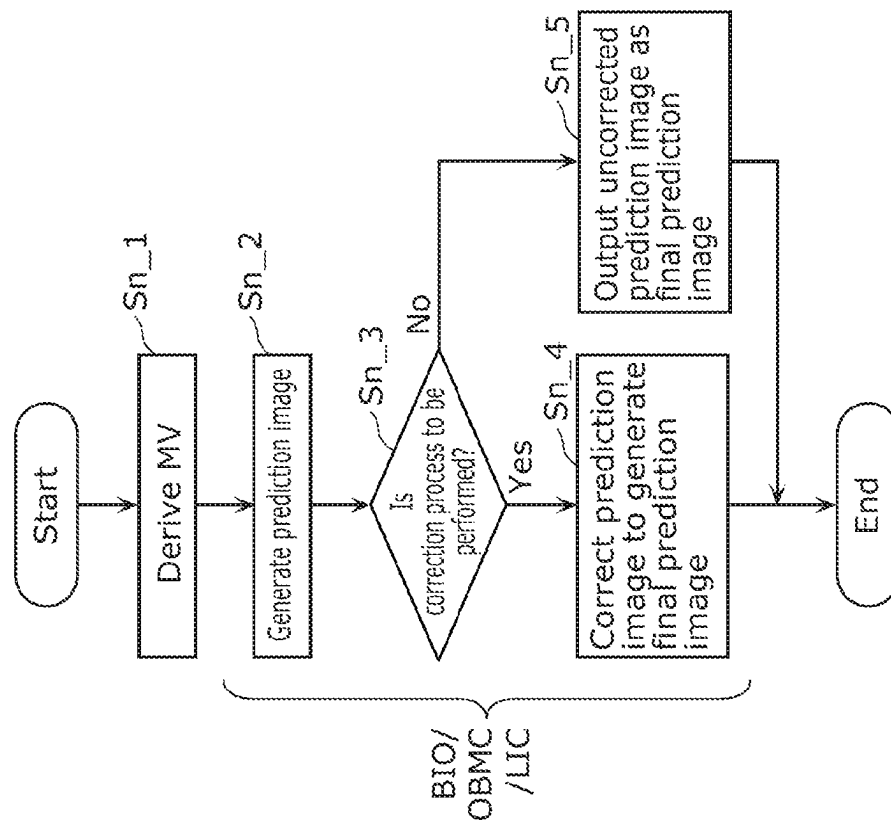


FIG. 61

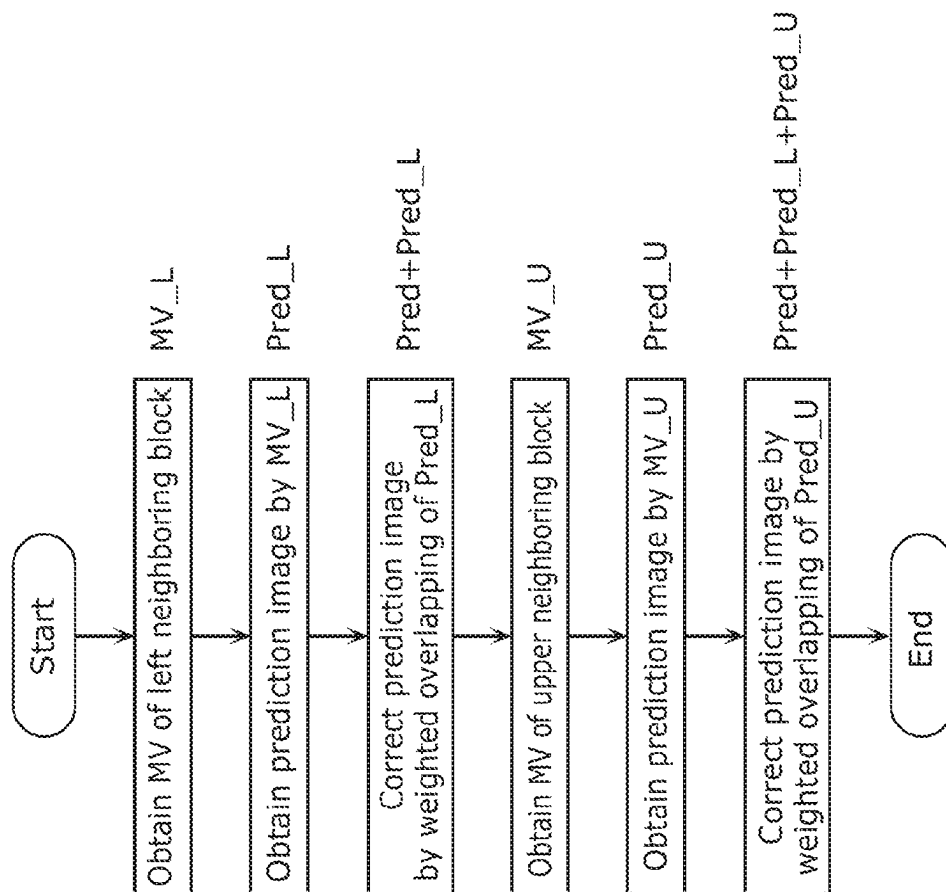


FIG. 62

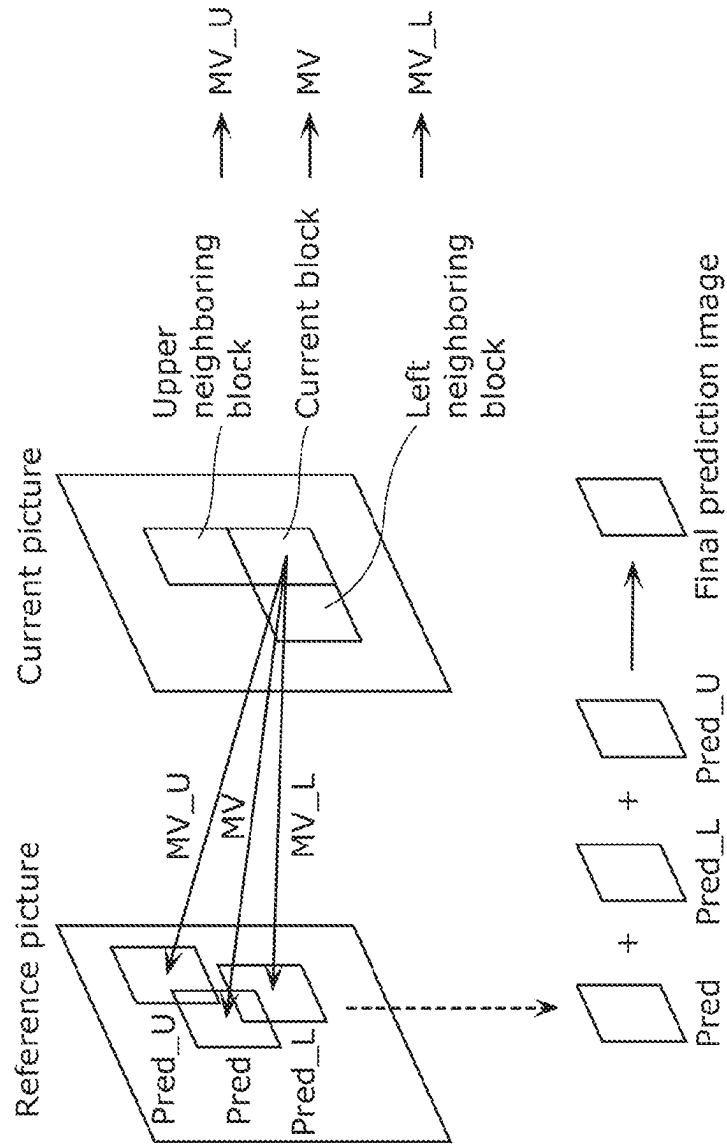


FIG. 63

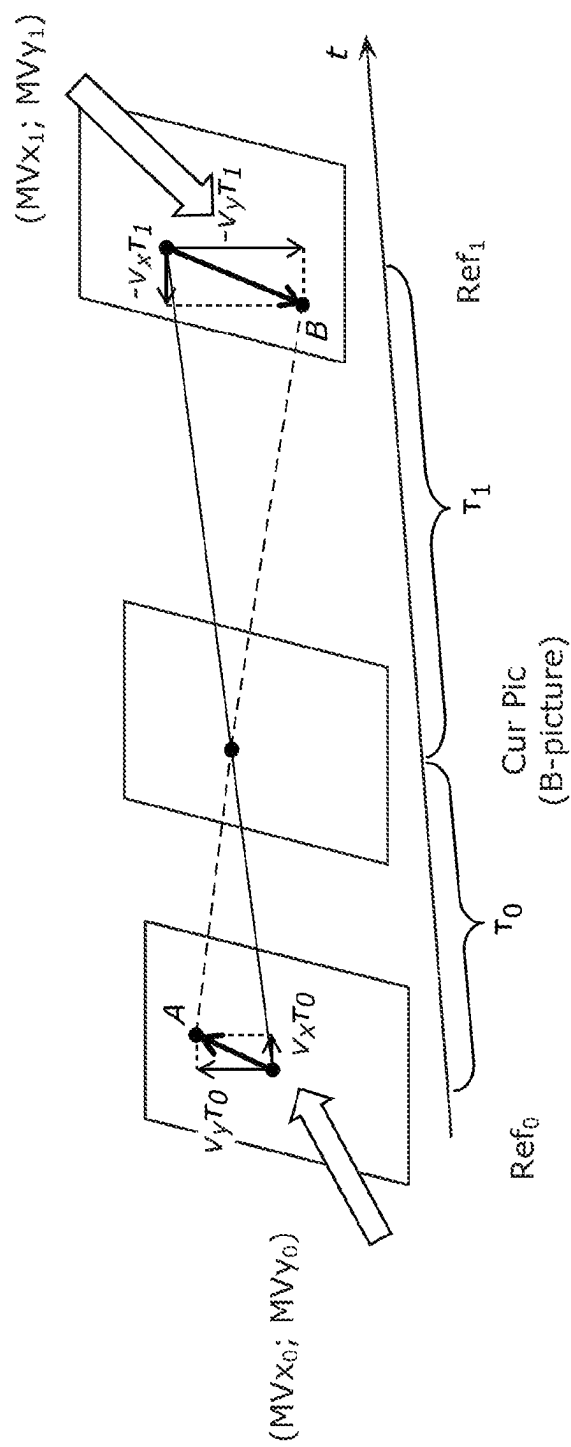


FIG. 64

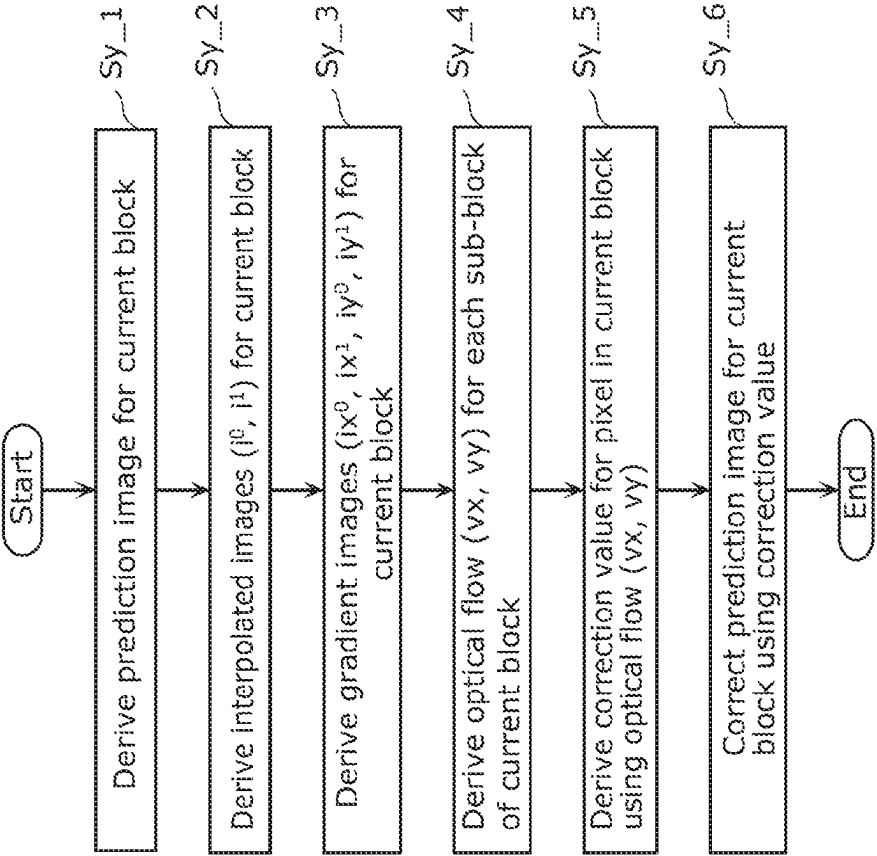


FIG. 65

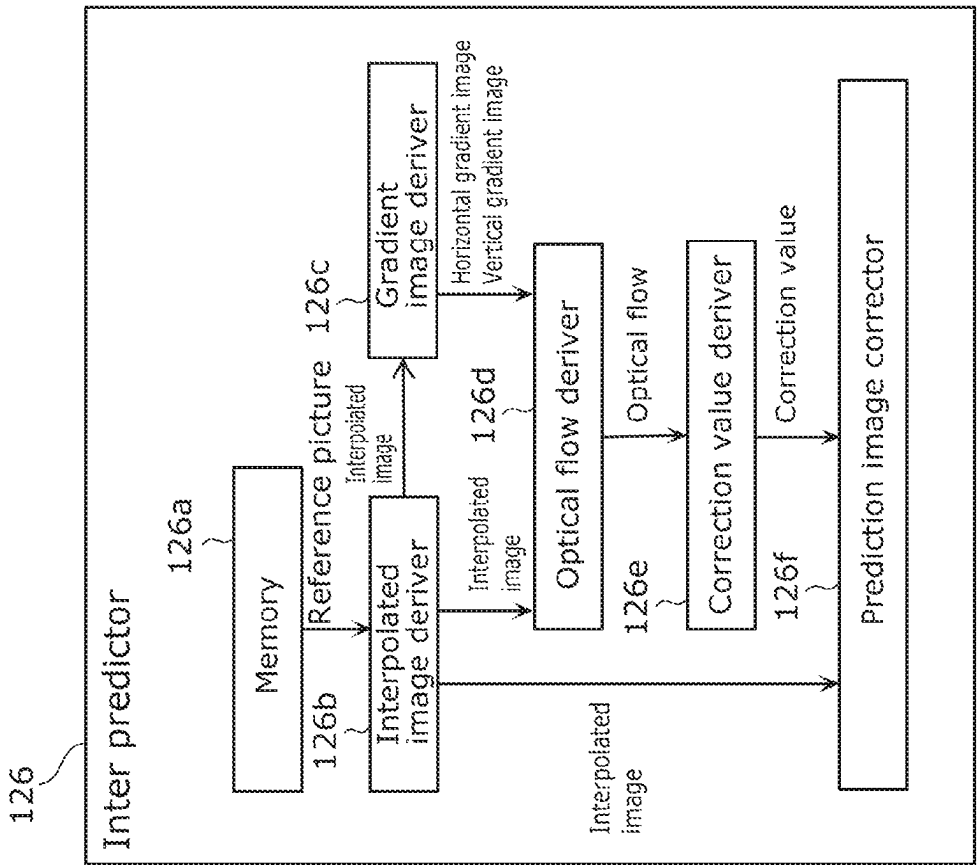


FIG. 66A

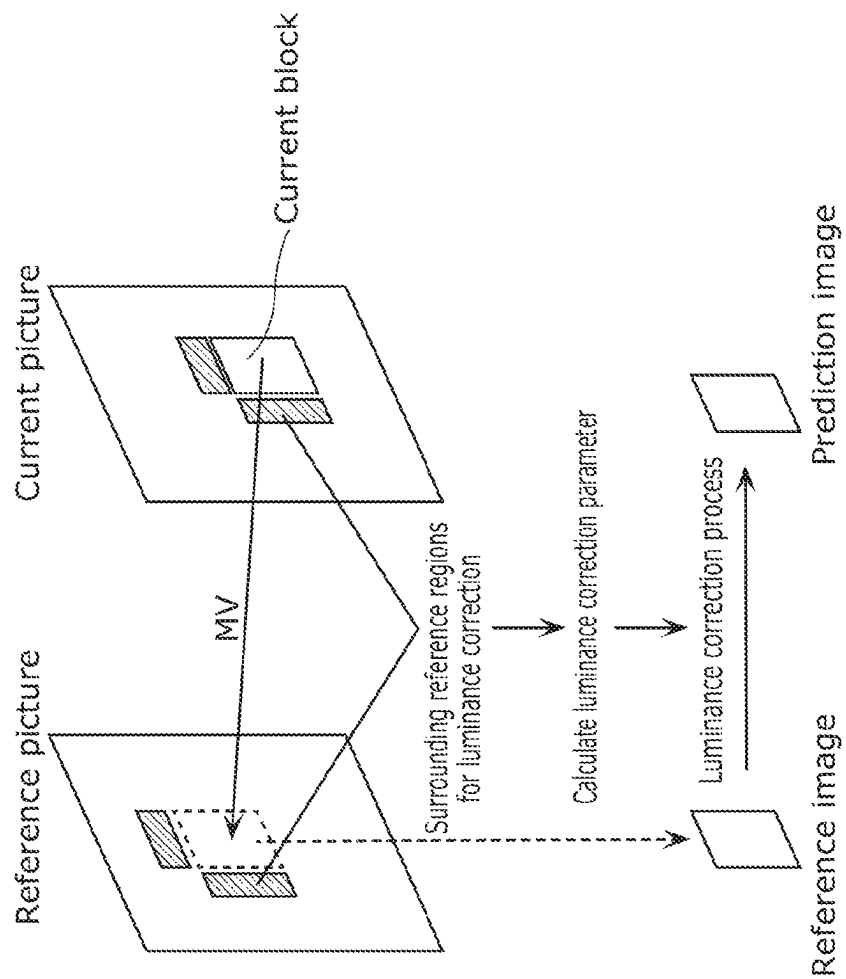


FIG. 66B

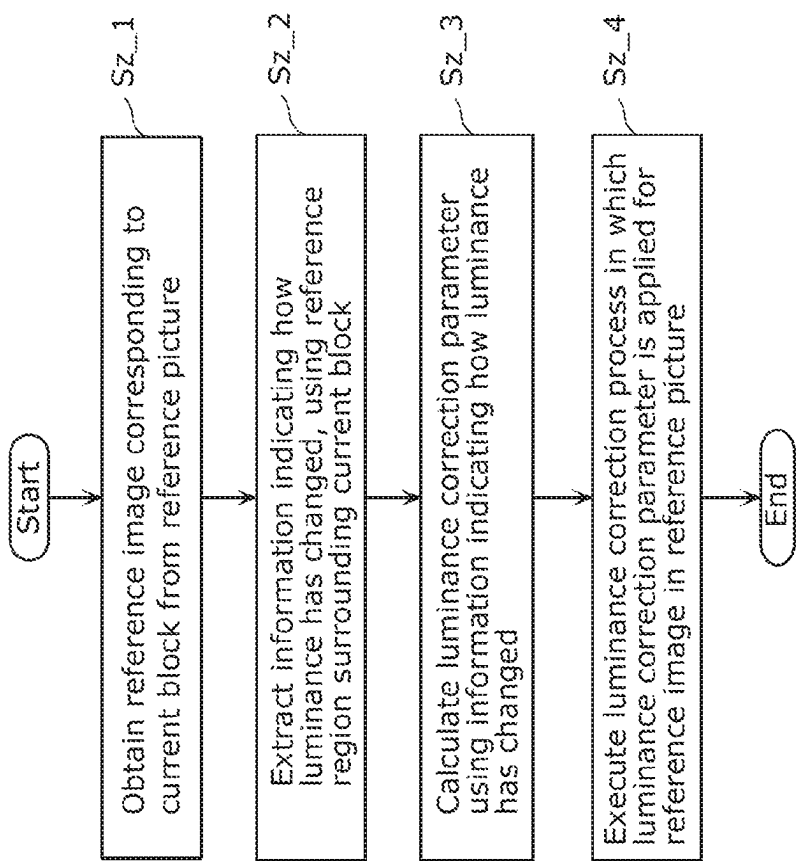


FIG. 67

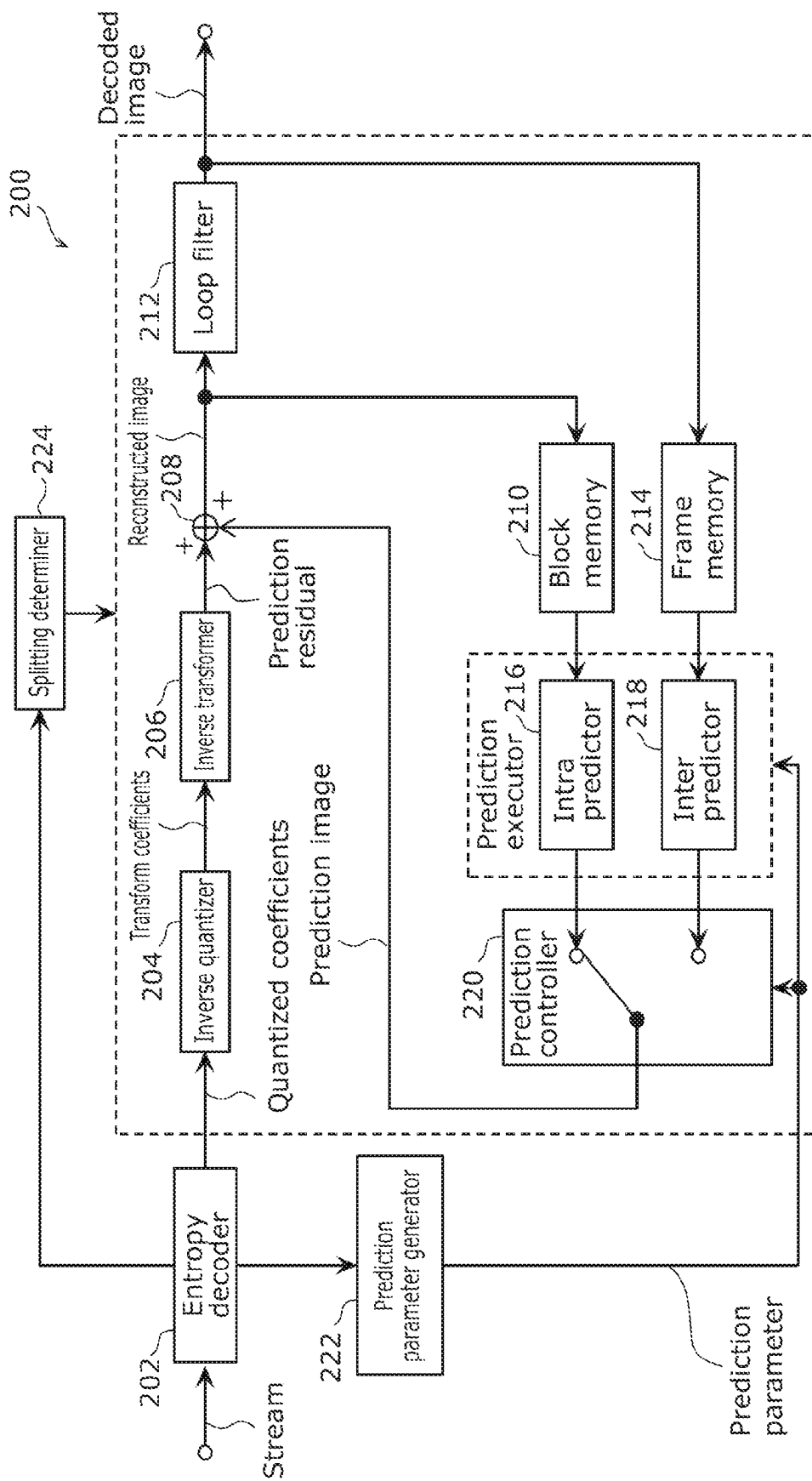


FIG. 68

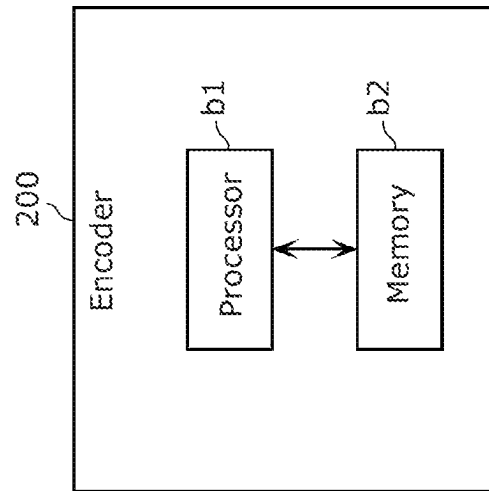


FIG. 69

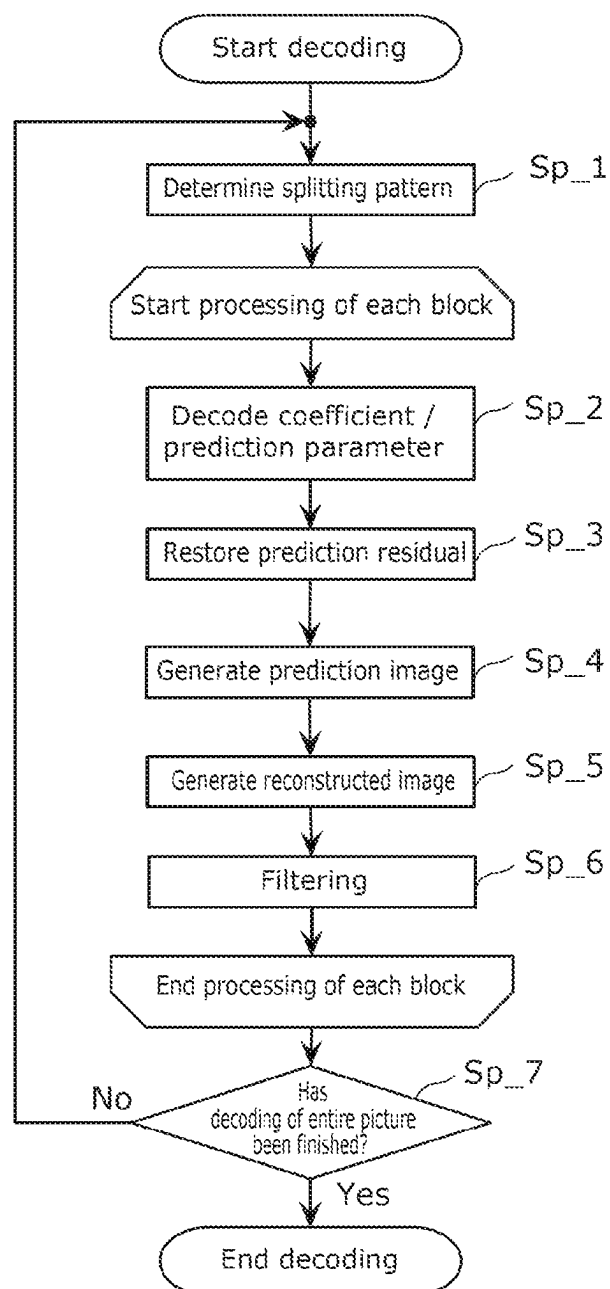


FIG. 70

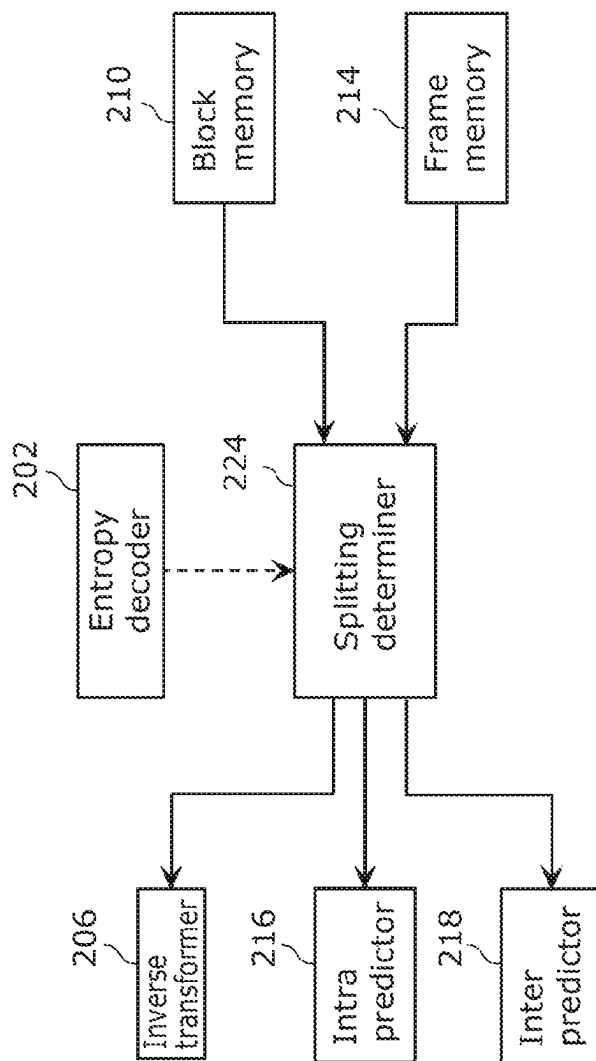


FIG. 71

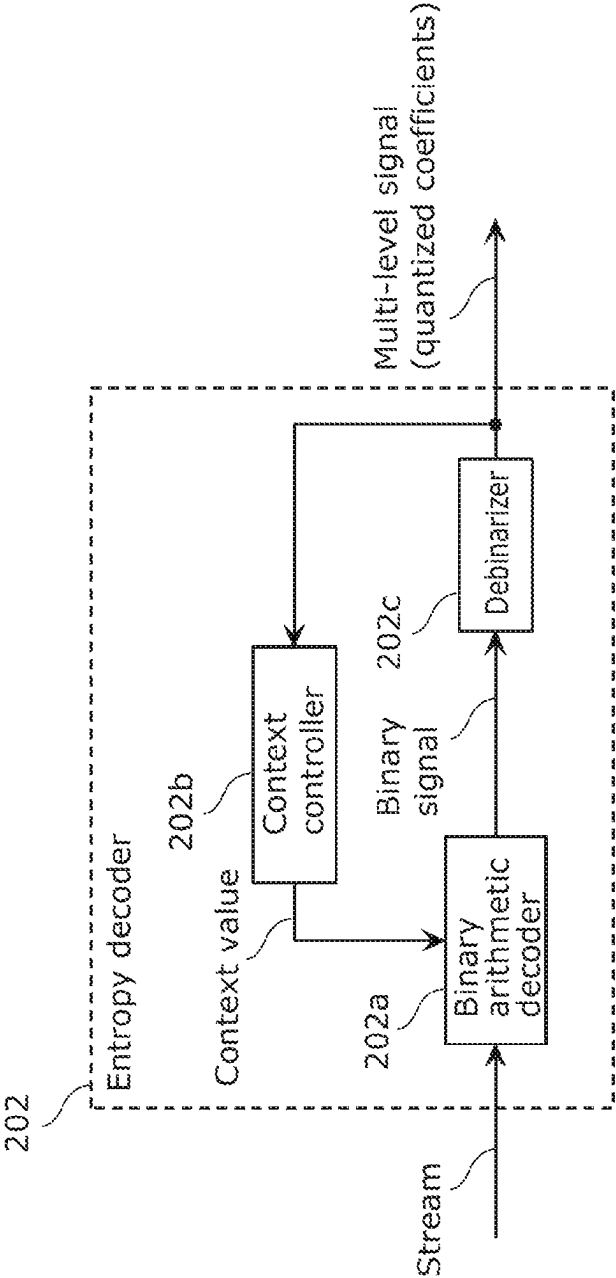


FIG. 72

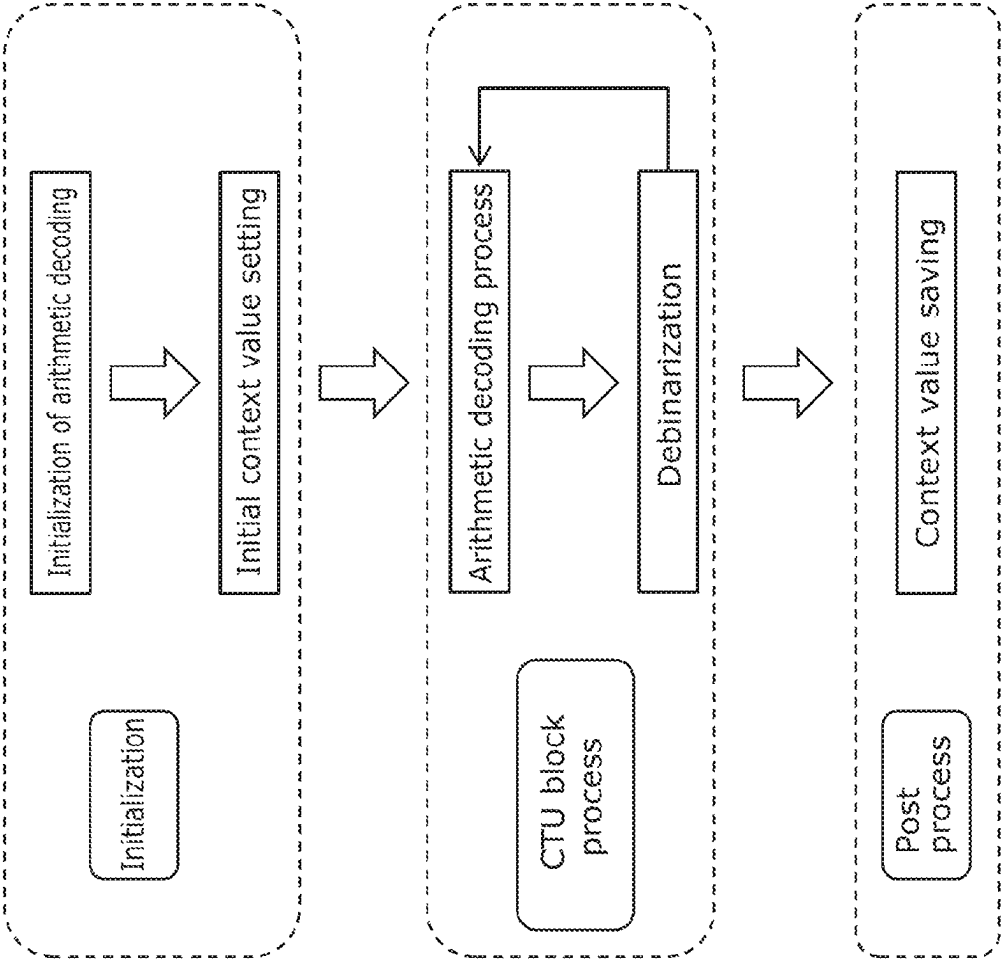


FIG. 73

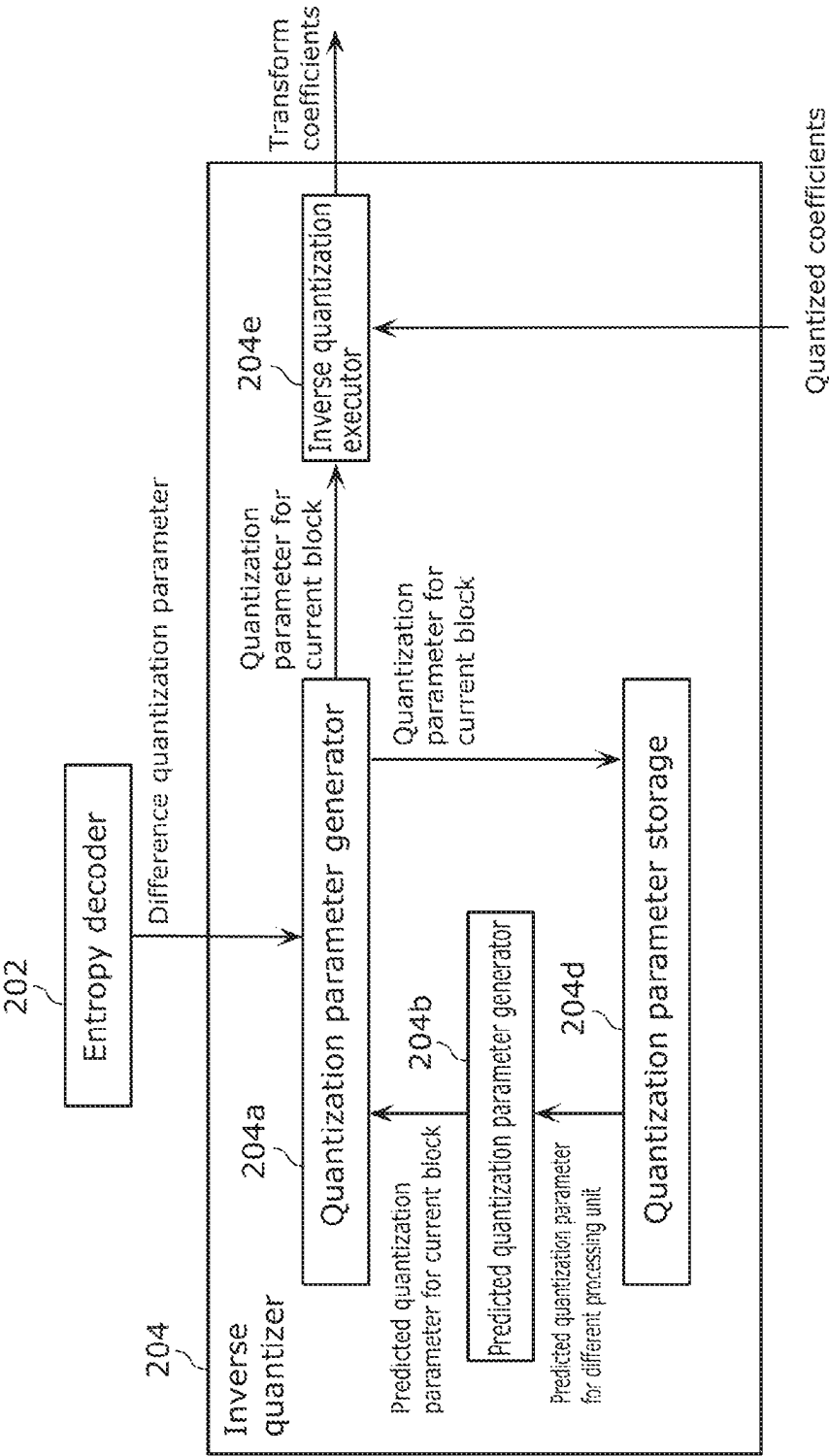


FIG. 74

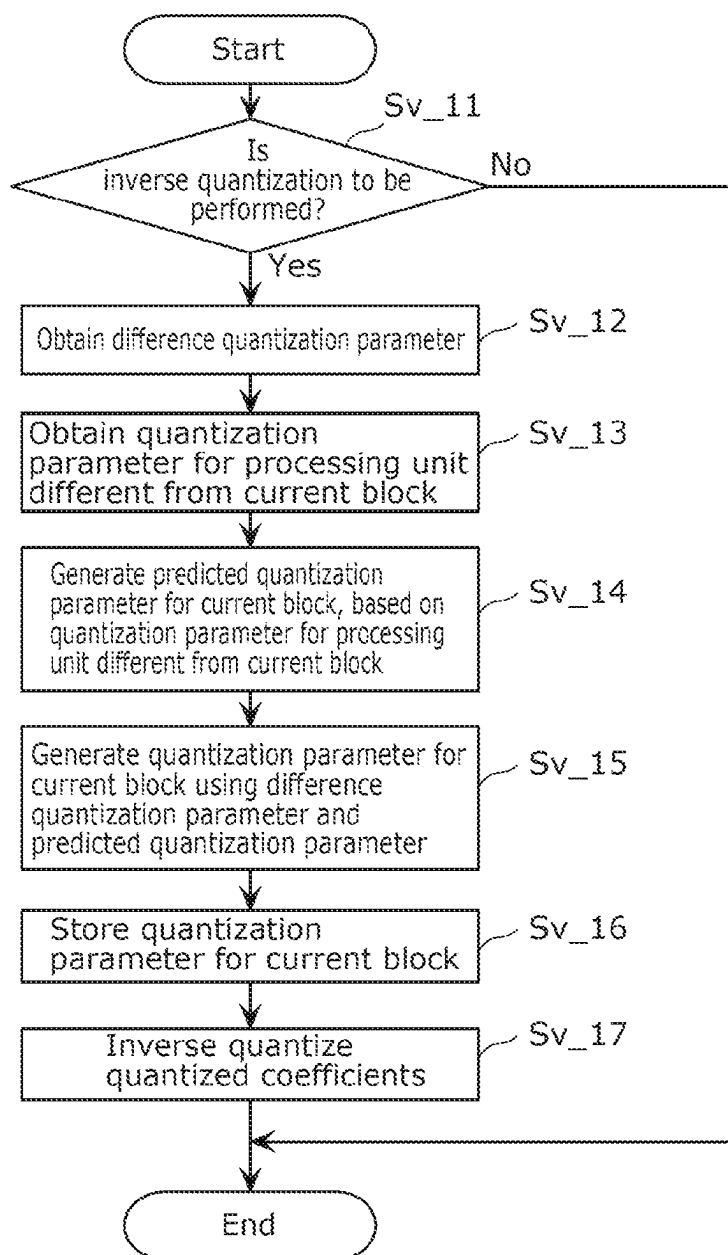


FIG. 75

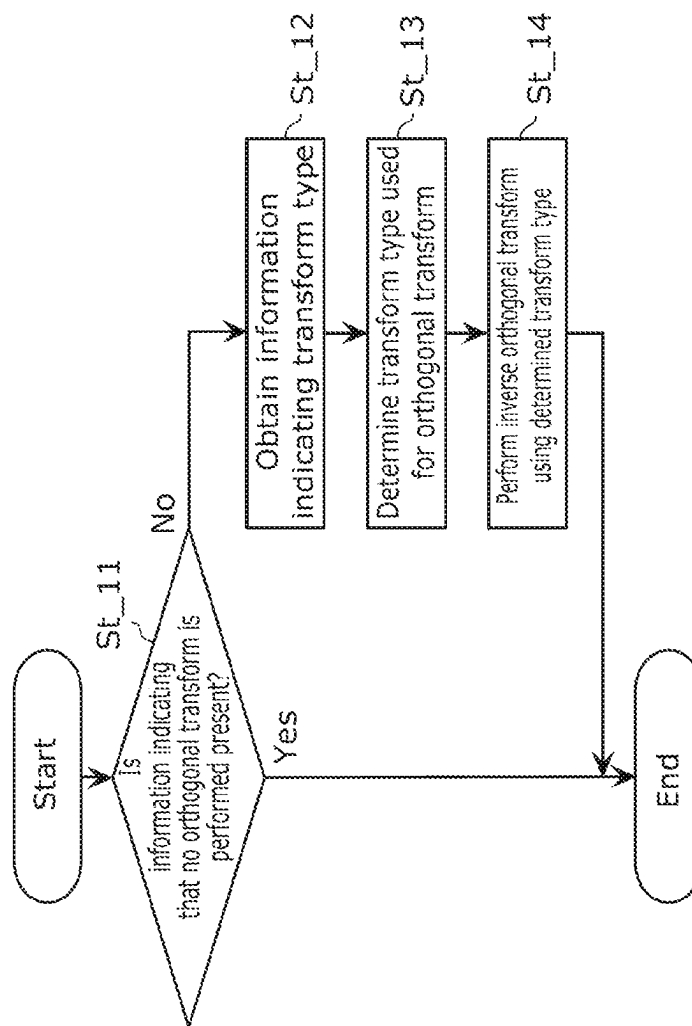


FIG. 76

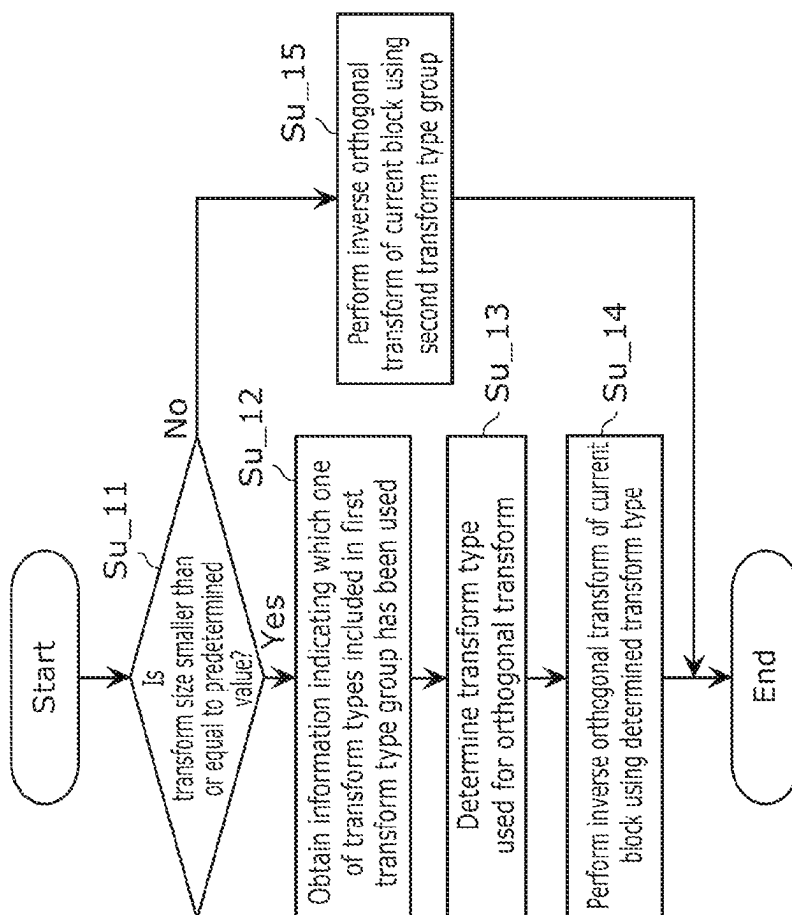


FIG. 77

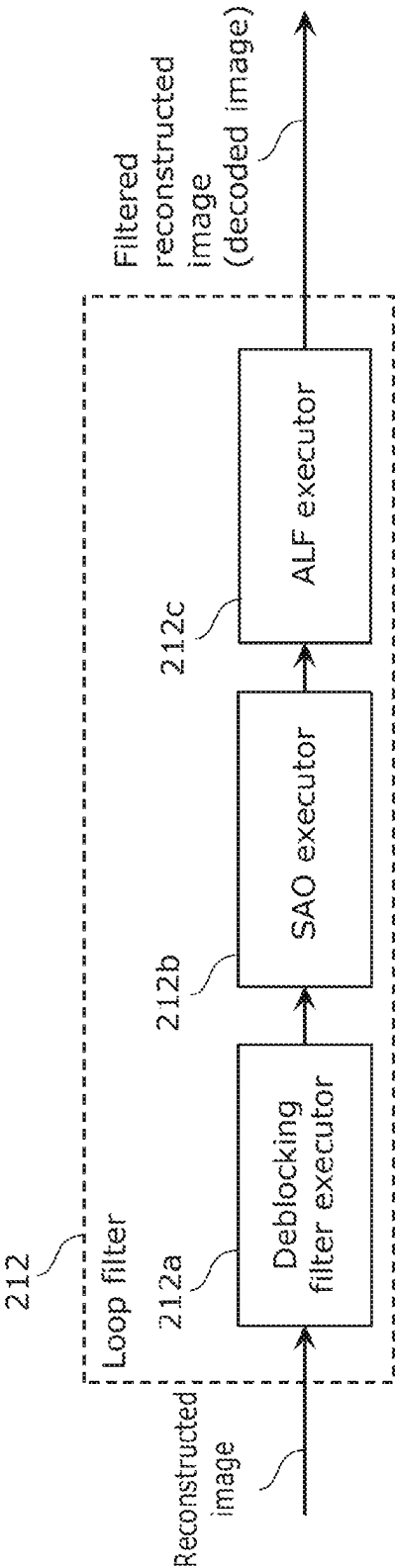


FIG. 78

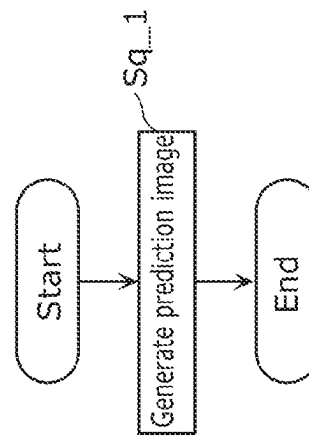


FIG. 79

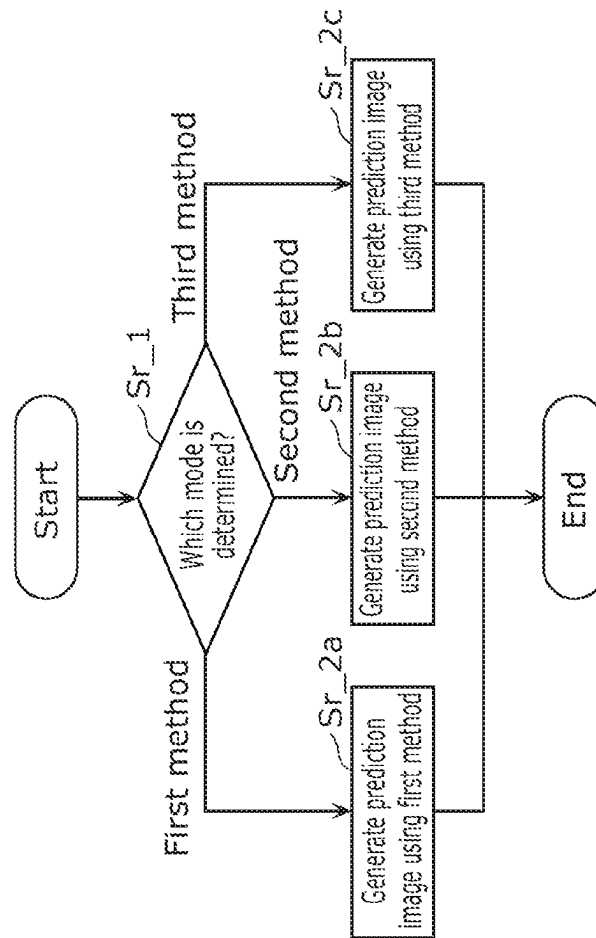


FIG. 80A

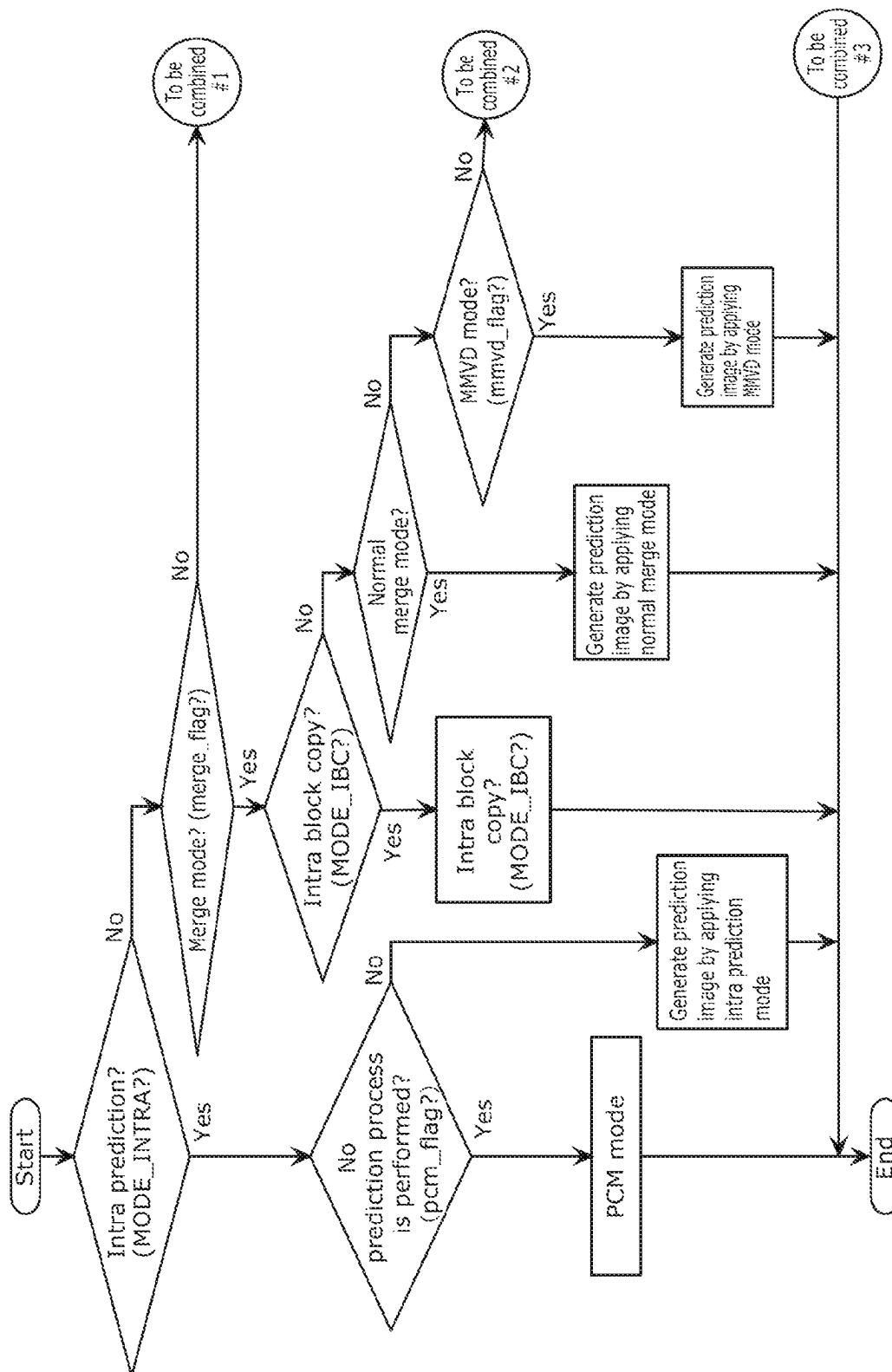


FIG. 80B

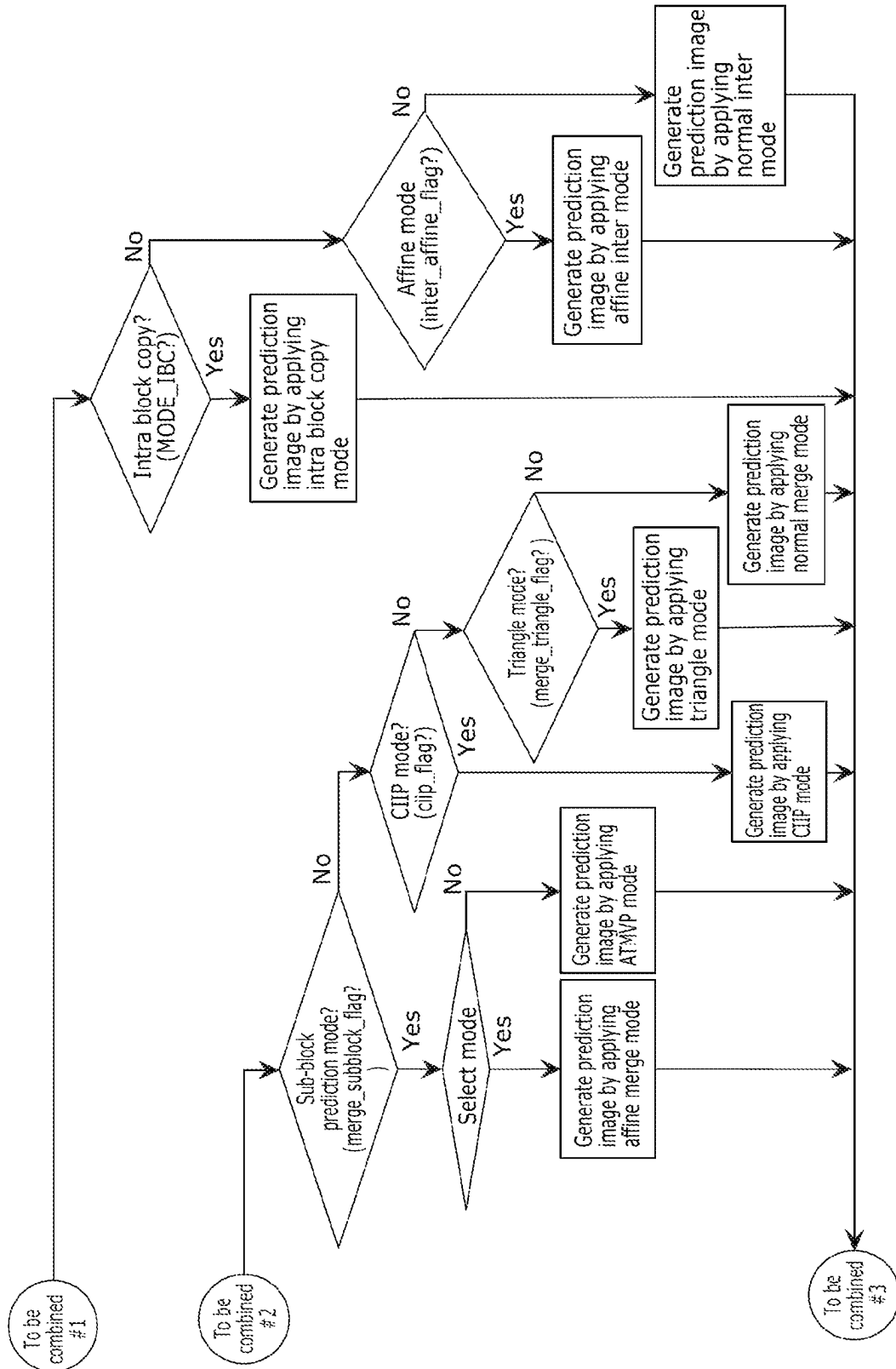


FIG. 81

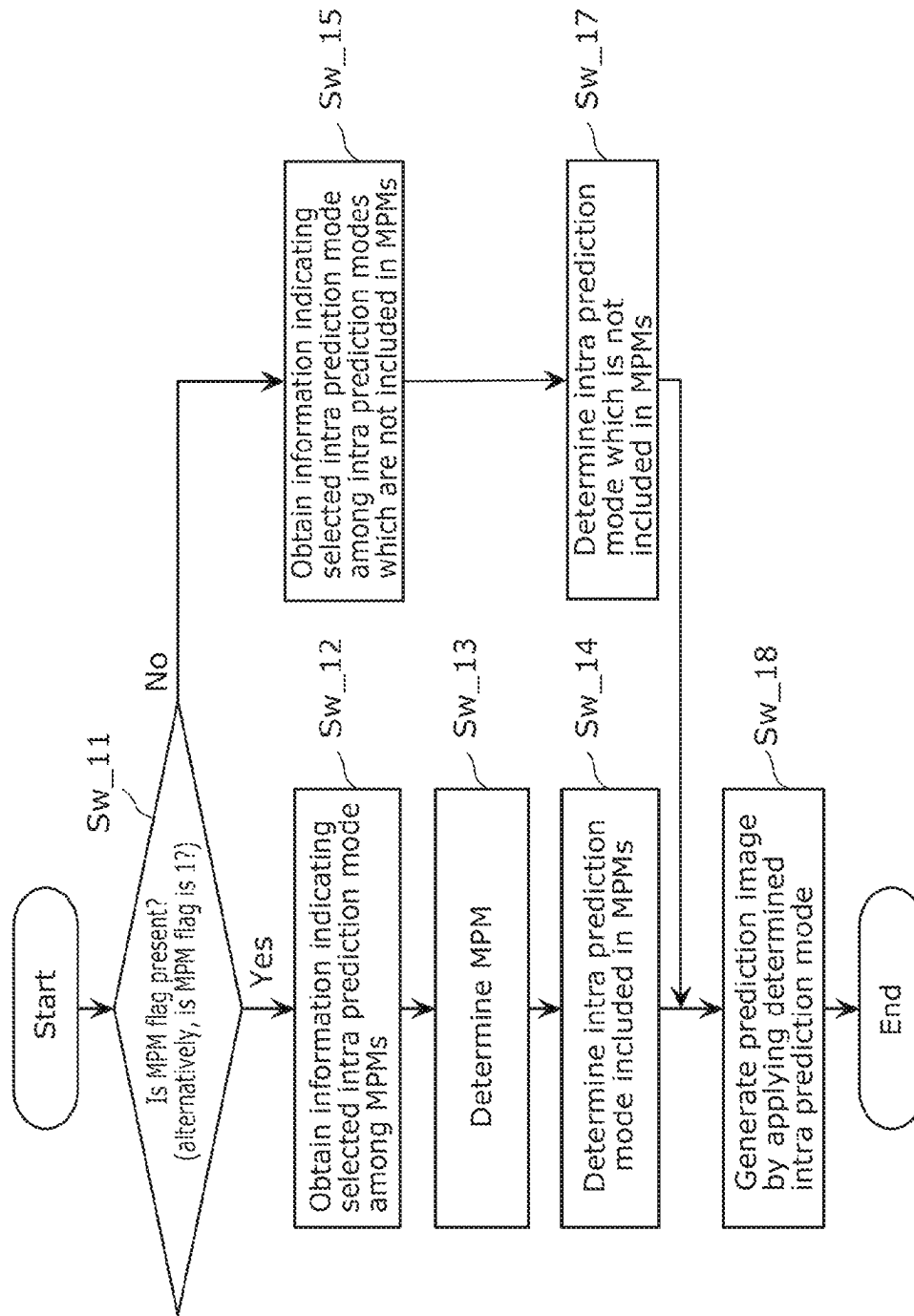


FIG. 82

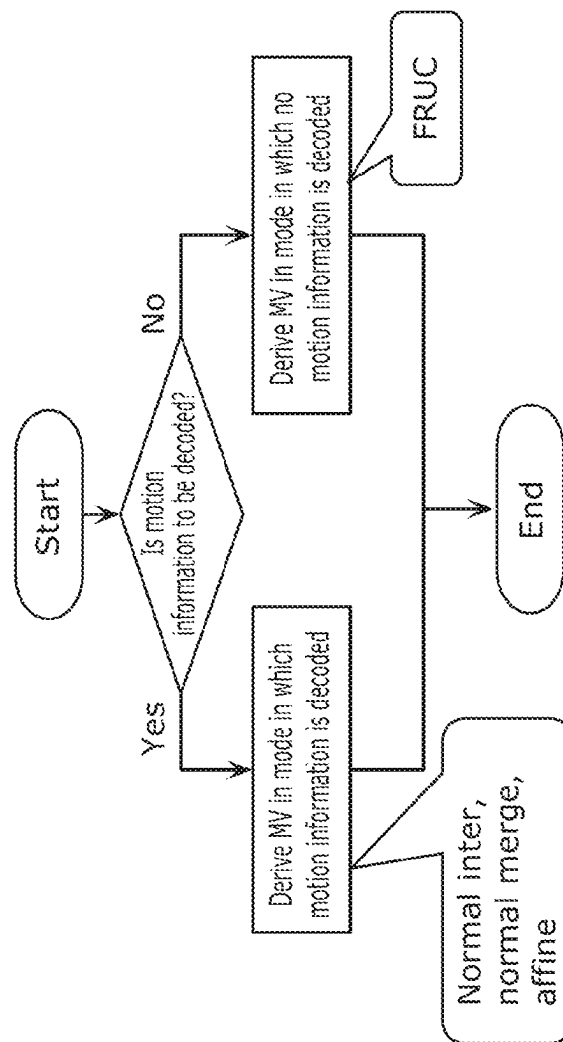


FIG. 83

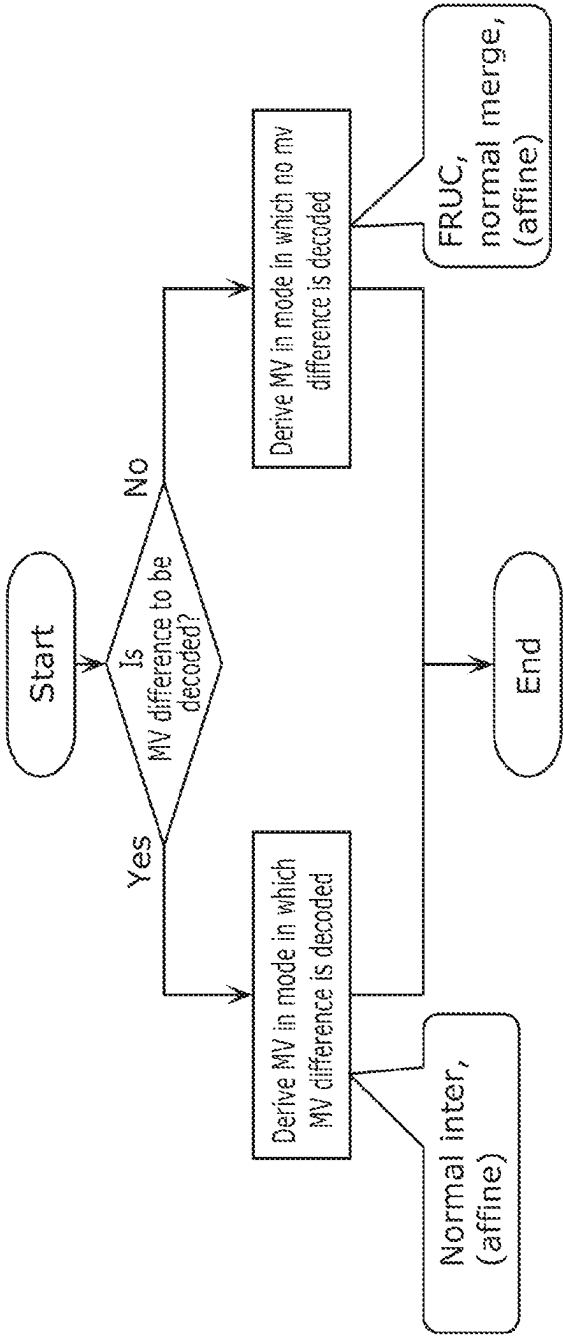


FIG. 84

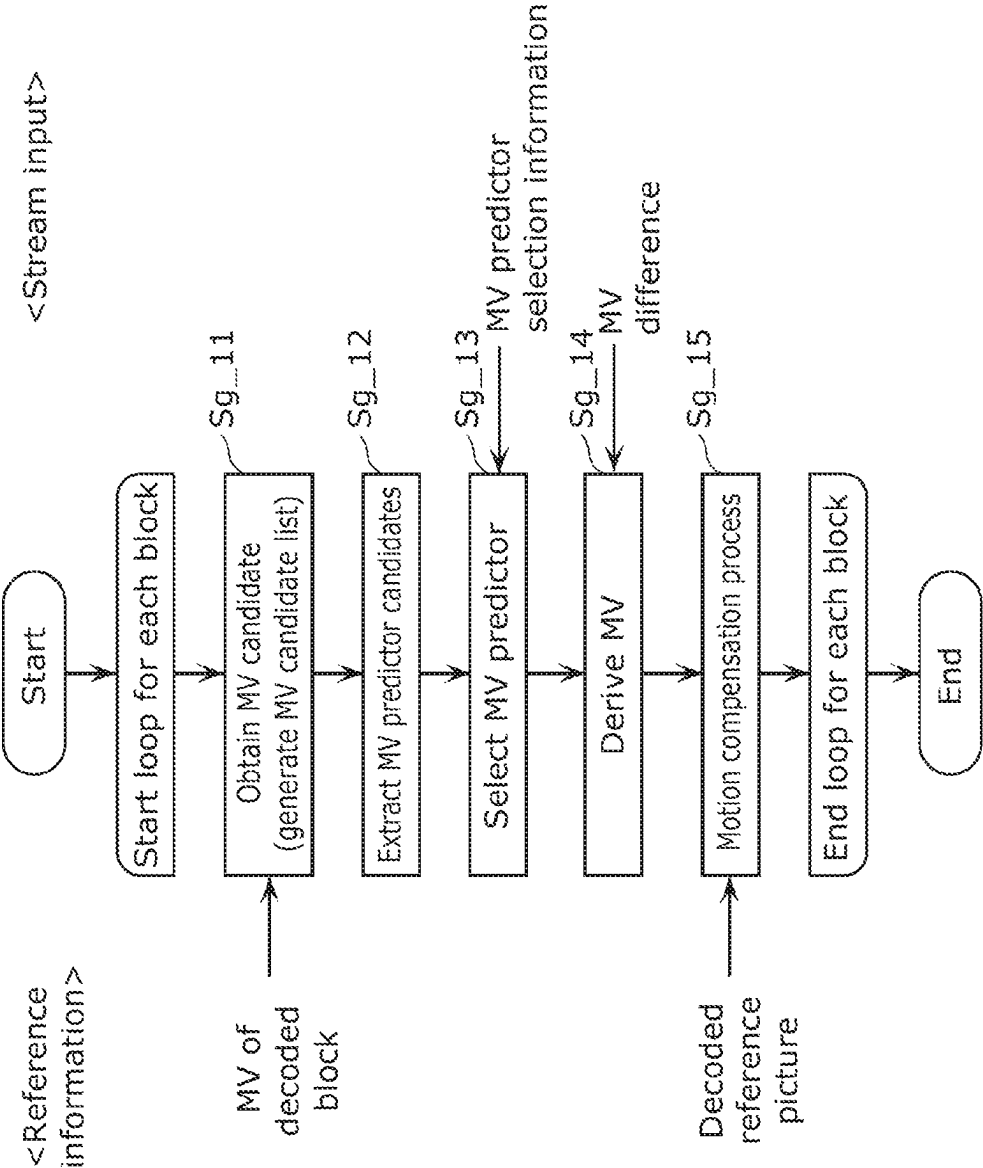


FIG. 85

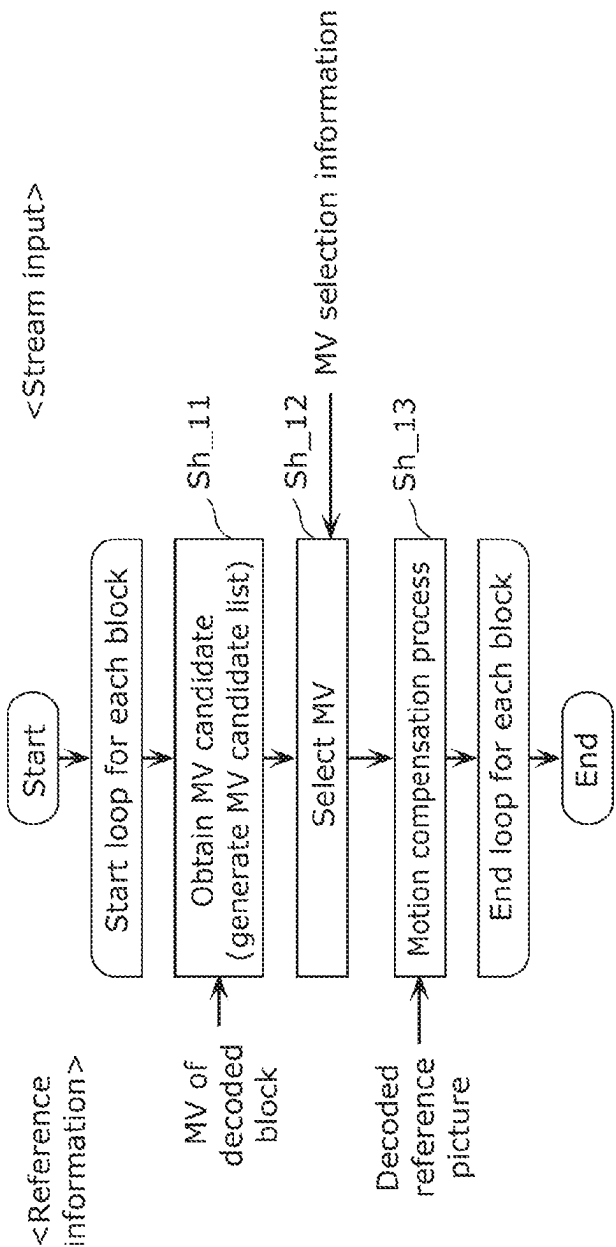


FIG. 86

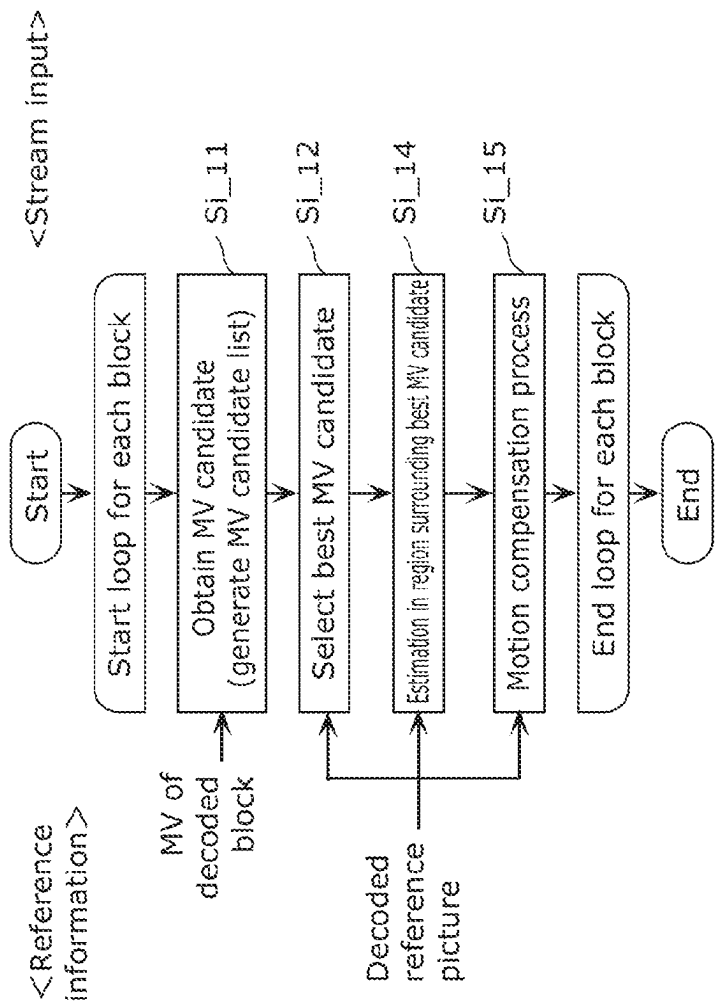


FIG. 87

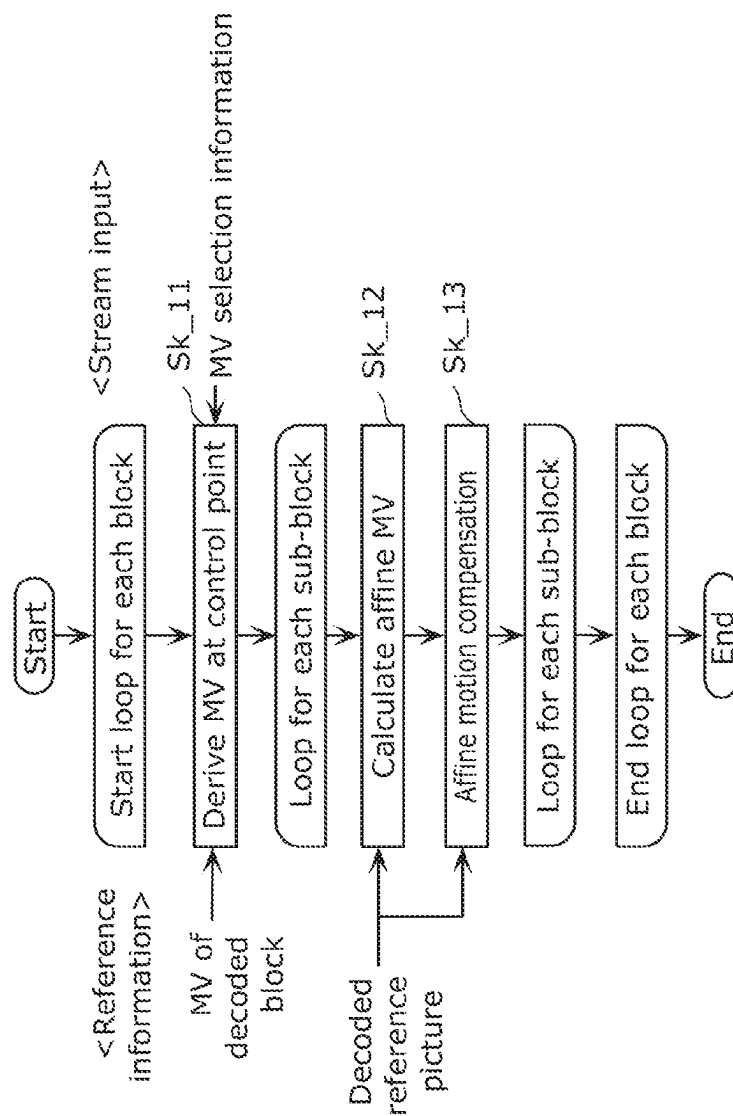


FIG. 88

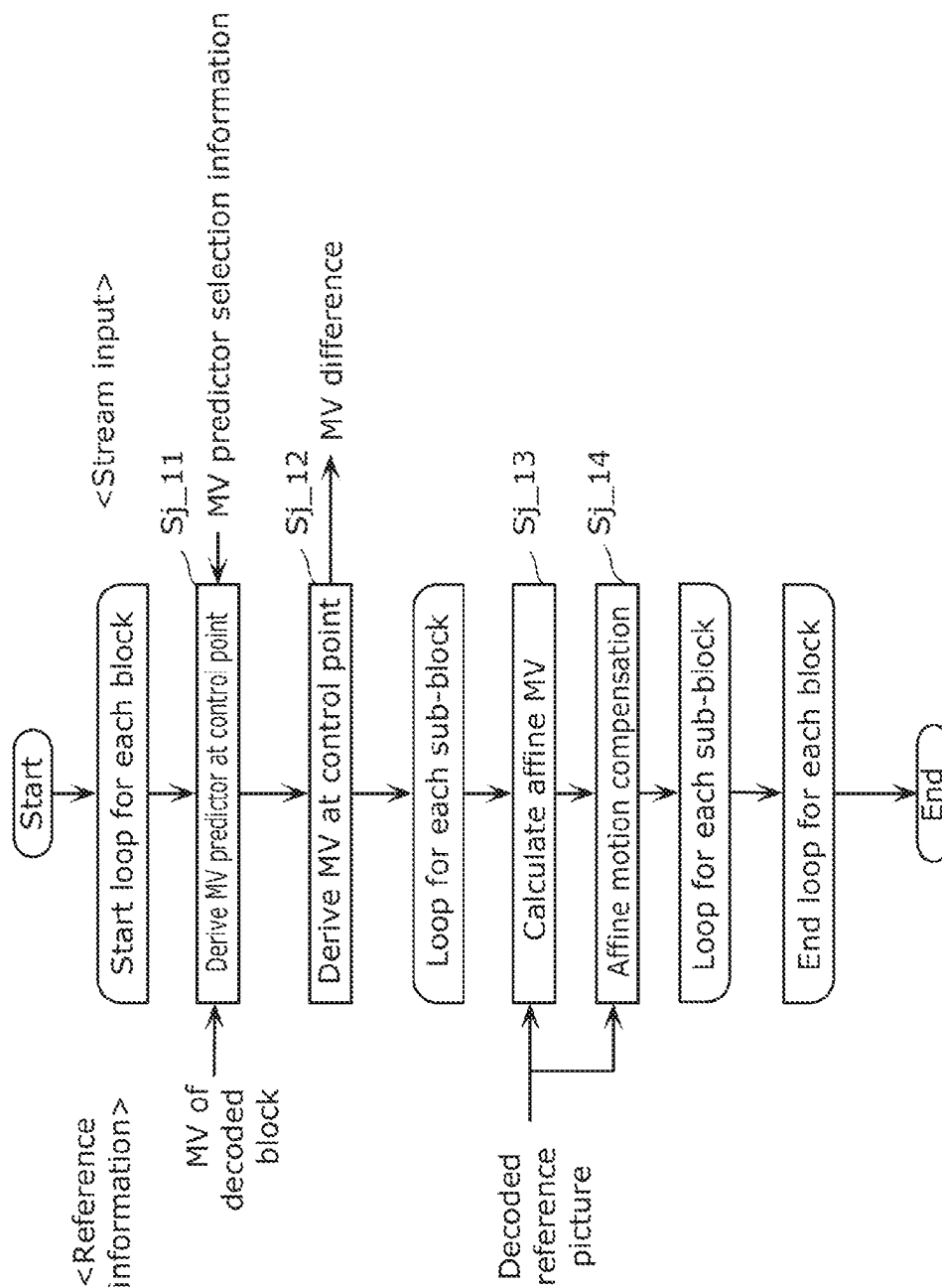


FIG. 89

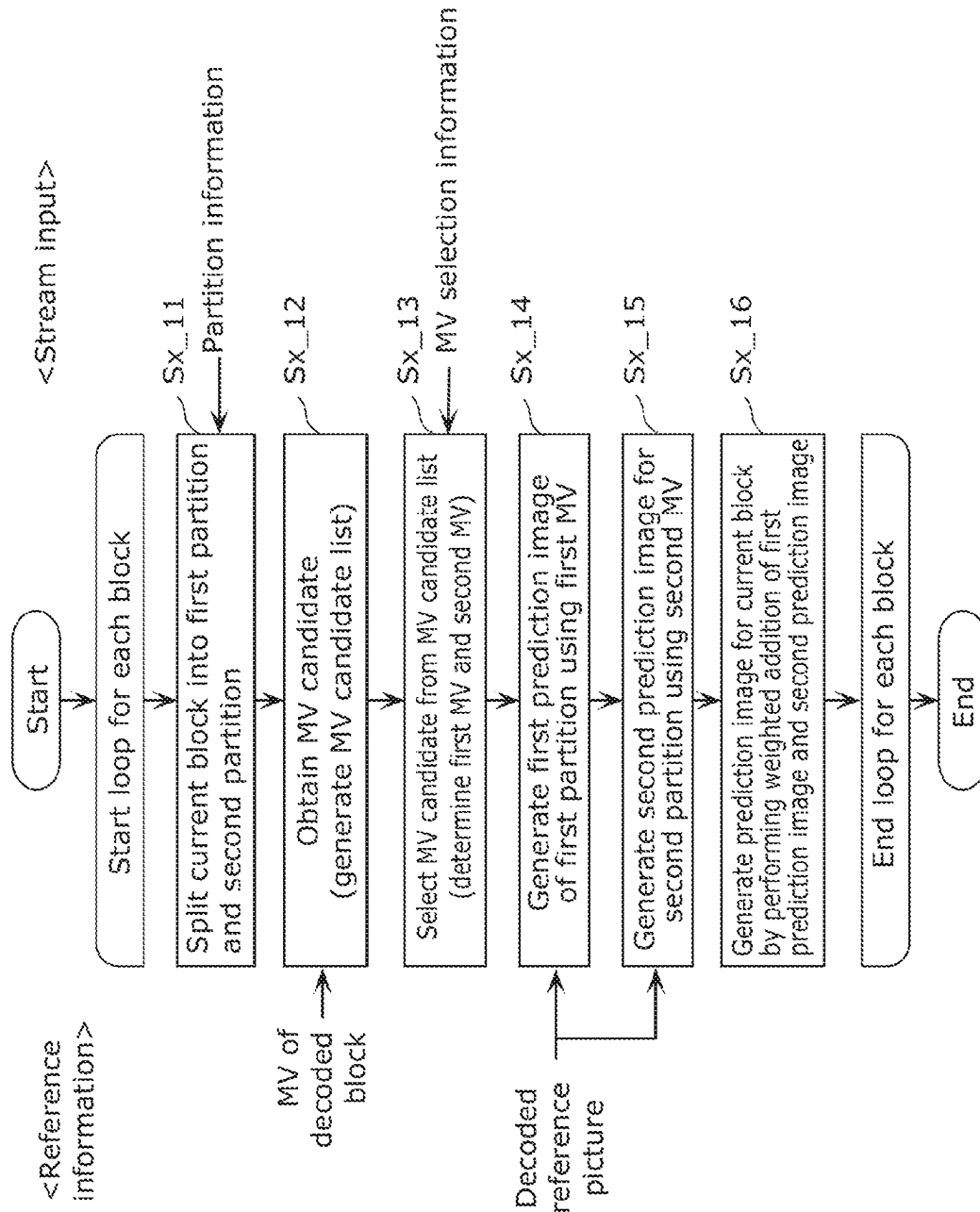


FIG. 90

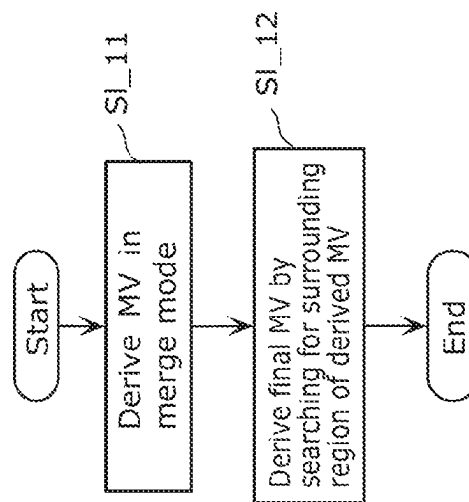


FIG. 91

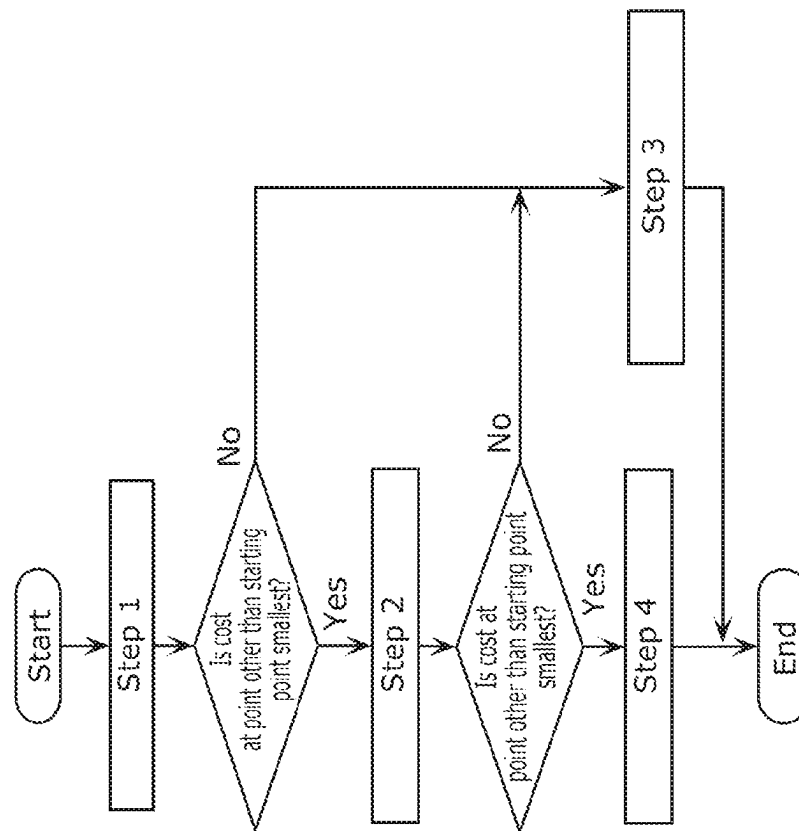


FIG. 92

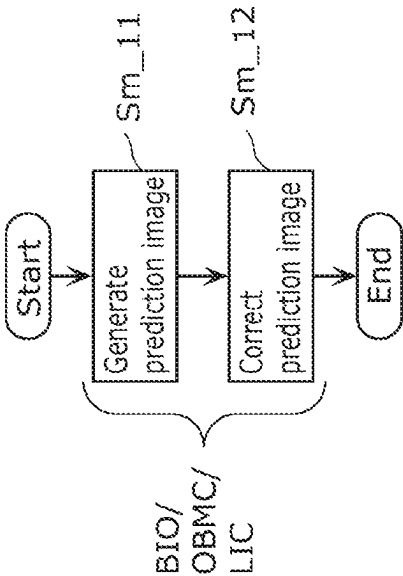


FIG. 93

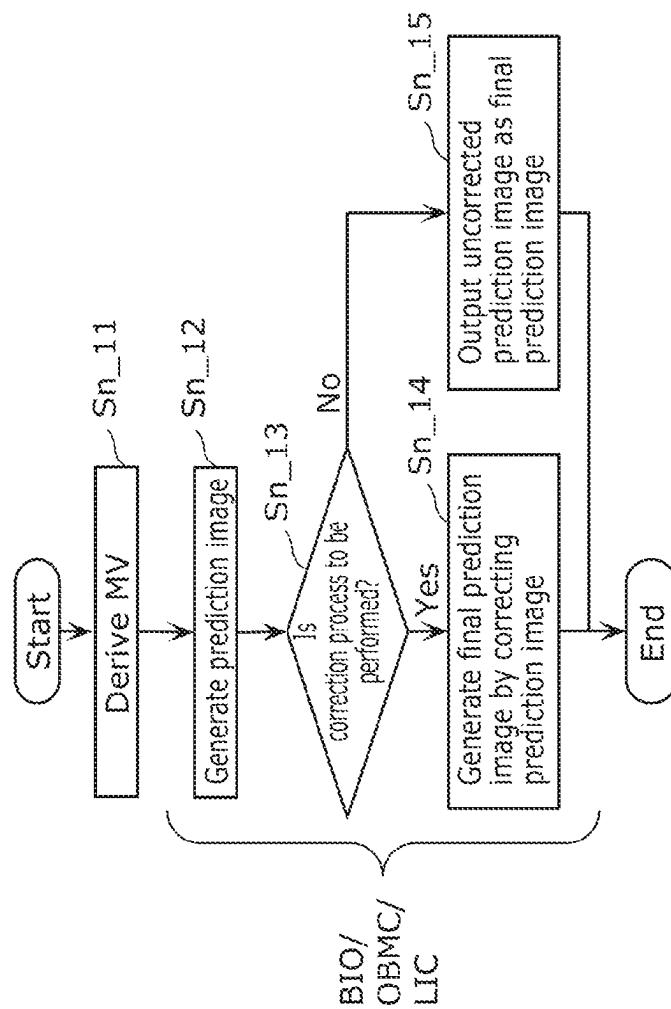


FIG. 94

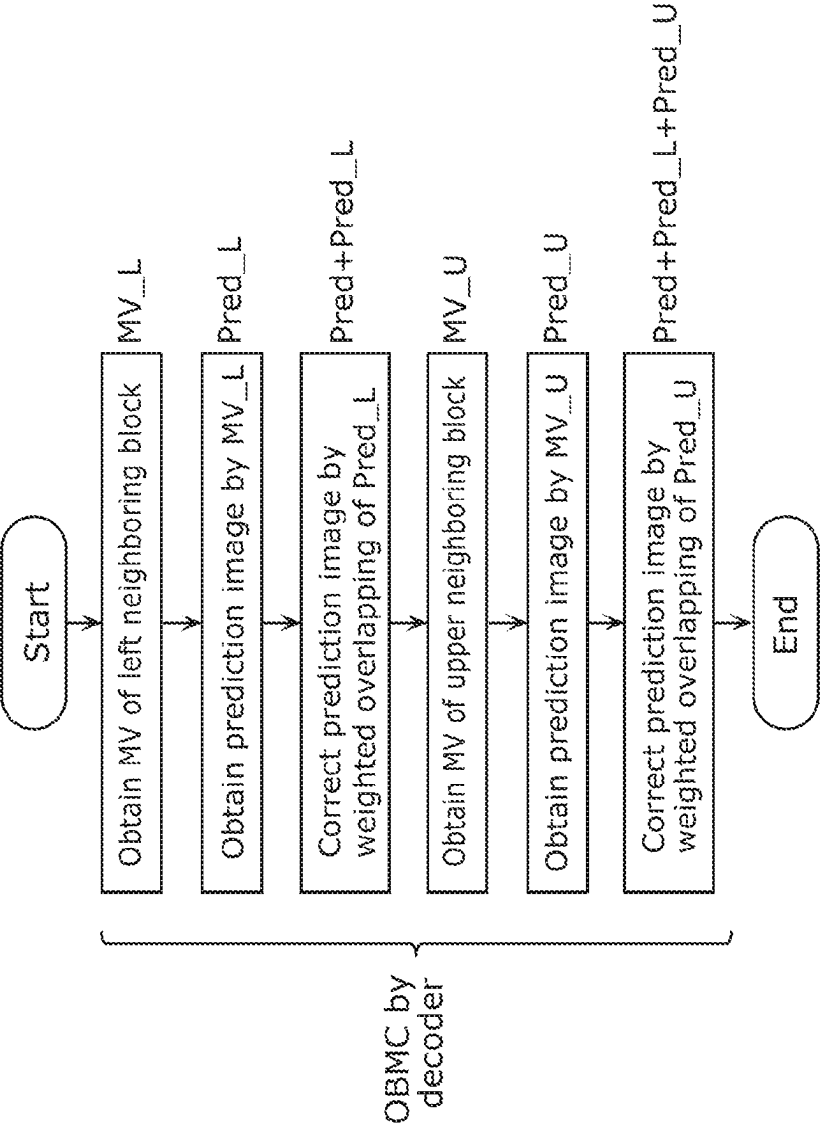


FIG. 95

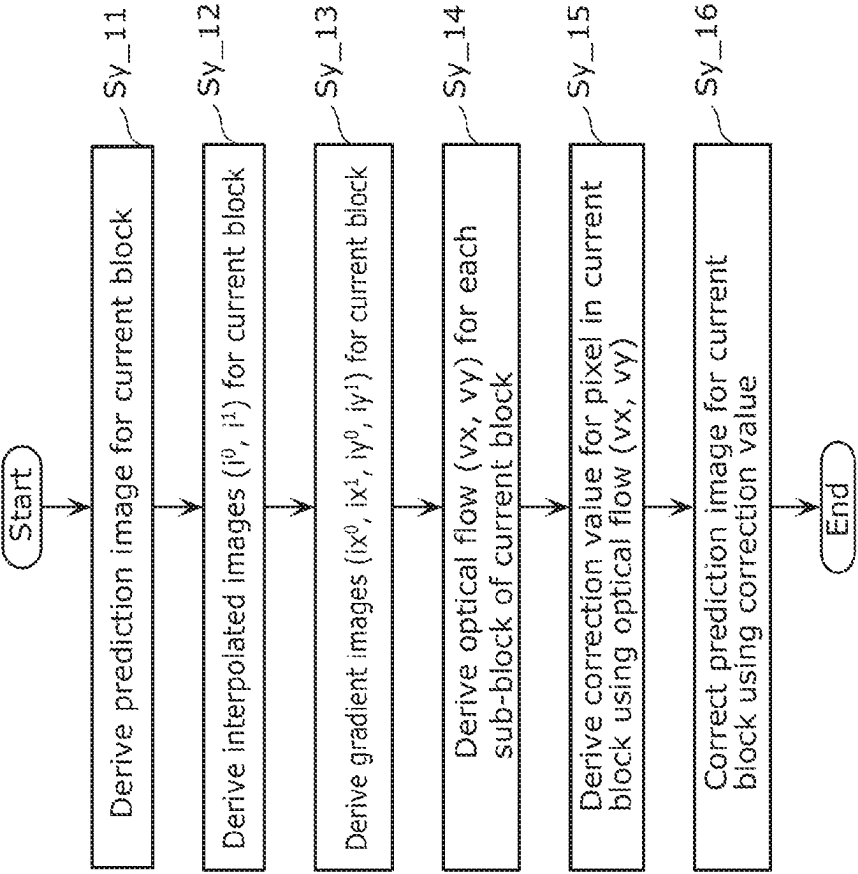


FIG. 96

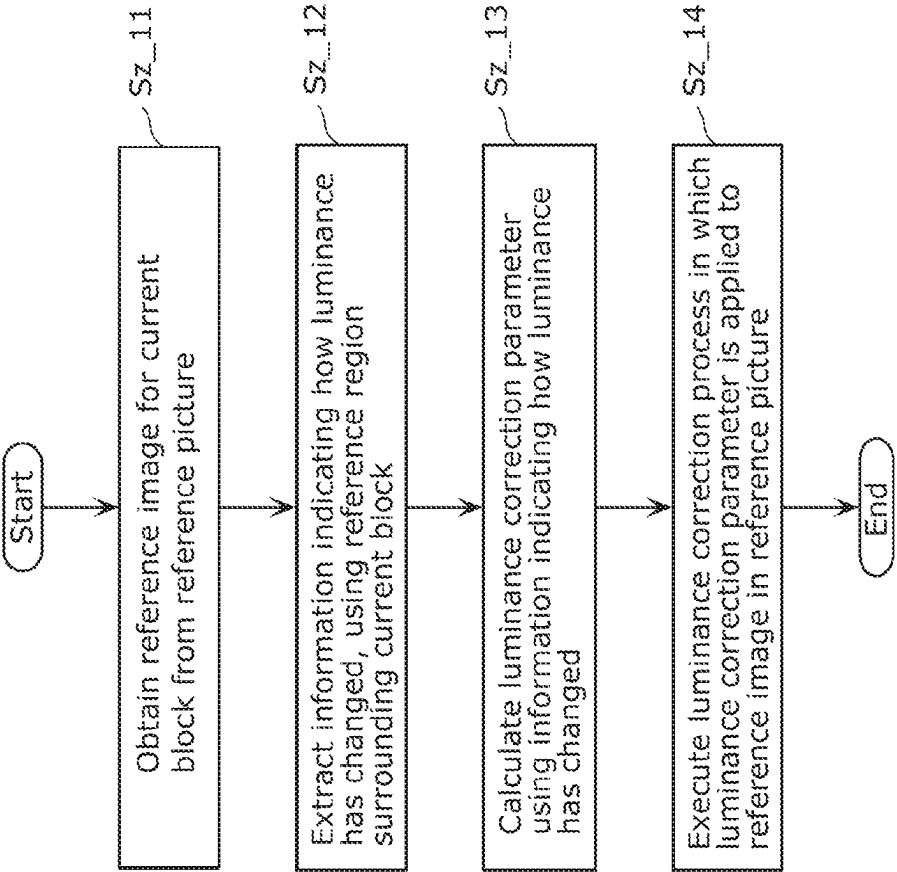


FIG. 97

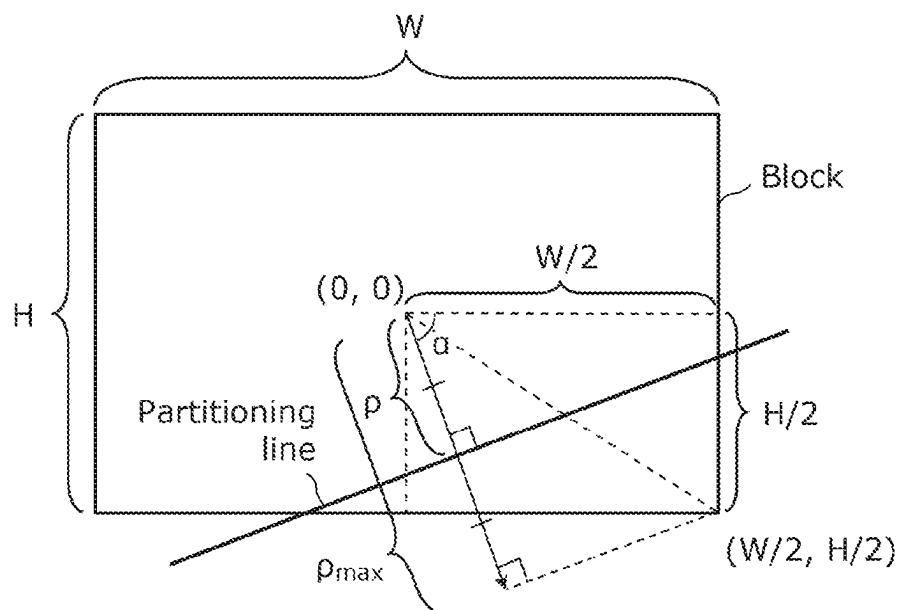


FIG. 98

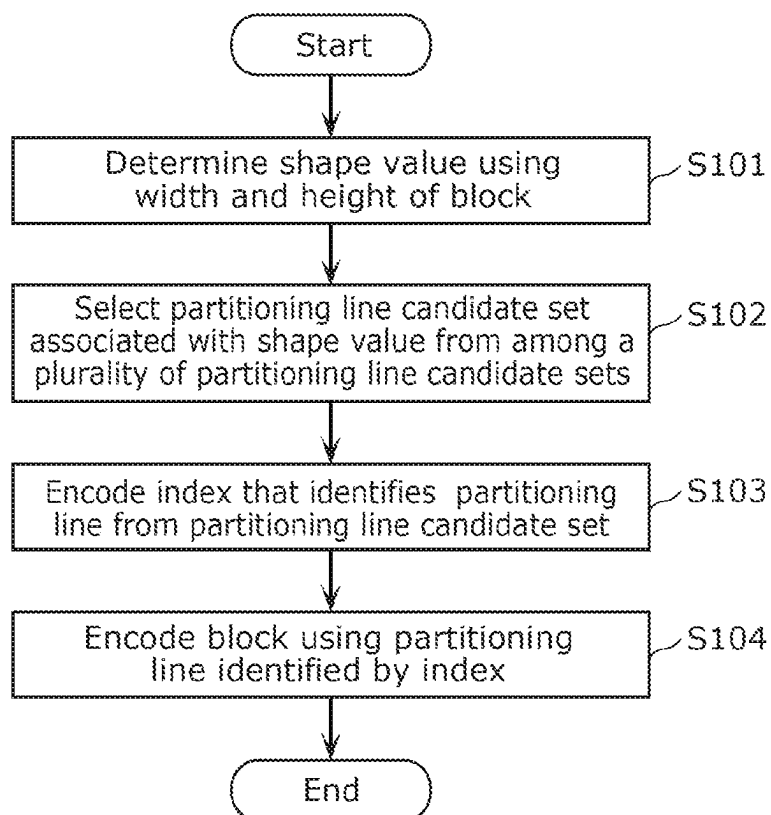


FIG. 99

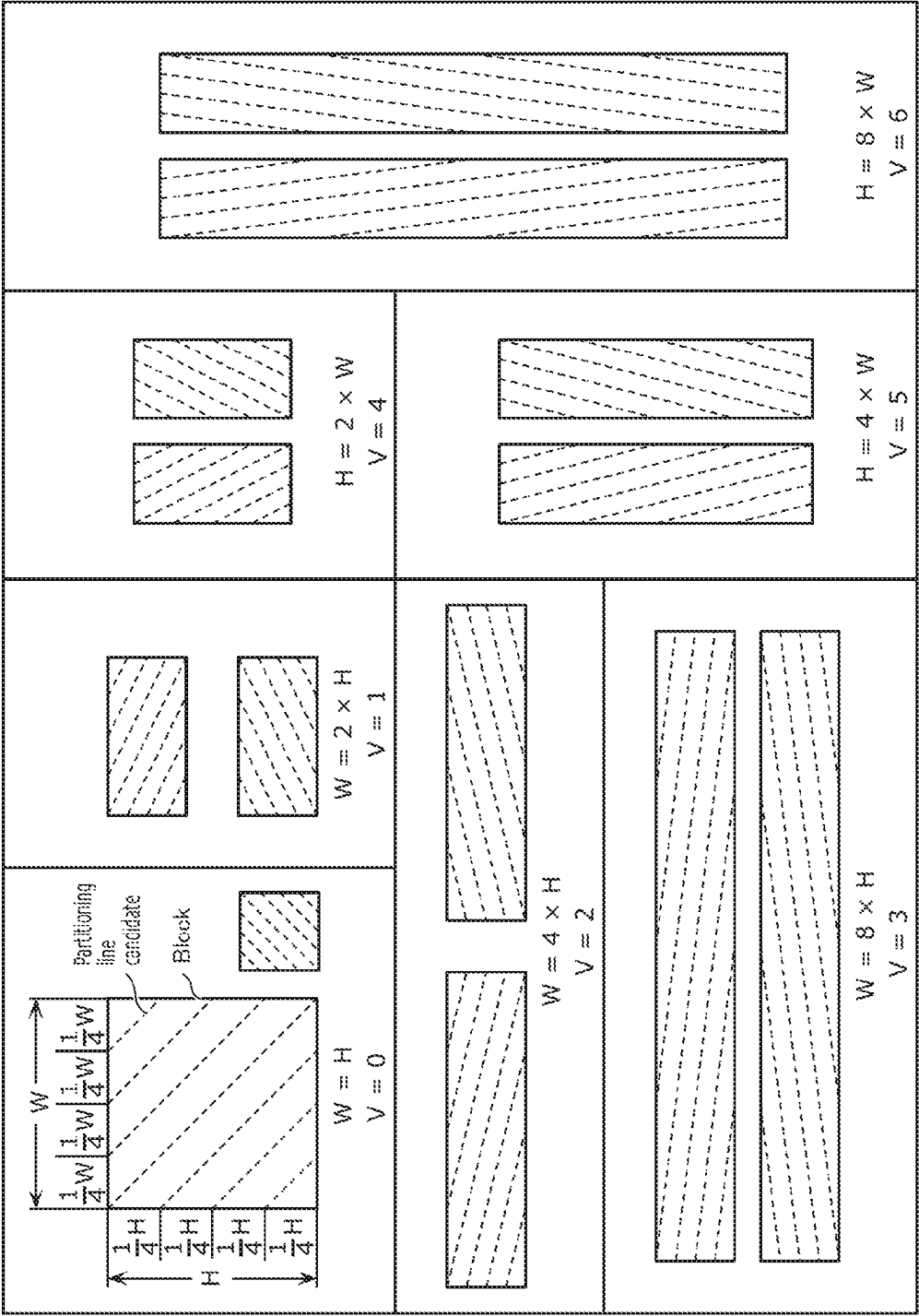


FIG. 100

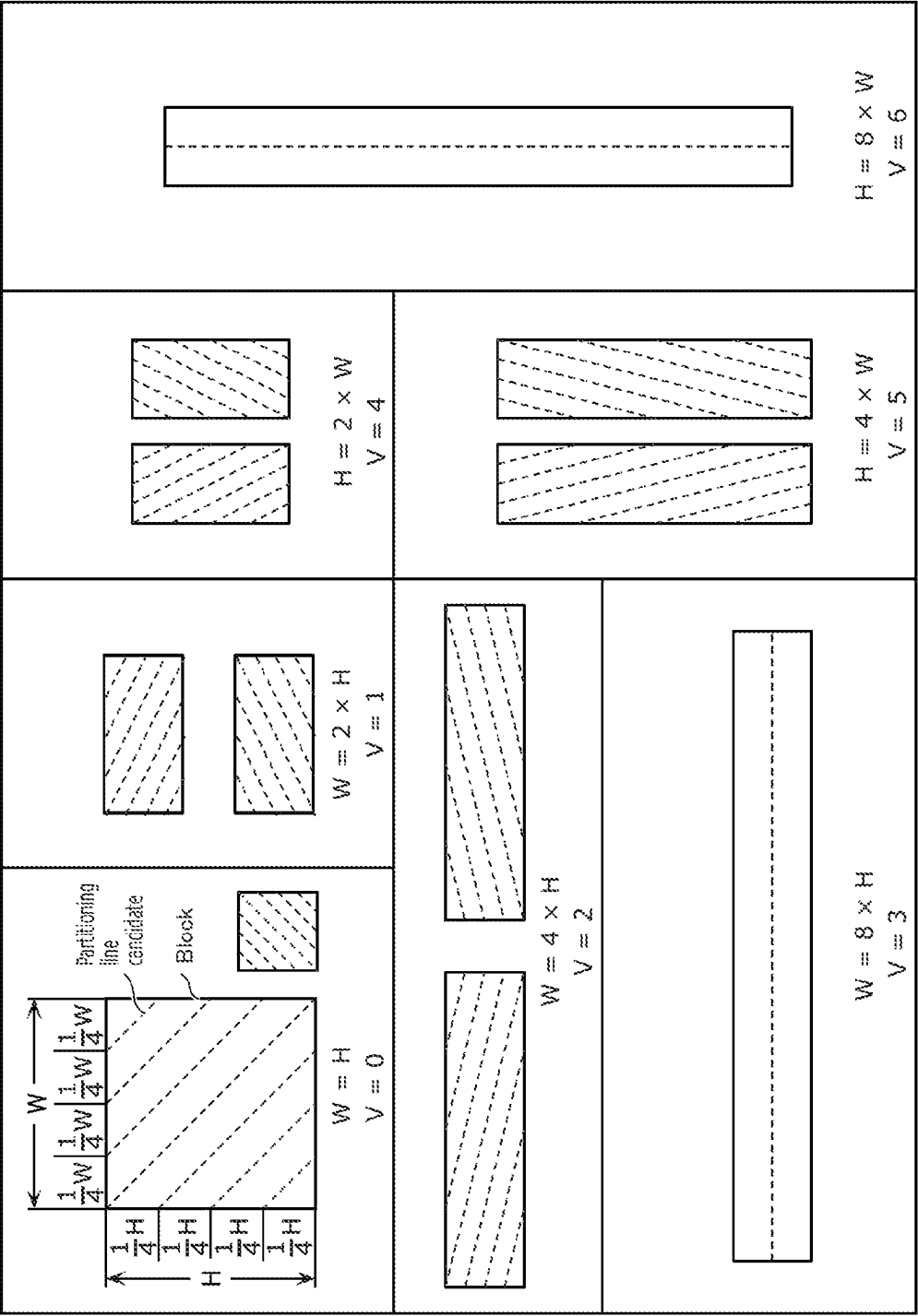


FIG. 101

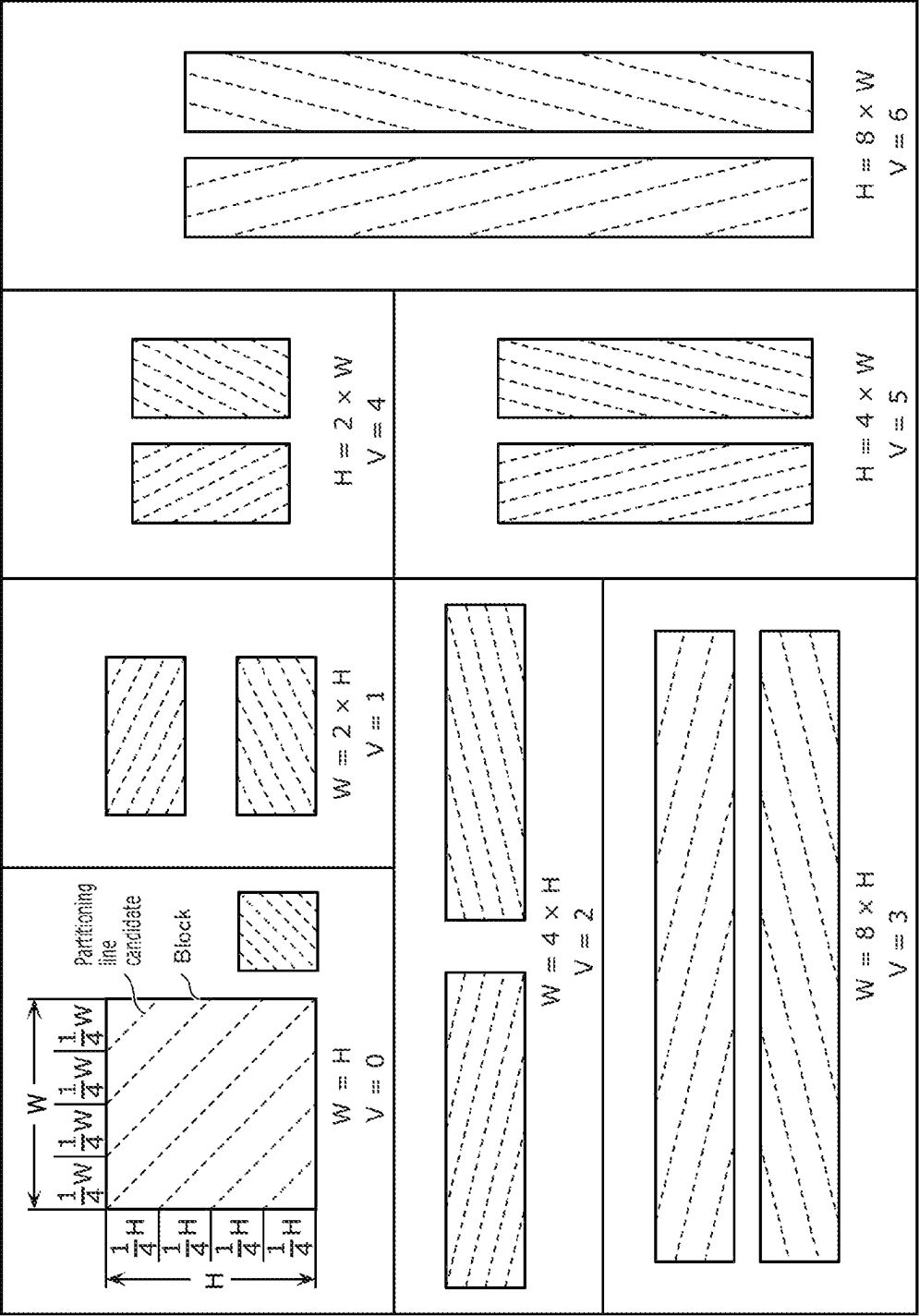


FIG. 102

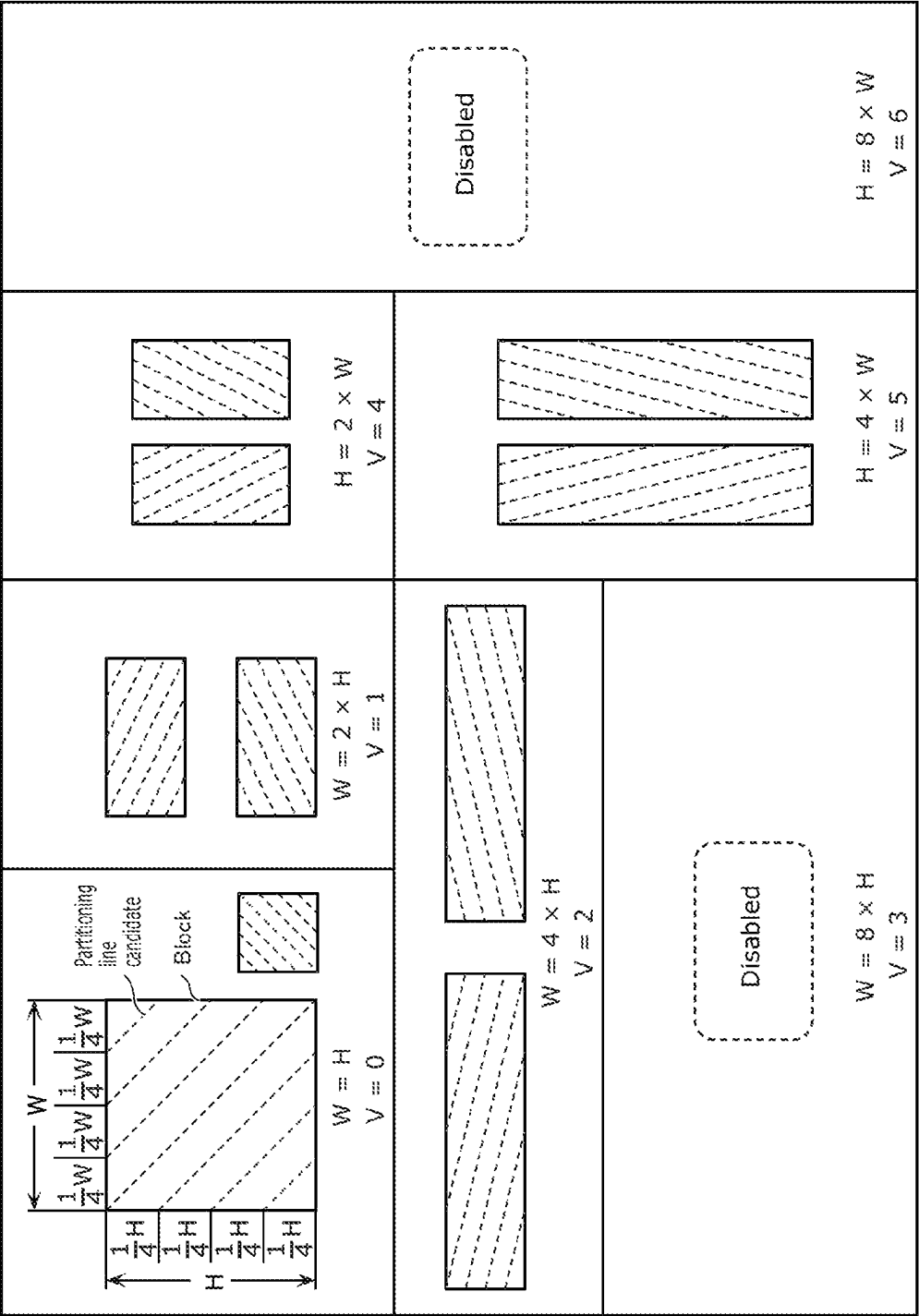


FIG. 103

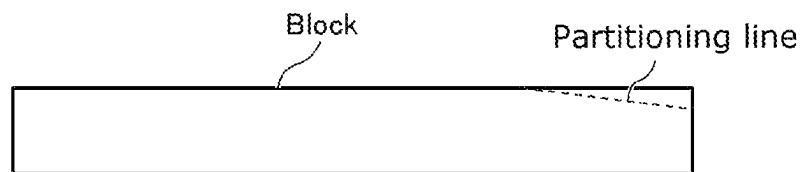


FIG. 104

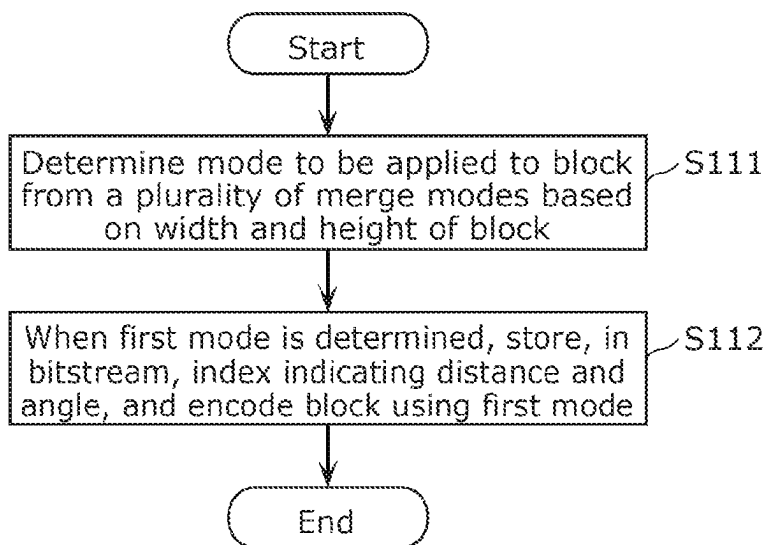


FIG. 105

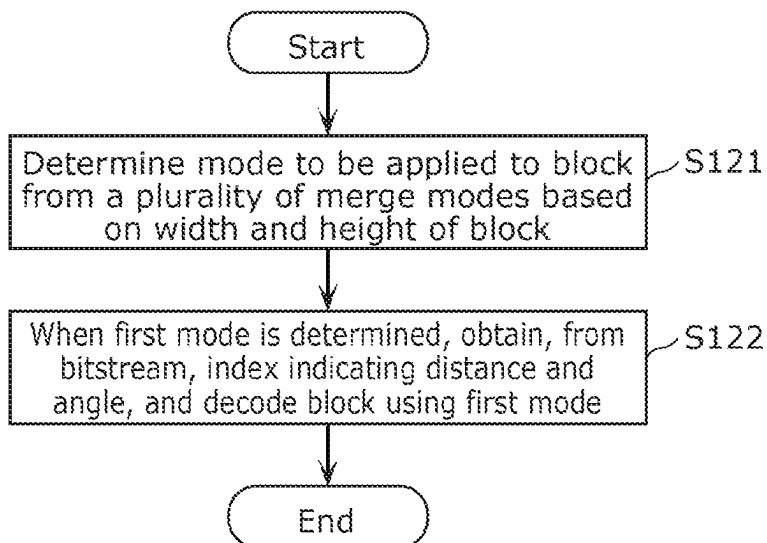


FIG. 106

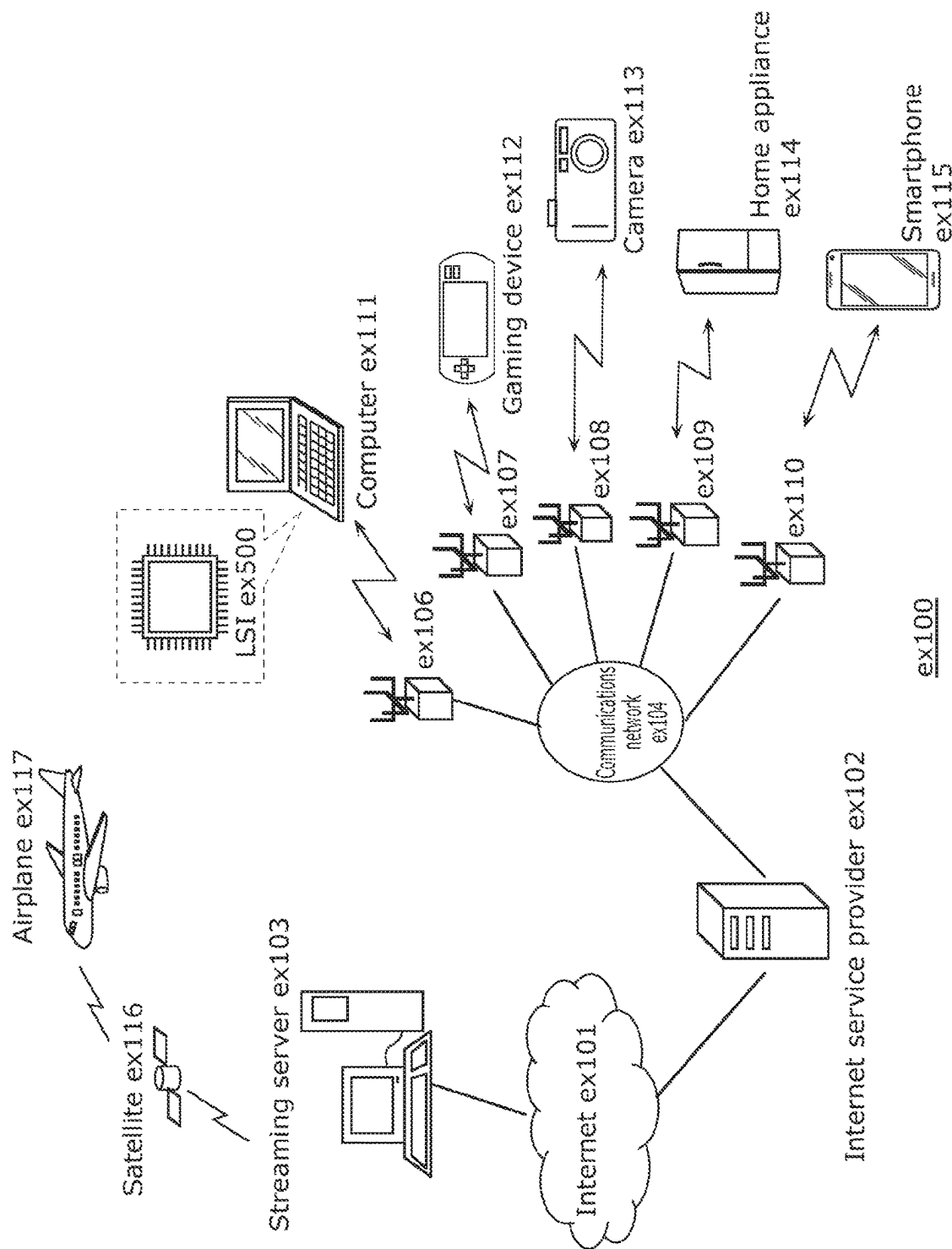


FIG. 107

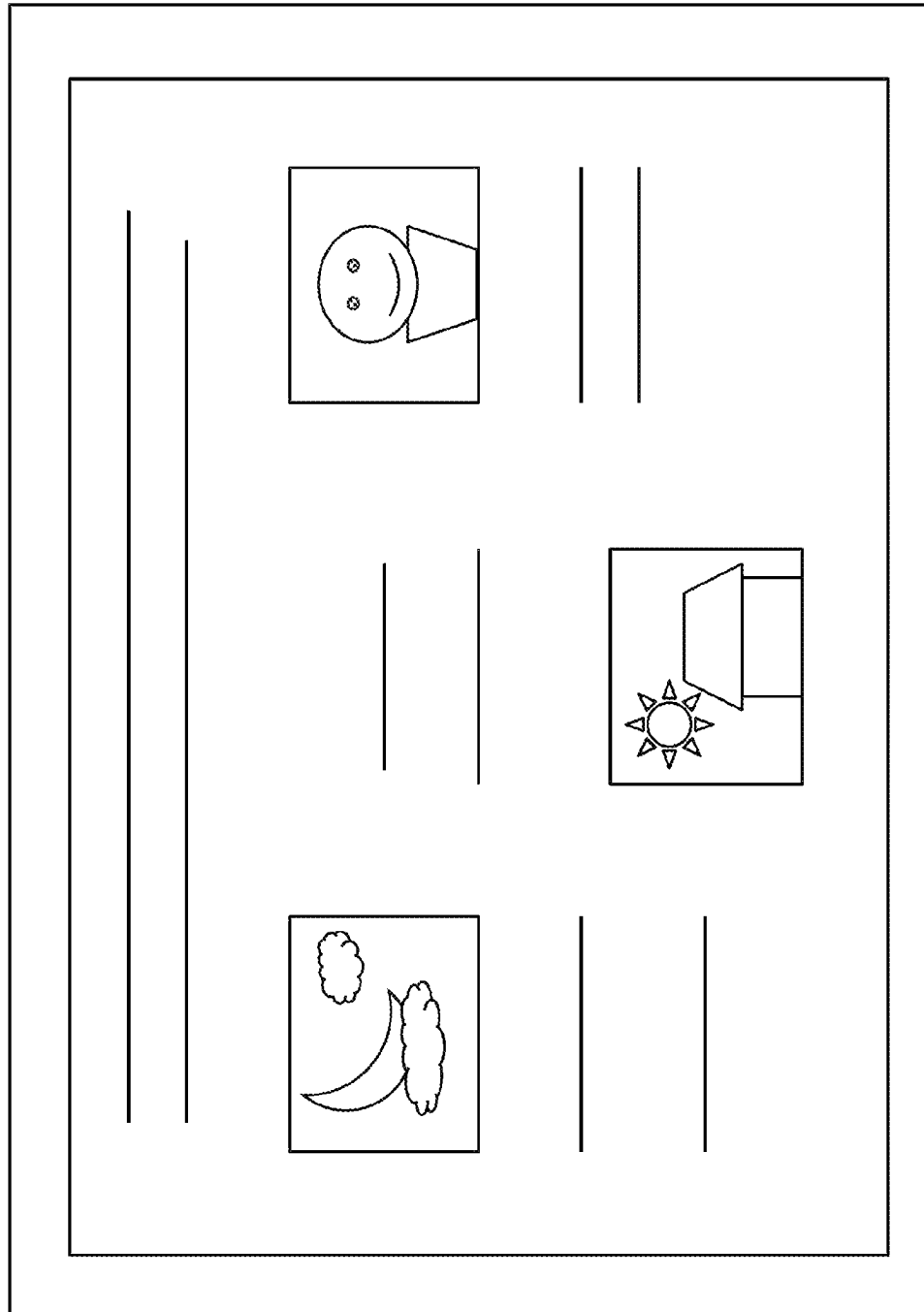


FIG. 108

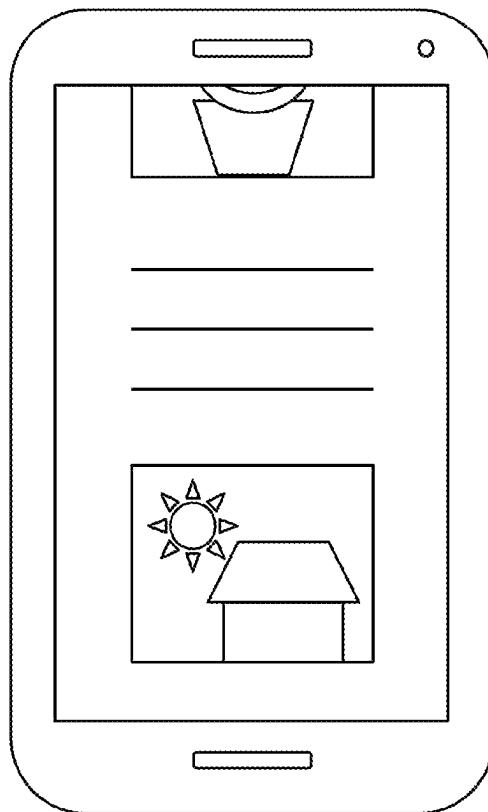


FIG. 109

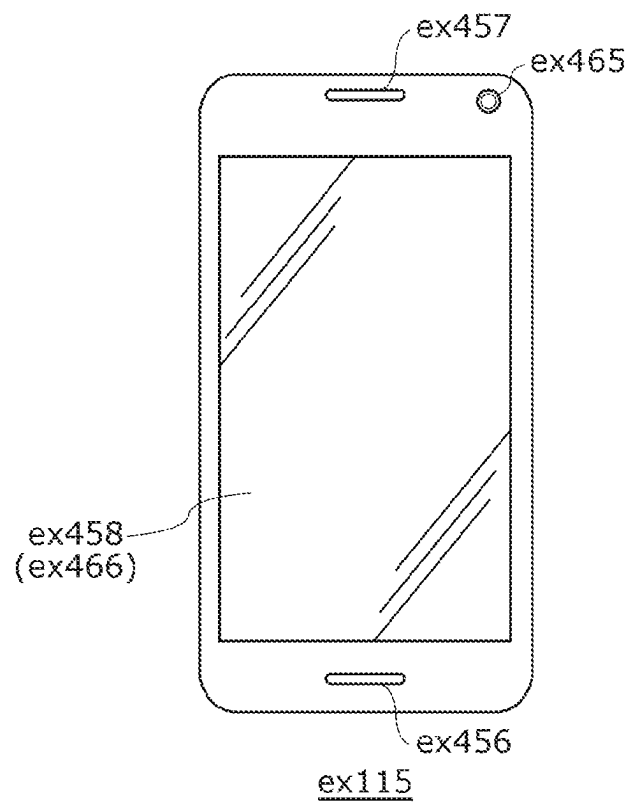
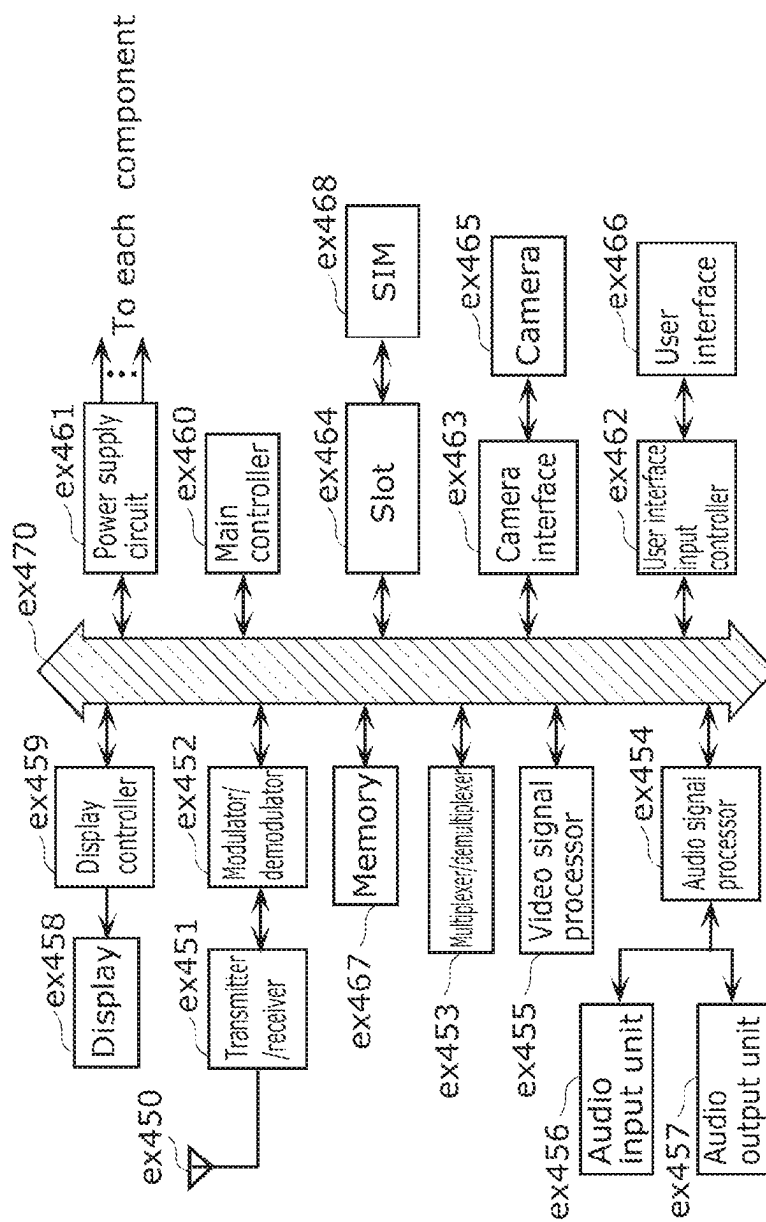


FIG. 110



1

ENCODING AND DECODING WITH MERGE MODE AND BLOCK PARTITION INDEX

CROSS REFERENCE TO RELATED APPLICATION

The present application is a continuation of U.S. application Ser. No. 17/727,299 filed on Apr. 22, 2022, which is a continuation of U.S. application Ser. No. 17/726,248 filed on Apr. 21, 2022, which is a continuation of U.S. application Ser. No. 17/725,084 filed on Apr. 20, 2022, which is a continuation of U.S. application Ser. No. 17/724,298 filed on Apr. 19, 2022, which is a continuation of U.S. application Ser. No. 17/723,324 filed on Apr. 18, 2022, which is a continuation of U.S. application Ser. No. 17/116,137 filed Dec. 9, 2020, which claims the benefit of U.S. Provisional Patent Application No. 62/947,185 filed Dec. 12, 2019. The entire disclosure of the above-identified application, including the specification, drawings and claims is incorporated herein by reference in its entirety.

FIELD

The present disclosure relates to video coding, and particularly to systems, constituent elements, and methods in video encoding and decoding.

BACKGROUND

With advancement in video coding technology, from H.261 and MPEG-1 to H.264/AVC (Advanced Video Coding), MPEG-LA, H.265/HEVC (High Efficiency Video Coding) and H.266/VVC (Versatile Video Codec), there remains a constant need to provide improvements and optimizations to the video coding technology to process an ever-increasing amount of digital video data in various applications. The present disclosure relates to further advancements, improvements and optimizations in video coding.

Note that Non-Patent Literature (NPL) 1 relates to one example of a conventional standard regarding the above-described video coding technology.

CITATION LIST

Non Patent Literature

NPL 1: H.265 (ISO/IEC 23008-2 HEVC)/HEVC (High Efficiency Video Coding)

SUMMARY

For example, an encoder according to an aspect of the present disclosure includes circuitry and memory connected to the circuitry, and the circuitry, in operation: determines a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, stores in a bitstream an index indicating a distance and an angle that define two partitions in the block, and encodes the block using the first mode; and disables storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

In video coding technology, new methods are desired to be proposed in order to improve coding efficiency, enhance image quality, and reduce circuit scale.

2

Each of embodiments, or each of part of constituent elements and methods in the present disclosure enables, for example, at least one of the following: improvement in coding efficiency, enhancement in image quality, reduction in processing amount of encoding/decoding, reduction in circuit scale, improvement in processing speed of encoding/decoding, etc. Alternatively, each of embodiments, or each of part of constituent elements and methods in the present disclosure enables, in encoding and decoding, appropriate selection of either an element such as a filter, a block, a size, a motion vector, a reference picture, and a reference block or an operation. It is to be noted that the present disclosure includes disclosure regarding configurations and methods which may provide advantages other than the above-described ones. Examples of such configurations and methods include a configuration or method for improving coding efficiency while reducing increase in processing amount.

Additional benefits and advantages according to an aspect of the present disclosure will become apparent from the specification and drawings. The benefits and/or advantages may be individually obtained by the various embodiments and features of the specification and drawings, and not all of which need to be provided in order to obtain one or more of such benefits and/or advantages.

It is to be noted that these general or specific aspects may be implemented using a system, an integrated circuit, a computer program, or a computer-readable recording medium such as a CD-ROM, or any combination of systems, methods, integrated circuits, computer programs, and recording media.

BRIEF DESCRIPTION OF DRAWINGS

These and other advantages and features will become apparent from the following description thereof taken in conjunction with the accompanying Drawings, by way of non-limiting examples of embodiments disclosed herein.

FIG. 1 is a schematic diagram illustrating one example of a configuration of a transmission system according to an embodiment.

FIG. 2 is a diagram illustrating one example of a hierarchical structure of data in a stream.

FIG. 3 is a diagram illustrating one example of a slice configuration.

FIG. 4 is a diagram illustrating one example of a tile configuration.

FIG. 5 is a diagram illustrating one example of an encoding structure in scalable encoding.

FIG. 6 is a diagram illustrating one example of an encoding structure in scalable encoding.

FIG. 7 is a block diagram illustrating one example of a functional configuration of an encoder according to an embodiment.

FIG. 8 is a block diagram illustrating a mounting example of the encoder.

FIG. 9 is a flow chart illustrating one example of an overall encoding process performed by the encoder.

FIG. 10 is a diagram illustrating one example of block splitting.

FIG. 11 is a diagram illustrating one example of a functional configuration of a splitter.

FIG. 12 is a diagram illustrating examples of splitting patterns.

FIG. 13A is a diagram illustrating one example of a syntax tree of a splitting pattern.

FIG. 13B is a diagram illustrating another example of a syntax tree of a splitting pattern.

3

FIG. 14 is a chart illustrating transform basis functions for each transform type.

FIG. 15 is a diagram illustrating examples of SVT.

FIG. 16 is a flow chart illustrating one example of a process performed by a transformer.

FIG. 17 is a flow chart illustrating another example of a process performed by the transformer.

FIG. 18 is a block diagram illustrating one example of a functional configuration of a quantizer.

FIG. 19 is a flow chart illustrating one example of quantization performed by the quantizer.

FIG. 20 is a block diagram illustrating one example of a functional configuration of an entropy encoder.

FIG. 21 is a diagram illustrating a flow of CABAC in the entropy encoder.

FIG. 22 is a block diagram illustrating one example of a functional configuration of a loop filter.

FIG. 23A is a diagram illustrating one example of a filter shape used in an adaptive loop filter (ALF).

FIG. 23B is a diagram illustrating another example of a filter shape used in an ALF.

FIG. 23C is a diagram illustrating another example of a filter shape used in an ALF.

FIG. 23D is a diagram illustrating an example where Y samples (first component) are used for a cross component ALF (CCALF) for Cb and a CCALF for Cr (components different from the first component).

FIG. 23E is a diagram illustrating a diamond shaped filter.

FIG. 23F is a diagram illustrating an example for a joint chroma CCALF (JC-CCALF).

FIG. 23G is a diagram illustrating an example for JC-CCALF weight index candidates.

FIG. 24 is a block diagram illustrating one example of a specific configuration of a loop filter which functions as a DBF.

FIG. 25 is a diagram illustrating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary.

FIG. 26 is a diagram for illustrating a block boundary on which a deblocking filter process is performed.

FIG. 27 is a diagram illustrating examples of Bs values.

FIG. 28 is a flow chart illustrating one example of a process performed by a predictor of the encoder.

FIG. 29 is a flow chart illustrating another example of a process performed by the predictor of the encoder.

FIG. 30 is a flow chart illustrating another example of a process performed by the predictor of the encoder.

FIG. 31 is a diagram illustrating one example of sixty-seven intra prediction modes used in intra prediction.

FIG. 32 is a flow chart illustrating one example of a process performed by an intra predictor.

FIG. 33 is a diagram illustrating examples of reference pictures.

FIG. 34 is a diagram illustrating examples of reference picture lists.

FIG. 35 is a flow chart illustrating a basic processing flow of inter prediction.

FIG. 36 is a flow chart illustrating one example of MV derivation.

FIG. 37 is a flow chart illustrating another example of MV derivation.

FIG. 38A is a diagram illustrating one example of categorization of modes for MV derivation.

FIG. 38B is a diagram illustrating one example of categorization of modes for MV derivation.

4

FIG. 39 is a flow chart illustrating an example of inter prediction by normal inter mode.

FIG. 40 is a flow chart illustrating an example of inter prediction by normal merge mode.

FIG. 41 is a diagram for illustrating one example of an MV derivation process by normal merge mode.

FIG. 42 is a diagram for illustrating one example of an MV derivation process by HMVP mode.

FIG. 43 is a flow chart illustrating one example of frame rate up conversion (FRUC).

FIG. 44 is a diagram for illustrating one example of pattern matching (bilateral matching) between two blocks located along a motion trajectory.

FIG. 45 is a diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture.

FIG. 46A is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which two control points are used.

FIG. 46B is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which three control points are used.

FIG. 47A is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode.

FIG. 47B is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode.

FIG. 47C is a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode.

FIG. 48A is a diagram for illustrating an affine mode in which two control points are used.

FIG. 48B is a diagram for illustrating an affine mode in which three control points are used.

FIG. 49A is a conceptual diagram for illustrating one example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other.

FIG. 49B is a conceptual diagram for illustrating another example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other.

FIG. 50 is a flow chart illustrating one example of the affine merge mode.

FIG. 51 is a flow chart illustrating one example of a process in affine inter mode.

FIG. 52A is a diagram for illustrating generation of two triangular prediction images.

FIG. 52B is a conceptual diagram illustrating examples of a first portion of a first partition and first and second sets of samples.

FIG. 52C is a conceptual diagram illustrating a first portion of a first partition.

FIG. 53 is a flow chart illustrating one example of a triangle mode.

FIG. 54 is a diagram illustrating one example of an ATMVP mode in which an MV is derived in units of a sub-block.

FIG. 55 is a diagram illustrating a relationship between a merge mode and dynamic motion vector refreshing (DMVR).

FIG. 56 is a conceptual diagram for illustrating one example of DMVR.

5

FIG. 57 is a conceptual diagram for illustrating another example of DMVR for determining an MV.

FIG. 58A is a diagram illustrating one example of motion estimation in DMVR.

FIG. 58B is a flow chart illustrating one example of motion estimation in DMVR.

FIG. 59 is a flow chart illustrating one example of generation of a prediction image.

FIG. 60 is a flow chart illustrating another example of generation of a prediction image.

FIG. 61 is a flow chart illustrating one example of a correction process of a prediction image by overlapped block motion compensation (OBMC).

FIG. 62 is a conceptual diagram for illustrating one example of a prediction image correction process by OBMC.

FIG. 63 is a diagram for illustrating a model assuming uniform linear motion.

FIG. 64 is a flow chart illustrating one example of inter prediction according to BIO.

FIG. 65 is a diagram illustrating one example of a functional configuration of an inter predictor which performs inter prediction according to BIO.

FIG. 66A is a diagram for illustrating one example of a prediction image generation method using a luminance correction process by local illumination compensation (LIC).

FIG. 66B is a flow chart illustrating one example of a prediction image generation method using a luminance correction process by LIC.

FIG. 67 is a block diagram illustrating a functional configuration of a decoder according to an embodiment.

FIG. 68 is a block diagram illustrating a mounting example of a decoder.

FIG. 69 is a flow chart illustrating one example of an overall decoding process performed by the decoder.

FIG. 70 is a diagram illustrating a relationship between a splitting determiner and other constituent elements.

FIG. 71 is a block diagram illustrating one example of a functional configuration of an entropy decoder.

FIG. 72 is a diagram illustrating a flow of CABAC in the entropy decoder.

FIG. 73 is a block diagram illustrating one example of a functional configuration of an inverse quantizer.

FIG. 74 is a flow chart illustrating one example of inverse quantization performed by the inverse quantizer.

FIG. 75 is a flow chart illustrating one example of a process performed by an inverse transformer.

FIG. 76 is a flow chart illustrating another example of a process performed by the inverse transformer.

FIG. 77 is a block diagram illustrating one example of a functional configuration of a loop filter.

FIG. 78 is a flow chart illustrating one example of a process performed by a predictor of the decoder.

FIG. 79 is a flow chart illustrating another example of a process performed by the predictor of the decoder.

FIG. 80A is a flow chart illustrating a portion of other example of a process performed by the predictor of the decoder.

FIG. 80B is a flow chart illustrating the remaining portion of the other example of the process performed by the predictor of the decoder.

FIG. 81 is a diagram illustrating one example of a process performed by an intra predictor of the decoder.

FIG. 82 is a flow chart illustrating one example of MV derivation in the decoder.

6

FIG. 83 is a flow chart illustrating another example of MV derivation in the decoder.

FIG. 84 is a flow chart illustrating an example of inter prediction by normal inter mode in the decoder.

FIG. 85 is a flow chart illustrating an example of inter prediction by normal merge mode in the decoder.

FIG. 86 is a flow chart illustrating an example of inter prediction by FRUC mode in the decoder.

FIG. 87 is a flow chart illustrating an example of inter prediction by affine merge mode in the decoder.

FIG. 88 is a flow chart illustrating an example of inter prediction by affine inter mode in the decoder.

FIG. 89 is a flow chart illustrating an example of inter prediction by triangle mode in the decoder.

FIG. 90 is a flow chart illustrating an example of motion estimation by DMVR in the decoder.

FIG. 91 is a flow chart illustrating one specific example of motion estimation by DMVR in the decoder.

FIG. 92 is a flow chart illustrating one example of generation of a prediction image in the decoder.

FIG. 93 is a flow chart illustrating another example of generation of a prediction image in the decoder.

FIG. 94 is a flow chart illustrating another example of correction of a prediction image by OBMC in the decoder.

FIG. 95 is a flow chart illustrating another example of correction of a prediction image by BIO in the decoder.

FIG. 96 is a flow chart illustrating another example of correction of a prediction image by LIC in the decoder.

FIG. 97 is a conceptual diagram illustrating a partitioning line used in a wedge mode.

FIG. 98 is a flow chart illustrating processing of encoding a block using the wedge mode.

FIG. 99 is a conceptual diagram illustrating an example of a plurality of partitioning line candidate sets.

FIG. 100 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 1.

FIG. 101 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 2.

FIG. 102 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 3.

FIG. 103 is a conceptual diagram illustrating an example of a partitioning line when the width-to-height ratio is high.

FIG. 104 is a flow chart illustrating operations of an encoder according to an embodiment.

FIG. 105 is a flow chart illustrating operations of a decoder according to the embodiment.

FIG. 106 is a diagram illustrating an overall configuration of a content providing system for implementing a content distribution service.

FIG. 107 is a diagram illustrating an example of a display screen of a web page.

FIG. 108 is a diagram illustrating an example of a display screen of a web page.

FIG. 109 is a diagram illustrating one example of a smartphone.

FIG. 110 is a diagram illustrating an example of a smartphone.

DESCRIPTION OF EMBODIMENTS

[Introduction]

For example, to encode a video, a prediction image of a block in a picture included in the video is generated, and a difference image between the prediction image and the original image of the block is encoded. To decode a video, a difference image is decoded, a prediction image is generated, and the prediction image and the difference image are

added to generate a reconstructed image of a block. This way, the code amount is reduced.

A prediction image of a block may be generated by splitting the block into two partitions and applying two different prediction processes to the two partitions. For example, a partitioning line may be determined, and the block may be split into two partitions along the partitioning line. The partitioning line can be flexibly determined by a distance from the center of the block and an angle corresponding to the partitioning line. This makes it possible to generate various prediction images.

However, there are cases where it is inappropriate to define two partitions by a distance and an angle. By eliminating such an operation, there is a possibility of reducing useless processing and enhancing the processing efficiency.

In view of the above, an encoder according to an aspect of the present disclosure is an encoder including circuitry and memory connected to the circuitry. In the encoder, the circuitry, in operation: determines a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, stores in a bitstream an index indicating a distance and an angle that define two partitions in the block, and encodes the block using the first mode; and disables storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the storing of the index indicating a distance and an angle in the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, the circuitry disables the storing of the index in the bitstream when (i) the width is 64 pixels and the height is 8 pixels or (ii) the width is 8 pixels and the height is 64 pixels.

By doing so, there is a possibility that the storing of the index indicating a distance and an angle in the bitstream is disabled when the width-to-height ratio or the height-to-width ratio is high, even if the block size is within an appropriate range. This way, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, a line that determines shapes of the two partitions in the block is defined by the distance and the angle, the distance is a distance between a center of the block and the line, and the angle is an angle related to the line.

This way, there is a possibility of appropriately defining the line for determining the shapes of two partitions.

For example, a decoder according to an aspect of the present disclosure is a decoder including circuitry and memory connected to the circuitry. In the decoder, the circuitry, in operation: determines a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, obtains from a bitstream an index indicating a distance and an angle that define two partitions in the block, and decodes the block using the first mode; and disables obtaining of the index from the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width

ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the obtaining of the index indicating a distance and an angle from the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, the circuitry disables the obtaining of the index from the bitstream when (i) the width is 64 pixels and the height is 8 pixels or (ii) the width is 8 pixels and the height is 64 pixels.

By doing so, there is a possibility that the obtaining of the index indicating a distance and an angle from the bitstream is disabled when the width-to-height ratio or the height-to-width ratio is high, even if the block size is within an appropriate range. This way, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, a line that determines shapes of the two partitions in the block is defined by the distance and the angle, the distance is a distance between a center of the block and the line, and the angle is an angle related to the line.

This way, there is a possibility of appropriately defining the line that determines the shapes of two partitions.

For example, an encoding method according to an aspect of the present disclosure is an encoding method including: determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, storing in a bitstream an index indicating a distance and an angle that define two partitions in the block, and encoding the block using the first mode; and disabling the storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the storing of the index indicating a distance and an angle in the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, a decoding method according to an aspect of the present disclosure is a decoding method including: determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, obtaining from a bitstream an index indicating a distance and an angle that define two partitions in the block, and decoding the block using the first mode; and disabling the obtaining of the index from the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the obtaining of the index indicating a distance and an angle from the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

For example, an encoder according to an aspect of the present disclosure includes an input terminal, a splitter, an intra predictor, an inter predictor, a loop filter, a transformer, a quantizer, an entropy encoder, and an output terminal.

A current picture is input to the input terminal. The splitter splits the current picture into a plurality of blocks.

The intra predictor generates a prediction signal of a current block included in the current picture, using a reference image included in the current picture. The inter predictor generates a prediction signal of a current block included in the current picture, using a reference image included in a reference picture different from the current picture. The loop filter applies a filter to a reconstructed block of a current block included in the current picture.

The transformer generates transform coefficients by transforming a prediction error between the original signal of a current block included in the current picture and the prediction signal generated by the intra predictor or the inter predictor. The quantizer generates quantized coefficients by quantizing the transform coefficients. The entropy encoder generates an encoded bitstream by applying variable-length encoding to the quantized coefficients. Then, the encoded bitstream that includes control information and the quantized coefficients to which the variable-length encoding has been applied is output from the output terminal.

For example, the inter predictor, in operation: determines a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; and when the mode determined is a first mode, stores in a bitstream an index indicating a distance and an angle that define two partitions in the block, and encodes the block using the first mode. Furthermore, the inter predictor disables storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

For example, a decoder according to an aspect of the present disclosure includes an input terminal, an entropy decoder, an inverse quantizer, an inverse transformer, an intra predictor, an inter predictor, a loop filter, and an output terminal.

An encoded bitstream is input to the input terminal. The entropy decoder derives quantized coefficients by applying variable-length decoding to the encoded bitstream. The inverse quantizer derives transform coefficients by inverse quantizing the quantized coefficients. The inverse transformer derives a prediction error by inverse transforming the transform coefficients.

The intra predictor generates a prediction signal of a current block included in a current picture, using a reference image included in the current picture. The inter predictor generates a prediction signal of a current block included in the current picture, using a reference image included in a reference picture different from the current picture.

The loop filter applies a filter to a reconstructed block of the current block included in the current picture. Then, the current picture is output from the output terminal.

For example, the inter predictor, in operation: determines a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; and when the mode determined is a first mode, obtains from a bitstream an index indicating a distance and an angle that define two partitions in the block, and decodes the block using the first mode. Furthermore, the inter predictor disables obtaining of the index from the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

In addition, these general or specific aspects may be implemented using a system, a device, a method, an integrated circuit, a computer program, or a non-transitory computer-readable recording medium such as a CD-ROM,

or any combination of systems, devices, methods, integrated circuits, computer programs, and recording media.

Definitions of Terms

The respective terms may be defined as indicated below as examples.

(1) Image

An image is a data unit configured with a set of pixels, is a picture or includes blocks smaller than a picture. Images include a still image in addition to a video.

(2) Picture

A picture is an image processing unit configured with a set of pixels, and is also referred to as a frame or a field.

(3) Block

A block is a processing unit which is a set of a particular number of pixels. The block is also referred to as indicated in the following examples. The shapes of blocks are not limited.

Examples include a rectangle shape of M×N pixels and a square shape of M×M pixels for the first place, and also include a triangular shape, a circular shape, and other shapes.

(Examples of Blocks)

slice/tile/brick

CTU/super block/basic splitting unit

VPDU/processing splitting unit for hardware

CU/processing block unit/prediction block unit (PU)/

orthogonal transform block unit (TU)/unit

sub-block

(4) Pixel/Sample

A pixel or sample is a smallest point of an image. Pixels or samples include not only a pixel at an integer position but also a pixel at a sub-pixel position generated based on a pixel at an integer position.

(5) Pixel Value/Sample Value

A pixel value or sample value is an eigen value of a pixel. Pixel or sample values naturally include a luma value, a chroma value, an RGB gradation level and also covers a depth value, or a binary value of 0 or 1.

(6) Flag

A flag indicates one or more bits, and may be, for example, a parameter or index represented by two or more bits. Alternatively, the flag may indicate not only a binary value represented by a binary number but also a multiple value represented by a number other than the binary number.

(7) Signal

A signal is the one symbolized or encoded to convey information. Signals include a discrete digital signal and an analog signal which takes a continuous value.

(8) Stream/Bitstream

A stream or bitstream is a digital data string or a digital data flow. A stream or bitstream may be one stream or may be configured with a plurality of streams having a plurality of hierarchical layers. A stream or bitstream may be transmitted in serial communication using a single transmission path, or may be transmitted in packet communication using a plurality of transmission paths.

(9) Difference

In the case of scalar quantity, it is only necessary that a simple difference (x-y) and a difference calculation be included. Differences include an absolute value of a difference (|x-y|), a squared difference (x²-y²), a square root of a difference (√(x-y)), a weighted difference (ax-by: a and b are constants), an offset difference (x-y+a: a is an offset).

11

(10) Sum

In the case of scalar quantity, it is only necessary that a simple sum ($x+y$) and a sum calculation be included. Sums include an absolute value of a sum ($|x+y|$), a squared sum (x^2+y^2), a square root of a sum ($\sqrt{x+y}$), a weighted difference ($ax+by$: a and b are constants), an offset sum ($x+y+a$: a is an offset).

(11) Based on

A phrase “based on something” means that a thing other than the something may be considered. In addition, “based on” may be used in a case in which a direct result is obtained or a case in which a result is obtained through an intermediate result.

(12) Used, Using

A phrase “something used” or “using something” means that a thing other than the something may be considered. In addition, “used” or “using” may be used in a case in which a direct result is obtained or a case in which a result is obtained through an intermediate result.

(13) Prohibit, Forbid

The term “prohibit” or “forbid” can be rephrased as “does not permit” or “does not allow”. In addition, “being not prohibited/forbidden” or “being permitted/allowed” does not always mean “obligation”.

(14) Limit, Restriction/Restrict/Restricted

The term “limit” or “restriction/restrict/restricted” can be rephrased as “does not permit/allow” or “being not permitted/allowed”. In addition, “being not prohibited/forbidden” or “being permitted/allowed” does not always mean “obligation”. Furthermore, it is only necessary that part of something be prohibited/forbidden quantitatively or qualitatively, and something may be fully prohibited/forbidden.

(15) Chroma

An adjective, represented by the symbols C_b and C_r , specifying that a sample array or single sample is representing one of the two color difference signals related to the primary colors. The term chroma may be used instead of the term chrominance.

(16) Luma

An adjective, represented by the symbol or subscript Y or L , specifying that a sample array or single sample is representing the monochrome signal related to the primary colors. The term luma may be used instead of the term luminance.

[Notes Related to the Descriptions]

In the drawings, same reference numbers indicate same or similar components. The sizes and relative locations of components are not necessarily drawn by the same scale.

Hereinafter, embodiments will be described with reference to the drawings. Note that the embodiments described below each show a general or specific example. The numerical values, shapes, materials, components, the arrangement and connection of the components, steps, the relation and order of the steps, etc., indicated in the following embodiments are mere examples, and are not intended to limit the scope of the claims.

Embodiments of an encoder and a decoder will be described below. The embodiments are examples of an encoder and a decoder to which the processes and/or configurations presented in the description of aspects of the present disclosure are applicable. The processes and/or configurations can also be implemented in an encoder and a decoder different from those according to the embodiments. For example, regarding the processes and/or configurations as applied to the embodiments, any of the following may be implemented:

12

(1) Any of the components of the encoder or the decoder according to the embodiments presented in the description of aspects of the present disclosure may be substituted or combined with another component presented anywhere in the description of aspects of the present disclosure.

(2) In the encoder or the decoder according to the embodiments, discretionary changes may be made to functions or processes performed by one or more components of the encoder or the decoder, such as addition, substitution, removal, etc., of the functions or processes. For example, any function or process may be substituted or combined with another function or process presented anywhere in the description of aspects of the present disclosure.

(3) In methods implemented by the encoder or the decoder according to the embodiments, discretionary changes may be made such as addition, substitution, and removal of one or more of the processes included in the method. For example, any process in the method may be substituted or combined with another process presented anywhere in the description of aspects of the present disclosure.

(4) One or more components included in the encoder or the decoder according to embodiments may be combined with a component presented anywhere in the description of aspects of the present disclosure, may be combined with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, and may be combined with a component that implements one or more processes implemented by a component presented in the description of aspects of the present disclosure.

(5) A component including one or more functions of the encoder or the decoder according to the embodiments, or a component that implements one or more processes of the encoder or the decoder according to the embodiments, may be combined or substituted with a component presented anywhere in the description of aspects of the present disclosure, with a component including one or more functions presented anywhere in the description of aspects of the present disclosure, or with a component that implements one or more processes presented anywhere in the description of aspects of the present disclosure.

(6) In methods implemented by the encoder or the decoder according to the embodiments, any of the processes included in the method may be substituted or combined with a process presented anywhere in the description of aspects of the present disclosure or with any corresponding or equivalent process.

(7) One or more processes included in methods implemented by the encoder or the decoder according to the embodiments may be combined with a process presented anywhere in the description of aspects of the present disclosure.

(8) The implementation of the processes and/or configurations presented in the description of aspects of the present disclosure is not limited to the encoder or the decoder according to the embodiments. For example, the processes and/or configurations may be implemented in a device used for a purpose different from the moving picture encoder or the moving picture decoder disclosed in the embodiments.

[System Configuration]

FIG. 1 is a schematic diagram illustrating one example of a configuration of a transmission system according to an embodiment.

13

Transmission system Trs is a system which transmits a stream generated by encoding an image and decodes the transmitted stream. Transmission system Trs like this includes, for example, encoder 100, network Nw, and decoder 200 as illustrated in FIG. 1.

An image is input to encoder 100. Encoder 100 generates a stream by encoding the input image, and outputs the stream to network Nw. The stream includes, for example, the encoded image and control information for decoding the encoded image. The image is compressed by the encoding.

It is to be noted that a previous image before being encoded and being input to encoder 100 is also referred to as the original image, the original signal, or the original sample. The image may be a video or a still image. The image is a generic concept of a sequence, a picture, and a block, and thus is not limited to a spatial region having a particular size and to a temporal region having a particular size unless otherwise specified. The image is an array of pixels or pixel values, and the signal representing the image or pixel values are also referred to as samples. The stream may be referred to as a bitstream, an encoded bitstream, a compressed bitstream, or an encoded signal. Furthermore, the encoder may be referred to as an image encoder or a video encoder. The encoding method performed by encoder 100 may be referred to as an encoding method, an image encoding method, or a video encoding method.

Network Nw transmits the stream generated by encoder 100 to decoder 200. Network Nw may be the Internet, the Wide Area Network (WAN), the Local Area Network (LAN), or any combination of these networks. Network Nw is not always limited to a bi-directional communication network, and may be a uni-directional communication network which transmits broadcast waves of digital terrestrial broadcasting, satellite broadcasting, or the like. Alternatively, network Nw may be replaced by a recording medium such as a Digital Versatile Disc (DVD) and a Blu-Ray Disc (BD)®, etc. on which a stream is recorded.

Decoder 200 generates, for example, a decoded image which is an uncompressed image by decoding a stream transmitted by network Nw. For example, the decoder decodes a stream according to a decoding method corresponding to an encoding method by encoder 100.

It is to be noted that the decoder may also be referred to as an image decoder or a video decoder, and that the decoding method performed by decoder 200 may also be referred to as a decoding method, an image decoding method, or a video decoding method.

[Data Structure]

FIG. 2 is a diagram illustrating one example of a hierarchical structure of data in a stream. A stream includes, for example, a video sequence. As illustrated in (a) of FIG. 2, the video sequence includes a video parameter set (VPS), a sequence parameter set (SPS), a picture parameter set (PPS), supplemental enhancement information (SEI), and a plurality of pictures.

In a video having a plurality of layers, a VPS includes: a coding parameter which is common between some of the plurality of layers; and a coding parameter related to some of the plurality of layers included in the video or an individual layer.

An SPS includes a parameter which is used for a sequence, that is, a coding parameter which decoder 200 refers to in order to decode the sequence. For example, the coding parameter may indicate the width or height of a picture. It is to be noted that a plurality of SPSs may be present.

14

A PPS includes a parameter which is used for a picture, that is, a coding parameter which decoder 200 refers to in order to decode each of the pictures in the sequence. For example, the coding parameter may include a reference value for the quantization width which is used to decode a picture and a flag indicating application of weighted prediction. It is to be noted that a plurality of PPSs may be present. Each of the SPS and the PPS may be simply referred to as a parameter set.

As illustrated in (b) of FIG. 2, a picture may include a picture header and at least one slice. A picture header includes a coding parameter which decoder 200 refers to in order to decode the at least one slice.

As illustrated in (c) of FIG. 2, a slice includes a slice header and at least one brick. A slice header includes a coding parameter which decoder 200 refers to in order to decode the at least one brick.

As illustrated in (d) of FIG. 2, a brick includes at least one coding tree unit (CTU).

It is to be noted that a picture may not include any slice and may include a tile group instead of a slice. In this case, the tile group includes at least one tile. In addition, a brick may include a slice.

A CTU is also referred to as a super block or a basis splitting unit. As illustrated in (e) of FIG. 2, a CTU like this includes a CTU header and at least one coding unit (CU). A CTU header includes a coding parameter which decoder 200 refers to in order to decode the at least one CU.

A CU may be split into a plurality of smaller CUs. As illustrated in (f) of FIG. 2, a CU includes a CU header, prediction information, and residual coefficient information. Prediction information is information for predicting the CU, and the residual coefficient information is information indicating a prediction residual to be described later. Although a CU is basically the same as a prediction unit (PU) and a transform unit (TU), it is to be noted that, for example, an SBT to be described later may include a plurality of TUs smaller than the CU. In addition, the CU may be processed for each virtual pipeline decoding unit (VPDU) included in the CU. The VPDU is, for example, a fixed unit which can be processed at one stage when pipeline processing is performed in hardware.

It is to be noted that a stream may not include part of the hierarchical layers illustrated in FIG. 2. The order of the hierarchical layers may be exchanged, or any of the hierarchical layers may be replaced by another hierarchical layer. Here, a picture which is a target for a process which is about to be performed by a device such as encoder 100 or decoder 200 is referred to as a current picture. A current picture means a current picture to be encoded when the process is an encoding process, and a current picture means a current picture to be decoded when the process is a decoding process. Likewise, for example, a CU or a block of CUs which is a target for a process which is about to be performed by a device such as encoder 100 or decoder 200 is referred to as a current block. A current block means a current block to be encoded when the process is an encoding process, and a current block means a current block to be decoded when the process is a decoding process.

[Picture Structure: Slice/Tile]

A picture may be configured with one or more slice units or tile units in order to decode the picture in parallel.

Slices are basic encoding units included in a picture. A picture may include, for example, one or more slices. In addition, a slice includes one or more successive coding tree units (CTUs).

15

FIG. 3 is a diagram illustrating one example of a slice configuration. For example, a picture includes 11×8 CTUs, and is split into four slices (slices 1 to 4). Slice 1 includes sixteen CTUs, slice 2 includes twenty-one CTUs, slice 3 includes twenty-nine CTUs, and slice 4 includes twenty-two CTUs. Here, each CTU in the picture belongs to one of the slices. The shape of each slice is a shape obtained by splitting the picture horizontally. A boundary of each slice does not need to coincide with an image end, and may coincide with any of the boundaries between CTUs in the image. The processing order of the CTUs in a slice (an encoding order or a decoding order) is, for example, a raster-scan order. A slice includes a slice header and encoded data. Features of the slice may be written in the slice header. The features include a CTU address of a top CTU in the slice, a slice type, etc.

A tile is a unit of a rectangular region included in a picture. Each of tiles may be assigned with a number referred to as TileId in raster-scan order.

FIG. 4 is a diagram illustrating one example of a tile configuration. For example, a picture includes 11×8 CTUs, and is split into four tiles of rectangular regions (tiles 1 to 4). When tiles are used, the processing order of CTUs is changed from the processing order in the case where no tile is used. When no tile is used, a plurality of CTUs in a picture are processed in raster-scan order. When a plurality of tiles are used, at least one CTU in each of the plurality of tiles is processed in raster-scan order. For example, as illustrated in FIG. 4, the processing order of the CTUs included in tile 1 is the order which starts from the left-end of the first column of tile 1 toward the right-end of the first column of tile 1 and then starts from the left-end of the second column of tile 1 toward the right-end of the second column of tile 1.

It is to be noted that one tile may include one or more slices, and one slice may include one or more tiles.

It is to be noted that a picture may be configured with one or more tile sets. A tile set may include one or more tile groups, or one or more tiles. A picture may be configured with only one of a tile set, a tile group, and a tile. For example, an order for scanning a plurality of tiles for each tile set in raster scan order is assumed to be a basic encoding order of tiles. A set of one or more tiles which are continuous in the basic encoding order in each tile set is assumed to be a tile group. Such a picture may be configured by splitter 102 (see FIG. 7) to be described later.

[Scalable Encoding]

FIGS. 5 and 6 are diagrams illustrating examples of scalable stream structures.

As illustrated in FIG. 5, encoder 100 may generate a temporally/spatially scalable stream by dividing each of a plurality of pictures into any of a plurality of layers and encoding the picture in the layer. For example, encoder 100 encodes the picture for each layer, thereby achieving scalability where an enhancement layer is present above a base layer. Such encoding of each picture is also referred to as scalable encoding. In this way, decoder 200 is capable of switching image quality of an image which is displayed by decoding the stream. In other words, decoder 200 determines up to which layer to decode based on internal factors such as the processing ability of decoder 200 and external factors such as a state of a communication bandwidth. As a result, decoder 200 is capable of decoding a content while freely switching between low resolution and high resolution. For example, the user of the stream watches a video of the stream halfway using a smartphone on the way to home, and continues watching the video at home on a device such as a TV connected to the Internet. It is to be noted that each of

16

the smartphone and the device described above includes decoder 200 having the same or different performances. In this case, when the device decodes layers up to the higher layer in the stream, the user can watch the video at high quality at home. In this way, encoder 100 does not need to generate a plurality of streams having different image qualities of the same content, and thus the processing load can be reduced.

Furthermore, the enhancement layer may include meta information based on statistical information on the image. Decoder 200 may generate a video whose image quality has been enhanced by performing super-resolution imaging on a picture in the base layer based on the metadata. Super-resolution imaging may be any of improvement in the SN ratio in the same resolution and increase in resolution. Metadata may include information for identifying a linear or a non-linear filter coefficient, as used in a super-resolution process, or information identifying a parameter value in a filter process, machine learning, or a least squares method used in super-resolution processing.

Alternatively, a configuration may be provided in which a picture is divided into, for example, tiles in accordance with, for example, the meaning of an object in the picture. In this case, decoder 200 may decode only a partial region in a picture by selecting a tile to be decoded. In addition, an attribute of the object (person, car, ball, etc.) and a position of the object in the picture (coordinates in identical images) may be stored as metadata. In this case, decoder 200 is capable of identifying the position of a desired object based on the metadata, and determining the tile including the object. For example, as illustrated in FIG. 6, the metadata may be stored using a data storage structure different from image data, such as SEI in HEVC. This metadata indicates, for example, the position, size, or color of a main object.

Metadata may be stored in units of a plurality of pictures, such as a stream, a sequence, or a random access unit. In this way, decoder 200 is capable of obtaining, for example, the time at which a specific person appears in the video, and by fitting the time information with picture unit information, is capable of identifying a picture in which the object is present and determining the position of the object in the picture.

[Encoder]

Next, encoder 100 according to this embodiment is described. FIG. 7 is a block diagram illustrating one example of a functional configuration of encoder 100 according to this embodiment. Encoder 100 encodes an image in units of a block.

As illustrated in FIG. 7, encoder 100 is an apparatus which encodes an image in units of a block, and includes splitter 102, subtractor 104, transformer 106, quantizer 108, entropy encoder 110, inverse quantizer 112, inverse transformer 114, adder 116, block memory 118, loop filter 120, frame memory 122, intra predictor 124, inter predictor 126, prediction controller 128, and prediction parameter generator 130. It is to be noted that intra predictor 124 and inter predictor 126 are configured as part of a prediction executor. [Mounting Example of Encoder]

FIG. 8 is a block diagram illustrating a mounting example of encoder 100. Encoder 100 includes processor a1 and memory a2. For example, the plurality of constituent elements of encoder 100 illustrated in FIG. 7 are mounted on processor a1 and memory a2 illustrated in FIG. 8.

Processor a1 is circuitry which performs information processing and is accessible to memory a2. For example, processor a1 is dedicated or general electronic circuitry which encodes an image. Processor a1 may be a processor such as a CPU. In addition, processor a1 may be an

17

aggregate of a plurality of electronic circuits. In addition, for example, processor a1 may take the roles of two or more constituent elements other than a constituent element for storing information out of the plurality of constituent elements of encoder 100 illustrated in FIG. 7, etc.

Memory a2 is dedicated or general memory for storing information that is used by processor a1 to encode the image. Memory a2 may be electronic circuitry, and may be connected to processor a1. In addition, memory a2 may be included in processor a1. In addition, memory a2 may be an aggregate of a plurality of electronic circuits. In addition, memory a2 may be a magnetic disc, an optical disc, or the like, or may be represented as storage, a recording medium, or the like. In addition, memory a2 may be non-volatile memory, or volatile memory.

For example, memory a2 may store an image to be encoded or a stream corresponding to an encoded image. In addition, memory a2 may store a program for causing processor a1 to encode an image.

In addition, for example, memory a2 may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of encoder 100 illustrated in FIG. 7. More specifically, memory a2 may take the roles of block memory 118 and frame memory 122 illustrated in FIG. 7. More specifically, memory a2 may store a reconstructed image (specifically, a reconstructed block, a reconstructed picture, or the like).

It is to be noted that, in encoder 100, not all of the plurality of constituent elements indicated in FIG. 7, etc. may be implemented, and not all the processes described above may be performed. Part of the constituent elements indicated in FIG. 7 may be included in another device, or part of the processes described above may be performed by another device.

Hereinafter, an overall flow of processes performed by encoder 100 is described, and then each of constituent elements included in encoder 100 is described.
[Overall Flow of Encoding Process]

FIG. 9 is a flow chart illustrating one example of an overall encoding process performed by encoder 100.

First, splitter 102 of encoder 100 splits each of pictures included in an original image into a plurality of blocks having a fixed size (128×128 pixels) (Step Sa_1). Splitter 102 then selects a splitting pattern for the fixed-size block (Step Sa_2). In other words, splitter 102 further splits the fixed-size block into a plurality of blocks which form the selected splitting pattern. Encoder 100 performs, for each of the plurality of blocks, Steps Sa_3 to Sa_9 for the block.

Prediction controller 128 and a prediction executor which is configured with intra predictor 124 and inter predictor 126 generate a prediction image of a current block (Step Sa_3). It is to be noted that the prediction image is also referred to as a prediction signal, a prediction block, or prediction samples.

Next, subtractor 104 generates the difference between a current block and a prediction image as a prediction residual (Step Sa_4). It is to be noted that the prediction residual is also referred to as a prediction error.

Next, transformer 106 transforms the prediction image and quantizer 108 quantizes the result, to generate a plurality of quantized coefficients (Step Sa_5).

Next, entropy encoder 110 encodes (specifically, entropy encodes) the plurality of quantized coefficients and a prediction parameter related to generation of a prediction image to generate a stream (Step Sa_6).

Next, inverse quantizer 112 performs inverse quantization of the plurality of quantized coefficients and inverse trans-

18

former 114 performs inverse transform of the result, to restore a prediction residual (Step Sa_7).

Next, adder 116 adds the prediction image to the restored prediction residual to reconstruct the current block (Step Sa_8). In this way, the reconstructed image is generated. It is to be noted that the reconstructed image is also referred to as a reconstructed block, and, in particular, that a reconstructed image generated by encoder 100 is also referred to as a local decoded block or a local decoded image.

When the reconstructed image is generated, loop filter 120 performs filtering of the reconstructed image as necessary (Step Sa_9).

Encoder 100 then determines whether encoding of the entire picture has been finished (Step Sa_10). When determining that the encoding has not yet been finished (No in Step Sa_10), processes from Step Sa_2 are executed repeatedly.

Although encoder 100 selects one splitting pattern for a fixed-size block, and encodes each block according to the splitting pattern in the above-described example, it is to be noted that each block may be encoded according to a corresponding one of a plurality of splitting patterns. In this case, encoder 100 may evaluate a cost for each of the plurality of splitting patterns, and, for example, may select the stream obtained by encoding according to the splitting pattern which yields the smallest cost as a stream which is output finally.

Alternatively, the processes in Steps Sa_1 to Sa_10 may be performed sequentially by encoder 100, or two or more of the processes may be performed in parallel or may be reordered.

The encoding process by encoder 100 is hybrid encoding using prediction encoding and transform encoding. In addition, prediction encoding is performed by an encoding loop configured with subtractor 104, transformer 106, quantizer 108, inverse quantizer 112, inverse transformer 114, adder 116, loop filter 120, block memory 118, frame memory 122, intra predictor 124, inter predictor 126, and prediction controller 128. In other words, the prediction executor configured with intra predictor 124 and inter predictor 126 is part of the encoding loop.

[Splitter]

Splitter 102 splits each of pictures included in the original image into a plurality of blocks, and outputs each block to subtractor 104. For example, splitter 102 first splits a picture into blocks of a fixed size (for example, 128×128 pixels). The fixed-size block is also referred to as a coding tree unit (CTU). Splitter 102 then splits each fixed-size block into blocks of variable sizes (for example, 64×64 pixels or smaller), based on recursive quadtree and/or binary tree block splitting. In other words, splitter 102 selects a splitting pattern. The variable-size block is also referred to as a coding unit (CU), a prediction unit (PU), or a transform unit (TU). It is to be noted that, in various kinds of mounting examples, there is no need to differentiate between CU, PU, and TU; all or some of the blocks in a picture may be processed in units of a CU, a PU, or a TU.

FIG. 10 is a diagram illustrating one example of block splitting according to this embodiment. In FIG. 10, the solid lines represent block boundaries of blocks split by quadtree block splitting, and the dashed lines represent block boundaries of blocks split by binary tree block splitting.

Here, block 10 is a square block having 128×128 pixels. This block 10 is first split into four square 64×64 pixel blocks (quadtree block splitting).

The upper-left 64×64 pixel block is further vertically split into two rectangle 32×64 pixel blocks, and the left 32×64

pixel block is further vertically split into two rectangle 16×64 pixel blocks (binary tree block splitting). As a result, the upper-left square 64×64 pixel block is split into two 16×64 pixel blocks **11** and **12** and one 32×64 pixel block **13**.

The upper-right square 64×64 pixel block is horizontally split into two rectangle 64×32 pixel blocks **14** and **15** (binary tree block splitting).

The lower-left square 64×64 pixel block is first split into four square 32×32 pixel blocks (quadtree block splitting). The upper-left block and the lower-right block among the four square 32×32 pixel blocks are further split. The upper-left square 32×32 pixel block is vertically split into two rectangle 16×32 pixel blocks, and the right 16×32 pixel block is further horizontally split into two 16×16 pixel blocks (binary tree block splitting). The lower-right 32×32 pixel block is horizontally split into two 32×16 pixel blocks (binary tree block splitting). The upper-right square 32×32 pixel block is horizontally split into two rectangle 32×16 pixel blocks (binary tree block splitting). As a result, the lower-left square 64×64 pixel block is split into rectangle 16×32 pixel block **16**, two square 16×16 pixel blocks **17** and **18**, two square 32×32 pixel blocks **19** and **20**, and two rectangle 32×16 pixel blocks **21** and **22**.

The lower-right 64×64 pixel block **23** is not split.

As described above, in FIG. **10**, block **10** is split into thirteen variable-size blocks **11** through **23** based on recursive quadtree and binary tree block splitting. Such splitting is also referred to as quad-tree plus binary tree splitting (QTBT).

It is to be noted that, in FIG. **10**, one block is split into four or two blocks (quadtree or binary tree block splitting), but splitting is not limited to these examples. For example, one block may be split into three blocks (ternary block splitting). Splitting including such ternary block splitting is also referred to as multi type tree (MBT) splitting.

FIG. **11** is a diagram illustrating one example of a functional configuration of splitter **102**. As illustrated in FIG. **11**, splitter **102** may include block splitting determiner **102a**. Block splitting determiner **102a** may perform the following processes as examples.

For example, block splitting determiner **102a** collects block information from either block memory **118** or frame memory **122**, and determines the above-described splitting pattern based on the block information. Splitter **102** splits the original image according to the splitting pattern, and outputs at least one block obtained by the splitting to subtractor **104**.

In addition, for example, block splitting determiner **102a** outputs a parameter indicating the above-described splitting pattern to transformer **106**, inverse transformer **114**, intra predictor **124**, inter predictor **126**, and entropy encoder **110**. Transformer **106** may transform a prediction residual based on the parameter. Intra predictor **124** and inter predictor **126** may generate a prediction image based on the parameter. In addition, entropy encoder **110** may entropy encode the parameter.

The parameter related to the splitting pattern may be written in a stream as indicated below as one example.

FIG. **12** is a diagram illustrating examples of splitting patterns. Examples of splitting patterns include: splitting into four regions (QT) in which a block is split into two regions both horizontally and vertically; splitting into three regions (HT or VT) in which a block is split in the same direction in a ratio of 1:2:1; splitting into two regions (HB or VB) in which a block is split in the same direction in a ratio of 1:1; and no splitting (NS).

It is to be noted that the splitting pattern does not have any block splitting direction in the case of splitting into four regions and no splitting, and that the splitting pattern has splitting direction information in the case of splitting into two regions or three regions.

FIGS. **13A** and **13B** are each a diagram illustrating one example of a syntax tree of a splitting pattern. In the example of FIG. **13A**, first, information indicating whether to perform splitting (S: Split flag) is present, and information indicating whether to perform splitting into four regions (QT: QT flag) is present next. Information indicating which one of splitting into three regions and two regions is to be performed (TT: TT flag or BT: BT flag) is present next, and lastly, information indicating a division direction (Ver: Vertical flag or Hor: Horizontal flag) is present. It is to be noted that each of at least one block obtained by splitting according to such a splitting pattern may be further split repeatedly in a similar process. In other words, as one example, whether splitting is performed, whether splitting into four regions is performed, which one of the horizontal direction and the vertical direction is the direction in which a splitting method is to be performed, which one of splitting into three regions and splitting into two regions is to be performed may be recursively determined, and the determination results may be encoded in a stream according to the encoding order disclosed by the syntax tree illustrated in FIG. **13A**.

In addition, although information items respectively indicating S, QT, TT, and Ver are arranged in the listed order in the syntax tree illustrated in FIG. **13A**, information items respectively indicating S, QT, Ver, and BT may be arranged in the listed order. In other words, in the example of FIG. **13B**, first, information indicating whether to perform splitting (S: Split flag) is present, and information indicating whether to perform splitting into four regions (QT: QT flag) is present next. Information indicating the splitting direction (Ver: Vertical flag or Hor: Horizontal flag) is present next, and lastly, information indicating which one of splitting into two regions and splitting into three regions is to be performed (BT: BT flag or TT: TT flag) is present.

It is to be noted that the splitting patterns described above are examples, and splitting patterns other than the described splitting patterns may be used, or part of the described splitting patterns may be used.

[Subtractor]

Subtractor **104** subtracts a prediction image (prediction image that is input from prediction controller **128**) from the original image in units of a block input from splitter **102** and split by splitter **102**. In other words, subtractor **104** calculates prediction residuals of a current block. Subtractor **104** then outputs the calculated prediction residuals to transformer **106**.

The original signal is an input signal which has been input to encoder **100** and represents an image of each picture included in a video (for example, a luma signal and two chroma signals).

[Transformer]

Transformer **106** transforms prediction residuals in spatial domain into transform coefficients in frequency domain, and outputs the transform coefficients to quantizer **108**. More specifically, transformer **106** applies, for example, a pre-defined discrete cosine transform (DCT) or discrete sine transform (DST) to prediction residuals in spatial domain.

It is to be noted that transformer **106** may adaptively select a transform type from among a plurality of transform types, and transform prediction residuals into transform coefficients by using a transform basis function corresponding to the selected transform type. This sort of transform is

also referred to as explicit multiple core transform (EMT) or adaptive multiple transform (AMT). In addition, a transform basis function is also simply referred to as a basis.

The transform types include, for example, DCT-II, DCT-V, DCT-VIII, DST-I, and DST-VII. It is to be noted that these transform types may be represented as DCT2, DCT5, DCT8, DST1, and DST7. FIG. 14 is a chart illustrating transform basis functions for each transform type. In FIG. 14, N indicates the number of input pixels. For example, selection of a transform type from among the plurality of transform types may depend on a prediction type (one of intra prediction and inter prediction), and may depend on an intra prediction mode.

Information indicating whether to apply such EMT or AMT (referred to as, for example, an EMT flag or an AMT flag) and information indicating the selected transform type is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

In addition, transformer 106 may re-transform the transform coefficients (which are transform results). Such re-transform is also referred to as adaptive secondary transform (AST) or non-separable secondary transform (NSST). For example, transformer 106 performs re-transform in units of a sub-block (for example, 4×4 pixel sub-block) included in a transform coefficient block corresponding to an intra prediction residual. Information indicating whether to apply NSST and information related to a transform matrix for use in NSST are normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

Transformer 106 may employ a separable transform and a non-separable transform. A separable transform is a method in which a transform is performed a plurality of times by separately performing a transform for each of directions according to the number of dimensions of inputs. A non-separable transform is a method of performing a collective transform in which two or more dimensions in multidimensional inputs are collectively regarded as a single dimension.

In one example of the non-separable transform, when an input is a 4×4 pixel block, the 4×4 pixel block is regarded as a single array including sixteen elements, and the transform applies a 16×16 transform matrix to the array.

In another example of the non-separable transform, an input block of 4×4 pixels is regarded as a single array including sixteen elements, and then a transform (hypercube gives transform) in which gives revolution is performed on the array a plurality of times may be performed.

In the transform in transformer 106, the transform types of transform basis functions to be transformed into the frequency domain according to regions in a CU can be switched. Examples include a spatially varying transform (SVT).

FIG. 15 is a diagram illustrating one example of SVT.

In SVT, as illustrated in FIG. 15, CUs are split into two equal regions horizontally or vertically, and only one of the regions is transformed into the frequency domain. A transform type can be set for each region. For example, DST7 and DST8 are used. For example, among the two regions obtained by splitting a CU vertically into two equal regions, DST7 and DCT8 may be used for the region at position 0. Alternatively, among the two regions, DST7 is used for the

region at position 1. Likewise, among the two regions obtained by splitting a CU horizontally into two equal regions, DST7 and DCT8 are used for the region at position 0. Alternatively, among the two regions, DST7 is used for the region at position 1. Although only one of the two regions in a CU is transformed and the other is not transformed in the example illustrated in FIG. 15, each of the two regions may be transformed. In addition, splitting method may include not only splitting into two regions but also splitting into four regions. In addition, the splitting method can be more flexible. For example, information indicating the splitting method may be encoded and may be signaled in the same manner as the CU splitting. It is to be noted that SVT is also referred to as sub-block transform (SBT).

The AMT and EMT described above may be referred to as MTS (multiple transform selection). When MTS is applied, a transform type that is DST7, DCT8, or the like can be selected, and the information indicating the selected transform type may be encoded as index information for each CU. There is another process referred to as IMTS (implicit MTS) as a process for selecting, based on the shape of a CU, a transform type to be used for orthogonal transform performed without encoding index information. When IMTS is applied, for example, when a CU has a rectangle shape, orthogonal transform of the rectangle shape is performed using DST7 for the short side and DST2 for the long side. In addition, for example, when a CU has a square shape, orthogonal transform of the rectangle shape is performed using DCT2 when MTS is effective in a sequence and using DST7 when MTS is ineffective in the sequence. DCT2 and DST7 are mere examples. Other transform types may be used, and it is also possible to change the combination of transform types for use to a different combination of transform types. IMTS may be used only for intra prediction blocks, or may be used for both intra prediction blocks and inter prediction block.

The three processes of MTS, SBT, and IMTS have been described above as selection processes for selectively switching transform types for use in orthogonal transform. However, all of the three selection processes may be made effective, or only part of the selection processes may be selectively made effective. Whether each of the selection processes is made effective can be identified based on flag information or the like in a header such as an SPS. For example, when all of the three selection processes are effective, one of the three selection processes is selected for each CU and orthogonal transform of the CU is performed. It is to be noted that the selection processes for selectively switching the transform types may be selection processes different from the above three selection processes, or each of the three selection processes may be replaced by another process as long as at least one of the following four functions [1] to [4] can be achieved. Function [1] is a function for performing orthogonal transform of the entire CU and encoding information indicating the transform type used in the transform. Function [2] is a function for performing orthogonal transform of the entire CU and determining the transform type based on a predetermined rule without encoding information indicating the transform type. Function [3] is a function for performing orthogonal transform of a partial region of a CU and encoding information indicating the transform type used in the transform. Function [4] is a function for performing orthogonal transform of a partial region of a CU and determining the transform type based on a predetermined rule without encoding information indicating the transform type used in the transform.

It is to be noted that whether each of MTS, IMTS, and SBT is applied may be determined for each processing unit. For example, whether each of MTS, IMTS, and SBT is applied may be determined for each sequence, picture, brick, slice, CTU, or CU.

It is to be noted that a tool for selectively switching transform types in the present disclosure may be rephrased by a method for selectively selecting a basis for use in a transform process, a selection process, or a process for selecting a basis. In addition, the tool for selectively switching transform types may be rephrased by a mode for adaptively selecting a transform type.

FIG. 16 is a flow chart illustrating one example of a process performed by transformer 106.

For example, transformer 106 determines whether to perform orthogonal transform (Step St_1). Here, when determining to perform orthogonal transform (Yes in Step St_1), transformer 106 selects a transform type for use in orthogonal transform from a plurality of transform types (Step St_2). Next, transformer 106 performs orthogonal transform by applying the selected transform type to the prediction residual of a current block (Step St_3). Transformer 106 then outputs information indicating the selected transform type to entropy encoder 110, so as to allow entropy encoder 110 to encode the information (Step St_4). On the other hand, when determining not to perform orthogonal transform (No in Step St_1), transformer 106 outputs information indicating that no orthogonal transform is performed, so as to allow entropy encoder 110 to encode the information (Step St_5). It is to be noted that whether to perform orthogonal transform in Step St_1 may be determined based on, for example, the size of a transform block, a prediction mode applied to the CU, etc. Alternatively, orthogonal transform may be performed using a predefined transform type without encoding information indicating the transform type for use in orthogonal transform.

FIG. 17 is a flow chart illustrating another example of a process performed by transformer 106. It is to be noted that the example illustrated in FIG. 17 is an example of orthogonal transform in the case where transform types for use in orthogonal transform are selectively switched as in the case of the example illustrated in FIG. 16.

As one example, a first transform type group may include DCT2, DST7, and DCT8. As another example, a second transform type group may include DCT2. The transform types included in the first transform type group and the transform types included in the second transform type group may partly overlap with each other, or may be totally different from each other.

More specifically, transformer 106 determines whether a transform size is smaller than or equal to a predetermined value (Step Su_1). Here, when determining that the transform size is smaller than or equal to the predetermined value (Yes in Step Su_1), transformer 106 performs orthogonal transform of the prediction residual of the current block using the transform type included in the first transform type group (Step Su_2). Next, transformer 106 outputs information indicating the transform type to be used among at least one transform type included in the first transform type group to entropy encoder 110, so as to allow entropy encoder 110 to encode the information (Step Su_3). On the other hand, when determining that the transform size is not smaller than or equal to the predetermined value (No in Step Su_1), transformer 106 performs orthogonal transform of the prediction residual of the current block using the second transform type group (Step Su_4).

In Step Su_3, the information indicating the transform type for use in orthogonal transform may be information indicating a combination of the transform type to be applied vertically in the current block and the transform type to be applied horizontally in the current block. The first type group may include only one transform type, and the information indicating the transform type for use in orthogonal transform may not be encoded. The second transform type group may include a plurality of transform types, and information indicating the transform type for use in orthogonal transform among the one or more transform types included in the second transform type group may be encoded.

Alternatively, a transform type may be determined based only on a transform size. It is to be noted that such determinations are not limited to the determination as to whether the transform size is smaller than or equal to the predetermined value, and other processes are also possible as long as the processes are for determining a transform type for use in orthogonal transform based on the transform size. [Quantizer]

Quantizer 108 quantizes the transform coefficients output from transformer 106. More specifically, quantizer 108 scans, in a determined scanning order, the transform coefficients of the current block, and quantizes the scanned transform coefficients based on quantization parameters (QP) corresponding to the transform coefficients. Quantizer 108 then outputs the quantized transform coefficients (hereinafter also referred to as quantized coefficients) of the current block to entropy encoder 110 and inverse quantizer 112.

A determined scanning order is an order for quantizing/inverse quantizing transform coefficients. For example, a determined scanning order is defined as ascending order of frequency (from low to high frequency) or descending order of frequency (from high to low frequency).

A quantization parameter (QP) is a parameter defining a quantization step (quantization width). For example, when the value of the quantization parameter increases, the quantization step also increases. In other words, when the value of the quantization parameter increases, an error in quantized coefficients (quantization error) increases.

In addition, a quantization matrix may be used for quantization. For example, several kinds of quantization matrices may be used correspondingly to frequency transform sizes such as 4×4 and 8×8, prediction modes such as intra prediction and inter prediction, and pixel components such as luma and chroma pixel components. It is to be noted that quantization means digitalizing values sampled at predetermined intervals correspondingly to predetermined levels. In this technical field, quantization may be represented as other expressions such as rounding and scaling.

Methods using quantization matrices include a method using a quantization matrix which has been set directly at the encoder 100 side and a method using a quantization matrix which has been set as a default (default matrix). At the encoder 100 side, a quantization matrix suitable for features of an image can be set by directly setting a quantization matrix. This case, however, has a disadvantage of increasing a coding amount for encoding the quantization matrix. It is to be noted that a quantization matrix to be used to quantize the current block may be generated based on a default quantization matrix or an encoded quantization matrix, instead of directly using the default quantization matrix or the encoded quantization matrix.

There is a method for quantizing a high-frequency coefficient and a low-frequency coefficient in the same manner

without using a quantization matrix. It is to be noted that this method is equivalent to a method using a quantization matrix (flat matrix) whose all coefficients have the same value.

The quantization matrix may be encoded, for example, at the sequence level, picture level, slice level, brick level, or CTU level.

When using a quantization matrix, quantizer **108** scales, for each transform coefficient, for example a quantization width which can be calculated based on a quantization parameter, etc., using the value of the quantization matrix. The quantization process performed without using any quantization matrix may be a process of quantizing transform coefficients based on a quantization width calculated based on a quantization parameter, etc. It is to be noted that, in the quantization process performed without using any quantization matrix, the quantization width may be multiplied by a predetermined value which is common for all the transform coefficients in a block.

FIG. **18** is a block diagram illustrating one example of a functional configuration of quantizer **108**.

For example, quantizer **108** includes difference quantization parameter generator **108a**, predicted quantization parameter generator **108b**, quantization parameter generator **108c**, quantization parameter storage **108d**, and quantization executor **108e**.

FIG. **19** is a flow chart illustrating one example of quantization performed by quantizer **108**.

As one example, quantizer **108** may perform quantization for each CU based on the flow chart illustrated in FIG. **19**. More specifically, quantization parameter generator **108c** determines whether to perform quantization (Step Sv_1). Here, when determining to perform quantization (Yes in Step Sv_1), quantization parameter generator **108c** generates a quantization parameter for a current block (Step Sv_2), and stores the quantization parameter into quantization parameter storage **108d** (Step Sv_3).

Next, quantization executor **108e** quantizes transform coefficients of the current block using the quantization parameter generated in Step Sv_2 (Step Sv_4). Predicted quantization parameter generator **108b** then obtains a quantization parameter for a processing unit different from the current block from quantization parameter storage **108d** (Step Sv_5). Predicted quantization parameter generator **108b** generates a predicted quantization parameter of the current block based on the obtained quantization parameter (Step Sv_6). Difference quantization parameter generator **108a** calculates the difference between the quantization parameter of the current block generated by quantization parameter generator **108c** and the predicted quantization parameter of the current block generated by predicted quantization parameter generator **108b** (Step Sv_7). The difference quantization parameter is generated by calculating the difference. Difference quantization parameter generator **108a** outputs the difference quantization parameter to entropy encoder **110**, so as to allow entropy encoder **110** to encode the difference quantization parameter (Step Sv_8).

It is to be noted that the difference quantization parameter may be encoded, for example, at the sequence level, picture level, slice level, brick level, or CTU level. In addition, the initial value of the quantization parameter may be encoded at the sequence level, picture level, slice level, brick level, or CTU level. At this time, the quantization parameter may be generated using the initial value of the quantization parameter and the difference quantization parameter.

It is to be noted that quantizer **108** may include a plurality of quantizers, and may apply dependent quantization in

which transform coefficients are quantized using a quantization method selected from a plurality of quantization methods.

[Entropy Encoder]

FIG. **20** is a block diagram illustrating one example of a functional configuration of entropy encoder **110**.

Entropy encoder **110** generates a stream by entropy encoding the quantized coefficients input from quantizer **108** and a prediction parameter input from prediction parameter generator **130**. For example, context-based adaptive binary arithmetic coding (CABAC) is used as the entropy encoding. More specifically, entropy encoder **110** includes binarizer **110a**, context controller **110b**, and binary arithmetic encoder **110c**. Binarizer **110a** performs binarization in which multi-level signals such as quantized coefficients and a prediction parameter are transformed into binary signals. Examples of binarization methods include truncated Rice binarization, exponential Golomb codes, and fixed length binarization. Context controller **110b** derives a context value according to a feature or a surrounding state of a syntax element, that is, an occurrence probability of a binary signal. Examples of methods for deriving a context value include bypass, referring to a syntax element, referring to an upper and left adjacent blocks, referring to hierarchical information, and others. Binary arithmetic encoder **110c** arithmetically encodes the binary signal using the derived context value.

FIG. **21** is a diagram illustrating a flow of CABAC in entropy encoder **110**.

First, initialization is performed in CABAC in entropy encoder **110**. In the initialization, initialization in binary arithmetic encoder **110c** and setting of an initial context value are performed. For example, binarizer **110a** and binary arithmetic encoder **110c** execute binarization and arithmetic encoding of a plurality of quantization coefficients in a CTU sequentially. At this time, context controller **110b** updates the context value each time arithmetic encoding is performed. Context controller **110b** then saves the context value as a post process. The saved context value is used, for example, to initialize the context value for the next CTU.

[Inverse Quantizer]

Inverse quantizer **112** inverse quantizes quantized coefficients which have been input from quantizer **108**. More specifically, inverse quantizer **112** inverse quantizes, in a determined scanning order, quantized coefficients of the current block. Inverse quantizer **112** then outputs the inverse quantized transform coefficients of the current block to inverse transformer **114**.

[Inverse Transformer]

Inverse transformer **114** restores prediction errors by inverse transforming the transform coefficients which have been input from inverse quantizer **112**. More specifically, inverse transformer **114** restores the prediction residuals of the current block by performing an inverse transform corresponding to the transform applied to the transform coefficients by transformer **106**. Inverse transformer **114** then outputs the restored prediction residuals to adder **116**.

It is to be noted that since information is normally lost in quantization, the restored prediction residuals do not match the prediction errors calculated by subtractor **104**. In other words, the restored prediction residuals normally include quantization errors.

[Adder]

Adder **116** reconstructs the current block by adding the prediction residuals which have been input from inverse transformer **114** and prediction images which have been input from prediction controller **128**. Consequently, a recon-

structed image is generated. Adder 116 then outputs the reconstructed image to block memory 118 and loop filter 120.

[Block Memory]

Block memory 118 is storage for storing a block which is included in a current picture and is referred to in intra prediction. More specifically, block memory 118 stores a reconstructed image output from adder 116.

[Frame Memory]

Frame memory 122 is, for example, storage for storing reference pictures for use in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory 122 stores a reconstructed image filtered by loop filter 120.

[Loop Filter]

Loop filter 120 applies a loop filter to a reconstructed image output by adder 116, and outputs the filtered reconstructed image to frame memory 122. A loop filter is a filter used in an encoding loop (in-loop filter). Examples of loop filters include, for example, an adaptive loop filter (ALF), a deblocking filter (DF or DBF), a sample adaptive offset (SAO), etc.

FIG. 22 is a block diagram illustrating one example of a functional configuration of loop filter 120.

For example, as illustrated in FIG. 22, loop filter 120 includes deblocking filter executor 120a, SAO executor 120b, and ALF executor 120c. Deblocking filter executor 120a performs a deblocking filter process of the reconstructed image. SAO executor 120b performs a SAO process of the reconstructed image after being subjected to the deblocking filter process. ALF executor 120c performs an ALF process of the reconstructed image after being subjected to the SAO process. The ALF and deblocking filter processes are described later in detail. The SAO process is a process for enhancing image quality by reducing ringing (a phenomenon in which pixel values are distorted like waves around an edge) and correcting deviation in pixel value. Examples of SAO processes include an edge offset process and a band offset process. It is to be noted that loop filter 120 does not always need to include all the constituent elements disclosed in FIG. 22, and may include only part of the constituent elements. In addition, loop filter 120 may be configured to perform the above processes in a processing order different from the one disclosed in FIG. 22.

[Loop Filter>Adaptive Loop Filter]

In an ALF, a least square error filter for removing compression artifacts is applied. For example, one filter selected from among a plurality of filters based on the direction and activity of local gradients is applied for each of 2x2 pixel sub-blocks in the current block.

More specifically, first, each sub-block (for example, each 2x2 pixel sub-block) is categorized into one out of a plurality of classes (for example, fifteen or twenty-five classes). The categorization of the sub-block is based on, for example, gradient directionality and activity. In a specific example, category index C (for example, C=5D+A) is calculated based on gradient directionality D (for example, 0 to 2 or 0 to 4) and gradient activity A (for example, 0 to 4). Then, based on category index C, each sub-block is categorized into one out of a plurality of classes.

For example, gradient directionality D is calculated by comparing gradients of a plurality of directions (for example, the horizontal, vertical, and two diagonal directions). Moreover, for example, gradient activity A is calculated by adding gradients of a plurality of directions and quantizing the result of the addition.

The filter to be used for each sub-block is determined from among the plurality of filters based on the result of such categorization.

The filter shape to be used in an ALF is, for example, a circular symmetric filter shape. FIG. 23A through FIG. 23C illustrate examples of filter shapes used in ALFs. FIG. 23A illustrates a 5x5 diamond shape filter, FIG. 23B illustrates a 7x7 diamond shape filter, and FIG. 23C illustrates a 9x9 diamond shape filter. Information indicating the filter shape is normally signaled at the picture level. It is to be noted that the signaling of such information indicating the filter shape does not necessarily need to be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, brick level, CTU level, or CU level).

The ON or OFF of the ALF is determined, for example, at the picture level or CU level. For example, the decision of whether to apply the ALF to luma may be made at the CU level, and the decision of whether to apply ALF to chroma may be made at the picture level. Information indicating ON or OFF of the ALF is normally signaled at the picture level or CU level. It is to be noted that the signaling of information indicating ON or OFF of the ALF does not necessarily need to be performed at the picture level or CU level, and may be performed at another level (for example, at the sequence level, slice level, brick level, or CTU level).

In addition, as described above, one filter is selected from the plurality of filters, and an ALF process of a sub-block is performed. A coefficient set of coefficients to be used for each of the plurality of filters (for example, up to the fifteenth or twenty-fifth filter) is normally signaled at the picture level. It is to be noted that the coefficient set does not always need to be signaled at the picture level, and may be signaled at another level (for example, the sequence level, slice level, brick level, CTU level, CU level, or sub-block level).

[Loop Filter>Cross Component Adaptive Loop Filter]

FIG. 23D is a diagram illustrating an example where Y samples (first component) are used for a cross component ALF (CCALF) for Cb and a CCALF for Cr (components different from the first component). FIG. 23E is a diagram illustrating a diamond shaped filter.

One example of CC-ALF operates by applying a linear, diamond shaped filter (FIGS. 23D, 23E) to a luma channel for each chroma component. The filter coefficients, for example, may be transmitted in the APS, scaled by a factor of 2^{10} , and rounded for fixed point representation. The application of the filters is controlled on a variable block size and signaled by a context-coded flag received for each block of samples. The block size along with a CC-ALF enabling flag is received at the slice-level for each chroma component. Syntax and semantics for CC-ALF are provided in the Appendix. In the contribution, the following block sizes (in chroma samples) were supported: 16x16, 32x32, 64x64, and 128x128.

[Loop Filter>Joint Chroma Cross Component Adaptive Loop Filter]

FIG. 23F is a diagram illustrating an example for a joint chroma CCALF (JC-CCALF).

One example of JC-CCALF, where only one CCALF filter will be used to generate one CCALF filtered output as a chroma refinement signal for one color component only, while a properly weighted version of the same chroma refinement signal will be applied to the other color component. In this way, the complexity of existing CCALF is reduced roughly by half.

The weight value is coded into a sign flag and a weight index. The weight index (denoted as weight_index) is coded into 3 bits, and specifies the magnitude of the JC-CCALF weight JcCcWeight. It cannot be equal to 0. The magnitude of JcCcWeight is determined as follows.

If weight_index is less than or equal to 4, JcCcWeight is equal to weight_index>>2.

Otherwise, JcCcWeight is equal to 4/(weight_index-4).

The block-level on/off control of ALF filtering for Cb and Cr are separate. This is the same as in CCALF, and two separate sets of block-level on/off control flags will be coded. Different from CCALF, herein, the Cb, Cr on/off control block sizes are the same, and thus, only one block size variable is coded.

[Loop Filter>Deblocking Filter]

In a deblocking filter process, loop filter 120 performs a filter process on a block boundary in a reconstructed image so as to reduce distortion which occurs at the block boundary.

FIG. 24 is a block diagram illustrating one example of a specific configuration of deblocking filter executor 120a.

For example, deblocking filter executor 120a includes: boundary determiner 1201; filter determiner 1203; filter executor 1205; process determiner 1208; filter characteristic determiner 1207; and switches 1202, 1204, and 1206.

Boundary determiner 1201 determines whether a pixel to be deblock filtered (that is, a current pixel) is present around a block boundary. Boundary determiner 1201 then outputs the determination result to switch 1202 and process determiner 1208.

In the case where boundary determiner 1201 has determined that a current pixel is present around a block boundary, switch 1202 outputs an unfiltered image to switch 1204. In the opposite case where boundary determiner 1201 has determined that no current pixel is present around a block boundary, switch 1202 outputs an unfiltered image to switch 1206. It is to be noted that the unfiltered image is an image configured with a current pixel and at least one surrounding pixel located around the current pixel.

Filter determiner 1203 determines whether to perform deblocking filtering of the current pixel, based on the pixel value of at least one surrounding pixel located around the current pixel. Filter determiner 1203 then outputs the determination result to switch 1204 and process determiner 1208.

In the case where filter determiner 1203 has determined to perform deblocking filtering of the current pixel, switch 1204 outputs the unfiltered image obtained through switch 1202 to filter executor 1205. In the opposite case where filter determiner 1203 has determined not to perform deblocking filtering of the current pixel, switch 1204 outputs the unfiltered image obtained through switch 1202 to switch 1206.

When obtaining the unfiltered image through switches 1202 and 1204, filter executor 1205 executes, for the current pixel, deblocking filtering having the filter characteristic determined by filter characteristic determiner 1207. Filter executor 1205 then outputs the filtered pixel to switch 1206.

Under control by process determiner 1208, switch 1206 selectively outputs a pixel which has not been deblock filtered and a pixel which has been deblock filtered by filter executor 1205.

Process determiner 1208 controls switch 1206 based on the results of determinations made by boundary determiner 1201 and filter determiner 1203. In other words, process determiner 1208 causes switch 1206 to output the pixel which has been deblock filtered when boundary determiner 1201 has determined that the current pixel is present around the block boundary and filter determiner 1203 has deter-

mined to perform deblocking filtering of the current pixel. In addition, in a case other than the above case, process determiner 1208 causes switch 1206 to output the pixel which has not been deblock filtered. A filtered image is output from switch 1206 by repeating output of a pixel in this way. It is to be noted that the configuration illustrated in FIG. 24 is one example of a configuration in deblocking filter executor 120a. Deblocking filter executor 120a may have another configuration.

FIG. 25 is a diagram illustrating an example of a deblocking filter having a symmetrical filtering characteristic with respect to a block boundary.

In a deblocking filter process, one of two deblocking filters having different characteristics, that is, a strong filter and a weak filter is selected using pixel values and quantization parameters, for example. In the case of the strong filter, pixels p0 to p2 and pixels q0 to q2 are present across a block boundary as illustrated in FIG. 25, the pixel values of the respective pixels q0 to q2 are changed to pixel values q'0 to q'2 by performing computations according to the expressions below.

$$q'0 = (p1 + 2 \times p0 + 2 \times q0 + 2 \times q1 + q2 + 4) / 8$$

$$q'1 = (p0 + q0 + q1 + q2 + 2) / 4$$

$$q'2 = (p0 + q0 + q1 + 3 \times q2 + 2 \times q3 + 4) / 8$$

It is to be noted that, in the above expressions, p0 to p2 and q0 to q2 are the pixel values of respective pixels p0 to p2 and pixels q0 to q2. In addition, q3 is the pixel value of neighboring pixel q3 located at the opposite side of pixel q2 with respect to the block boundary. In addition, in the right side of each of the expressions, coefficients which are multiplied with the respective pixel values of the pixels to be used for deblocking filtering are filter coefficients.

Furthermore, in the deblocking filtering, clipping may be performed so that the calculated pixel values do not change over a threshold value. In the clipping process, the pixel values calculated according to the above expressions are clipped to a value obtained according to "a pre-computation pixel value $\pm 2 \times$ a threshold value" using the threshold value determined based on a quantization parameter. In this way, it is possible to prevent excessive smoothing.

FIG. 26 is a diagram for illustrating one example of a block boundary on which a deblocking filter process is performed. FIG. 27 is a diagram illustrating examples of Bs values.

The block boundary on which the deblocking filter process is performed is, for example, a boundary between CUs, PUs, or TUs having 8x8 pixel blocks as illustrated in FIG. 26. The deblocking filter process is performed, for example, in units of four rows or four columns. First, boundary strength (Bs) values are determined as indicated in FIG. 27 for block P and block Q illustrated in FIG. 26.

According to the Bs values in FIG. 27, whether to perform deblocking filter processes of block boundaries belonging to the same image using different strengths may be determined. The deblocking filter process for a chroma signal is performed when a Bs value is 2. The deblocking filter process for a luma signal is performed when a Bs value is 1 or more and a determined condition is satisfied. It is to be noted that conditions for determining Bs values are not limited to those indicated in FIG. 27, and a Bs value may be determined based on another parameter.

[Predictor (Intra Predictor, Inter Predictor, Prediction Controller)]

FIG. 28 is a flow chart illustrating one example of a process performed by a predictor of encoder 100. It is to be noted that the predictor, as one example, includes all or part of the following constituent elements: intra predictor 124; inter predictor 126; and prediction controller 128. The prediction executor includes, for example, intra predictor 124 and inter predictor 126.

The predictor generates a prediction image of a current block (Step Sb_1). It is to be noted that the prediction image is, for example, an intra prediction image (intra prediction signal) or an inter prediction image (inter prediction signal). More specifically, the predictor generates the prediction image of the current block using a reconstructed image which has been already obtained for another block through generation of a prediction image, generation of a prediction residual, generation of quantized coefficients, restoring of a prediction residual, and addition of a prediction image.

The reconstructed image may be, for example, an image in a reference picture or an image of an encoded block (that is, the other block described above) in a current picture which is the picture including the current block. The encoded block in the current picture is, for example, a neighboring block of the current block.

FIG. 29 is a flow chart illustrating another example of a process performed by the predictor of encoder 100.

The predictor generates a prediction image using a first method (Step Sc_1a), generates a prediction image using a second method (Step Sc_1b), and generates a prediction image using a third method (Step Sc_1c). The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.

Next, the predictor evaluates the prediction images generated in Steps Sc_1a, Sc_1b, and Sc_1c (Step Sc_2). For example, the predictor calculates costs C for the prediction images generated in Step Sc_1a, Sc_1b, and Sc_1c, and evaluates the prediction images by comparing the costs C of the prediction images. It is to be noted that cost C is calculated according to an expression of an R-D optimization model, for example, $C=D+\lambda \times R$. In this expression, D indicates compression artifacts of a prediction image, and is represented as, for example, a sum of absolute differences between the pixel value of a current block and the pixel value of a prediction image. In addition, R indicates a bit rate of a stream. In addition, λ indicates, for example, a multiplier according to the method of Lagrange multiplier.

The predictor then selects one of the prediction images generated in Steps Sc_1a, Sc_1b, and Sc_1c (Step Sc_3). In other words, the predictor selects a method or a mode for obtaining a final prediction image. For example, the predictor selects the prediction image having the smallest cost C, based on costs C calculated for the prediction images. Alternatively, the evaluation in Step Sc_2 and the selection of the prediction image in Step Sc_3 may be made based on a parameter which is used in an encoding process.

Encoder 100 may transform information for identifying the selected prediction image, the method, or the mode into a stream. The information may be, for example, a flag or the like. In this way, decoder 200 is capable of generating a prediction image according to the method or the mode selected by encoder 100, based on the information. It is to be noted that, in the example illustrated in FIG. 29, the pre-

dictor selects any of the prediction images after the prediction images are generated using the respective methods. However, the predictor may select a method or a mode based on a parameter for use in the above-described encoding process before generating prediction images, and may generate a prediction image according to the method or mode selected.

For example, the first method and the second method may be intra prediction and inter prediction, respectively, and the predictor may select a final prediction image for a current block from prediction images generated according to the prediction methods.

FIG. 30 is a flow chart illustrating another example of a process performed by the predictor of encoder 100.

First, the predictor generates a prediction image using intra prediction (Step Sd_1a), and generates a prediction image using inter prediction (Step Sd_1b). It is to be noted that the prediction image generated by intra prediction is also referred to as an intra prediction image, and the prediction image generated by inter prediction is also referred to as an inter prediction image.

Next, the predictor evaluates each of the intra prediction image and the inter prediction image (Step Sd_2). Cost C described above may be used in the evaluation. The predictor may then select the prediction image for which the smallest cost C has been calculated among the intra prediction image and the inter prediction image, as the final prediction image for the current block (Step Sd_3). In other words, the prediction method or the mode for generating the prediction image for the current block is selected.

[Intra Predictor]

Intra predictor 124 generates a prediction image (that is, intra prediction image) of a current block by performing intra prediction (also referred to as intra frame prediction) of the current block by referring to a block or blocks in the current picture which is or are stored in block memory 118. More specifically, intra predictor 124 generates an intra prediction image by performing intra prediction by referring to pixel values (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction image to prediction controller 128.

For example, intra predictor 124 performs intra prediction by using one mode from among a plurality of intra prediction modes which have been predefined. The intra prediction modes normally include one or more non-directional prediction modes and a plurality of directional prediction modes.

The one or more non-directional prediction modes include, for example, planar prediction mode and DC prediction mode defined in the H.265/HEVC standard.

The plurality of directional prediction modes include, for example, the thirty-three directional prediction modes defined in the H.265/HEVC standard. It is to be noted that the plurality of directional prediction modes may further include thirty-two directional prediction modes in addition to the thirty-three directional prediction modes (for a total of sixty-five directional prediction modes). FIG. 31 is a diagram illustrating sixty-seven intra prediction modes in total used in intra prediction (two non-directional prediction modes and sixty-five directional prediction modes). The solid arrows represent the thirty-three directions defined in the H.265/HEVC standard, and the dashed arrows represent the additional thirty-two directions (the two non-directional prediction modes are not illustrated in FIG. 31).

In various kinds of mounting examples, a luma block may be referred to in intra prediction of a chroma block. In other

words, a chroma component of the current block may be predicted based on a luma component of the current block. Such intra prediction is also referred to as cross-component linear model (CCLM). The intra prediction mode for a chroma block in which such a luma block is referred to (also referred to as, for example, a CCLM mode) may be added as one of the intra prediction modes for chroma blocks.

Intra predictor **124** may correct intra-predicted pixel values based on horizontal/vertical reference pixel gradients. The intra prediction which accompanies this sort of correcting is also referred to as position dependent intra prediction combination (PDPC). Information indicating whether to apply PDPC (referred to as, for example, a PDPC flag) is normally signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, or CTU level).

FIG. **32** is a flow chart illustrating one example of a process performed by intra predictor **124**.

Intra predictor **124** selects one intra prediction mode from a plurality of intra prediction modes (Step Sw_1). Intra predictor **124** then generates a prediction image according to the selected intra prediction mode (Step Sw_2). Next, intra predictor **124** determines most probable modes (MPMs) (Step Sw_3). MPMs include, for example, six intra prediction modes. Two modes among the six intra prediction modes may be planar mode and DC prediction mode, and the other four modes may be directional prediction modes. Intra predictor **124** determines whether the intra prediction mode selected in Step Sw_1 is included in the MPMs (Step Sw_4).

Here, when determining that the intra prediction mode selected in Step Sw_1 is included in the MPMs (Yes in Step Sw_4), intra predictor **124** sets an MPM flag to 1 (Step Sw_5), and generates information indicating the selected intra prediction mode among the MPMs (Step Sw_6). It is to be noted that the MPM flag set to 1 and the information indicating the intra prediction mode are encoded as prediction parameters by entropy encoder **110**.

When determining that the selected intra prediction mode is not included in the MPMs (No in Step Sw_4), intra predictor **124** sets the MPM flag to 0 (Step Sw_7). Alternatively, intra predictor **124** does not set any MPM flag. Intra predictor **124** then generates information indicating the selected intra prediction mode among at least one intra prediction mode which is not included in the MPMs (Step Sw_8). It is to be noted that the MPM flag set to 0 and the information indicating the intra prediction mode are encoded as prediction parameters by entropy encoder **110**. The information indicating the intra prediction mode indicates, for example, any one of 0 to 60.

[Inter Predictor]

Inter predictor **126** generates a prediction image (inter prediction image) by performing inter prediction (also referred to as inter frame prediction) of the current block by referring to a block or blocks in a reference picture which is different from the current picture and is stored in frame memory **122**. Inter prediction is performed in units of a current block or a current sub-block in the current block. The sub-block is included in the block and is a unit smaller than the block. The size of the sub-block may be 4×4 pixels, 8×8 pixels, or another size. The size of the sub-block may be switched for a unit such as slice, brick, picture, etc.

For example, inter predictor **126** performs motion estimation in a reference picture for a current block or a current sub-block, and finds out a reference block or a reference

sub-block which best matches the current block or current sub-block. Inter predictor **126** then obtains motion information (for example, a motion vector) which compensates a motion or a change from the reference block or the reference sub-block to the current block or the current sub-block. Inter predictor **126** generates an inter prediction image of the current block or the current sub-block by performing motion compensation (or motion prediction) based on the motion information. Inter predictor **126** outputs the generated inter prediction image to prediction controller **128**.

The motion information used in motion compensation may be signaled as inter prediction images in various forms. For example, a motion vector may be signaled. As another example, the difference between a motion vector and a motion vector predictor may be signaled.

[Reference Picture List]

FIG. **33** is a diagram illustrating examples of reference pictures. FIG. **34** is a conceptual diagram illustrating examples of reference picture lists. Each reference picture list is a list indicating at least one reference picture stored in frame memory **122**. It is to be noted that, in FIG. **33**, each of rectangles indicates a picture, each of arrows indicates a picture reference relationship, the horizontal axis indicates time, I, P, and B in the rectangles indicate an intra prediction picture, a uni-prediction picture, and a bi-prediction picture, respectively, and numerals in the rectangles indicate a decoding order. As illustrated in FIG. **33**, the decoding order of the pictures is an order of I0, P1, B2, B3, and B4, and the display order of the pictures is an order of I0, B3, B2, B4, and P1. As illustrated in FIG. **34**, the reference picture list is a list representing reference picture candidates. For example, one picture (or a slice) may include at least one reference picture list. For example, one reference picture list is used when a current picture is a uni-prediction picture, and two reference picture lists are used when a current picture is a bi-prediction picture. In the examples of FIGS. **33** and **34**, picture B3 which is current picture currPic has two reference picture lists which are the L0 list and the L1 list. When current picture currPic is picture B3, reference picture candidates for current picture currPic are I0, P1, and B2, and the reference picture lists (which are the L0 list and the L1 list) indicate these pictures. Inter predictor **126** or prediction controller **128** specifies which picture in each reference picture list is to be actually referred to in form of a reference picture index refIdxLx. In FIG. **34**, reference pictures P1 and B2 are specified by reference picture indices refIdxL0 and refIdxL1.

Such a reference picture list may be generated for each unit such as a sequence, picture, slice, brick, CTU, or CU. In addition, among reference pictures indicated in reference picture lists, a reference picture index indicating a reference picture to be referred to in inter prediction may be signaled at the sequence level, picture level, slice level, brick level, CTU level, or CU level. In addition, a common reference picture list may be used in a plurality of inter prediction modes.

[Basic Flow of Inter Prediction]

FIG. **35** is a flow chart illustrating a basic processing flow of inter prediction.

First, inter predictor **126** generates a prediction signal (Steps Se_1 to Se_3). Next, subtractor **104** generates the difference between a current block and a prediction image as a prediction residual (Step Se_4).

Here, in the generation of the prediction image, inter predictor **126** generates the prediction image through, for example, determination of a motion vector (MV) of the current block (Steps Se_1 and Se_2) and motion compen-

35

sation (Step Se_3). Furthermore, in determination of an MV, inter predictor 126 determines the MV through, for example, selection of a motion vector candidate (MV candidate) (Step Se_1) and derivation of an MV (Step Se_2). The selection of the MV candidate is made by means of, for example, inter predictor 126 generating an MV candidate list and selecting at least one MV candidate from the MV candidate list. It is to be noted that MVs derived in the past may be added to the MV candidate list. Alternatively, in derivation of an MV, inter predictor 126 may further select at least one MV candidate from the at least one MV candidate, and determine the selected at least one MV candidate as the MV for the current block. Alternatively, inter predictor 126 may determine the MV for the current block by performing estimation in a reference picture region specified by each of the selected at least one MV candidate. It is to be noted that the estimation in the reference picture region may be referred to as motion estimation.

In addition, although Steps Se_1 to Se_3 are performed by inter predictor 126 in the above-described example, a process that is, for example, Step Se_1, Step Se_2, or the like may be performed by another constituent element included in encoder 100.

It is to be noted that an MV candidate list may be generated for each process in inter prediction mode, or a common MV candidate list may be used in a plurality of inter prediction modes. The processes in Steps Se_3 and Se_4 correspond to Steps Sa_3 and Sa_4 illustrated in FIG. 9, respectively. The process in Step Se_3 corresponds to the process in Step Sd_1b in FIG. 30.

[MV Derivation Flow]

FIG. 36 is a flow chart illustrating one example of MV derivation.

Inter predictor 126 may derive an MV for a current block in a mode for encoding motion information (for example, an MV). In this case, for example, the motion information may be encoded as a prediction parameter, and may be signaled. In other words, the encoded motion information is included in a stream.

Alternatively, inter predictor 126 may derive an MV in a mode in which motion information is not encoded. In this case, no motion information is included in the stream.

Here, MV derivation modes include a normal inter mode, a normal merge mode, a FRUC mode, an affine mode, etc. which are described later. Modes in which motion information is encoded among the modes include the normal inter mode, the normal merge mode, the affine mode (specifically, an affine inter mode and an affine merge mode), etc. It is to be noted that motion information may include not only an MV but also MV predictor selection information which is described later. Modes in which no motion information is encoded include the FRUC mode, etc. Inter predictor 126 selects a mode for deriving an MV of the current block from the plurality of modes, and derives the MV of the current block using the selected mode.

FIG. 37 is a flow chart illustrating another example of MV derivation.

Inter predictor 126 may derive an MV for a current block in a mode in which an MV difference is encoded. In this case, for example, the MV difference is encoded as a prediction parameter, and is signaled. In other words, the encoded MV difference is included in a stream. The MV difference is the difference between the MV of the current block and the MV predictor. It is to be noted that the MV predictor is a motion vector predictor.

36

Alternatively, inter predictor 126 may derive an MV in a mode in which no MV difference is encoded. In this case, no encoded MV difference is included in the stream.

Here, as described above, the MV derivation modes include the normal inter mode, the normal merge mode, the FRUC mode, the affine mode, etc. which are described later. Modes in which an MV difference is encoded among the modes include the normal inter mode, the affine mode (specifically, the affine inter mode), etc. Modes in which no MV difference is encoded include the FRUC mode, the normal merge mode, the affine mode (specifically, the affine merge mode), etc. Inter predictor 126 selects a mode for deriving an MV of the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

[MV Derivation Modes]

FIGS. 38A and 38B are each a diagram illustrating one example of categorization of modes for MV derivation. For example, as illustrated in FIG. 38A, MV derivation modes are roughly categorized into three modes according to whether to encode motion information and whether to encode MV differences. The three modes are inter mode, merge mode, and frame rate up-conversion (FRUC) mode. The inter mode is a mode in which motion estimation is performed, and in which motion information and an MV difference are encoded. For example, as illustrated in FIG. 38B, the inter mode includes affine inter mode and normal inter mode. The merge mode is a mode in which no motion estimation is performed, and in which an MV is selected from an encoded surrounding block and an MV for the current block is derived using the MV. The merge mode is a mode in which, basically, motion information is encoded and no MV difference is encoded. For example, as illustrated in FIG. 38B, the merge modes include normal merge mode (also referred to as normal merge mode or regular merge mode), merge with motion vector difference (MMVD) mode, combined inter merge/intra prediction (CIIP) mode, triangle mode, ATMVP mode, and affine merge mode. Here, an MV difference is encoded exceptionally in the MMVD mode among the modes included in the merge modes. It is to be noted that the affine merge mode and the affine inter mode are modes included in the affine modes. The affine mode is a mode for deriving, as an MV of a current block, an MV of each of a plurality of sub-blocks included in the current block, assuming affine transform. The FRUC mode is a mode which is for deriving an MV of the current block by performing estimation between encoded regions, and in which neither motion information nor any MV difference is encoded. It is to be noted that the respective modes will be described later in detail.

It is to be noted that the categorization of the modes illustrated in FIGS. 38A and 38B are examples, and categorization is not limited thereto. For example, when an MV difference is encoded in CIIP mode, the CIIP mode is categorized into inter modes.

[MV Derivation>Normal Inter Mode]

The normal inter mode is an inter prediction mode for deriving an MV of a current block by finding out a block similar to the image of the current block from a reference picture region specified by an MV candidate. In this normal inter mode, an MV difference is encoded.

FIG. 39 is a flow chart illustrating an example of inter prediction by normal inter mode.

First, inter predictor 126 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially

37

surrounding the current block (Step Sg_1). In other words, inter predictor 126 generates an MV candidate list.

Next, inter predictor 126 extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Sg_1, as motion vector predictor candidates according to a predetermined priority order (Step Sg_2). It is to be noted that the priority order is determined in advance for each of the N MV candidates.

Next, inter predictor 126 selects one MV predictor candidate from the N MV predictor candidates as the MV predictor for the current block (Step Sg_3). At this time, inter predictor 126 encodes, in a stream, MV predictor selection information for identifying the selected MV predictor. In other words, inter predictor 126 outputs the MV predictor selection information as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

Next, inter predictor 126 derives an MV of a current block by referring to an encoded reference picture (Step Sg_4). At this time, inter predictor 126 further encodes, in the stream, the difference value between the derived MV and the MV predictor as an MV difference. In other words, inter predictor 126 outputs the MV difference as a prediction parameter to entropy encoder 110 through prediction parameter generator 130. It is to be noted that the encoded reference picture is a picture including a plurality of blocks which have been reconstructed after being encoded.

Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sg_5). The processes in Steps Sg_1 to Sg_5 are executed on each block. For example, when the processes in Steps Sg_1 to Sg_5 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal inter mode finishes. For example, when the processes in Steps Sg_1 to Sg_5 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal inter mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sg_1 to Sg_5, and inter prediction of the slice using the normal inter mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal inter mode may finish when the processes in Steps Sg_1 to Sg_5 are executed on part of the blocks in the picture.

It is to be noted that the prediction image is an inter prediction signal as described above. In addition, information indicating the inter prediction mode (normal inter mode in the above example) used to generate the prediction image is, for example, encoded as a prediction parameter in an encoded signal.

It is to be noted that the MV candidate list may be also used as a list for use in another mode. In addition, the processes related to the MV candidate list may be applied to processes related to the list for use in another mode. The processes related to the MV candidate list include, for example, extraction or selection of an MV candidate from the MV candidate list, reordering of MV candidates, or deletion of an MV candidate.

[MV Derivation>Normal Merge Mode]

The normal merge mode is an inter prediction mode for selecting an MV candidate from an MV candidate list as an MV for a current block, thereby deriving the MV. It is to be noted that the normal merge mode is a merge mode in a narrow meaning and is also simply referred to as a merge

38

mode. In this embodiment, the normal merge mode and the merge mode are distinguished, and the merge mode is used in a broad meaning.

FIG. 40 is a flow chart illustrating an example of inter prediction by normal merge mode.

First, inter predictor 126 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sh_1). In other words, inter predictor 126 generates an MV candidate list.

Next, inter predictor 126 selects one MV candidate from the plurality of MV candidates obtained in Step Sh_1, thereby deriving an MV for the current block (Step Sh_2). At this time, inter predictor 126 encodes, in a stream, MV selection information for identifying the selected MV candidate. In other words, inter predictor 126 outputs the MV selection information as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Sh_3). The processes in Steps Sh_1 to Sh_3 are executed, for example, on each block. For example, when the processes in Steps Sh_1 to Sh_3 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal merge mode finishes. In addition, when the processes in Steps Sh_1 to Sh_3 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal merge mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sh_1 to Sh_3, and inter prediction of the slice using the normal merge mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal merge mode may finish when the processes in Steps Sh_1 to Sh_3 are executed on part of the blocks in the picture.

In addition, information indicating the inter prediction mode (normal merge mode in the above example) used to generate the prediction image is, for example, encoded as a prediction parameter in a stream.

FIG. 41 is a diagram for illustrating one example of an MV derivation process for a current picture by normal merge mode.

First, inter predictor 126 generates an MV candidate list in which MV candidates are registered. Examples of MV candidates include: spatially neighboring MV candidates which are MVs of a plurality of encoded blocks located spatially surrounding a current block; temporally neighboring MV candidates which are MVs of surrounding blocks on which the position of a current block in an encoded reference picture is projected; combined MV candidates which are MVs generated by combining the MV value of a spatially neighboring MV predictor and the MV value of a temporally neighboring MV predictor; and a zero MV candidate which is an MV having a zero value.

Next, inter predictor 126 selects one MV candidate from a plurality of MV candidates registered in an MV candidate list, and determines the MV candidate as the MV of the current block.

Furthermore, entropy encoder 110 writes and encodes, in a stream, merge_idx which is a signal indicating which MV candidate has been selected.

It is to be noted that the MV candidates registered in the MV candidate list described in FIG. 41 are examples. The number of MV candidates may be different from the number of MV candidates in the diagram, the MV candidate list may

be configured in such a manner that some of the kinds of the MV candidates in the diagram may not be included, or that one or more MV candidates other than the kinds of MV candidates in the diagram are included.

A final MV may be determined by performing a dynamic motion vector refreshing (DMVR) to be described later using the MV of the current block derived by normal merge mode. It is to be noted that, in normal merge mode, no MV difference is encoded, but an MV difference is encoded. In MMVD mode, one MV candidate is selected from an MV candidate list as in the case of normal merge mode, an MV difference is encoded. As illustrated in FIG. 38B, MMVD may be categorized into merge modes together with normal merge mode. It is to be noted that the MV difference in MMVD mode does not always need to be the same as the MV difference for use in inter mode. For example, MV difference derivation in MMVD mode may be a process that requires a smaller amount of processing than the amount of processing required for MV difference derivation in inter mode.

In addition, a combined inter merge/intra prediction (CIIP) mode may be performed. The mode is for overlapping a prediction image generated in inter prediction and a prediction image generated in intra prediction to generate a prediction image for a current block.

It is to be noted that the MV candidate list may be referred to as a candidate list. In addition, merge_idx is MV selection information.

[MV Derivation>HMVP Mode]

FIG. 42 is a diagram for illustrating one example of an MV derivation process for a current picture by HMVP merge mode.

In normal merge mode, an MV for, for example, a CU which is a current block is determined by selecting one MV candidate from an MV candidate list generated by referring to an encoded block (for example, a CU). Here, another MV candidate may be registered in the MV candidate list. The mode in which such another MV candidate is registered is referred to as HMVP mode.

In HMVP mode, MV candidates are managed using a first-in first-out (FIFO) buffer for HMVP, separately from the MV candidate list for normal merge mode.

In FIFO buffer, motion information such as MVs of blocks processed in the past are stored newest first. In the management of the FIFO buffer, each time when one block is processed, the MV for the newest block (that is the CU processed immediately before) is stored in the FIFO buffer, and the MV of the oldest CU (that is, the CU processed earliest) is deleted from the FIFO buffer. In the example illustrated in FIG. 42, HMVP1 is the MV for the newest block, and HMVP5 is the MV for the oldest MV.

Inter predictor 126 then, for example, checks whether each MV managed in the FIFO buffer is an MV different from all the MV candidates which have been already registered in the MV candidate list for normal merge mode starting from HMVP1. When determining that the MV is different from all the MV candidates, inter predictor 126 may add the MV managed in the FIFO buffer in the MV candidate list for normal merge mode as an MV candidate. At this time, the MV candidate registered from the FIFO buffer may be one or more.

By using the HMVP mode in this way, it is possible to add not only the MV of a block which neighbors the current block spatially or temporally but also an MV for a block processed in the past. As a result, the variation of MV

candidates for normal merge mode is expanded, which increases the probability that coding efficiency can be increased.

It is to be noted that the MV may be motion information. In other words, information stored in the MV candidate list and the FIFO buffer may include not only MV values but also reference picture information, reference directions, the numbers of pictures, etc. In addition, the block is, for example, a CU.

It is to be noted that the MV candidate list and the FIFO buffer illustrated in FIG. 42 are examples. The MV candidate list and FIFO buffer may be different in size from those in FIG. 42, or may be configured to register MV candidates in an order different from the one in FIG. 42. In addition, the process described here is common between encoder 100 and decoder 200.

It is to be noted that the HMVP mode can be applied for modes other than the normal merge mode. For example, it is also excellent that motion information such as MVs of blocks processed in affine mode in the past may be stored newest first, and may be used as MV candidates. The mode obtained by applying HMVP mode to affine mode may be referred to as history affine mode.

[MV Derivation>FRUC Mode]

Motion information may be derived at the decoder 200 side without being signaled from the encoder 100 side. For example, motion information may be derived by performing motion estimation at the decoder 200 side. At this time, at the decoder 200 side, motion estimation is performed without using any pixel value in a current block. Modes in which motion estimation is performed at the decoder 200 side in this way include a frame rate up-conversion (FRUC) mode, a pattern matched motion vector derivation (PMMVD) mode, etc.

One example of a FRUC process is illustrated in FIG. 43. First, a list which indicates, as MV candidates, MVs for encoded blocks each of which neighbors the current block spatially or temporally is generated by referring to the MVs (the list may be an MV candidate list, and be also used as the MV candidate list for normal merge mode) (Step Si_1). Next, a best MV candidate is selected from the plurality of MV candidates registered in the MV candidate list (Step Si_2). For example, the evaluation values of the respective MV candidates included in the MV candidate list are calculated, and one MV candidate is selected as the best MV candidate based on the evaluation values. Based on the selected best MV candidate, a motion vector for the current block is then derived (Step Si_4). More specifically, for example, the selected best MV candidate is directly derived as the MV for the current block. In addition, for example, the MV for the current block may be derived using pattern matching in a surrounding region of a position which is included in a reference picture and corresponds to the selected best MV candidate. In other words, estimation using the pattern matching in a reference picture and the evaluation values may be performed in the surrounding region of the best MV candidate, and when there is an MV that yields a better evaluation value, the best MV candidate may be updated to the MV that yields the better evaluation value, and the updated MV may be determined as the final MV for the current block. Update to the MV that yields the better evaluation value may not be performed.

Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the encoded reference picture (Step Si_5). The processes in Steps Si_1 to Si_5 are executed, for example, on each block. For example,

41

when the processes in Steps Si_1 to Si_5 are executed on each of all the blocks in the slice, inter prediction of the slice using the FRUC mode finishes. For example, when the processes in Steps Si_1 to Si_5 are executed on each of all the blocks in the picture, inter prediction of the picture using the FRUC mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Si_1 to Si_5, and inter prediction of the slice using the FRUC mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the FRUC mode may finish when the processes in Steps Si_1 to Si_5 are executed on part of the blocks included in the picture.

Each sub-block may be processed similarly to the above-described case of processing each block.

Evaluation values may be calculated according to various kinds of methods. For example, a comparison is made between a reconstructed image in a region in a reference picture corresponding to an MV and a reconstructed image in a determined region (the region may be, for example, a region in another reference picture or a region in a neighboring block of a current picture, as indicated below). The difference between the pixel values of the two reconstructed images may be used for an evaluation value of the MV. It is to be noted that an evaluation value may be calculated using information other than the value of the difference.

Next, pattern matching is described in detail. First, one MV candidate included in an MV candidate list (also referred to as a merge list) is selected as a starting point for estimation by pattern matching. As the pattern matching, either a first pattern matching or a second pattern matching may be used. The first pattern matching and the second pattern matching may be referred to as bilateral matching and template matching, respectively.

[MV Derivation>FRUC>Bilateral Matching]

In the first pattern matching, the pattern matching is performed between two blocks which are located along a motion trajectory of a current block and included in two different reference pictures. Accordingly, in the first pattern matching, a region in another reference picture located along the motion trajectory of the current block is used as a determined region for calculating the evaluation value of the above-described MV candidate.

FIG. 44 is a diagram for illustrating one example of the first pattern matching (bilateral matching) between the two blocks in the two reference pictures located along the motion trajectory. As illustrated in FIG. 44, in the first pattern matching, two motion vectors (MV0, MV1) are derived by estimating a pair which best matches among pairs of two blocks which are included in the two different reference pictures (Ref0, Ref1) and located along the motion trajectory of the current block (Cur block). More specifically, a difference between the reconstructed image at a specified position in the first encoded reference picture (Ref0) specified by an MV candidate and the reconstructed image at a specified position in the second encoded reference picture (Ref1) specified by a symmetrical MV obtained by scaling the MV candidate at a display time interval is derived for the current block, and an evaluation value is calculated using the value of the obtained difference. It is excellent to select, as the best MV, the MV candidate which yields the best evaluation value among the plurality of MV candidates.

In the assumption of a continuous motion trajectory, the motion vectors (MV0, MV1) specifying the two reference blocks are proportional to temporal distances (TD0, TD1) between the current picture (Cur Pic) and the two reference pictures (Ref0, Ref1). For example, when the current picture

42

is temporally located between the two reference pictures and the temporal distances from the current picture to the respective two reference pictures are equal to each other, mirror-symmetrical bi-directional MVs are derived in the first pattern matching.

[MV Derivation>FRUC>Template Matching]

In the second pattern matching (template matching), pattern matching is performed between a block in a reference picture and a template in the current picture (the template is a block neighboring the current block in the current picture (the neighboring block is, for example, an upper and/or left neighboring block(s))). Accordingly, in the second pattern matching, the block neighboring the current block in the current picture is used as the determined region for calculating the evaluation value of the above-described MV candidate.

FIG. 45 is a diagram for illustrating one example of pattern matching (template matching) between a template in a current picture and a block in a reference picture. As illustrated in FIG. 45, in the second pattern matching, the MV for the current block (Cur block) is derived by estimating, in the reference picture (Ref0), the block which best matches the block neighboring the current block in the current picture (Cur Pic). More specifically, the difference between a reconstructed image in an encoded region which neighbors both left and above or either left or above and a reconstructed image which is in a corresponding region in the encoded reference picture (Ref0) and is specified by an MV candidate is derived, and an evaluation value is calculated using the value of the obtained difference. It is excellent to select, as the best MV candidate, the MV candidate which yields the best evaluation value among the plurality of MV candidates.

Such information indicating whether to apply the FRUC mode (referred to as, for example, a FRUC flag) may be signaled at the CU level. In addition, when the FRUC mode is applied (for example, when a FRUC flag is true), information indicating an applicable pattern matching method (either the first pattern matching or the second pattern matching) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

[MV Derivation>Affine Mode]

The affine mode is a mode for generating an MV using affine transform. For example, an MV may be derived in units of a sub-block based on motion vectors of a plurality of neighboring blocks. This mode is also referred to as an affine motion compensation prediction mode.

FIG. 46A is a diagram for illustrating one example of MV derivation in units of a sub-block based on MVs of a plurality of neighboring blocks. In FIG. 46A, the current block includes sixteen 4x4 pixel sub-blocks. Here, motion vector v_0 at an upper-left corner control point in the current block is derived based on an MV of a neighboring block, and likewise, motion vector v_1 at an upper-right corner control point in the current block is derived based on an MV of a neighboring sub-block. Two motion vectors v_0 and v_1 are projected according to an expression (1A) indicated below, and motion vectors (v_x, v_y) for the respective sub-blocks in the current block are derived.

43

[MATH 1]

$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x - \frac{(v_{1y} - v_{0y})}{w}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{1x} - v_{0x})}{w}y + v_{0y} \end{cases} \quad (1A)$$

Here, x and y indicate the horizontal position and the vertical position of the sub-block, respectively, and w indicates a predetermined weighting coefficient.

Such information indicating the affine mode (for example, referred to as an affine flag) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

In addition, the affine mode may include several modes for different methods for deriving MVs at the upper-left and upper-right corner control points. For example, the affine modes include two modes which are the affine inter mode (also referred to as an affine normal inter mode) and the affine merge mode.

FIG. 46B is a diagram for illustrating one example of MV derivation in units of a sub-block in affine mode in which three control points are used. In FIG. 46B, the current block includes, for example, sixteen 4x4 pixel sub-blocks. Here, motion vector v_0 at an upper-left corner control point in the current block is derived based on an MV of a neighboring block. Here, motion vector v_1 at an upper-right corner control point in the current block is derived based on an MV of a neighboring block, and likewise, motion vector v_2 at a lower-left corner control point for the current block is derived based on an MV of a neighboring block. Three motion vectors v_0 , v_1 , and v_2 are projected according to an expression (1B) indicated below, and motion vectors (v_x , v_y) for the respective sub-blocks in the current block are derived.

[MATH 2]

$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x + \frac{(v_{2x} - v_{0x})}{h}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{2y} - v_{0y})}{h}y + v_{0y} \end{cases} \quad (1B)$$

Here, x and y indicate the horizontal position and the vertical position of the sub-block, respectively, and each of w and h indicates a predetermined weighting coefficient. Here, w may indicate the width of a current block, and h may indicate the height of the current block.

Affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level. It is to be noted that information indicating the number of control points in affine mode used at the CU level may be signaled at another level (for example, the sequence level, picture level, slice level, brick level, CTU level, or sub-block level).

In addition, such an affine mode in which three control points are used may include different methods for deriving MVs at the upper-left, upper-right, and lower-left corner control points. For example, the affine modes in which three control points are used include two modes which are affine inter mode and affine merge mode, as in the case of affine modes in which two control points are used.

44

It is to be noted that, in the affine modes, the size of each sub-block included in the current block may not be limited to 4x4 pixels, and may be another size. For example, the size of each sub-block may be 8x8 pixels.

5 [MV Derivation>Affine Mode>Control Point]

FIGS. 47A, 47B, and 47C are each a conceptual diagram for illustrating one example of MV derivation at control points in an affine mode.

As illustrated in FIG. 47A, in the affine mode, for example, MV predictors at respective control points for a current block are calculated based on a plurality of MVs corresponding to blocks encoded according to the affine mode among encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) which neighbor the current block. More specifically, encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) are checked in the listed order, and the first effective block encoded according to the affine mode is identified. The MV at each control point for the current block is calculated based on the plurality of MVs corresponding to the identified block.

For example, as illustrated in FIG. 47B, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used, motion vectors v_3 and v_4 projected at the upper-left corner position and the upper-right corner position of the encoded block including block A are derived. Motion vector v_0 at the upper-left control point and motion vector v_1 at the upper-right control point for the current block are then calculated from derived motion vectors v_3 and v_4 .

For example, as illustrated in FIG. 47C, when block A which neighbors to the left of the current block has been encoded according to an affine mode in which three control points are used, motion vectors v_3 , v_4 , and v_5 projected at the upper-left corner position, the upper-right corner position, and the lower-left corner position of the encoded block including block A are derived. Motion vector v_0 at the upper-left control point for the current block, motion vector v_1 at the upper-right control point for the current block, and motion vector v_2 at the lower-left control point for the current block are then calculated from derived motion vectors v_3 , v_4 , and v_5 .

The MV derivation methods illustrated in FIGS. 47A to 47C may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

FIGS. 48A and 48B are each a conceptual diagram for illustrating another example of MV derivation at control points in affine mode.

FIG. 48A is a diagram for illustrating an affine mode in which two control points are used.

In the affine mode, as illustrated in FIG. 48A, an MV selected from MVs at encoded block A, block B, and block C which neighbor the current block is used as motion vector v_0 at the upper-left corner control point for the current block. Likewise, an MV selected from MVs of encoded block D and block E which neighbor the current block is used as motion vector v_1 at the upper-right corner control point for the current block.

FIG. 48B is a diagram for illustrating an affine mode in which three control points are used.

In the affine mode, as illustrated in FIG. 48B, an MV selected from MVs at encoded block A, block B, and block

C which neighbor the current block is used as motion vector v_0 at the upper-left corner control point for the current block. Likewise, an MV selected from MVs of encoded block D and block E which neighbor the current block is used as motion vector v_1 at the upper-right corner control point for the current block. Furthermore, an MV selected from MVs of encoded block F and block G which neighbor the current block is used as motion vector v_2 at the lower-left corner control point for the current block.

It is to be noted that the MV derivation methods illustrated in FIGS. 48A and 48B may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

Here, when affine modes in which different numbers of control points (for example, two and three control points) are used may be switched and signaled at the CU level, the number of control points for an encoded block and the number of control points for a current block may be different from each other.

FIGS. 49A and 49B are each a conceptual diagram for illustrating one example of a method for MV derivation at control points when the number of control points for an encoded block and the number of control points for a current block are different from each other.

For example, as illustrated in FIG. 49A, a current block has three control points at the upper-left corner, the upper-right corner, and the lower-left corner, and block A which neighbors to the left of the current block has been encoded according to an affine mode in which two control points are used. In this case, motion vectors v_3 and v_4 projected at the upper-left corner position and the upper-right corner position in the encoded block including block A are derived. Motion vector v_0 at the upper-left corner control point and motion vector v_1 at the upper-right corner control point for the current block are then calculated from derived motion vectors v_3 and v_4 . Furthermore, motion vector v_2 at the lower-left corner control point is calculated from derived motion vectors v_0 and v_1 .

For example, as illustrated in FIG. 49B, a current block has two control points at the upper-left corner and the upper-right corner, and block A which neighbors to the left of the current block has been encoded according to an affine mode in which three control points are used. In this case, motion vectors v_3 , v_4 , and v_5 projected at the upper-left corner position in the encoded block including block A, the upper-right corner position in the encoded block, and the lower-left corner position in the encoded block are derived. Motion vector v_0 at the upper-left corner control point for the current block and motion vector v_1 at the upper-right corner control point for the current block are then calculated from derived motion vectors v_3 , v_4 , and v_5 .

It is to be noted that the MV derivation methods illustrated in FIGS. 49A and 49B may be used in the MV derivation at each control point for the current block in Step Sk_1 illustrated in FIG. 50 described later, or may be used for MV predictor derivation at each control point for the current block in Step Sj_1 illustrated in FIG. 51 described later.

[MV Derivation>Affine Mode>Affine Merge Mode]
FIG. 50 is a flow chart illustrating one example of the affine merge mode.

In the affine merge mode, first, inter predictor 126 derives MVs at respective control points for a current block (Step Sk_1). The control points are an upper-left corner point of the current block and an upper-right corner point of the current block as illustrated in FIG. 46A, or an upper-left

corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. 46B. At this time, inter predictor 126 may encode MV selection information for identifying two or three derived MVs in a stream.

For example, when MV derivation methods illustrated in FIGS. 47A to 47C are used, as illustrated in FIG. 47A, inter predictor 126 checks encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) in the listed order, and identifies the first effective block encoded according to the affine mode.

Inter predictor 126 derives the MV at the control point using the identified first effective block encoded according to the identified affine mode. For example, when block A is identified and block A has two control points, as illustrated in FIG. 47B, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block from motion vectors v_3 and v_4 at the upper-left corner of the encoded block including block A and the upper-right corner of the encoded block. For example, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block by projecting motion vectors v_3 and v_4 at the upper-left corner and the upper-right corner of the encoded block onto the current block.

Alternatively, when block A is identified and block A has three control points, as illustrated in FIG. 47C, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block, motion vector v_1 at the upper-right corner control point of the current block, and motion vector v_2 at the lower-left corner control point of the current block from motion vectors v_3 , v_4 , and v_5 at the upper-left corner of the encoded block including block A, the upper-right corner of the encoded block, and the lower-left corner of the encoded block. For example, inter predictor 126 calculates motion vector v_0 at the upper-left corner control point of the current block, motion vector v_1 at the upper-right corner control point of the current block, and motion vector v_2 at the lower-left corner control point of the current block by projecting motion vectors v_3 , v_4 , and v_5 at the upper-left corner, the upper-right corner, and the lower-left corner of the encoded block onto the current block.

It is to be noted that, as illustrated in FIG. 49A described above, MVs at three control points may be calculated when block A is identified and block A has two control points, and that, as illustrated in FIG. 49B described above, MVs at two control points may be calculated when block A is identified and block A has three control points.

Next, inter predictor 126 performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor 126 calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression (1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step Sk_2). Inter predictor 126 then performs motion compensation of the sub-blocks using these affine MVs and encoded reference pictures (Step Sk_3). When the processes in Steps Sk_2 and Sk_3 are executed for each of all the sub-blocks included in the current block, the process for generating a prediction image using the affine merge mode for the current block finishes. In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

It is to be noted that the above-described MV candidate list may be generated in Step Sk_1. The MV candidate list

may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. 47A to 47C, the MV derivation methods illustrated in FIGS. 48A and 48B, the MV derivation methods illustrated in FIGS. 49A and 49B, and other MV derivation methods.

It is to be noted that MV candidate lists may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

It is to be noted that, for example, an MV candidate list including MV candidates in an affine merge mode in which two control points are used and an affine merge mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine merge mode in which two control points are used and an MV candidate list including MV candidates in the affine merge mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine merge mode in which two control points are used and the affine merge mode in which three control points are used may be generated. The MV candidate(s) may be, for example, MVs for encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left), or an MV for an effective block among the blocks.

It is to be noted that index indicating one of the MVs in an MV candidate list may be transmitted as MV selection information.

[MV Derivation>Affine Mode>Affine Inter Mode]

FIG. 51 is a flow chart illustrating one example of an affine inter mode.

In the affine inter mode, first, inter predictor 126 derives MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) of respective two or three control points for a current block (Step Sj_1). The control points are an upper-left corner point for the current block, an upper-right corner point of the current block, and a lower-left corner point for the current block as illustrated in FIG. 46A or FIG. 46B.

For example, when the MV derivation methods illustrated in FIGS. 48A and 48B are used, inter predictor 126 derives the MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) at respective two or three control points for the current block by selecting MVs of any of the blocks among encoded blocks in the vicinity of the respective control points for the current block illustrated in either FIG. 48A or FIG. 48B. At this time, inter predictor 126 encodes, in a stream, MV predictor selection information for identifying the selected two or three MV predictors.

For example, inter predictor 126 may determine, using a cost evaluation or the like, the block from which an MV as an MV predictor at a control point is selected from among encoded blocks neighboring the current block, and may write, in a bitstream, a flag indicating which MV predictor has been selected. In other words, inter predictor 126 outputs, as a prediction parameter, the MV predictor selection information such as a flag to entropy encoder 110 through prediction parameter generator 130.

Next, inter predictor 126 performs motion estimation (Steps Sj_3 and Sj_4) while updating the MV predictor selected or derived in Step Sj_1 (Step Sj_2). In other words, inter predictor 126 calculates, as an affine MV, an MV of each of sub-blocks which corresponds to an updated MV predictor, using either the expression (1A) or expression (1B) described above (Step Sj_3). Inter predictor 126 then performs motion compensation of the sub-blocks using these

affine MVs and encoded reference pictures (Step Sj_4). The processes in Steps Sj_3 and Sj_4 are executed on all the blocks in the current block each time an MV predictor is updated in Step Sj_2. As a result, for example, inter predictor 126 determines the MV predictor which yields the smallest cost as the MV at a control point in a motion estimation loop (Step Sj_5). At this time, inter predictor 126 further encodes, in the stream, the difference value between the determined MV and the MV predictor as an MV difference. In other words, inter predictor 126 outputs the MV difference as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

Lastly, inter predictor 126 generates a prediction image for the current block by performing motion compensation of the current block using the determined MV and the encoded reference picture (Step Sj_6).

It is to be noted that the above-described MV candidate list may be generated in Step Sj_1. The MV candidate list may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. 47A to 47C, the MV derivation methods illustrated in FIGS. 48A and 48B, the MV derivation methods illustrated in FIGS. 49A and 49B, and other MV derivation methods.

It is to be noted that the MV candidate list may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

It is to be noted that, for example, an MV candidate list including MV candidates in an affine inter mode in which two control points are used and an affine inter mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine inter mode in which two control points are used and an MV candidate list including MV candidates in the affine inter mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine inter mode in which two control points are used and the affine inter mode in which three control points are used may be generated. The MV candidate(s) may be, for example, MVs for encoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left), or an MV for an effective block among the blocks.

It is to be noted that index indicating one of the MV candidates in an MV candidate list may be transmitted as MV predictor selection information.

[MV Derivation>Triangle Mode]

Inter predictor 126 generates one rectangular prediction image for a rectangular current block in the above example. However, inter predictor 126 may generate a plurality of prediction images each having a shape different from a rectangle for the rectangular current block, and may combine the plurality of prediction images to generate the final rectangular prediction image. The shape different from a rectangle may be, for example, a triangle.

FIG. 52A is a diagram for illustrating generation of two triangular prediction images.

Inter predictor 126 generates a triangular prediction image by performing motion compensation of a first partition having a triangular shape in a current block by using a first MV of the first partition, to generate a triangular prediction image. Likewise, inter predictor 126 generates a triangular prediction image by performing motion compensation of a second partition having a triangular shape in a current block by using a second MV of the second partition, to generate a

triangular prediction image. Inter predictor 126 then generates a prediction image having the same rectangular shape as the rectangular shape of the current block by combining these prediction images.

It is to be noted that a first prediction image having a rectangular shape corresponding to a current block may be generated as a prediction image for a first partition, using a first MV. In addition, a second prediction image having a rectangular shape corresponding to a current block may be generated as a prediction image for a second partition, using a second MV. A prediction image for the current block may be generated by performing a weighted addition of the first prediction image and the second prediction image. It is to be noted that the part which is subjected to the weighted addition may be a partial region across the boundary between the first partition and the second partition.

FIG. 52B is a conceptual diagram for illustrating examples of a first portion of a first partition which overlaps with a second partition, and first and second sets of samples which may be weighted as part of a correction process. The first portion may be, for example, one fourth of the width or height of the first partition. In another example, the first portion may have a width corresponding to N samples adjacent to an edge of the first partition, where N is an integer greater than zero, and N may be, for example, the integer 2. As illustrated, the left example of FIG. 52B shows a rectangular partition having a rectangular portion with a width which is one fourth of the width of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The center example of FIG. 52B shows a rectangular partition having a rectangular portion with a height which is one fourth of the height of the first partition, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion. The right example of FIG. 52B shows a triangular partition having a polygonal portion with a height which corresponds to two samples, with the first set of samples including samples outside of the first portion and samples inside of the first portion, and the second set of samples including samples within the first portion.

The first portion may be a portion of the first partition which overlaps with an adjacent partition. FIG. 52C is a conceptual diagram for illustrating a first portion of a first partition, which is a portion of the first partition that overlaps with a portion of an adjacent partition. For ease of illustration, a rectangular partition having an overlapping portion with a spatially adjacent rectangular partition is shown. Partitions having other shapes, such as triangular partitions, may be employed, and the overlapping portions may overlap with a spatially or temporally adjacent partition.

In addition, although an example is given in which a prediction image is generated for each of two partitions using inter prediction, a prediction image may be generated for at least one partition using intra prediction.

FIG. 53 is a flow chart illustrating one example of a triangle mode.

In the triangle mode, first, inter predictor 126 splits the current block into the first partition and the second partition (Step Sx_1). At this time, inter predictor 126 may encode, in a stream, partition information which is information related to the splitting into the partitions as a prediction parameter. In other words, inter predictor 126 may output the partition information as the prediction parameter to entropy encoder 110 through prediction parameter generator 130.

First, inter predictor 126 obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of encoded blocks temporally or spatially surrounding the current block (Step Sx_2). In other words, inter predictor 126 generates an MV candidate list.

Inter predictor 126 then selects the MV candidate for the first partition and the MV candidate for the second partition as a first MV and a second MV, respectively, from the plurality of MV candidates obtained in Step Sx_2 (Step Sx_3). At this time, inter predictor 126 encodes, in a stream, MV selection information for identifying the selected MV candidate, as a prediction parameter. In other words, inter predictor 126 outputs the MV selection information as a prediction parameter to entropy encoder 110 through prediction parameter generator 130.

Next, inter predictor 126 generates a first prediction image by performing motion compensation using the selected first MV and an encoded reference picture (Step Sx_4). Likewise, inter predictor 126 generates a second prediction image by performing motion compensation using the selected second MV and an encoded reference picture (Step Sx_5).

Lastly, inter predictor 126 generates a prediction image for the current block by performing a weighted addition of the first prediction image and the second prediction image (Step Sx_6).

It is to be noted that, although the first partition and the second partition are triangles in the example illustrated in FIG. 52A, the first partition and the second partition may be trapezoids, or other shapes different from each other. Furthermore, although the current block includes two partitions in the example illustrated in FIG. 52A, the current block may include three or more partitions.

In addition, the first partition and the second partition may overlap with each other. In other words, the first partition and the second partition may include the same pixel region. In this case, a prediction image for a current block may be generated using a prediction image in the first partition and a prediction image in the second partition.

In addition, although the example in which the prediction image is generated for each of the two partitions using inter prediction has been illustrated, a prediction image may be generated for at least one partition using intra prediction.

It is to be noted that the MV candidate list for selecting the first MV and the MV candidate list for selecting the second MV may be different from each other, or the MV candidate list for selecting the first MV may be also used as the MV candidate list for selecting the second MV.

It is to be noted that partition information may include an index indicating the splitting direction in which at least a current block is split into a plurality of partitions. The MV selection information may include an index indicating the selected first MV and an index indicating the selected second MV. One index may indicate a plurality of pieces of information. For example, one index collectively indicating a part or the entirety of partition information and a part or the entirety of MV selection information may be encoded. [MV Derivation>ATMVP Mode]

FIG. 54 is a diagram illustrating one example of an ATMVP mode in which an MV is derived in units of a sub-block.

The ATMVP mode is a mode categorized into the merge mode. For example, in the ATMVP mode, an MV candidate for each sub-block is registered in an MV candidate list for use in normal merge mode.

More specifically, in the ATMVP mode, first, as illustrated in FIG. 54, a temporal MV reference block associated with

51

a current block is identified in an encoded reference picture specified by an MV (MV0) of a neighboring block located at the lower-left position with respect to the current block. Next, in each sub-block in the current block, the MV used to encode the region corresponding to the sub-block in the temporal MV reference block is identified. The MV identified in this way is included in an MV candidate list as an MV candidate for the sub-block in the current block. When the MV candidate for each sub-block is selected from the MV candidate list, the sub-block is subjected to motion compensation in which the MV candidate is used as the MV for the sub-block. In this way, a prediction image for each sub-block is generated.

Although the block located at the lower-left position with respect to the current block is used as a surrounding MV reference block in the example illustrated in FIG. 54, it is to be noted that another block may be used. In addition, the size of the sub-block may be 4×4 pixels, 8×8 pixels, or another size. The size of the sub-block may be switched for a unit such as a slice, brick, picture, etc.

[Motion Estimation>DMVR]

FIG. 55 is a diagram illustrating a relationship between a merge mode and DMVR.

Inter predictor 126 derives an MV for a current block according to the merge mode (Step SI_1). Next, inter predictor 126 determines whether to perform estimation of an MV that is motion estimation (Step SI_2). Here, when determining not to perform motion estimation (No in Step SI_2), inter predictor 126 determines the MV derived in Step SI_1 as the final MV for the current block (Step SI_4). In other words, in this case, the MV for the current block is determined according to the merge mode.

When determining to perform motion estimation in Step SI_1 (Yes in Step SI_2), inter predictor 126 derives the final MV for the current block by estimating a surrounding region of the reference picture specified by the MV derived in Step SI_1 (Step SI_3). In other words, in this case, the MV for the current block is determined according to the DMVR.

FIG. 56 is a conceptual diagram for illustrating another example of DMVR for determining an MV.

First, in the merge mode for example, MV candidates (L0 and L1) are selected for the current block. A reference pixel is identified from a first reference picture (L0) which is an encoded picture in the L0 list according to the MV candidate (L0). Likewise, a reference pixel is identified from a second reference picture (L1) which is an encoded picture in the L1 list according to the MV candidate (L1). A template is generated by calculating an average of these reference pixels.

Next, each of the surrounding regions of MV candidates of the first reference picture (L0) and the second reference picture (L1) are estimated using the template, and the MV which yields the smallest cost is determined to be the final MV. It is to be noted that the cost may be calculated, for example, using a difference value between each of the pixel values in the template and a corresponding one of the pixel values in the estimation region, the values of MV candidates, etc.

Exactly the same processes described here do not always need to be performed. Any process for enabling derivation of the final MV by estimation in surrounding regions of MV candidates may be used.

FIG. 57 is a conceptual diagram for illustrating another example of DMVR for determining an MV. Unlike the example of DMVR illustrated in FIG. 56, in the example illustrated in FIG. 57, costs are calculated without generating any template.

52

First, inter predictor 126 estimates a surrounding region of a reference block included in each of reference pictures in the L0 list and L1 list, based on an initial MV which is an MV candidate obtained from each MV candidate list. For example, as illustrated in FIG. 57, the initial MV corresponding to the reference block in the L0 list is InitMV_L0, and the initial MV corresponding to the reference block in the L1 list is InitMV_L1. In motion estimation, inter predictor 126 firstly sets a search position for the reference picture in the L0 list. Based on the position indicated by the vector difference indicating the search position to be set, specifically, the initial MV (that is, InitMV_L0), the vector difference to the search position is MVd_L0. Inter predictor 126 then determines the estimation position in the reference picture in the L1 list. This search position is indicated by the vector difference to the search position from the position indicated by the initial MV (that is, InitMV_L1). More specifically, inter predictor 126 determines the vector difference as MVd_L1 by mirroring of MVd_L0. In other words, inter predictor 126 determines the position which is symmetrical with respect to the position indicated by the initial MV to be the search position in each reference picture in the L0 list and the L1 list. Inter predictor 126 calculates, for each search position, the total sum of the absolute differences (SADs) between values of pixels at search positions in blocks as a cost, and finds out the search position that yields the smallest cost.

FIG. 58A is a diagram illustrating one example of motion estimation in DMVR, and FIG. 58B is a flow chart illustrating one example of the motion estimation.

First, in Step 1, inter predictor 126 calculates the cost between the search position (also referred to as a starting point) indicated by the initial MV and eight surrounding search positions. Inter predictor 126 then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at the search position other than the starting point is the smallest, inter predictor 126 changes a target to the search position at which the smallest cost is obtained, and performs the process in Step 2. When the cost at the starting point is the smallest, inter predictor 126 skips the process in Step 2 and performs the process in Step 3.

In Step 2, inter predictor 126 performs the search similar to the process in Step 1, regarding, as a new starting point, the search position after the target change according to the result of the process in Step 1. Inter predictor 126 then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at the search position other than the starting point is the smallest, inter predictor 126 performs the process in Step 4. When the cost at the starting point is the smallest, inter predictor 126 performs the process in Step 3.

In Step 4, inter predictor 126 regards the search position at the starting point as the final search position, and determines the difference between the position indicated by the initial MV and the final search position to be a vector difference.

In Step 3, inter predictor 126 determines the pixel position at sub-pixel accuracy at which the smallest cost is obtained, based on the costs at the four points located at upper, lower, left, and right positions with respect to the starting point in Step 1 or Step 2, and regards the pixel position as the final search position. The pixel position at the sub-pixel accuracy is determined by performing weighted addition of each of the four upper, lower, left, and right vectors ((0, 1), (0, -1), (-1, 0), and (1, 0)), using, as a weight, the cost at a

corresponding one of the four search positions. Inter predictor **126** then determines the difference between the position indicated by the initial MV and the final search position to be the vector difference.

[Motion Compensation>BIO/OBMC/LIC]

Motion compensation involves a mode for generating a prediction image, and correcting the prediction image. The mode is, for example, BIO, OBMC, and LIC to be described later.

FIG. **59** is a flow chart illustrating one example of generation of a prediction image.

Inter predictor **126** generates a prediction image (Step Sm_1), and corrects the prediction image according to any of the modes described above (Step Sm_2).

FIG. **60** is a flow chart illustrating another example of generation of a prediction image.

Inter predictor **126** derives an MV of a current block (Step Sn_1). Next, inter predictor **126** generates a prediction image using the MV (Step Sn_2), and determines whether to perform a correction process (Step Sn_3). Here, when determining to perform a correction process (Yes in Step Sn_3), inter predictor **126** generates the final prediction image by correcting the prediction image (Step Sn_4). It is to be noted that, in LIC described later, luminance and chrominance may be corrected in Step Sn_4. When determining not to perform a correction process (No in Step Sn_3), inter predictor **126** outputs the prediction image as the final prediction image without correcting the prediction image (Step Sn_5).

[Motion Compensation>OBMC]

It is to be noted that an inter prediction image may be generated using motion information for a neighboring block in addition to motion information for the current block obtained by motion estimation. More specifically, an inter prediction image may be generated for each sub-block in a current block by performing weighted addition of a prediction image based on the motion information obtained by motion estimation (in a reference picture) and a prediction image based on the motion information of the neighboring block (in the current picture). Such inter prediction (motion compensation) is also referred to as overlapped block motion compensation (OBMC) or an OBMC mode.

In OBMC mode, information indicating a sub-block size for OBMC (referred to as, for example, an OBMC block size) may be signaled at the sequence level. Moreover, information indicating whether to apply the OBMC mode (referred to as, for example, an OBMC flag) may be signaled at the CU level. It is to be noted that the signaling of such information does not necessarily need to be performed at the sequence level and CU level, and may be performed at another level (for example, at the picture level, slice level, brick level, CTU level, or sub-block level).

The OBMC mode will be described in further detail. FIGS. **61** and **62** are a flow chart and a conceptual diagram for illustrating an outline of a prediction image correction process performed by OBMC.

First, as illustrated in FIG. **62**, a prediction image (Pred) by normal motion compensation is obtained using an MV assigned to a current block. In FIG. **62**, the arrow "MV" points a reference picture, and indicates what the current block of the current picture refers to in order to obtain the prediction image.

Next, a prediction image (Pred_L) is obtained by applying a motion vector (MV_L) which has been already derived for the encoded block neighboring to the left of the current block to the current block (re-using the motion vector for the current block). The motion vector (MV_L) is indicated by an

arrow "MV_L" indicating a reference picture from a current block. A first correction of a prediction image is performed by overlapping two prediction images Pred and Pred_L. This provides an effect of blending the boundary between neighboring blocks.

Likewise, a prediction image (Pred_U) is obtained by applying an MV (MV_U) which has been already derived for the encoded block neighboring above the current block to the current block (re-using the MV for the current block). The MV (MV_U) is indicated by an arrow "MV_U" indicating a reference picture from a current block. A second correction of a prediction image is performed by overlapping the prediction image Pred_U to the prediction images (for example, Pred and Pred_L) on which the first correction has been performed. This provides an effect of blending the boundary between neighboring blocks. The prediction image obtained by the second correction is the one in which the boundary between the neighboring blocks has been blended (smoothed), and thus is the final prediction image of the current block.

Although the above example is a two-path correction method using left and upper neighboring blocks, it is to be noted that the correction method may be three- or more-path correction method using also the right neighboring block and/or the lower neighboring block.

It is to be noted that the region in which such overlapping is performed may be only part of a region near a block boundary instead of the pixel region of the entire block.

It is to be noted that the prediction image correction process according to OBMC for obtaining one prediction image Pred from one reference picture by overlapping additional prediction images Pred_L and Pred_U has been described above. However, when a prediction image is corrected based on a plurality of reference images, a similar process may be applied to each of the plurality of reference pictures. In such a case, after corrected prediction images are obtained from the respective reference pictures by performing OBMC image correction based on the plurality of reference pictures, the obtained corrected prediction images are further overlapped to obtain the final prediction image.

It is to be noted that, in OBMC, a current block unit may be a PU or a sub-block unit obtained by further splitting the PU.

One example of a method for determining whether to apply OBMC is a method for using an obmc_flag which is a signal indicating whether to apply OBMC. As one specific example, encoder **100** may determine whether the current block belongs to a region having complicated motion. Encoder **100** sets the obmc_flag to a value of "1" when the block belongs to a region having complicated motion and applies OBMC when encoding, and sets the obmc_flag to a value of "0" when the block does not belong to a region having complicated motion and encodes the block without applying OBMC. Decoder **200** switches between application and non-application of OBMC by decoding the obmc_flag written in a stream.

[Motion Compensation>BIO]

Next, an MV derivation method is described. First, a mode for deriving an MV based on a model assuming uniform linear motion is described. This mode is also referred to as a bi-directional optical flow (BIO) mode. In addition, this bi-directional optical flow may be written as BDOF instead of BIO.

FIG. **63** is a diagram for illustrating a model assuming uniform linear motion. In FIG. **63**, (vx, vy) indicates a velocity vector, and τ_0 and τ_1 indicate temporal distances between a current picture (Cur Pic) and two reference

55

pictures (Ref0, Ref1). (MVx0, MVy0) indicates an MV corresponding to reference picture Ref0, and (MVx1, MVy1) indicates an MV corresponding to reference picture Ref1.

Here, under the assumption of uniform linear motion exhibited by a velocity vector (vx, vy), (MVx0, MVy0) and (MVx1, MVy1) are represented as (vxτ0, vyτ0) and (−vxτ1, −vyτ1), respectively, and the following optical flow equation (2) is given.

[MATH 3]

$$\partial I^{(k)} / \partial t + v_x \partial I^{(k)} / \partial x + v_y \partial I^{(k)} / \partial y = 0 \quad (2)$$

Here, I(k) denotes a luma value from reference image k (k=0, 1) after motion compensation. This optical flow equation shows that the sum of (i) the time derivative of the luma value, (ii) the product of the horizontal velocity and the horizontal component of the spatial gradient of a reference image, and (iii) the product of the vertical velocity and the vertical component of the spatial gradient of a reference image is equal to zero. A motion vector of each block obtained from, for example, an MV candidate list may be corrected in units of a pixel, based on a combination of the optical flow equation and Hermite interpolation.

It is to be noted that a motion vector may be derived on the decoder 200 side using a method other than deriving a motion vector based on a model assuming uniform linear motion. For example, a motion vector may be derived in units of a sub-block based on MVs of a plurality of neighboring blocks.

FIG. 64 is a flow chart illustrating one example of inter prediction according to BIO. FIG. 65 is a diagram illustrating one example of a functional configuration of inter predictor 126 which performs inter prediction according to BIO.

As illustrated in FIG. 65, inter predictor 126 includes, for example, memory 126a, interpolated image deriver 126b, gradient image deriver 126c, optical flow deriver 126d, correction value deriver 126e, and prediction image corrector 126f. It is to be noted that memory 126a may be frame memory 122.

Inter predictor 126 derives two motion vectors (M0, M1), using two reference pictures (Ref0, Ref1) different from the picture (Cur Pic) including a current block. Inter predictor 126 then derives a prediction image for the current block using the two motion vectors (M0, M1) (Step Sy_1). It is to be noted that motion vector M0 is motion vector (MVx0, MVy0) corresponding to reference picture Ref0, and motion vector M1 is motion vector (MVx1, MVy1) corresponding to reference picture Ref1.

Next, interpolated image deriver 126b derives interpolated image I⁰ for the current block, using motion vector M0 and reference picture L0 by referring to memory 126a. Next, interpolated image deriver 126b derives interpolated image I¹ for the current block, using motion vector M1 and reference picture L1 by referring to memory 126a (Step Sy_2). Here, interpolated image I⁰ is an image included in reference picture Ref0 and to be derived for the current block, and interpolated image I¹ is an image included in reference picture Ref1 and to be derived for the current block. Each of interpolated image I⁰ and interpolated image I¹ may be the same in size as the current block. Alternatively, each of interpolated image I⁰ and interpolated image I¹ may be an image larger than the current block. Furthermore, interpolated image I⁰ and interpolated image I¹ may include a prediction image obtained by using motion vectors (M0, M1) and reference pictures (L0, L1) and applying a motion compensation filter.

56

In addition, gradient image deriver 126c derives gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹) of the current block, from interpolated image I⁰ and interpolated image I¹ (Step Sy_3). It is to be noted that the gradient images in the horizontal direction are (Ix⁰, Ix¹), and the gradient images in the vertical direction are (Iy⁰, Iy¹). Gradient image deriver 126c may derive each gradient image by, for example, applying a gradient filter to the interpolated images. It is only necessary that a gradient image indicate the amount of spatial change in pixel value along the horizontal direction or the vertical direction.

Next, optical flow deriver 126d derives, for each sub-block of the current block, an optical flow (vx, vy) which is a velocity vector, using the interpolated images (I⁰, I¹) and the gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹) (Step Sy_4). The optical flow indicates coefficients for correcting the amount of spatial pixel movement, and may be referred to as a local motion estimation value, a corrected motion vector, or a corrected weighting vector. As one example, a sub-block may be 4×4 pixel sub-CU. It is to be noted that the optical flow derivation may be performed for each pixel unit, or the like, instead of being performed for each sub-block.

Next, inter predictor 126 corrects a prediction image for the current block using the optical flow (vx, vy). For example, correction value deriver 126e derives a correction value for the value of a pixel included in a current block, using the optical flow (vx, vy) (Step Sy_5). Prediction image corrector 126f may then correct the prediction image for the current block using the correction value (Step Sy_6). It is to be noted that the correction value may be derived in units of a pixel, or may be derived in units of a plurality of pixels or in units of a sub-block.

It is to be noted that the BIO process flow is not limited to the process disclosed in FIG. 64. Only part of the processes disclosed in FIG. 64 may be performed, or a different process may be added or used as a replacement, or the processes may be executed in a different processing order.

[Motion Compensation>LIC]

Next, one example of a mode for generating a prediction image (prediction) using a local illumination compensation (LIC) is described.

FIG. 66A is a diagram for illustrating one example of a prediction image generation method using a luminance correction process performed by LIC. FIG. 66B is a flow chart illustrating one example of a prediction image generation method using the LIC.

First, inter predictor 126 derives an MV from an encoded reference picture, and obtains a reference image corresponding to the current block (Step Sz_1).

Next, inter predictor 126 extracts, for the current block, information indicating how the luma value has changed between the current block and the reference picture (Step Sz_2). This extraction is performed based on the luma pixel values of the encoded left neighboring reference region (surrounding reference region) and the encoded upper neighboring reference region (surrounding reference region) in the current picture, and the luma pixel values at the corresponding positions in the reference picture specified by the derived MVs. Inter predictor 126 calculates a luminance correction parameter, using the information indicating how the luma value has changed (Step Sz_3).

Inter predictor 126 generates a prediction image for the current block by performing a luminance correction process in which the luminance correction parameter is applied to the reference image in the reference picture specified by the MV (Step Sz_4). In other words, the prediction image which

57

is the reference image in the reference picture specified by the MV is subjected to the correction based on the luminance correction parameter. In this correction, luminance may be corrected, or chrominance may be corrected. In other words, a chrominance correction parameter may be calculated using information indicating how chrominance has changed, and a chrominance correction process may be performed.

It is to be noted that the shape of the surrounding reference region illustrated in FIG. 66A is one example; another shape may be used.

Moreover, although the process in which a prediction image is generated from a single reference picture has been described here, cases in which a prediction image is generated from a plurality of reference pictures can be described in the same manner. The prediction image may be generated after performing a luminance correction process of the reference images obtained from the reference pictures in the same manner as described above.

One example of a method for determining whether to apply LIC is a method for using a `lic_flag` which is a signal indicating whether to apply the LIC. As one specific example, encoder 100 determines whether the current block belongs to a region having a luminance change. Encoder 100 sets the `lic_flag` to a value of "1" when the block belongs to a region having a luminance change and applies LIC when encoding, and sets the `lic_flag` to a value of "0" when the block does not belong to a region having a luminance change and performs encoding without applying LIC. Decoder 200 may decode the `lic_flag` written in the stream and decode the current block by switching between application and non-application of LIC in accordance with the flag value.

One example of a different method of determining whether to apply a LIC process is a determining method in accordance with whether a LIC process has been applied to a surrounding block. As one specific example, when a current block has been processed in merge mode, inter predictor 126 determines whether an encoded surrounding block selected in MV derivation in merge mode has been encoded using LIC. Inter predictor 126 performs encoding by switching between application and non-application of LIC according to the result. It is to be noted that, also in this example, the same processes are applied to processes at the decoder 200 side.

The luminance correction (LIC) process has been described with reference to FIGS. 66A and 66B, and is further described below.

First, inter predictor 126 derives an MV for obtaining a reference image corresponding to a current block from a reference picture which is an encoded picture.

Next, inter predictor 126 extracts information indicating how the luma value of the reference picture has been changed to the luma value of the current picture, using the luma pixel values of encoded surrounding reference regions which neighbor to the left of and above the current block and the luma pixel values in the corresponding positions in the reference pictures specified by MVs, and calculates a luminance correction parameter. For example, it is assumed that the luma pixel value of a given pixel in the surrounding reference region in the current picture is p_0 , and that the luma pixel value of the pixel corresponding to the given pixel in the surrounding reference region in the reference picture is p_1 . Inter predictor 126 calculates coefficients A and B for optimizing $A \times p_1 + B = p_0$ as the luminance correction parameter for a plurality of pixels in the surrounding reference region.

Next, inter predictor 126 performs a luminance correction process using the luminance correction parameter for the

58

reference image in the reference picture specified by the MV, to generate a prediction image for the current block. For example, it is assumed that the luma pixel value in the reference image is p_2 , and that the luminance-corrected luma pixel value of the prediction image is p_3 . Inter predictor 126 generates the prediction image after being subjected to the luminance correction process by calculating $A \times p_2 + B = p_3$ for each of the pixels in the reference image.

For example, a region having a determined number of pixels extracted from each of an upper neighboring pixel and a left neighboring pixel may be used as a surrounding reference region. In addition, the surrounding reference region is not limited to a region which neighbors the current block, and may be a region which does not neighbor the current block. In the example illustrated in FIG. 66A, the surrounding reference region in the reference picture may be a region specified by another MV in a current picture, from a surrounding reference region in the current picture. For example, the other MV may be an MV in a surrounding reference region in the current picture.

Although operations performed by encoder 100 have been described here, it is to be noted that decoder 200 performs similar operations.

It is to be noted that LIC may be applied not only to luma but also to chroma. At this time, a correction parameter may be derived individually for each of Y, Cb, and Cr, or a common correction parameter may be used for any of Y, Cb, and Cr.

In addition, the LIC process may be applied in units of a sub-block. For example, a correction parameter may be derived using a surrounding reference region in a current sub-block and a surrounding reference region in a reference sub-block in a reference picture specified by an MV of the current sub-block.

[Prediction Controller]

Prediction controller 128 selects one of an intra prediction image (an image or a signal output from intra predictor 124) and an inter prediction image (an image or a signal output from inter predictor 126), and outputs the selected prediction image to subtractor 104 and adder 116.

[Prediction Parameter Generator]

Prediction parameter generator 130 may output information related to intra prediction, inter prediction, selection of a prediction image in prediction controller 128, etc. as a prediction parameter to entropy encoder 110. Entropy encoder 110 may generate a stream, based on the prediction parameter which is input from prediction parameter generator 130 and quantized coefficients which are input from quantizer 108. The prediction parameter may be used in decoder 200. Decoder 200 may receive and decode the stream, and perform the same processes as the prediction processes performed by intra predictor 124, inter predictor 126, and prediction controller 128. The prediction parameter may include (i) a selection prediction signal (for example, an MV, a prediction type, or a prediction mode used by intra predictor 124 or inter predictor 126), or (ii) an optional index, a flag, or a value which is based on a prediction process performed in each of intra predictor 124, inter predictor 126, and prediction controller 128, or which indicates the prediction process.

[Decoder]

Next, decoder 200 capable of decoding a stream output from encoder 100 described above is described. FIG. 67 is a block diagram illustrating a functional configuration of decoder 200 according to this embodiment. Decoder 200 is an apparatus which decodes a stream that is an encoded image in units of a block.

59

As illustrated in FIG. 67, decoder 200 includes entropy decoder 202, inverse quantizer 204, inverse transformer 206, adder 208, block memory 210, loop filter 212, frame memory 214, intra predictor 216, inter predictor 218, prediction controller 220, prediction parameter generator 222, and splitting determiner 224. It is to be noted that intra predictor 216 and inter predictor 218 are configured as part of a prediction executor.

[Mounting Example of Decoder]

FIG. 68 is a block diagram illustrating a mounting example of decoder 200. Decoder 200 includes processor b1 and memory b2. For example, the plurality of constituent elements of decoder 200 illustrated in FIG. 67 are mounted on processor b1 and memory b2 illustrated in FIG. 68.

Processor b1 is circuitry which performs information processing and is accessible to memory b2. For example, processor b1 is a dedicated or general electronic circuit which decodes a stream. Processor b1 may be a processor such as a CPU. In addition, processor b1 may be an aggregate of a plurality of electronic circuits. In addition, for example, processor b1 may take the roles of two or more constituent elements other than a constituent element for storing information out of the plurality of constituent elements of decoder 200 illustrated in FIG. 67, etc.

Memory b2 is dedicated or general memory for storing information that is used by processor b1 to decode a stream. Memory b2 may be electronic circuitry, and may be connected to processor b1. In addition, memory b2 may be included in processor b1. In addition, memory b2 may be an aggregate of a plurality of electronic circuits. In addition, memory b2 may be a magnetic disc, an optical disc, or the like, or may be represented as a storage, a recording medium, or the like. In addition, memory b2 may be non-volatile memory, or volatile memory.

For example, memory b2 may store an image or a stream. In addition, memory b2 may store a program for causing processor b1 to decode a stream.

In addition, for example, memory b2 may take the roles of two or more constituent elements for storing information out of the plurality of constituent elements of decoder 200 illustrated in FIG. 67, etc. More specifically, memory b2 may take the roles of block memory 210 and frame memory 214 illustrated in FIG. 67. More specifically, memory b2 may store a reconstructed image (specifically, a reconstructed block, a reconstructed picture, or the like).

It is to be noted that, in decoder 200, not all of the plurality of constituent elements illustrated in FIG. 67, etc. may be implemented, and not all the processes described above may be performed. Part of the constituent elements indicated in FIG. 67, etc. may be included in another device, or part of the processes described above may be performed by another device.

Hereinafter, an overall flow of the processes performed by decoder 200 is described, and then each of the constituent elements included in decoder 200 is described. It is to be noted that, some of the constituent elements included in decoder 200 perform the same processes as performed by some of the constituent elements included in encoder 100, and thus the same processes are not repeatedly described in detail. For example, inverse quantizer 204, inverse transformer 206, adder 208, block memory 210, frame memory 214, intra predictor 216, inter predictor 218, prediction controller 220, and loop filter 212 included in decoder 200 perform similar processes as performed by inverse quantizer 112, inverse transformer 114, adder 116, block memory 118,

60

frame memory 122, intra predictor 124, inter predictor 126, prediction controller 128, and loop filter 120 included in encoder 100, respectively.

[Overall Flow of Decoding Process]

FIG. 69 is a flow chart illustrating one example of an overall decoding process performed by decoder 200.

First, splitting determiner 224 in decoder 200 determines a splitting pattern of each of a plurality of fixed-size blocks (128×128 pixels) included in a picture, based on a parameter which is input from entropy decoder 202 (Step Sp_1). This splitting pattern is a splitting pattern selected by encoder 100. Decoder 200 then performs processes of Steps Sp_2 to Sp_6 for each of a plurality of blocks of the splitting pattern.

Entropy decoder 202 decodes (specifically, entropy decodes) encoded quantized coefficients and a prediction parameter of a current block (Step Sp_2).

Next, inverse quantizer 204 performs inverse quantization of the plurality of quantized coefficients and inverse transformer 206 performs inverse transform of the result, to restore prediction residuals of the current block (Step Sp_3).

Next, the prediction executor including all or part of intra predictor 216, inter predictor 218, and prediction controller 220 generates a prediction image of the current block (Step Sp_4).

Next, adder 208 adds the prediction image to a prediction residual to generate a reconstructed image (also referred to as a decoded image block) of the current block (Step Sp_5).

When the reconstructed image is generated, loop filter 212 performs filtering of the reconstructed image (Step Sp_6).

Decoder 200 then determines whether decoding of the entire picture has been finished (Step Sp_7). When determining that the decoding has not yet been finished (No in Step Sp_7), decoder 200 repeatedly executes the processes starting with Step Sp_1.

It is to be noted that the processes of these Steps Sp_1 to Sp_7 may be performed sequentially by decoder 200, or two or more of the processes may be performed in parallel. The processing order of the two or more of the processes may be modified.

[Splitting Determiner]

FIG. 70 is a diagram illustrating a relationship between splitting determiner 224 and other constituent elements. Splitting determiner 224 may perform the following processes as examples.

For example, splitting determiner 224 collects block information from block memory 210 or frame memory 214, and furthermore obtains a parameter from entropy decoder 202. Splitting determiner 224 may then determine the splitting pattern of a fixed-size block, based on the block information and the parameter. Splitting determiner 224 may then output information indicating the determined splitting pattern to inverse transformer 206, intra predictor 216, and inter predictor 218. Inverse transformer 206 may perform inverse transform of transform coefficients, based on the splitting pattern indicated by the information from splitting determiner 224. Intra predictor 216 and inter predictor 218 may generate a prediction image, based on the splitting pattern indicated by the information from splitting determiner 224.

[Entropy Decoder]

FIG. 71 is a block diagram illustrating one example of a functional configuration of entropy decoder 202.

Entropy decoder 202 generates quantized coefficients, a prediction parameter, and a parameter related to a splitting pattern, by entropy decoding the stream. For example, CABAC is used in the entropy decoding. More specifically,

61

entropy decoder **202** includes, for example, binary arithmetic decoder **202a**, context controller **202b**, and debinarizer **202c**. Binary arithmetic decoder **202a** arithmetically decodes the stream using a context value derived by context controller **202b** to a binary signal. Context controller **202b** derives a context value according to a feature or a surrounding state of a syntax element, that is, an occurrence probability of a binary signal, in the same manner as performed by context controller **110b** of encoder **100**. Debinarizer **202c** performs debinarization for transforming the binary signal output from binary arithmetic decoder **202a** to a multi-level signal indicating quantized coefficients as described above. This binarization is performed according to the binarization method described above.

With this, entropy decoder **202** outputs quantized coefficients of each block to inverse quantizer **204**. Entropy decoder **202** may output a prediction parameter included in a stream (see FIG. 1) to intra predictor **216**, inter predictor **218**, and prediction controller **220**. Intra predictor **216**, inter predictor **218**, and prediction controller **220** are capable of executing the same prediction processes as those performed by intra predictor **124**, inter predictor **126**, and prediction controller **128** at the encoder **100** side.

[Entropy Decoder]

FIG. 72 is a diagram illustrating a flow of CABAC in entropy decoder **202**.

First, initialization is performed in CABAC in entropy decoder **202**. In the initialization, initialization in binary arithmetic decoder **202a** and setting of an initial context value are performed. Binary arithmetic decoder **202a** and debinarizer **202c** then execute arithmetic decoding and debinarization of, for example, encoded data of a CTU. At this time, context controller **202b** updates the context value each time arithmetic decoding is performed. Context controller **202b** then saves the context value as a post process. The saved context value is used, for example, to initialize the context value for the next CTU.

[Inverse Quantizer]

Inverse quantizer **204** inverse quantizes quantized coefficients of a current block which are inputs from entropy decoder **202**. More specifically, inverse quantizer **204** inverse quantizes the quantized coefficients of the current block, based on quantization parameters corresponding to the quantized coefficients. Inverse quantizer **204** then outputs the inverse quantized transform coefficients (that are transform coefficients) of the current block to inverse transformer **206**.

FIG. 73 is a block diagram illustrating one example of a functional configuration of inverse quantizer **204**.

Inverse quantizer **204** includes, for example, quantization parameter generator **204a**, predicted quantization parameter generator **204b**, quantization parameter storage **204d**, and inverse quantization executor **204e**.

FIG. 74 is a flow chart illustrating one example of inverse quantization performed by inverse quantizer **204**.

Inverse quantizer **204** may perform an inverse quantization process as one example for each CU based on the flow illustrated in FIG. 74. More specifically, quantization parameter generator **204a** determines whether to perform inverse quantization (Step Sv_11). Here, when determining to perform inverse quantization (Yes in Step Sv_11), quantization parameter generator **204a** obtains a difference quantization parameter for the current block from entropy decoder **202** (Step Sv_12).

Next, predicted quantization parameter generator **204b** then obtains a quantization parameter for a processing unit different from the current block from quantization parameter

62

storage **204d** (Step Sv_13). Predicted quantization parameter generator **204b** generates a predicted quantization parameter of the current block based on the obtained quantization parameter (Step Sv_14).

Quantization parameter generator **204a** then adds the difference quantization parameter for the current block obtained from entropy decoder **202** and the predicted quantization parameter for the current block generated by predicted quantization parameter generator **204b** (Step Sv_15). This addition generates a quantization parameter for the current block. In addition, quantization parameter generator **204a** stores the quantization parameter for the current block in quantization parameter storage **204d** (Step Sv_16).

Next, inverse quantization executor **204e** inverse quantizes the quantized coefficients of the current block into transform coefficients, using the quantization parameter generated in Step Sv_15 (Step Sv_17).

It is to be noted that the difference quantization parameter may be decoded at the bit sequence level, picture level, slice level, brick level, or CTU level. In addition, the initial value of the quantization parameter may be decoded at the sequence level, picture level, slice level, brick level, or CTU level. At this time, the quantization parameter may be generated using the initial value of the quantization parameter and the difference quantization parameter.

It is to be noted that inverse quantizer **204** may include a plurality of inverse quantizers, and may inverse quantize the quantized coefficients using an inverse quantization method selected from a plurality of inverse quantization methods.

[Inverse Transformer]

Inverse transformer **206** restores prediction residuals by inverse transforming the transform coefficients which are inputs from inverse quantizer **204**.

For example, when information parsed from a stream indicates that EMT or AMT is to be applied (for example, when an AMT flag is true), inverse transformer **206** inverse transforms the transform coefficients of the current block based on information indicating the parsed transform type.

Moreover, for example, when information parsed from a stream indicates that NSST is to be applied, inverse transformer **206** applies a secondary inverse transform to the transform coefficients.

FIG. 75 is a flow chart illustrating one example of a process performed by inverse transformer **206**.

For example, inverse transformer **206** determines whether information indicating that no orthogonal transform is performed is present in a stream (Step St_11). Here, when determining that no such information is present (No in Step St_11), inverse transformer **206** obtains information indicating the transform type decoded by entropy decoder **202** (Step St_12). Next, based on the information, inverse transformer **206** determines the transform type used for the orthogonal transform in encoder **100** (Step St_13). Inverse transformer **206** then performs inverse orthogonal transform using the determined transform type (Step St_14).

FIG. 76 is a flow chart illustrating another example of a process performed by inverse transformer **206**.

For example, inverse transformer **206** determines whether a transform size is smaller than or equal to a predetermined value (Step Su_11). Here, when determining that the transform size is smaller than or equal to a predetermined value (Yes in Step Su_11), inverse transformer **206** obtains, from entropy decoder **202**, information indicating which transform type has been used by encoder **100** among at least one transform type included in the first transform type group

(Step Su₁₂). It is to be noted that such information is decoded by entropy decoder 202 and output to inverse transformer 206.

Based on the information, inverse transformer 206 determines the transform type used for the orthogonal transform in encoder 100 (Step Su₁₃). Inverse transformer 206 then inverse orthogonal transforms the transform coefficients of the current block using the determined transform type (Step Su₁₄). When determining that a transform size is not smaller than or equal to the predetermined value (No in Step Su₁₁), inverse transformer 206 inverse transforms the transform coefficients of the current block using the second transform type group (Step Su₁₅).

It is to be noted that the inverse orthogonal transform by inverse transformer 206 may be performed according to the flow illustrated in FIG. 75 or FIG. 76 for each TU as one example. In addition, inverse orthogonal transform may be performed by using a predefined transform type without decoding information indicating a transform type used for orthogonal transform. In addition, the transform type is specifically DST7, DCT8, or the like. In inverse orthogonal transform, an inverse transform basis function corresponding to the transform type is used.

[Adder]

Adder 208 reconstructs the current block by adding a prediction residual which is an input from inverse transformer 206 and a prediction image which is an input from prediction controller 220. In other words, a reconstructed image of the current block is generated. Adder 208 then outputs the reconstructed image of the current block to block memory 210 and loop filter 212.

[Block Memory]

Block memory 210 is storage for storing a block which is included in a current picture and is referred to in intra prediction. More specifically, block memory 210 stores a reconstructed image output from adder 208.

[Loop Filter]

Loop filter 212 applies a loop filter to the reconstructed image generated by adder 208, and outputs the filtered reconstructed image to frame memory 214 and a display device, etc.

When information indicating ON or OFF of an ALF parsed from a stream indicates that an ALF is ON, one filter from among a plurality of filters is selected based on the direction and activity of local gradients, and the selected filter is applied to the reconstructed image.

FIG. 77 is a block diagram illustrating one example of a functional configuration of loop filter 212. It is to be noted that loop filter 212 has a configuration similar to the configuration of loop filter 120 of encoder 100.

For example, as illustrated in FIG. 77, loop filter 212 includes deblocking filter executor 212a, SAO executor 212b, and ALF executor 212c. Deblocking filter executor 212a performs a deblocking filter process of the reconstructed image. SAO executor 212b performs a SAO process of the reconstructed image after being subjected to the deblocking filter process. ALF executor 212c performs an ALF process of the reconstructed image after being subjected to the SAO process. It is to be noted that loop filter 212 does not always need to include all the constituent elements disclosed in FIG. 77, and may include only part of the constituent elements. In addition, loop filter 212 may be configured to perform the above processes in a processing order different from the one disclosed in FIG. 77.

[Frame Memory]

Frame memory 214 is, for example, storage for storing reference pictures for use in inter prediction, and is also

referred to as a frame buffer. More specifically, frame memory 214 stores a reconstructed image filtered by loop filter 212.

[Predictor (Intra Predictor, Inter Predictor, Prediction Controller)]

FIG. 78 is a flow chart illustrating one example of a process performed by a predictor of decoder 200. It is to be noted that the prediction executor includes all or part of the following constituent elements: intra predictor 216; inter predictor 218; and prediction controller 220. The prediction executor includes, for example, intra predictor 216 and inter predictor 218.

The predictor generates a prediction image of a current block (Step Sq₁). This prediction image is also referred to as a prediction signal or a prediction block. It is to be noted that the prediction signal is, for example, an intra prediction signal or an inter prediction signal. More specifically, the predictor generates the prediction image of the current block using a reconstructed image which has been already obtained for another block through generation of a prediction image, restoration of a prediction residual, and addition of a prediction image. The predictor of decoder 200 generates the same prediction image as the prediction image generated by the predictor of encoder 100. In other words, the prediction images are generated according to a method common between the predictors or mutually corresponding methods.

The reconstructed image may be, for example, an image in a reference picture, or an image of a decoded block (that is, the other block described above) in a current picture which is the picture including the current block. The decoded block in the current picture is, for example, a neighboring block of the current block.

FIG. 79 is a flow chart illustrating another example of a process performed by the predictor of decoder 200.

The predictor determines either a method or a mode for generating a prediction image (Step Sr₁). For example, the method or mode may be determined based on, for example, a prediction parameter, etc.

When determining a first method as a mode for generating a prediction image, the predictor generates a prediction image according to the first method (Step Sr_{2a}). When determining a second method as a mode for generating a prediction image, the predictor generates a prediction image according to the second method (Step Sr_{2b}). When determining a third method as a mode for generating a prediction image, the predictor generates a prediction image according to the third method (Step Sr_{2c}).

The first method, the second method, and the third method may be mutually different methods for generating a prediction image. Each of the first to third methods may be an inter prediction method, an intra prediction method, or another prediction method. The above-described reconstructed image may be used in these prediction methods.

FIG. 80A and FIG. 80B illustrate a flow chart illustrating another example of a process performed by a predictor of decoder 200.

The predictor may perform a prediction process according to the flow illustrated in FIG. 80A and FIG. 80B as one example. It is to be noted that intra block copy illustrated in FIG. 80A and FIG. 80B is one mode which belongs to inter prediction, and in which a block included in a current picture is referred to as a reference image or a reference block. In other words, no picture different from the current picture is referred to in intra block copy. In addition, the PCM mode

illustrated in FIG. 80A is one mode which belongs to intra prediction, and in which no transform and quantization is performed.

[Intra Predictor]

Intra predictor **216** performs intra prediction by referring to a block in a current picture stored in block memory **210**, based on the intra prediction mode parsed from the stream, to generate a prediction image of a current block (that is, an intra prediction image). More specifically, intra predictor **216** performs intra prediction by referring to pixel values (for example, luma and/or chroma values) of a block or blocks neighboring the current block to generate an intra prediction image, and then outputs the intra prediction image to prediction controller **220**.

It is to be noted that when an intra prediction mode in which a luma block is referred to in intra prediction of a chroma block is selected, intra predictor **216** may predict the chroma component of the current block based on the luma component of the current block.

Moreover, when information parsed from a stream indicates that PDPC is to be applied, intra predictor **216** corrects intra predicted pixel values based on horizontal/vertical reference pixel gradients.

FIG. 81 is a diagram illustrating one example of a process performed by intra predictor **216** of decoder **200**.

Intra predictor **216** firstly determines whether an MPM flag indicating 1 is present in the stream (Step Sw_11). Here, when determining that the MPM flag indicating 1 is present (Yes in Step Sw_11), intra predictor **216** obtains, from entropy decoder **202**, information indicating the intra prediction mode selected in encoder **100** among MPMs (Step Sw_12). It is to be noted that such information is decoded by entropy decoder **202** and output to intra predictor **216**. Next, intra predictor **216** determines an MPM (Step Sw_13). MPMs include, for example, six intra prediction modes. Intra predictor **216** then determines the intra prediction mode which is included in a plurality of intra prediction modes included in the MPMs and is indicated by the information obtained in Step Sw_12 (Step Sw_14).

When determining that no MPM flag indicating 1 is present (No in Step Sw_11), intra predictor **216** obtains information indicating the intra prediction mode selected in encoder **100** (Step Sw_15). In other words, intra predictor **216** obtains, from entropy decoder **202**, information indicating the intra prediction mode selected in encoder **100** from among at least one intra prediction mode which is not included in the MPMs. It is to be noted that such information is decoded by entropy decoder **202** and output to intra predictor **216**. Intra predictor **216** then determines the intra prediction mode which is not included in a plurality of intra prediction modes included in the MPMs and is indicated by the information obtained in Step Sw_15 (Step Sw_17).

Intra predictor **216** generates a prediction image according to the intra prediction mode determined in Step Sw_14 or Step Sw_17 (Step Sw_18).

[Inter Predictor]

Inter predictor **218** predicts the current block by referring to a reference picture stored in frame memory **214**. Prediction is performed in units of a current block or a current sub-block in the current block. It is to be noted that the sub-block is included in the block and is a unit smaller than the block. The size of the sub-block may be 4×4 pixels, 8×8 pixels, or another size. The size of the sub-block may be switched for a unit such as a slice, brick, picture, etc.

For example, inter predictor **218** generates an inter prediction image of a current block or a current sub-block by performing motion compensation using motion information

(for example, an MV) parsed from a stream (for example, a prediction parameter output from entropy decoder **202**), and outputs the inter prediction image to prediction controller **220**.

When the information parsed from the stream indicates that the OBMC mode is to be applied, inter predictor **218** generates the inter prediction image using motion information of a neighboring block in addition to motion information of the current block obtained through motion estimation.

Moreover, when the information parsed from the stream indicates that the FRUC mode is to be applied, inter predictor **218** derives motion information by performing motion estimation in accordance with the pattern matching method (bilateral matching or template matching) parsed from the stream. Inter predictor **218** then performs motion compensation (prediction) using the derived motion information.

Moreover, when the BIO mode is to be applied, inter predictor **218** derives an MV based on a model assuming uniform linear motion. In addition, when the information parsed from the stream indicates that the affine mode is to be applied, inter predictor **218** derives an MV for each sub-block, based on the MVs of a plurality of neighboring blocks.

[MV Derivation Flow]

FIG. 82 is a flow chart illustrating one example of MV derivation in decoder **200**.

Inter predictor **218** determines, for example, whether to decode motion information (for example, an MV). For example, inter predictor **218** may make the determination according to the prediction mode included in the stream, or may make the determination based on other information included in the stream. Here, when determining to decode motion information, inter predictor **218** derives an MV for a current block in a mode in which the motion information is decoded. When determining not to decode motion information, inter predictor **218** derives an MV in a mode in which no motion information is decoded.

Here, MV derivation modes include a normal inter mode, a normal merge mode, a FRUC mode, an affine mode, etc. which are described later. Modes in which motion information is decoded among the modes include the normal inter mode, the normal merge mode, the affine mode (specifically, an affine inter mode and an affine merge mode), etc. It is to be noted that motion information may include not only an MV but also MV predictor selection information which is described later. Modes in which no motion information is decoded include the FRUC mode, etc. Inter predictor **218** selects a mode for deriving an MV for the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

FIG. 83 is a flow chart illustrating another example of MV derivation in decoder **200**.

For example, inter predictor **218** may determine whether to decode an MV difference, that is for example, may make the determination according to the prediction mode included in the stream, or may make the determination based on other information included in the stream. Here, when determining to decode an MV difference, inter predictor **218** may derive an MV for a current block in a mode in which the MV difference is decoded. In this case, for example, the MV difference included in the stream is decoded as a prediction parameter.

When determining not to decode any MV difference, inter predictor **218** derives an MV in a mode in which no MV difference is decoded. In this case, no encoded MV difference is included in the stream.

67

Here, as described above, the MV derivation modes include the normal inter mode, the normal merge mode, the FRUC mode, the affine mode, etc. which are described later. Modes in which an MV difference is encoded among the modes include the normal inter mode and the affine mode (specifically, the affine inter mode), etc. Modes in which no MV difference is encoded include the FRUC mode, the normal merge mode, the affine mode (specifically, the affine merge mode), etc. Inter predictor **218** selects a mode for deriving an MV for the current block from the plurality of modes, and derives the MV for the current block using the selected mode.

[MV Derivation>Normal Inter Mode]

For example, when information parsed from a stream indicates that the normal inter mode is to be applied, inter predictor **218** derives an MV based on the information parsed from the stream and performs motion compensation (prediction) using the MV.

FIG. **84** is a flow chart illustrating an example of inter prediction by normal inter mode in decoder **200**.

Inter predictor **218** of decoder **200** performs motion compensation for each block. At this time, first, inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Sg_11). In other words, inter predictor **218** generates an MV candidate list.

Next, inter predictor **218** extracts N (an integer of 2 or larger) MV candidates from the plurality of MV candidates obtained in Step Sg_11, as motion vector predictor candidates (also referred to as MV predictor candidates) according to the predetermined ranks in priority order (Step Sg_12). It is to be noted that the ranks in priority order are determined in advance for the respective N MV predictor candidates.

Next, inter predictor **218** decodes the MV predictor selection information from the input stream, and selects one MV predictor candidate from the N MV predictor candidates as the MV predictor for the current block using the decoded MV predictor selection information (Step Sg_13).

Next, inter predictor **218** decodes an MV difference from the input stream, and derives an MV for the current block by adding a difference value which is the decoded MV difference and the selected MV predictor (Step Sg_14).

Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Sg_15). The processes in Steps Sg_11 to Sg_15 are executed on each block. For example, when the processes in Steps Sg_11 to Sg_15 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal inter mode finishes. For example, when the processes in Steps Sg_11 to Sg_15 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal inter mode finishes. It is to be noted that not all the blocks included in the slice may be subjected to the processes in Steps Sg_11 to Sg_15, and inter prediction of the slice using the normal inter mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal inter mode may finish when the processes in Steps Sg_11 to Sg_15 are executed on part of the blocks in the picture.

[MV Derivation>Normal Merge Mode]

For example, when information parsed from a stream indicates that the normal merge mode is to be applied, inter predictor **218** derives an MV and performs motion compensation (prediction) using the MV.

68

FIG. **85** is a flow chart illustrating an example of inter prediction by normal merge mode in decoder **200**.

At this time, first, inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step Sh_11). In other words, inter predictor **218** generates an MV candidate list.

Next, inter predictor **218** selects one MV candidate from the plurality of MV candidates obtained in Step Sh_11, thereby deriving an MV for the current block (Step Sh_12). More specifically, inter predictor **218** obtains MV selection information included as a prediction parameter in a stream, and selects the MV candidate identified by the MV selection information as the MV for the current block.

Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Sh_13). The processes in Steps Sh_11 to Sh_13 are executed, for example, on each block. For example, when the processes in Steps Sh_11 to Sh_13 are executed on each of all the blocks in the slice, inter prediction of the slice using the normal merge mode finishes. In addition, when the processes in Steps Sh_11 to Sh_13 are executed on each of all the blocks in the picture, inter prediction of the picture using the normal merge mode finishes. It is to be noted that not all the blocks included in the slice are subjected to the processes in Steps Sh_11 to Sh_13, and inter prediction of the slice using the normal merge mode may finish when part of the blocks are subjected to the processes. Likewise, inter prediction of the picture using the normal merge mode may finish when the processes in Steps Sh_11 to Sh_13 are executed on part of the blocks in the picture.

[MV Derivation>FRUC Mode]

For example, when information parsed from a stream indicates that the FRUC mode is to be applied, inter predictor **218** derives an MV in the FRUC mode and performs motion compensation (prediction) using the MV. In this case, the motion information is derived at the decoder **200** side without being signaled from the encoder **100** side. For example, decoder **200** may derive the motion information by performing motion estimation. In this case, decoder **200** performs motion estimation without using any pixel value in a current block.

FIG. **86** is a flow chart illustrating an example of inter prediction by FRUC mode in decoder **200**.

First, inter predictor **218** generates a list indicating MVs of decoded blocks spatially or temporally neighboring the current block by referring to the MVs as MV candidates (the list is an MV candidate list, and may be used also as an MV candidate list for normal merge mode (Step Si_11)). Next, a best MV candidate is selected from the plurality of MV candidates registered in the MV candidate list (Step Si_12). For example, inter predictor **218** calculates the evaluation value of each MV candidate included in the MV candidate list, and selects one of the MV candidates as the best MV candidate based on the evaluation values. Based on the selected best MV candidate, inter predictor **218** then derives an MV for the current block (Step Si_14). More specifically, for example, the selected best MV candidate is directly derived as the MV for the current block. In addition, for example, the MV for the current block may be derived using pattern matching in a surrounding region of a position which is included in a reference picture and corresponds to the selected best MV candidate. In other words, estimation using the pattern matching in a reference picture and the

evaluation values may be performed in the surrounding region of the best MV candidate, and when there is an MV that yields a better evaluation value, the best MV candidate may be updated to the MV that yields the better evaluation value, and the updated MV may be determined as the final MV for the current block. Update to the MV that yields the better evaluation value may not be performed.

Lastly, inter predictor **218** generates a prediction image for the current block by performing motion compensation of the current block using the derived MV and the decoded reference picture (Step Si_15). The processes in Steps Si_11 to Si_15 are executed, for example, on each block. For example, when the processes in Steps Si_11 to Si_15 are executed on each of all the blocks in the slice, inter prediction of the slice using the FRUC mode finishes. For example, when the processes in Steps Si_11 to Si_15 are executed on each of all the blocks in the picture, inter prediction of the picture using the FRUC mode finishes. Each sub-block may be processed similarly to the above-described case of processing each block.

[MV Derivation>Affine Merge Mode]

For example, when information parsed from a stream indicates that the affine merge mode is to be applied, inter predictor **218** derives an MV in the affine merge mode and performs motion compensation (prediction) using the MV.

FIG. **87** is a flow chart illustrating an example of inter prediction by the affine merge mode in decoder **200**.

In the affine merge mode, first, inter predictor **218** derives MVs at respective control points for a current block (Step Sk_11). The control points are an upper-left corner point of the current block and an upper-right corner point of the current block as illustrated in FIG. **46A**, or an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. **46B**.

For example, when the MV derivation methods illustrated in FIGS. **47A** to **47C** are used, as illustrated in FIG. **47A**, inter predictor **218** checks decoded block A (left), block B (upper), block C (upper-right), block D (lower-left), and block E (upper-left) in this order, and identifies the first effective block decoded according to the affine mode.

Inter predictor **218** derives the MV at the control point using the identified first effective block decoded according to the affine mode. For example, when block A is identified and block A has two control points, as illustrated in FIG. **47B**, inter predictor **218** calculates motion vector v_0 at the upper-left corner control point of the current block and motion vector v_1 at the upper-right corner control point of the current block by projecting motion vectors v_3 and v_4 at the upper-left corner and the upper-right corner of the decoded block including block A onto the current block. In this way, the MV at each control point is derived.

It is to be noted that, as illustrated in FIG. **49A**, MVs at three control points may be calculated when block A is identified and block A has two control points, and that, as illustrated in FIG. **49B**, MVs at two control points may be calculated when block A is identified and when block A has three control points.

In addition, when MV selection information is included as a prediction parameter in a stream, inter predictor **218** may derive the MV at each control point for the current block using the MV selection information.

Next, inter predictor **218** performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor **218** calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression

(1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step Sk_12). Inter predictor **218** then performs motion compensation of the sub-blocks using these affine MVs and decoded reference pictures (Step Sk_13). When the processes in Steps Sk_12 and Sk_13 are executed for each of all the sub-blocks included in the current block, the inter prediction using the affine merge mode for the current block finishes. In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

It is to be noted that the above-described MV candidate list may be generated in Step Sk_11. The MV candidate list may be, for example, a list including MV candidates derived using a plurality of MV derivation methods for each control point. The plurality of MV derivation methods may be any combination of the MV derivation methods illustrated in FIGS. **47A** to **47C**, the MV derivation methods illustrated in FIGS. **48A** and **48B**, the MV derivation methods illustrated in FIGS. **49A** and **49B**, and other MV derivation methods.

It is to be noted that an MV candidate list may include MV candidates in a mode in which prediction is performed in units of a sub-block, other than the affine mode.

It is to be noted that, for example, an MV candidate list including MV candidates in an affine merge mode in which two control points are used and an affine merge mode in which three control points are used may be generated as an MV candidate list. Alternatively, an MV candidate list including MV candidates in the affine merge mode in which two control points are used and an MV candidate list including MV candidates in the affine merge mode in which three control points are used may be generated separately. Alternatively, an MV candidate list including MV candidates in one of the affine merge mode in which two control points are used and the affine merge mode in which three control points are used may be generated.

[MV Derivation>Affine Inter Mode]

For example, when information parsed from a stream indicates that the affine inter mode is to be applied, inter predictor **218** derives an MV in the affine inter mode and performs motion compensation (prediction) using the MV.

FIG. **88** is a flow chart illustrating an example of inter prediction by the affine inter mode in decoder **200**.

In the affine inter mode, first, inter predictor **218** derives MV predictors (v_0 , v_1) or (v_0 , v_1 , v_2) of respective two or three control points for a current block (Step Sj_11). The control points are an upper-left corner point of the current block, an upper-right corner point of the current block, and a lower-left corner point of the current block as illustrated in FIG. **46A** or FIG. **46B**.

Inter predictor **218** obtains MV predictor selection information included as a prediction parameter in the stream, and derives the MV predictor at each control point for the current block using the MV identified by the MV predictor selection information. For example, when the MV derivation methods illustrated in FIGS. **48A** and **48B** are used, inter predictor **218** derives the motion vector predictors (v_0 , v_1) or (v_0 , v_1 , v_2) at control points for the current block by selecting the MV of the block identified by the MV predictor selection information among decoded blocks in the vicinity of the respective control points for the current block illustrated in either FIG. **48A** or FIG. **48B**.

Next, inter predictor **218** obtains each MV difference included as a prediction parameter in the stream, and adds the MV predictor at each control point for the current block and the MV difference corresponding to the MV predictor (Step Sj_12). In this way, the MV at each control point for the current block is derived.

71

Next, inter predictor **218** performs motion compensation of each of a plurality of sub-blocks included in the current block. In other words, inter predictor **218** calculates an MV for each of the plurality of sub-blocks as an affine MV, using either two motion vectors v_0 and v_1 and the above expression (1A) or three motion vectors v_0 , v_1 , and v_2 and the above expression (1B) (Step S_{j_13}). Inter predictor **218** then performs motion compensation of the sub-blocks using these affine MVs and decoded reference pictures (Step S_{j_14}). When the processes in Steps S_{j_13} and S_{j_14} are executed for each of all the sub-blocks included in the current block, the inter prediction using the affine merge mode for the current block finishes. In other words, motion compensation of the current block is performed to generate a prediction image of the current block.

It is to be noted that the above-described MV candidate list may be generated in Step S_{j_11} as in Step S_{k_11}. [MV Derivation>Triangle Mode]

For example, when information parsed from a stream indicates that the triangle mode is to be applied, inter predictor **218** derives an MV in the triangle mode and performs motion compensation (prediction) using the MV.

FIG. **89** is a flow chart illustrating an example of inter prediction by the triangle mode in decoder **200**.

In the triangle mode, first, inter predictor **218** splits the current block into a first partition and a second partition (Step S_{x_11}). At this time, inter predictor **218** may obtain, from the stream, partition information which is information related to the splitting as a prediction parameter. Inter predictor **218** may then split a current block into a first partition and a second partition according to the partition information.

Next, first, inter predictor **218** obtains a plurality of MV candidates for a current block based on information such as MVs of a plurality of decoded blocks temporally or spatially surrounding the current block (Step S_{x_12}). In other words, inter predictor **218** generates an MV candidate list.

Inter predictor **218** then selects the MV candidate for the first partition and the MV candidate for the second partition as a first MV and a second MV, respectively, from the plurality of MV candidates obtained in Step S_{x_11} (Step S_{x_13}). At this time, inter predictor **218** may obtain, from the stream, MV selection information for identifying each selected MV candidate, as a prediction parameter. Inter predictor **218** may then select the first MV and the second MV according to the MV selection information.

Next, inter predictor **218** generates a first prediction image by performing motion compensation using the selected first MV and a decoded reference picture (Step S_{x_14}). Likewise, inter predictor **218** generates a second prediction image by performing motion compensation using the selected second MV and a decoded reference picture (Step S_{x_15}).

Lastly, inter predictor **218** generates a prediction image for the current block by performing a weighted addition of the first prediction image and the second prediction image (Step S_{x_16}).

[Motion Estimation>DMVR]

For example, information parsed from a stream indicates that DMVR is to be applied, inter predictor **218** performs motion estimation using DMVR.

FIG. **90** is a flow chart illustrating an example of motion estimation by DMVR in decoder **200**.

Inter predictor **218** derives an MV for a current block according to the merge mode (Step S_{l_11}). Next, inter predictor **218** derives the final MV for the current block by searching the region surrounding the reference picture indi-

72

cated by the MV derived in S_{l_11} (Step S_{l_12}). In other words, the MV of the current block is determined according to the DMVR.

FIG. **91** is a flow chart illustrating a specific example of motion estimation by DMVR in decoder **200**.

First, in Step **1** illustrated in FIG. **58A**, inter predictor **218** calculates the cost between the search position (also referred to as a starting point) indicated by the initial MV and eight surrounding search positions. Inter predictor **218** then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at one of the search positions other than the starting point is the smallest, inter predictor **218** changes a target to the search position at which the smallest cost is obtained, and performs the process in Step **2** illustrated in FIG. **58A**. When the cost at the starting point is the smallest, inter predictor **218** skips the process in Step **2** illustrated in FIG. **58A** and performs the process in Step **3**.

In Step **2** illustrated in FIG. **58A**, inter predictor **218** performs search similar to the process in Step **1**, regarding the search position after the target change as a new starting point according to the result of the process in Step **1**. Inter predictor **218** then determines whether the cost at each of the search positions other than the starting point is the smallest. Here, when determining that the cost at one of the search positions other than the starting point is the smallest, inter predictor **218** performs the process in Step **4**. When the cost at the starting point is the smallest, inter predictor **218** performs the process in Step **3**.

In Step **4**, inter predictor **218** regards the search position at the starting point as the final search position, and determines the difference between the position indicated by the initial MV and the final search position to be a vector difference.

In Step **3** illustrated in FIG. **58A**, inter predictor **218** determines the pixel position at sub-pixel accuracy at which the smallest cost is obtained, based on the costs at the four points located at upper, lower, left, and right positions with respect to the starting point in Step **1** or Step **2**, and regards the pixel position as the final search position. The pixel position at the sub-pixel accuracy is determined by performing weighted addition of each of the four upper, lower, left, and right vectors ((0, 1), (0, -1), (-1, 0), and (1, 0)), using, as a weight, the cost at a corresponding one of the four search positions. Inter predictor **218** then determines the difference between the position indicated by the initial MV and the final search position to be the vector difference.

[Motion Compensation>BIO/OBMC/LIC]

For example, when information parsed from a stream indicates that correction of a prediction image is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image based on the mode for the correction. The mode is, for example, one of BIO, OBMC, and LIC described above.

FIG. **92** is a flow chart illustrating one example of generation of a prediction image in decoder **200**.

Inter predictor **218** generates a prediction image (Step S_{m_11}), and corrects the prediction image according to any of the modes described above (Step S_{m_12}).

FIG. **93** is a flow chart illustrating another example of generation of a prediction image in decoder **200**.

Inter predictor **218** derives an MV for a current block (Step S_{n_11}). Next, inter predictor **218** generates a prediction image using the MV (Step S_{n_12}), and determines whether to perform a correction process (Step S_{n_13}). For example, inter predictor **218** obtains a prediction parameter included in the stream, and determines whether to perform

a correction process based on the prediction parameter. This prediction parameter is, for example, a flag indicating whether each of the above-described modes is to be applied. Here, when determining to perform a correction process (Yes in Step Sn_13), inter predictor **218** generates the final prediction image by correcting the prediction image (Step Sn_14). It is to be noted that, in LIC, the luminance and chrominance of the prediction image may be corrected in Step Sn_14. When determining not to perform a correction process (No in Step Sn_13), inter predictor **218** outputs the final prediction image without correcting the prediction image (Step Sn_15).

[Motion Compensation>OBMC]

For example, when information parsed from a stream indicates that OBMC is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image according to the OBMC.

FIG. 94 is a flow chart illustrating an example of correction of a prediction image by OBMC in decoder **200**. It is to be noted that the flow chart in FIG. 94 indicates the correction flow of a prediction image using the current picture and the reference picture illustrated in FIG. 62.

First, as illustrated in FIG. 62, inter predictor **218** obtains a prediction image (Pred) by normal motion compensation using an MV assigned to the current block.

Next, inter predictor **218** obtains a prediction image (Pred_L) by applying a motion vector (MV_L) which has been already derived for the decoded block neighboring to the left of the current block to the current block (re-using the motion vector for the current block). Inter predictor **218** then performs a first correction of a prediction image by overlapping two prediction images Pred and Pred_L. This provides an effect of blending the boundary between neighboring blocks.

Likewise, inter predictor **218** obtains a prediction image (Pred_U) by applying an MV (MV_U) which has been already derived for the decoded block neighboring above the current block to the current block (re-using the motion vector for the current block). Inter predictor **218** then performs a second correction of the prediction image by overlapping the prediction image Pred_U to the prediction images (for example, Pred and Pred_L) on which the first correction has been performed. This provides an effect of blending the boundary between neighboring blocks. The prediction image obtained by the second correction is the one in which the boundary between the neighboring blocks has been blended (smoothed), and thus is the final prediction image of the current block.

[Motion Compensation>BIO]

For example, when information parsed from a stream indicates that BIO is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image according to the BIO.

FIG. 95 is a flow chart illustrating an example of correction of a prediction image by the BIO in decoder **200**.

As illustrated in FIG. 63, inter predictor **218** derives two motion vectors (M0, M1), using two reference pictures (Ref0, Ref1) different from the picture (Cur Pic) including a current block. Inter predictor **218** then derives a prediction image for the current block using the two motion vectors (M0, M1) (Step Sy_11). It is to be noted that motion vector M0 is a motion vector (MVx₀, MVy₀) corresponding to reference picture Ref0, and motion vector M1 is a motion vector (MVx₁, MVy₁) corresponding to reference picture Ref1.

Next, inter predictor **218** derives interpolated image I⁰ for the current block using motion vector M0 and reference

picture L0. In addition, inter predictor **218** derives interpolated image I¹ for the current block using motion vector M1 and reference picture L1 (Step Sy_12). Here, interpolated image I⁰ is an image included in reference picture Ref0 and to be derived for the current block, and interpolated image I¹ is an image included in reference picture Ref1 and to be derived for the current block. Each of interpolated image I⁰ and interpolated image I¹ may be the same in size as the current block. Alternatively, each of interpolated image I⁰ and interpolated image I¹ may be an image larger than the current block. Furthermore, interpolated image I⁰ and interpolated image I¹ may include a prediction image obtained by using motion vectors (M0, M1) and reference pictures (L0, L1) and applying a motion compensation filter.

In addition, inter predictor **218** derives gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹) of the current block, from interpolated image I⁰ and interpolated image I¹ (Step Sy_13). It is to be noted that the gradient images in the horizontal direction are (Ix⁰, Ix¹), and the gradient images in the vertical direction are (Iy⁰, Iy¹). Inter predictor **218** may derive the gradient images by, for example, applying a gradient filter to the interpolated images. The gradient images may be the ones each of which indicates the amount of spatial change in pixel value along the horizontal direction or the amount of spatial change in pixel value along the vertical direction.

Next, inter predictor **218** derives, for each sub-block of the current block, an optical flow (vx, vy) which is a velocity vector, using the interpolated images (I⁰, I¹) and the gradient images (Ix⁰, Ix¹, Iy⁰, Iy¹) (Step Sy_14). As one example, a sub-block may be 4×4 pixel sub-CU.

Next, inter predictor **218** corrects a prediction image for the current block using the optical flow (vx, vy). For example, inter predictor **218** derives a correction value for the value of a pixel included in a current block, using the optical flow (vx, vy) (Step Sy_15). Inter predictor **218** may then correct the prediction image for the current block using the correction value (Step Sy_16). It is to be noted that the correction value may be derived in units of a pixel, or may be derived in units of a plurality of pixels or in units of a sub-block.

It is to be noted that the BIO process flow is not limited to the process disclosed in FIG. 95. Only part of the processes disclosed in FIG. 95 may be performed, or a different process may be added or used as a replacement, or the processes may be executed in a different processing order.

[Motion Compensation>LIC]

For example, when information parsed from a stream indicates that LIC is to be performed, upon generating a prediction image, inter predictor **218** corrects the prediction image according to the LIC.

FIG. 96 is a flow chart illustrating an example of correction of a prediction image by the LIC in decoder **200**.

First, inter predictor **218** obtains a reference image corresponding to a current block from a decoded reference picture using an MV (Step Sz_11).

Next, inter predictor **218** extracts, for the current block, information indicating how the luma value has changed between the current picture and the reference picture (Step Sz_12). This extraction is performed based on the luma pixel values for the decoded left neighboring reference region (surrounding reference region) and the decoded upper neighboring reference region (surrounding reference region), and the luma pixel values at the corresponding positions in the reference picture specified by the derived MV's. Inter pre-

75

dictor **218** calculates a luminance correction parameter, using the information indicating how the luma value changed (Step Sz_13).

Inter predictor **218** generates a prediction image for the current block by performing a luminance correction process in which the luminance correction parameter is applied to the reference image in the reference picture specified by the MV (Step Sz_14). In other words, the prediction image which is the reference image in the reference picture specified by the MV is subjected to the correction based on the luminance correction parameter. In this correction, luminance may be corrected, or chrominance may be corrected. [Prediction Controller]

Prediction controller **220** selects either an intra prediction image or an inter prediction image, and outputs the selected image to adder **208**. As a whole, the configurations, functions, and processes of prediction controller **220**, intra predictor **216**, and inter predictor **218** at the decoder **200** side may correspond to the configurations, functions, and processes of prediction controller **128**, intra predictor **124**, and inter predictor **126** at the encoder **100** side. [Wedge Mode (GEO)]

Here, the outline of the wedge mode is described. The wedge mode is one of a plurality of merge modes, and is also called a geometric partitioning mode. It is also called GPM mode or GEO mode.

FIG. 97 is a conceptual diagram illustrating a partitioning line used in the wedge mode. As described above in relation to the triangle mode illustrated in FIG. 52A, FIG. 52B, and FIG. 52C, one of the first partition and the second partition obtained by splitting the current block may be a trapezoid, and these partitions may have shapes different from each other. Such shapes, i.e., non-rectangular shapes other than triangles, can be used in the wedge mode.

In the wedge mode, as illustrated in FIG. 97, the block is split into two along a partitioning line defined by distance ρ from the center of the block and angle α of a normal line.

Distance ρ is the shortest distance from the center of the block to the partitioning line. Angle α represents the direction from the center of the block toward the partitioning line in the shortest distance.

More specifically, angle α may be the angle between the right direction which is the reference direction and the direction from the center of the block toward the partitioning line in the shortest distance. Angle α may take, for example, any one of 20 angles. Distance ρ may take, for example, any one of the following four distances: 0 (zero), $\rho_{max}/4$, $\rho_{max}/2$, and $\rho_{max} \times 3/4$. Here, ρ_{max} is the shortest distance from the center of the block to a straight line which is in parallel to the partitioning line and touches the block.

Note that this is a mere example of how to define the partitioning line, and the partitioning line may be defined in a different manner.

As can be understood from above, in the wedge mode, there are as many block splitting methods available as there are combinations of possible distance ρ and possible angle α . The coding efficiency can be improved by selecting an optimal one from the block splitting methods available. [Example of Wedge Mode]

FIG. 98 is a flow chart illustrating an example of processing in which encoder **100** encodes a block using the wedge mode. While the wedge mode in the present example is basically the same as the wedge mode described above, the manner in which the partitioning line is defined in the present example is different from the manner described above.

76

First, encoder **100** determines a shape value (V) using the width (W) and the height (H) of the block (S101). The shape value is a value that indicates the shape of the block, and may be calculated by Equation (3), for example.

[MATH. 4]

$$V = \begin{cases} \text{Log2}(W/H), & \text{if } W \geq H \\ 3_ \text{Log2}(H/W), & \text{otherwise} \end{cases} \quad (3)$$

The shape value may be calculated by an equation different from Equation (3), so long as it is a value uniquely determined based on the width and height of the block.

Next, encoder **100** selects a partitioning line candidate set associated with the shape value from among a plurality of partitioning line candidate sets (S102). Each of the plurality of partitioning line candidate sets includes a plurality of partitioning line candidates, that is, a plurality of candidates for the partitioning line. An example of the plurality of partitioning line candidate sets will be described later.

Next, encoder **100** encodes an index that identifies a partitioning line from the partitioning line candidate set (S103).

For example, encoder **100** evaluates the cost of each of the partitioning line candidates included in the partitioning line candidate set, and selects a partitioning line candidate with the least cost as the partitioning line. Encoder **100** then encodes an index that identifies the partitioning line by encoding an index associated with the partitioning line candidate selected as the partitioning line, among a plurality of indices associated with the partitioning line candidates.

The indices associated with the partitioning line candidates may each indicate a distance and an angle corresponding to the partitioning line candidate. The index encoded by encoder **100** may indicate a distance and an angle corresponding to the partitioning line.

Encoder **100** then encodes the block using the partitioning line identified by the index (S104).

Specifically, encoder **100** splits the block into two partitions along the partitioning line identified by the index. Encoder **100** then applies two prediction processes to the two partitions to generate a prediction image of the block. Then, encoder **100** encodes the block by encoding the difference between the original image of the block and the prediction image of the block. A smoothing process may be applied to the boundary of the two partitions to remove distortion.

FIG. 99 is a conceptual diagram illustrating an example of a plurality of partitioning line candidate sets. Each partitioning line candidate set is associated with a shape value (V) in the range from 0 to 6. In this example, each partitioning line candidate set includes 14 partitioning line candidates for a given shape value. Each partitioning line candidate can be identified by an index that indicates a distance and an angle corresponding to the partitioning line candidate. Here, the distance corresponds to one of shift values on seven levels from a diagonal line, and the angle corresponds to one of the two diagonal lines.

That is to say, the 14 partitioning line candidates are determined by 7 distances and 2 angles. The index may be a single value in a range from 0 to 13, for example. Alternatively, the index may include two values; a value indicating a distance and a value indicating an angle.

[Variation 1 of Wedge Mode]

For example, depending on the shape value, the total number of partitioning line candidates included in the partitioning line candidate set may be different from the normal number.

FIG. 100 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 1. For example, only one partitioning line candidate may be enabled when the shape value is 3 or 6, i.e., when the width-to-height ratio or the height-to-width ratio is 8. The block whose width-to-height ratio or height-to-width ratio is 8 is, for example, a block having 64×8 pixels or 8×64 pixels.

[Variation 2 of Wedge Mode]

For example, depending on the shape value, the angles corresponding to partitioning line candidates may be different from the angle corresponding to a diagonal line.

FIG. 101 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 2. For example, the angles used when the shape value is 3 or 6 may be the same as the angles used when the shape value is 2 or 5. That is to say, the angles used when the width-to-height ratio or the height-to-width ratio is 8 may be the same as the angles used when the width-to-height ratio or the height-to-width ratio is 4. The block whose width-to-height ratio or height-to-width ratio is 8 is, for example, a block having 64×8 pixels or 8×64 pixels. The block whose width-to-height ratio or height-to-width ratio is 4 is, for example, a block having 64×16 pixels, 16×64 pixels, 32×8 pixels, or 8×32 pixels.

[Variation 3 of Wedge Mode]

For example, the wedge mode may be disabled depending on the shape value. Specifically, the wedge mode may be disabled when the width-to-height ratio or the height-to-width ratio is greater than or equal to a threshold. The threshold may be 8, for example. Alternatively, the wedge mode may be disabled depending on the shape or size of the block. For example, the wedge mode may be disabled for a block having 64×8 pixels or 8×64 pixels.

FIG. 102 is a conceptual diagram illustrating a plurality of partitioning line candidate sets according to Variation 3. For example, the wedge mode may be disabled when the shape value is 3 or 6. That is to say, the wedge mode may be disabled when the width-to-height ratio or the height-to-width ratio is 8. The wedge mode may be enabled when the shape value is neither 3 nor 6. That is to say, the wedge mode may be enabled when the width-to-height ratio and the height-to-width ratio are both less than 8.

Here, disabling the wedge mode means prohibiting the wedge mode, whereas enabling the wedge mode means permitting the wedge mode. That is to say, the wedge mode may be prohibited or permitted based on the width and the height of the block. For example, the wedge mode may be prohibited when the width-to-height ratio or the height-to-width ratio is greater than or equal to a threshold. The wedge mode may be permitted when the width-to-height ratio and the height-to-width ratio are both less than the threshold.

When the wedge mode is enabled, whether or not to use the wedge mode may be determined based on an R-D cost or a coding parameter, for example. In other words, when the wedge mode is enabled, the wedge mode may be used or may be not used. When the wedge mode is disabled, the wedge mode is not used.

Note that, for example, a width-to-height ratio of at least 8 means that the width is at least 8 times as much as the height. Also, a height-to-width ratio of at least 8 means that the height is at least 8 times as much as the width. When the wedge mode is disabled, the index identifying the partition-

ing line need not be encoded or decoded. That is to say, in that case, storage of the index in a bitstream and obtainment of the index from the bitstream may be disabled.

[Supplemental Information on Wedge Mode]

In the examples illustrated in FIG. 98 to FIG. 102, there is a possibility that the total number of partitioning line candidates becomes less than that in the example illustrated in FIG. 97, thus reducing labor for determining hardware compatibility.

For example, in the example illustrated in FIG. 97, distance ρ and angle α are used for block splitting, and there are many variations of values that distance ρ and angle α can take. Thus, there is a risk of an increase in the code amount of the signal that specifies distance ρ and angle α .

In contrast, in the examples illustrated in FIG. 98 to FIG. 102, basically, an angle equal to the angle corresponding to the diagonal line is used as the angle corresponding to the partitioning line. Therefore, the partitioning line can be identified simply by (i) distance ρ which can take a value from seven levels and (ii) a value which can take a binary value indicating either diagonally right up or diagonally left up. Accordingly, the code amount can be reduced.

Also, a single index can indicate distance ρ and a value which can take a binary value indicating either diagonally right up or diagonally left up. Accordingly, the code amount can be further reduced.

When the width-to-height ratio or the height-to-width ratio is high, there is a risk of impairing the effect achieved by applying two prediction processes to the two partitions of the block in the wedge mode.

Specifically, when the width-to-height ratio or the height-to-width ratio is high, the partitioning line is highly likely to be long in the longitudinal direction of the block. Application of the smoothing process to such a partitioning line to remove distortion causes significant intermingling of the two partitions of the block split along the partitioning line, thereby impairing the effect achieved by applying two prediction processes to the two partitions of the block.

In addition, in the wedge mode, there is a possibility that the partitioning line is determined in the vicinity of the edge of the block. When the width-to-height ratio or the height-to-width ratio is high and the partitioning line is determined in the vicinity of the edge of the block, there is a risk that most of the result of one of the two prediction processes is lost due to the smoothing process.

FIG. 103 is a conceptual diagram illustrating an example of a partitioning line determined when the width-to-height ratio is high. As illustrated in FIG. 103, there is a possibility that the partitioning line is determined in the vicinity of the edge of the block when the width-to-height ratio is high. In the example as illustrated in FIG. 103, there is a risk that most of the result of a prediction process on the upper right partition is lost due to the smoothing process.

Consequently, as described above, when the width-to-height ratio or the height-to-width ratio is high, there is a risk of impairing the effect achieved by applying two prediction processes to the two partitions of the block in the wedge mode. In view of this, as in the example illustrated in FIG. 102, the wedge mode may be disabled when the width-to-height ratio or the height-to-width ratio is high. This way, there is a possibility of disabling useless processing and enhancing the processing efficiency.

Further, when, for example, a large block is split into two non-rectangular partitions and two prediction processes are performed on the two partitions, there is a risk that the distortion at the boundary between the two non-rectangular partitions becomes visible, and that the processing load of

the smoothing process performed thereafter becomes large. Also, when a small block is split into two non-rectangular partitions and two prediction processes are performed on the two partitions, there is a risk that the results of the two prediction processes are intermingled and lost due to the smoothing process performed thereafter. In view of the above, the wedge mode may be permitted for blocks having an appropriate block size.

Specifically, the wedge mode may be permitted for blocks having 8×8 pixels to 64×64 pixels and prohibited for the other blocks. Likewise, the triangle mode described with reference to FIG. 52A etc. may also be permitted for blocks having 8×8 pixels to 64×64 pixels and prohibited for the other blocks for the same reason.

For example, for a block having 8×64 pixels or 64×8 pixels, the wedge mode is not prohibited by the block size, but may be prohibited by the ratio. Note that in the triangle mode in which the block is split along a diagonal line, the partitioning line is not determined in the vicinity of the edge of the block. Thus, the triangle mode need not be prohibited by the ratio.

Furthermore, as in the example illustrated in FIG. 100, when the width-to-height ratio or the height-to-width ratio is high, the number of partitioning line candidates included in the partitioning line candidate set may be reduced. For example, when the width-to-height ratio or the height-to-width ratio is high, the number of partitioning line candidates included in the partitioning line candidate set may be reduced to avoid the partitioning line from being determined in the vicinity of the edge of the block. This way, there is a possibility of not performing useless processing, thereby enhancing the processing efficiency.

Also, as in the example illustrated in FIG. 101, when the width-to-height ratio or the height-to-width ratio is high, the respective angles of the partitioning line candidates included in the partitioning line candidate set may be determined to avoid the results of the two prediction processes from being lost in the smoothing process. Specifically, the angle of each partitioning line candidate may be determined to avoid the partitioning line candidate from becoming excessively long in the longitudinal direction of the block.

The inter prediction process of encoder 100 described above can be performed as the inter prediction process of decoder 200 in a similar manner. For example, “encode” and “encoding” described above may be replaced with “decode” and “decoding”. Also, for example, encoder 100 may perform the operation of storing the index that identifies the partitioning line in a bitstream, and decoder 200 may perform the operation of obtaining the index that identifies the partitioning line from a bitstream.

Not all of the processes described with reference to the examples illustrated in FIG. 97 to FIG. 102 need to be performed. That is to say, only some of the processes described with reference to the examples illustrated in FIG. 97 to FIG. 102 may be performed. For example, one or more of the examples illustrated in FIG. 100 to FIG. 102 may be applied to the example illustrated in FIG. 97.

[Representative Example of Configuration and Processing]

The following describes a representative example of the configurations and processes of encoder 100 and decoder 200 described above.

FIG. 104 is a flow chart illustrating operations of encoder 100. For example, encoder 100 includes circuitry and memory connected to the circuitry. The circuitry and the memory included in encoder 100 may correspond to processor a1 and memory a2 illustrated in FIG. 8. The circuitry of encoder 100, in operation, performs the following:

For example, the circuitry of encoder 100 determines a mode to be applied to a block from a plurality of merge modes based on the width of the block and the height of the block (S111). Subsequently, when the mode determined is a first mode, the circuitry of encoder 100 stores, in a bitstream, an index indicating a distance and an angle that define two partitions in the block, and encodes the block using the first mode (S112).

Here, the circuitry of encoder 100 disables the storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width. Specifically, the circuitry of encoder 100 may determine, as the mode to be applied to the block, a mode different from the first mode among the plurality of merge modes when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the storing of the index indicating a distance and an angle in the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

Also, for example, the circuitry of encoder 100 may disable the storing of the index in the bitstream when (i) the width is 64 pixels and the height is 8 pixels or (ii) the width is 8 pixels and the height is 64 pixels. By doing so, there is a possibility that the storing of the index indicating a distance and an angle in the bitstream is disabled when the width-to-height ratio or the height-to-width ratio is high, even if the block size is within an appropriate range. This way, there is a possibility of reducing useless processing and enhancing the processing efficiency.

Furthermore, for example, a line that determines the shapes of two partitions in the block may be defined by a distance and an angle. Here, the distance may be a distance between the center of the block and the line, and the angle may be an angle related to the line. With this, there is a possibility of appropriately defining the line that determines the shapes of two partitions.

Note that the block may be a rectangular image block. Entropy encoder 110 or inter predictor 126 of encoder 100, or a combination thereof, may perform the above-described operations as the circuitry of encoder 100.

FIG. 105 is a flow chart illustrating operations of decoder 200. For example, decoder 200 includes circuitry and memory connected to the circuitry. The circuitry and the memory included in decoder 200 may correspond to processor b1 and memory b2 illustrated in FIG. 68. The circuitry of decoder 200, in operation, performs the following:

For example, the circuitry of decoder 200 determines a mode to be applied to a block from a plurality of merge modes based on the width of the block and the height of the block (S121). Subsequently, when the mode determined is a first mode, the circuitry of decoder 200 obtains, from a bitstream, an index indicating a distance and an angle that define two partitions in the block, and decodes the block using the first mode (S122).

Here, the circuitry of decoder 200 disables the obtaining of the index from the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width. Specifically, the circuitry of decoder 200 may determine, as the mode to be applied to the block, a mode different from the first mode from the

plurality of merge modes when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

This way, there is a possibility of reducing useless processing and enhancing the processing efficiency. Specifically, when the width-to-height ratio or the height-to-width ratio is high, it may be inappropriate to define two partitions by a distance and an angle. In such a case, by disabling the obtaining of the index indicating a distance and an angle from the bitstream, there is a possibility of reducing useless processing and enhancing the processing efficiency.

Also, for example, the circuitry of decoder **200** may disable the obtaining of the index from the bitstream when (i) the width is 64 pixels and the height is 8 pixels or (ii) the width is 8 pixels and the height is 64 pixels. By doing so, there is a possibility that the obtaining of the index indicating a distance and an angle from the bitstream is disabled when the width-to-height ratio or the height-to-width ratio is high, even if the block size is within an appropriate range. This way, there is a possibility of reducing useless processing and enhancing the processing efficiency.

Furthermore, for example, a line that determines the shapes of two partitions in the block may be defined by a distance and an angle. Here, the distance may be a distance between the center of the block and the line, and the angle may be an angle related to the line. With this, there is a possibility of appropriately defining the line that determines the shapes of two partitions.

Note that the block may be a rectangular image block. Entropy decoder **202** or inter predictor **218** of decoder **200**, or a combination thereof, may perform the above-described operations as the circuitry of decoder **200**.

Other Examples

Encoder **100** and decoder **200** in each example described above may be used as an image encoder and an image decoder, respectively, or may be used as a video encoder and a video decoder, respectively.

Alternatively, each of encoder **100** and decoder **200** may be used as an inter prediction device. That is to say, encoder **100** and decoder **200** may only correspond to inter predictor **126** and inter predictor **218**, respectively.

Encoder **100** may include an input terminal and an output terminal. For example, one or more pictures are input to the input terminal of encoder **100**, and an encoded bitstream is output from the output terminal of encoder **100**. Decoder **200** may also include an input terminal and an output terminal. For example, an encoded bitstream is input to the input terminal of decoder **200**, and one or more pictures are output from the output terminal of decoder **200**. The encoded bitstream may include control information and quantization coefficients to which variable-length encoding has been applied.

At least part of each example described above may be used as an encoding method, a decoding method, a prediction method, or a method other than these.

Each constituent element may be configured in the form of a dedicated hardware product, or may be implemented by executing a software program suitable for the constituent element. Each constituent element may be implemented by a program execution unit such as a CPU or a processor that reads and executes a software program recorded on a recording medium such as a hard disk or a semiconductor memory.

Specifically, each of encoder **100** and decoder **200** may include processing circuitry and storage that is electrically connected to the processing circuitry and is accessible from

the processing circuitry. For example, the processing circuitry corresponds to processor **a1** or **b1**, and the storage corresponds to memory **a2** or **b2**.

The processing circuitry includes at least one of a dedicated hardware product or a program execution unit, and performs processing using the storage. When the processing circuitry includes a program execution unit, the storage stores a software program which is performed by the program execution unit.

The software for implementing, for example, above-described encoder **100** or decoder **200** is a program such as the following:

For example, this program may cause a computer to perform an encoding method including: determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, storing in a bitstream an index indicating a distance and an angle that define two partitions in the block, and encoding the block using the first mode; and disabling the storing of the index in the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

Also, for example, this program may cause a computer to perform a decoding method including: determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block; when the mode determined is a first mode, obtaining from a bitstream an index indicating a distance and an angle that define two partitions in the block, and decoding the block using the first mode; and disabling the obtaining of the index from the bitstream when (i) the width is at least 8 times as much as the height or (ii) the height is at least 8 times as much as the width.

Each constituent element may be a circuit as described above. These circuits may form a single circuit as a whole, or may be individual circuits. Each constituent element may be implemented by a general-purpose processor or a dedicated processor.

Processing performed by a particular constituent element may be performed by another constituent element. The order in which processes are performed may be changed, or a plurality of processes may be performed in parallel. An encoding decoding device may include encoder **100** and decoder **200**.

Further, ordinal numbers such as first and second used in the description may be changed as necessary. The ordinal numbers of the constituent elements etc. may be renewed or removed.

While aspects of encoder **100** and decoder **200** have been described above based on more than one example, the aspects of encoder **100** and decoder **200** are not limited to these examples. The aspects of encoder **100** and decoder **200** may also encompass various modifications to the examples that are conceivable to a person skilled in the art and embodiments implemented by combining constituent elements in different examples without departing from the essence of the present disclosure.

One or more aspects disclosed herein may be implemented in combination with at least part of other aspects according to the present disclosure. In addition, part of the configuration of any of the devices, part of syntaxes, and part of the processes indicated in any of the flow charts according to one or more aspects disclosed herein, etc. may be implemented in combination with other aspects.

[Implementations and Applications]

As described in each of the above embodiments, each functional or operational block may typically be realized as

an MPU (micro processing unit) and memory, for example. Moreover, processes performed by each of the functional blocks may be realized as a program execution unit, such as a processor which reads and executes software (a program) recorded on a recording medium such as ROM. The software may be distributed. The software may be recorded on a variety of recording media such as semiconductor memory. Note that each functional block can also be realized as hardware (dedicated circuit).

The processing described in each of the embodiments may be realized via integrated processing using a single apparatus (system), and, alternatively, may be realized via decentralized processing using a plurality of apparatuses. Moreover, the processor that executes the above-described program may be a single processor or a plurality of processors. In other words, integrated processing may be performed, and, alternatively, decentralized processing may be performed.

Embodiments of the present disclosure are not limited to the above exemplary embodiments; various modifications may be made to the exemplary embodiments, the results of which are also included within the scope of the embodiments of the present disclosure.

Next, application examples of the moving picture encoding method (image encoding method) and the moving picture decoding method (image decoding method) described in each of the above embodiments will be described, as well as various systems that implement the application examples. Such a system may be characterized as including an image encoder that employs the image encoding method, an image decoder that employs the image decoding method, or an image encoder-decoder that includes both the image encoder and the image decoder. Other configurations of such a system may be modified on a case-by-case basis.

Usage Examples

FIG. 106 illustrates an overall configuration of content providing system ex100 suitable for implementing a content distribution service. The area in which the communication service is provided is divided into cells of desired sizes, and base stations ex106, ex107, ex108, ex109, and ex110, which are fixed wireless stations in the illustrated example, are located in respective cells.

In content providing system ex100, devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 are connected to internet ex101 via internet service provider ex102 or communications network ex104 and base stations ex106 through ex110. Content providing system ex100 may combine and connect any of the above devices. In various implementations, the devices may be directly or indirectly connected together via a telephone network or near field communication, rather than via base stations ex106 through ex110. Further, streaming server ex103 may be connected to devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 via, for example, internet ex101. Streaming server ex103 may also be connected to, for example, a terminal in a hotspot in airplane ex117 via satellite ex116.

Note that instead of base stations ex106 through ex110, wireless access points or hotspots may be used. Streaming server ex103 may be connected to communications network ex104 directly instead of via internet ex101 or internet service provider ex102, and may be connected to airplane ex117 directly instead of via satellite ex116.

Camera ex113 is a device capable of capturing still images and video, such as a digital camera. Smartphone ex115 is a smartphone device, cellular phone, or personal handyphone system (PHS) phone that can operate under the mobile communications system standards of the 2G, 3G, 3.9G, and 4G systems, as well as the next-generation 5G system.

Home appliance ex114 is, for example, a refrigerator or a device included in a home fuel cell cogeneration system.

In content providing system ex100, a terminal including an image and/or video capturing function is capable of, for example, live streaming by connecting to streaming server ex103 via, for example, base station ex106. When live streaming, a terminal (e.g., computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, or a terminal in airplane ex117) may perform the encoding processing described in the above embodiments on still-image or video content captured by a user via the terminal, may multiplex video data obtained via the encoding and audio data obtained by encoding audio corresponding to the video, and may transmit the obtained data to streaming server ex103. In other words, the terminal functions as the image encoder according to one aspect of the present disclosure.

Streaming server ex103 streams transmitted content data to clients that request the stream. Client examples include computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, and terminals inside airplane ex117, which are capable of decoding the above-described encoded data. Devices that receive the streamed data decode and reproduce the received data. In other words, the devices may each function as the image decoder, according to one aspect of the present disclosure.

[Decentralized Processing]

Streaming server ex103 may be realized as a plurality of servers or computers between which tasks such as the processing, recording, and streaming of data are divided. For example, streaming server ex103 may be realized as a content delivery network (CDN) that streams content via a network connecting multiple edge servers located throughout the world. In a CDN, an edge server physically near a client is dynamically assigned to the client. Content is cached and streamed to the edge server to reduce load times. In the event of, for example, some type of error or change in connectivity due, for example, to a spike in traffic, it is possible to stream data stably at high speeds, since it is possible to avoid affected parts of the network by, for example, dividing the processing between a plurality of edge servers, or switching the streaming duties to a different edge server and continuing streaming.

Decentralization is not limited to just the division of processing for streaming; the encoding of the captured data may be divided between and performed by the terminals, on the server side, or both. In one example, in typical encoding, the processing is performed in two loops. The first loop is for detecting how complicated the image is on a frame-by-frame or scene-by-scene basis, or detecting the encoding load. The second loop is for processing that maintains image quality and improves encoding efficiency. For example, it is possible to reduce the processing load of the terminals and improve the quality and encoding efficiency of the content by having the terminals perform the first loop of the encoding and having the server side that received the content perform the second loop of the encoding. In such a case, upon receipt of a decoding request, it is possible for the encoded data resulting from the first loop performed by one

terminal to be received and reproduced on another terminal in approximately real time. This makes it possible to realize smooth, real-time streaming.

In another example, camera ex113 or the like extracts a feature amount from an image, compresses data related to the feature amount as metadata, and transmits the compressed metadata to a server. For example, the server determines the significance of an object based on the feature amount and changes the quantization accuracy accordingly to perform compression suitable for the meaning (or content significance) of the image. Feature amount data is particularly effective in improving the precision and efficiency of motion vector prediction during the second compression pass performed by the server. Moreover, encoding that has a relatively low processing load, such as variable length coding (VLC), may be handled by the terminal, and encoding that has a relatively high processing load, such as context-adaptive binary arithmetic coding (CABAC), may be handled by the server.

In yet another example, there are instances in which a plurality of videos of approximately the same scene are captured by a plurality of terminals in, for example, a stadium, shopping mall, or factory. In such a case, for example, the encoding may be decentralized by dividing processing tasks between the plurality of terminals that captured the videos and, if necessary, other terminals that did not capture the videos, and the server, on a per-unit basis. The units may be, for example, groups of pictures (GOP), pictures, or tiles resulting from dividing a picture. This makes it possible to reduce load times and achieve streaming that is closer to real time.

Since the videos are of approximately the same scene, management and/or instructions may be carried out by the server so that the videos captured by the terminals can be cross-referenced. Moreover, the server may receive encoded data from the terminals, change the reference relationship between items of data, or correct or replace pictures themselves, and then perform the encoding. This makes it possible to generate a stream with increased quality and efficiency for the individual items of data.

Furthermore, the server may stream video data after performing transcoding to convert the encoding format of the video data. For example, the server may convert the encoding format from MPEG to VP (e.g., VP9), and may convert H.264 to H.265.

In this way, encoding can be performed by a terminal or one or more servers. Accordingly, although the device that performs the encoding is referred to as a “server” or “terminal” in the following description, some or all of the processes performed by the server may be performed by the terminal, and likewise some or all of the processes performed by the terminal may be performed by the server. This also applies to decoding processes.

[3D, Multi-angle]

There has been an increase in usage of images or videos combined from images or videos of different scenes concurrently captured, or of the same scene captured from different angles, by a plurality of terminals such as camera ex113 and/or smartphone ex115. Videos captured by the terminals are combined based on, for example, the separately obtained relative positional relationship between the terminals, or regions in a video having matching feature points.

In addition to the encoding of two-dimensional moving pictures, the server may encode a still image based on scene analysis of a moving picture, either automatically or at a point in time specified by the user, and transmit the encoded

still image to a reception terminal. Furthermore, when the server can obtain the relative positional relationship between the video capturing terminals, in addition to two-dimensional moving pictures, the server can generate three-dimensional geometry of a scene based on video of the same scene captured from different angles. The server may separately encode three-dimensional data generated from, for example, a point cloud and, based on a result of recognizing or tracking a person or object using three-dimensional data, may select or reconstruct and generate a video to be transmitted to a reception terminal, from videos captured by a plurality of terminals.

This allows the user to enjoy a scene by freely selecting videos corresponding to the video capturing terminals, and allows the user to enjoy the content obtained by extracting a video at a selected viewpoint from three-dimensional data reconstructed from a plurality of images or videos. Furthermore, as with video, sound may be recorded from relatively different angles, and the server may multiplex audio from a specific angle or space with the corresponding video, and transmit the multiplexed video and audio.

In recent years, content that is a composite of the real world and a virtual world, such as virtual reality (VR) and augmented reality (AR) content, has also become popular. In the case of VR images, the server may create images from the viewpoints of both the left and right eyes, and perform encoding that tolerates reference between the two viewpoint images, such as multi-view coding (MVC), and, alternatively, may encode the images as separate streams without referencing. When the images are decoded as separate streams, the streams may be synchronized when reproduced, so as to recreate a virtual three-dimensional space in accordance with the viewpoint of the user.

In the case of AR images, the server superimposes virtual object information existing in a virtual space onto camera information representing a real-world space, based on a three-dimensional position or movement from the perspective of the user. The decoder may obtain or store virtual object information and three-dimensional data, generate two-dimensional images based on movement from the perspective of the user, and then generate superimposed data by seamlessly connecting the images. Alternatively, the decoder may transmit, to the server, motion from the perspective of the user in addition to a request for virtual object information. The server may generate superimposed data based on three-dimensional data stored in the server, in accordance with the received motion, and encode and stream the generated superimposed data to the decoder. Note that superimposed data includes, in addition to RGB values, an a value indicating transparency, and the server sets the a value for sections other than the object generated from three-dimensional data to, for example, 0, and may perform the encoding while those sections are transparent. Alternatively, the server may set the background to a determined RGB value, such as a chroma key, and generate data in which areas other than the object are set as the background.

Decoding of similarly streamed data may be performed by the client (i.e., the terminals), on the server side, or divided therebetween. In one example, one terminal may transmit a reception request to a server, the requested content may be received and decoded by another terminal, and a decoded signal may be transmitted to a device having a display. It is possible to reproduce high image quality data by decentralizing processing and appropriately selecting content regardless of the processing ability of the communications terminal itself. In yet another example, while a TV, for example, is receiving image data that is large in size, a region of a

picture, such as a tile obtained by dividing the picture, may be decoded and displayed on a personal terminal or terminals of a viewer or viewers of the TV. This makes it possible for the viewers to share a big-picture view as well as for each viewer to check his or her assigned area, or inspect a region in further detail up close.

In situations in which a plurality of wireless connections are possible over near, mid, and far distances, indoors or outdoors, it may be possible to seamlessly receive content using a streaming system standard such as MPEG-DASH. The user may switch between data in real time while freely selecting a decoder or display apparatus including the user's terminal, displays arranged indoors or outdoors, etc. Moreover, using, for example, information on the position of the user, decoding can be performed while switching which terminal handles decoding and which terminal handles the displaying of content. This makes it possible to map and display information, while the user is on the move in route to a destination, on the wall of a nearby building in which a device capable of displaying content is embedded, or on part of the ground. Moreover, it is also possible to switch the bit rate of the received data based on the accessibility to the encoded data on a network, such as when encoded data is cached on a server quickly accessible from the reception terminal, or when encoded data is copied to an edge server in a content delivery service.

[Web Page Optimization]

FIG. 107 illustrates an example of a display screen of a web page on computer ex111, for example. FIG. 108 illustrates an example of a display screen of a web page on smartphone ex115, for example. As illustrated in FIG. 107 and FIG. 108, a web page may include a plurality of image links that are links to image content, and the appearance of the web page differs depending on the device used to view the web page. When a plurality of image links are viewable on the screen, until the user explicitly selects an image link, or until the image link is in the approximate center of the screen or the entire image link fits in the screen, the display apparatus (decoder) may display, as the image links, still images included in the content or I pictures; may display video such as an animated gif using a plurality of still images or I pictures; or may receive only the base layer, and decode and display the video.

When an image link is selected by the user, the display apparatus performs decoding while giving the highest priority to the base layer. Note that if there is information in the HTML code of the web page indicating that the content is scalable, the display apparatus may decode up to the enhancement layer. Further, in order to guarantee real-time reproduction, before a selection is made or when the bandwidth is severely limited, the display apparatus can reduce delay between the point in time at which the leading picture is decoded and the point in time at which the decoded picture is displayed (that is, the delay between the start of the decoding of the content to the displaying of the content) by decoding and displaying only forward reference pictures (I picture, P picture, forward reference B picture). Still further, the display apparatus may purposely ignore the reference relationship between pictures, and coarsely decode all B and P pictures as forward reference pictures, and then perform normal decoding as the number of pictures received over time increases.

[Autonomous Driving]

When transmitting and receiving still image or video data such as two- or three-dimensional map information for autonomous driving or assisted driving of an automobile, the reception terminal may receive, in addition to image data

belonging to one or more layers, information on, for example, the weather or road construction as metadata, and associate the metadata with the image data upon decoding. Note that metadata may be assigned per layer and, alternatively, may simply be multiplexed with the image data.

In such a case, since the automobile, drone, airplane, etc., containing the reception terminal is mobile, the reception terminal may seamlessly receive and perform decoding while switching between base stations among base stations ex106 through ex110 by transmitting information indicating the position of the reception terminal. Moreover, in accordance with the selection made by the user, the situation of the user, and/or the bandwidth of the connection, the reception terminal may dynamically select to what extent the metadata is received, or to what extent the map information, for example, is updated.

In content providing system ex100, the client may receive, decode, and reproduce, in real time, encoded information transmitted by the user.

[Streaming of Individual Content]

In content providing system ex100, in addition to high image quality, long content distributed by a video distribution entity, unicast or multicast streaming of low image quality, and short content from an individual are also possible. Such content from individuals is likely to further increase in popularity. The server may first perform editing processing on the content before the encoding processing, in order to refine the individual content. This may be achieved using the following configuration, for example.

In real time while capturing video or image content, or after the content has been captured and accumulated, the server performs recognition processing based on the raw data or encoded data, such as capture error processing, scene search processing, meaning analysis, and/or object detection processing. Then, based on the result of the recognition processing, the server—either when prompted or automatically—edits the content, examples of which include: correction such as focus and/or motion blur correction; removing low-priority scenes such as scenes that are low in brightness compared to other pictures, or out of focus; object edge adjustment; and color tone adjustment. The server encodes the edited data based on the result of the editing. It is known that excessively long videos tend to receive fewer views. Accordingly, in order to keep the content within a specific length that scales with the length of the original video, the server may, in addition to the low-priority scenes described above, automatically clip out scenes with low movement, based on an image processing result. Alternatively, the server may generate and encode a video digest based on a result of an analysis of the meaning of a scene.

There may be instances in which individual content may include content that infringes a copyright, moral right, portrait rights, etc. Such instance may lead to an unfavorable situation for the creator, such as when content is shared beyond the scope intended by the creator. Accordingly, before encoding, the server may, for example, edit images so as to blur faces of people in the periphery of the screen or blur the inside of a house, for example. Further, the server may be configured to recognize the faces of people other than a registered person in images to be encoded, and when such faces appear in an image, may apply a mosaic filter, for example, to the face of the person. Alternatively, as pre- or post-processing for encoding, the user may specify, for copyright reasons, a region of an image including a person or a region of the background to be processed. The server may process the specified region by, for example, replacing the region with a different image, or blurring the region. If

the region includes a person, the person may be tracked in the moving picture, and the person's head region may be replaced with another image as the person moves.

Since there is a demand for real-time viewing of content produced by individuals, which tends to be small in data size, the decoder first receives the base layer as the highest priority, and performs decoding and reproduction, although this may differ depending on bandwidth. When the content is reproduced two or more times, such as when the decoder receives the enhancement layer during decoding and reproduction of the base layer, and loops the reproduction, the decoder may reproduce a high image quality video including the enhancement layer. If the stream is encoded using such scalable encoding, the video may be low quality when in an unselected state or at the start of the video, but it can offer an experience in which the image quality of the stream progressively increases in an intelligent manner. This is not limited to just scalable encoding; the same experience can be offered by configuring a single stream from a low quality stream reproduced for the first time and a second stream encoded using the first stream as a reference.

Other Implementation and Application Examples

The encoding and decoding may be performed by LSI (large scale integration circuitry) ex500 (see FIG. 106), which is typically included in each terminal. LSI ex500 may be configured of a single chip or a plurality of chips. Software for encoding and decoding moving pictures may be integrated into some type of a recording medium (such as a CD-ROM, a flexible disk, or a hard disk) that is readable by, for example, computer ex111, and the encoding and decoding may be performed using the software. Furthermore, when smartphone ex115 is equipped with a camera, video data obtained by the camera may be transmitted. In this case, the video data is coded by LSI ex500 included in smartphone ex115.

Note that LSI ex500 may be configured to download and activate an application. In such a case, the terminal first determines whether it is compatible with the scheme used to encode the content, or whether it is capable of executing a specific service. When the terminal is not compatible with the encoding scheme of the content, or when the terminal is not capable of executing a specific service, the terminal first downloads a codec or application software and then obtains and reproduces the content.

Aside from the example of content providing system ex100 that uses internet ex101, at least the moving picture encoder (image encoder) or the moving picture decoder (image decoder) described in the above embodiments may be implemented in a digital broadcasting system. The same encoding processing and decoding processing may be applied to transmit and receive broadcast radio waves superimposed with multiplexed audio and video data using, for example, a satellite, even though this is geared toward multicast, whereas unicast is easier with content providing system ex100.

[Hardware Configuration]

FIG. 109 illustrates further details of smartphone ex115 shown in FIG. 106. FIG. 110 illustrates a configuration example of smartphone ex115. Smartphone ex115 includes antenna ex450 for transmitting and receiving radio waves to and from base station ex110, camera ex465 capable of capturing video and still images, and display ex458 that displays decoded data, such as video captured by camera ex465 and video received by antenna ex450. Smartphone ex115 further includes user interface ex466 such as a touch

panel, audio output unit ex457 such as a speaker for outputting speech or other audio, audio input unit ex456 such as a microphone for audio input, memory ex467 capable of storing decoded data such as captured video or still images, recorded audio, received video or still images, and mail, as well as decoded data, and slot ex464 which is an interface for SIM ex468 for authorizing access to a network and various data. Note that external memory may be used instead of memory ex467.

Main controller ex460, which comprehensively controls display ex458 and user interface ex466, power supply circuit ex461, user interface input controller ex462, video signal processor ex455, camera interface ex463, display controller ex459, modulator/demodulator ex452, multiplexer/demultiplexer ex453, audio signal processor ex454, slot ex464, and memory ex467 are connected via bus ex470.

When the user turns on the power button of power supply circuit ex461, smartphone ex115 is powered on into an operable state, and each component is supplied with power from a battery pack.

Smartphone ex115 performs processing for, for example, calling and data transmission, based on control performed by main controller ex460, which includes a CPU, ROM, and RAM. When making calls, an audio signal recorded by audio input unit ex456 is converted into a digital audio signal by audio signal processor ex454, to which spread spectrum processing is applied by modulator/demodulator ex452 and digital-analog conversion and frequency conversion processing are applied by transmitter/receiver ex451, and the resulting signal is transmitted via antenna ex450. The received data is amplified, frequency converted, and analog-digital converted, inverse spread spectrum processed by modulator/demodulator ex452, converted into an analog audio signal by audio signal processor ex454, and then output from audio output unit ex457. In data transmission mode, text, still-image, or video data is transmitted by main controller ex460 via user interface input controller ex462 based on operation of user interface ex466 of the main body, for example. Similar transmission and reception processing is performed. In data transmission mode, when sending a video, still image, or video and audio, video signal processor ex455 compression encodes, by the moving picture encoding method described in the above embodiments, a video signal stored in memory ex467 or a video signal input from camera ex465, and transmits the encoded video data to multiplexer/demultiplexer ex453. Audio signal processor ex454 encodes an audio signal recorded by audio input unit ex456 while camera ex465 is capturing a video or still image, and transmits the encoded audio data to multiplexer/demultiplexer ex453. Multiplexer/demultiplexer ex453 multiplexes the encoded video data and encoded audio data using a determined scheme, modulates and converts the data using modulator/demodulator (modulator/demodulator circuit) ex452 and transmitter/receiver ex451, and transmits the result via antenna ex450.

When a video appended in an email or a chat, or a video linked from a web page, is received, for example, in order to decode the multiplexed data received via antenna ex450, multiplexer/demultiplexer ex453 demultiplexes the multiplexed data to divide the multiplexed data into a bitstream of video data and a bitstream of audio data, supplies the encoded video data to video signal processor ex455 via synchronous bus ex470, and supplies the encoded audio data to audio signal processor ex454 via synchronous bus ex470. Video signal processor ex455 decodes the video signal using a moving picture decoding method corresponding to the moving picture encoding method described in the above

embodiments, and video or a still image included in the linked moving picture file is displayed on display ex458 via display controller ex459. Audio signal processor ex454 decodes the audio signal and outputs audio from audio output unit ex457. Since real-time streaming is becoming increasingly popular, there may be instances in which reproduction of the audio may be socially inappropriate, depending on the user's environment. Accordingly, as an initial value, a configuration in which only video data is reproduced, i.e., the audio signal is not reproduced, may be preferable; and audio may be synchronized and reproduced only when an input is received from the user clicking video data, for instance.

Although smartphone ex115 was used in the above example, three other implementations are conceivable: a transceiver terminal including both an encoder and a decoder; a transmitter terminal including only an encoder; and a receiver terminal including only a decoder. In the description of the digital broadcasting system, an example is given in which multiplexed data obtained as a result of video data being multiplexed with audio data is received or transmitted. The multiplexed data, however, may be video data multiplexed with data other than audio data, such as text data related to the video. Further, the video data itself rather than multiplexed data may be received or transmitted.

Although main controller ex460 including a CPU is described as controlling the encoding or decoding processes, various terminals often include GPUs. Accordingly, a configuration is acceptable in which a large area is processed at once by making use of the performance ability of the GPU via memory shared by the CPU and GPU, or memory including an address that is managed so as to allow common usage by the CPU and GPU. This makes it possible to shorten encoding time, maintain the real-time nature of streaming, and reduce delay. In particular, processing relating to motion estimation, deblocking filtering, sample adaptive offset (SAO), and transformation/quantization can be

effectively carried out by the GPU, instead of the CPU, in units of pictures, for example, all at once.

INDUSTRIAL APPLICABILITY

The present disclosure is applicable to, for example, television receivers, digital video recorders, car navigation systems, mobile phones, digital cameras, digital video cameras, video conference systems, and electron mirrors.

The invention claimed is:

1. An encoding method comprising:

determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block;

in a case where the mode determined is a first mode, storing, in a bitstream, an index indicating a distance and an angle that define two partitions in the block, and encoding the block using the first mode; and

disabling the first mode and storing of the index in the bitstream in a case where (i) the width is 64 pixels and the height is 8 pixels or (ii) the height is 64 pixels and the width is 8 pixels, wherein

the first mode is a geometric partitioning mode (GPM).

2. A decoding method comprising:

determining a mode to be applied to a block from a plurality of merge modes based on a width of the block and a height of the block;

in a case where the mode determined is a first mode, obtaining, from a bitstream, an index indicating a distance and an angle that define two partitions in the block, and decoding the block using the first mode; and disabling the first mode and obtaining of the index from the bitstream in a case where (i) the width is 64 pixels and the height is 8 pixels or (ii) the height is 64 pixels and the width is 8 pixels, wherein

the first mode is a geometric partitioning mode (GPM).

* * * * *